OptVerse Mathematical Programming Solver User Manual

Release 3.6.0

Huawei OptVerse Team

Aug 10, 2026

CONTENTS

1 OptVerse Solver Introduction **1**
- 1.1 MP Overview . . . 1
- 1.2 CP Overview . . . 2
- 1.3 Contact Information . . . 2

2 Installation Instructions **3**
- 2.1 Supported Platforms . . . 3
- 2.2 Acquisition Channels . . . 3
- 2.3 Installation Guide . . . 3

3 Command Line Tools **7**
- 3.1 OptVerse Solver Command Line Tool Usage . . . 7
- 3.2 Parameter Description . . . 8
- 3.3 Usage Examples . . . 9
- 3.4 Understanding Logs . . . 9

4 OptVerse Solver Quick Start **15**
- 4.1 OptVerse MILP Solver Quick Start . . . 15
- 4.2 OptVerse CP Solver Quick Start . . . 52

5 File Formats **65**
- 5.1 MPS File Format Description . . . 65
- 5.2 LP File Format Description . . . 73
- 5.3 FZN File Format Description . . . 82

6 Attributes References **87**
- 6.1 OptVerse MP Attributes . . . 87
- 6.2 Attribute Examples . . . 104

7 Parameters References **105**
- 7.1 MIP Parameters References . . . 105
- 7.2 Parameter Examples . . . 116
- 7.3 CP Parameters References . . . 117

8 API References **121**
- 8.1 OPTVEnv . . . 121
- 8.2 OptVerse MP Solver API References . . . 130
- 8.3 Enum . . . 394
- 8.4 OptVerse CP Solver API References . . . 396

9 SDK Recordings **481**

9.1 Record 481
9.2 Replay 482

10 Network Flow Planning Problems **483**
10.1 Problem Definition 483
10.2 Problem Example 483

CHAPTER

ONE

OPTVERSE SOLVER INTRODUCTION

OptVerse is a high-performance, general-purpose optimization solver developed by Huawei Cloud.

This user manual provides a concise overview of the functionalities of the OptVerse Mathematical Programming (MP) and Constraint Programming (CP) solvers, along with *installation instructions*, and usage guidance.

The solver can be used in a *Linux* environment and supports the following invocation methods:

- *Command Line*: Solves problems by reading standard format model files, with support for common parameter configurations.
  - OptVerse MP Solver provides APIs for *C++*, *Python*, and *Java*.
  - OptVerse CP Solver provides APIs for *C++* and *Python*.

1.1 MP Overview

At the current stage, the OptVerse solver offers the ability to solve linear programming (LP), quadratically constrained quadratic programming (QCQP), and mixed-integer linear programming (MILP) problems, with a commitment to support more types of optimization problem solving.

Considering the general form of (mixed-integer) linear programming:

$$\begin{aligned} \text{minimize} \quad & c_0 + c^T x \\ \text{subject to} \quad & L \leq Ax \leq U \\ & l \leq x \leq u \\ & x_i \in \mathbb{Z}, i \in \mathrm{I} \end{aligned}$$

where

$$\begin{aligned} c_0 &\in \mathbb{R} \\ c, x, l, u &\in \mathbb{R}^n \\ L, U &\in \mathbb{R}^m \\ A &\in \mathbb{R}^{m \times n} \end{aligned}$$

and I is the index set specifying variables x to take integer values.

The OptVerse MP Solver provides the following solving algorithms:

- For LP problems, the OptVerse solver offers primal/dual simplex and interior point methods (non-deterministic and deterministic algorithms), as well as a concurrent solver.
- For *network flow planning problems* with specific structural features, the OptVerse solver automatically selects the network simplex method for solving.

- For convex QP or QCQP problems (where there are quadratic terms $x'Qx$ in objective or constraints), the OptVerse solver will automatically detect the problem as quadratic and solve it using a barrier method.
- For mixed-integer programming problems (when the set I is non-empty), the OptVerse solver primarily uses the branch and bound method and integrates various industrial heuristic strategies to improve solving efficiency.
- For SOCP problems (where there are second order cone constraints $\|A_i x+b_i\| \leq c_i^T x+d_i$ where i is an integer), the OptVerse solver will automatically detect the problem as an SOCP model and solve it using a barrier method.
- For general nonlinear programming problems (where there are nonlinearities in objective or constraints), the OptVerse solver will automatically detect the problem as a nonlinear programming (NLP) model and solve it using a barrier method for local minima.

Note

The community version can only be used for academic purposes, for commercial uses please contact the solver team.

1.2 CP Overview

OptVerse CP solver offers the ability to solve constraint programming (CP) problems. OptVerse CP solver uses the CP-SAT technique based on the lazy clause generation method and integrates various industrial heuristic strategies to improve solving efficiency.

The solver supports a variety of constraint types, covering the following five categories:

- Logical Constraints: These include logical relationships (such as AND, OR, etc.) between Boolean variables, suitable for modelling decision logic.
- Linear Constraints: These involve linear equations or inequalities with integer or Boolean variables, commonly used for resource balancing and quantity restrictions.
- Global Constraints: These provide highly expressive composite constraints, such as Table (table constraint), AllDifferent (all different constraint), and Element (element index constraint), which can significantly simplify models and accelerate the solving process.
- Problem-specific Constraints: These are specialized constraints tailored for typical application scenarios (such as job scheduling, vehicle routing, 3D bin packing, resource allocation, etc.), enhancing modelling efficiency and solving performance.
- Computational Constraints: These support constraints that involve complex function expressions, such as LinMax (linear maximum) and LinProb (linear product), suitable for advanced modelling needs that require embedded computational logic.

1.3 Contact Information

The OptVerse solver offers end-to-end decision optimization solutions, including problem modelling, model optimization, algorithm tuning, and service deployment. Whether you are looking to improve existing solving solutions, develop alternatives, or apply mathematical optimization tools in new projects, feel free to contact us anytime.

The OptVerse solver is evolving rapidly, continuously improving solving performance and stability, and introducing more user-friendly features. For questions, suggestions, and requirements during the use of the OptVerse solver, you can email: optverse@huawei.com

CHAPTER

TWO

INSTALLATION INSTRUCTIONS

2.1 Supported Platforms

OptVerse currently supports the Linux operating system and has been tested and confirmed to run smoothly on the following systems:

- **Ubuntu**: Ubuntu 18.04, Ubuntu 20.04, Ubuntu 22.04
- **CentOS**: CentOS 8, CentOS 9
- For lower versions of Linux systems, the requirements include *glibc >= 2.33* and *libstdc++ >= 6.0.26*
- We support Python versions 3.10 - 3.13

2.2 Acquisition Channels

2.2.1 Offline Version

- **OptVerse Software Package**: Users can contact the *OptVerse Solver Team* to obtain the package, and a dedicated person will get in touch with you.

2.3 Installation Guide

2.3.1 Offline Version Installation

1. Copy the `optverse` solver installation package `optverse.x.x.x.tar.gz` to a specified path (e.g., `/opt/`). Let's assume the path is `<installdir>`.
2. Unpack the installation package:

```
tar -xzvf optverse.x.x.x.tar.gz
```

This will create a new path `<installdir>/optverse` with the following structure:

```
./
├── bin/
│   └── optverse
├── dist/
│   ├── optvpy-X.X.X-cp310-cp310-linux_x86_64.whl
```

(continues on next page)

(continued from previous page)

```
    ├── optvpy-X.X.X-cp311-cp311-linux_x86_64.whl
    ├── optvpy-X.X.X-cp312-cp312-linux_x86_64.whl
    └── optvpy-X.X.X-cp313-cp313-linux_x86_64.whl
├── examples/
    ├── c++/
        ├── feasibility-relax.cpp
        ├── lp-sensitivity-analysis.cpp
        ├── lp1.cpp
        ├── lp2.cpp
        ├── lp3.cpp
        ├── milp1.cpp
        └── milp2.cpp
    ├── CMakeLists.txt
    ├── data/
        ├── lp-example.lp
        ├── milp-example.mps
        └── testNetwork.mps
    └── python/
        ├── feasibility-relax.py
        ├── lp-sensitivity-analysis.py
        ├── lp1.py
        ├── lp2.py
        ├── lp3.py
        ├── milp1.py
        └── milp2.py
├── include/
    └── optv_c++.h
└── lib/
    ├── libiomp5.so
    └── liboptverse.so
```

3. Set `LD_LIBRARY_PATH`:

```
echo 'export LD_LIBRARY_PATH="<installdir>/optverse/lib:${LD_LIBRARY_PATH}"' >> ~/.bashrc
source ~/.bashrc
```

4. Navigate to the `<installdir>/optverse/bin` path and execute `./optverse --version`. OptVerse solver is successfully installed if the following information shows up:

```
OptVerse Optimizer version 3.6.0 (Internal Release)
Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
```

Note

This product integrates both the Mathematical Programming (MP) solver and the Constraint Programming (CP) solver into a single installation package, enabling one-time deployment with both solvers ready to use.

2.3.2 Python SDK Installation Guide

The wheel file supports installing the Python SDKs for both the MP solver and the CP solver simultaneously. Install it with:

```
pip3 install optvpy-x.x.x-cp3x-cp3xm-linux_x86_64.whl
```

2.3.3 Obtaining the License

The SDK and CLI require a license. Contact the *OptVerse Solver Team* to obtain the license. A dedicated person will be in contact with you.

2.3.4 Using the License

- Place the license somewhere on your machine, for example, `/opt/Optverse.lic`
- Point the `OPTV_LICENSE_FILE` environment variable to the license file: `export OPTV_LICENSE_FILE=/opt/Optverse.lic`

Note

For frequent usage, it is convenient to export this variable in your `.bashrc` file.

```
echo "export OPTV_LICENSE_FILE=/opt/Optverse.lic" >> ~/.bashrc
source ~/.bashrc
cd <installdir>/optverse/bin
./optverse ../examples/data/lp-example.lp
```

CHAPTER

THREE

COMMAND LINE TOOLS

3.1 OptVerse Solver Command Line Tool Usage

3.1.1 Displaying Help and Version Information

Usage:

```
optverse [flag]   # Display help or version information
```

Optional flags are detailed in the *parameter description*.

3.1.2 Solving Problems

Usage:

```
optverse [options]* filename # Set solving parameters, read model file, and solve
```

The optional solving parameter list can be found in *parameter description*.

Currently supported file formats include:

- `.mps`, `.lp` Supported problem formats
- `.gz`, `.z`, `.Z` Supported compressed formats
- `.fzn` Supported problem formats

Note

- `.fzn` format does not support compression.
- For detailed information about the `.mps` format, refer to the *MPS description*, and for `.lp` format, refer to the *LP description*.
- For the `.fzn` format, refer to the *FZN description*.

3.2 Parameter Description

-h, --help Display help information.

-v, --version Display version information.

--method=<int> Solver algorithm.

Allowed values

Value	Meaning
-1	Auto
0	Primal simplex
1	Dual simplex
2	Interior-point (IPM)
3	Deterministic IPM
4	Concurrent

Default
-1

--timeLimit=<double> Solver time limit (sec).

Range
$[0.0, 1.79769 \times 10^{308}]$

Default
1.79769×10^{308}

--memLimit=<int> Solver memory limit (MB).

Range
$[100, 2147483647]$

Default
2147483647

--threads=<int> Solver threads.

Range
[1, platform threads]

Default
Platform physical cores

--gap=<double> MIP target gap.

Range
$[0.0, 1.79769 \times 10^{308}]$

Default
0.0001

--logFile=<string> Solver log path.

Default
"./optv.log"

--resultFile=<string> Solution file path.

Default
"./<filename>.sol"

- If no input parameters are specified, the default behaviour is to display help information.
- The problem model file must be specified as the last input parameter.
- For *Network Flow Problems*, the OptVerse solver will automatically use the network simplex method for solving.

3.3 Usage Examples

- If no parameter is specified, the help information is displayed by default

```
./optverse
```

- Display help information

```
./optverse -h
./optverse --help
```

- Display version information

```
./optverse -v
./optverse --version
```

- Solve the problem in the file `test-problem.mps` with default settings.

```
./optverse test-problem.mps
```

- Solve the problem in the file `test-problem.mps`, specifying the log file as `./test-problem.log`, and the result file as `./test-problem.sol`.

```
./optverse --logFile=./test-problem.log --resultFile=./test-problem.sol␣
↪test-problem.mps
```

- Solve the compressed format problem file `test-problem.mps.gz`, setting a time limit of 7200 seconds, a memory limit of 32GB, a target gap of 0.01%, and using 4 threads for parallel solving.

```
./optverse --timeLimit=7200 --memLimit=32768 --gap=0.0001 --threads=4 test-
↪problem.mps.gz
```

3.4 Understanding Logs

3.4.1 LP Logs

Solve the LP problem file `lp-example.lp` with default parameters. Specify the result file as `lp-example.sol`:

```
./optverse --resultFile=lp-example.sol examples/data/lp-example.lp
```

The obtained log is as follows:

```
Set parameter "logFile" to "optv.log"
Set parameter "resultFile" to "lp-example.sol"
OptVerse Optimizer version "3.6.0"
Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.

Read problem /home/solver/optverse/examples/data/lp-example.lp
Read time: 0.00s

Optimize an LP model
  8 rows, 20 columns and 36 nonzeros

Presolve problem
Presolve time: 0.00s
After presolve:
  4 rows, 12 columns and 12 nonzeros

     Time    Iteration     Objective   Primal Num.Inf.  Dual Num.Inf.
     0.0s            0  0.000000e+00                4              0
     0.0s            4  1.380000e+02                0              0

Postsolving
Postsolve finished

     Time    Iteration     Objective   Primal Num.Inf.  Dual Num.Inf.
     0.0s            0  1.380000e+02                0              0

Solve results
  Status               Optimal solution found
  Objective            1.380000000000e+02
  Simplex iteration    4
  Time                 0.01

Write best solution /home/solver/optverse/lp-example.sol
```

The preceding logs contain the following information:

- Parameter settings, including logFile and resultFile. For details about parameters, see *OptVerse MP Parameters*
- Version and copyright information.
- Problem read time.
- Problem information, including the number of rows, number of columns, and number of non-zero elements.
- Preprocessing time and problem scale after preprocessing, including the number of new rows, columns, and non-zero elements.
- Simplex algorithm iteration log, such as time, number of iteration steps, current target value, and violation of primal and dual feasibility.
- Post-processing information.
- Overview of solution results, including:
  - Solution status, for example, “Optimal solution found”, “Problem is infeasible”, “Problem is unbounded”, “Time limit reached”, “Numerical error”, “Symbolic error”, “Unknown”
  - Optimal target value
  - Total number of iterations

  - Total time elapsed

- Alarm error information such as warnings and errors.

3.4.2 MILP Logs

Solve the MILP problem from the file `milp-example.mps` with default parameters. Specify the result file as *milp-example.sol*:

```
./optverse --resultFile=milp-example.sol examples/data/milp-example.mps
```

The obtained log is as follows:

```
Set parameter "logFile" to "optv.log"
Set parameter "resultFile" to "milp-example.sol"
OptVerse Optimizer version "3.6.0" (Internal Release)
Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.

Read problem /home/solver/optverse/examples/data/milp-example.mps
Read time: 0.00s

Optimize an MILP model
  8 rows, 20 columns (20 binary, 0 integer, 0 continuous) and 36 nonzeros

Presolve problem
Presolve time: 0.00s
After presolve:
  8 rows, 20 columns (20 binary, 0 integer, 0 continuous) and 36 nonzeros

Start parallel solving, using up to 6 threads

    Time    Solved      Open      LPIter     BestBound       BestSol       Gap
  H   0.0s         0         0          0   0.000000e+00   2.980000e+02     100%
  H   0.0s         0         0          0   1.380000e+02   2.880000e+02   52.08%
      0.0s         0         0          0   1.380000e+02   2.880000e+02   52.08%
      0.0s         0         0          9   2.420000e+02   2.880000e+02   15.97%
  H   0.0s         1         0          9   2.420000e+02   2.420000e+02    0.00%
      0.0s         1         0          9   2.420000e+02   2.420000e+02    0.00%

Solve results
  Status               Optimal solution found
  Best solution        2.420000000000e+02
  Best bound           2.420000000000e+02
  Gap                  0.0000%
  Node                 1
  LP iteration         9
  Time                 0.01

Write best solution /home/solver/optverse/milp-example.sol
```

The preceding logs contain the following information:

- Parameter settings, in this example: `logFile` and `resultFile`. See *Parameters* for a full list of parameters.
- Version and copyright information.

- Problem read time.
- Original problem size
- Preprocessing time and problem size after preprocessing
- Start serial, parallel solve (thread count), for example:
  - Start sequential solving
  - Start parallel solving, using up to 4 threads
- Only H and * may appear in the first column (before the Time column)
- Time, Solved, Open, LPIter, BestBound, BestSol, and Gap
- Overview of solution results, including:
  - Solution status. Only "Optimal solution found", "Time limit reached", "Memory limit reached", and "Problem is infeasible" can be displayed.
  - Best primal bound and best dual bound
  - Gap
  - Number of nodes
  - Total number of LP iterations
  - Total time elapsed
- Runtime information such as warnings and errors

3.4.3 CP Logs

Solve the CP problem file `fzn-example.fzn` with default parameters:

```
./optverse --cpThreads=1 examples/data/fzn-example.fzn
```

The obtained log is as follows:

```
Set parameter "cpThreads" to "1"
CP model is created with number of variables: 12.
CP model is created with number of constraints: 10.
CP solver is executed with the following parameters:
time limit: 9.223372036854776e+18 sec
deterministic time limit: 9.223372036854776e+18
number of threads: 1
deterministic mode: 1
random seed: 1
abs gap: 1e-06
relative gap: 0.0

    Time    Solved      Open    It/Node    BestBound       BestSol       Gap
    0.0s         0          0       --   3.450000e+02         --          --
 H   0.0s         0          0       --   3.450000e+02   3.450000e+02    0.00%

 Solve results
    Status               Optimal solution found
    Best solution        3.450000000000e+02
```

(continues on next page)

(continued from previous page)

```
Best bound          3.450000000000e+02
Gap                 0.0000%
Node                0
Time                0.00
```

The preceding logs contain the following information:

- Parameter settings, in this example `cpThreads` is used to set the number of threads
- Original problem size
- CP solver parameters
- Start solve:
  - Only H may appear in the first column (before the Time column)
  - Time, Solved, Open, It/Node, BestBound, BestSol, and Gap
- Overview of solution results, including:
  - Solution status. Only “Optimal solution found”, “Problem is infeasible”, “Input model is feasible”, “Input model is invalid”, “Interrupted by user”, and “Unknown” can be displayed.
  - Best primal bound and best dual bound
  - Gap
  - Number of nodes
  - Total Time Required
- Runtime information such as warnings and errors

CHAPTER

FOUR

OPTVERSE SOLVER QUICK START

4.1 OptVerse MILP Solver Quick Start

This chapter provides several basic examples demonstrating how to use the various language interfaces of the OptVerse MILP Solver to model and solve mathematical programming problems.

4.1.1 C++ MP API

This section presents a simple C++ example to demonstrate how to write a program using the OptVerse MP Solver's C++ API.

This example illustrates how to:

- Set solver parameters
- Construct a model
- Write the constructed problem to file
- Optimize the model
- Print solution information

The mathematical formulation for the problem to be solved is as follows:

$$
\begin{aligned}
\text{minimize} \quad & -x - 14y - 6z, \\
\text{subject to} \quad & 2x + y + z \leq 3 \\
& 3y + z \leq 6, \\
& 0 \leq x \leq 1 \\
& 0 \leq y \\
& 0 \leq z \leq 3 \\
\text{where} \quad & x,\ y \in \mathbb{Z},\ z \in \mathbb{R}.
\end{aligned}
$$

```
/*
 * Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
 * Description: OptVerse c++ interface example - build and solve an MILP problem
 */
#include "optv_c++.h"
using namespace std;

int main()
{
```

(continues on next page)

(continued from previous page)

```
    try {
        OPTVEnv env("milp1.log");
        // Before model construction: set parameters through environment
        env.Set(OPTVDblParam::TIME_LIMIT, 7200);

        OPTVModel model(env);
        // After model construction: set parameters through model
        model.Set(OPTVStrParam::RESULT_FILE, "milp1.sol");

        // Add variables to the model
        OPTVVar x = model.AddVar(0, 1, -1, OPTV_BINARY, "x");
        OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");
        OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z");

        // Add constraint to the model
        OPTVConstr c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0");
        OPTVConstr c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1");

        model.Write("milp1.lp");
        model.Optimize();

        if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
            cout << "Model Status: " << model.Get(OPTVIntAttr::STATUS) << endl;
            cout << "Best solution: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;

            cout << "x = " << x.Get(OPTVDblAttr::X) << endl;
            cout << "y = " << model.GetVar(1).Get(OPTVDblAttr::X) << endl;
            cout << "z = " << model.GetVarByName("z").Get(OPTVDblAttr::X) << endl;
            cout << "c0 = " << model.GetConstr(0).Get(OPTVDblAttr::X) << endl;
            cout << "c1 = " << model.GetConstrByName("c1").Get(OPTVDblAttr::X) << endl;
        } else {
            cout << "No feasible solution available!" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Include the Header

To provide access to the C++ interface, include the header file `optv_c++.h`.

```
#include "optv_c++.h"
```

Create the Environment

The first step is to create the environment object. The name of the log file is set by providing it to the constructor.

```
OPTVEnv env("milp1.log");
```

Configuring Parameters Before Model Construction

To edit solver parameters **before** calling the *OPTVModel* constructor, a user must set the parameter through the environment object. In this example, we demonstrate setting the solver time limit through the environment, prior to creation of the model object.

```
env.Set(OPTVDblParam::TIME_LIMIT, 7200);
```

Configuring Parameters After Model Construction

To edit solver parameters **after** calling the *OPTVModel* constructor, a user must set the parameter through the model object. In this example, we demonstrate setting the solution file through the model.

```
model.Set(OPTVStrParam::RESULT_FILE, "milp1.sol");
```

Creating the Model

After creating the environment, the next step is to create the model. Each OptVerse model is an optimization problem consisting of a set of variables, a set of constraints, and associated attributes (i.e. variable or constraint bounds, solution status, variable or constraint names, etc.).

Construct An Empty Model

If a user wishes to construct a model from scratch using API calls, they will first need to create an empty model.

```
OPTVModel model(env);
```

Loading Model from File

If the `.mps`[1] or `.lp`[2] extension is passed as the second parameter, the problem file is read. This allows solving or modifying a pre-existing model through the API.

```
OPTVEnv env("fileRead.log");
OPTVModel model(env, fileName);
```

For more details regarding this mode, refer to the example `milp2.cpp`

Add Variables to Model

If an empty model was created in the previous step, variables and constraints **must** be added to the model before it can be solved. Alternatively, a model loaded from a file can be expanded through the addition of variables or constraints. The *OPTVModel::AddVar()* method allows adding a variable to a model object. The first and second parameters are the lower and upper bounds respectively. The third parameter is the objective coefficient. The fourth parameter is the variable type, one of: *OPTV_CONTINUOUS*, *OPTV_INTEGER*, or *OPTV_BINARY*. The fifth parameter is the name.

```
OPTVVar x = model.AddVar(0, 1, -1, OPTV_BINARY, "x");
OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");
OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z");
```

If all created variables are *OPTV_CONTINUOUS*, the model is an LP problem, and the program automatically invokes the linear programming algorithm to solve the problem. For details, see the sample code `lp1.cpp`.

Add Constraints to Model

After adding variables, a user may now add constraints to the model. In the OptVerse API, constraint expressions are written as double-sided. This means the user must provide both lower and upper bounds to the constraint object upon creation. The first parameter is the constraint expression of type *OPTVLinExpr*. The second parameter and third parameters are the lower and upper bounds respectively. The fourth parameter is the name of the constraint.

```
OPTVConstr c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0");
OPTVConstr c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1");
```

Model Optimization

Once the model is constructed and any necessary parameters have been set, the model can be solved by invoking *OPTVModel::Optimize()*. After optimization, the solution status and optimal solution are saved in the *OPTVModel* object.

```
model.Optimize();
```

[1] For a description of the MPS format, see *File Formats*.
[2] For a description of the LP format, see *File Formats*.

Querying the Solution

To output the solution after optimization, the *OPTVModel::Get()* method can be used to query various attributes of the model. First, we check the model attribute *OPTVIntAttr::SOL_COUNT* to determine if a feasible solution has been found. Next, we demonstrate querying the model status through *OPTVIntAttr::STATUS*. We obtain the best found objective value by using *OPTVDblAttr::OBJ_VAL*. Finally, we demonstrate three ways in which the value of a variable or constraint can be obtained.

```
if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
    cout << "Model Status: " << model.Get(OPTVIntAttr::STATUS) << endl;
    cout << "Best solution: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;

    cout << "x = " << x.Get(OPTVDblAttr::X) << endl;
    cout << "y = " << model.GetVar(1).Get(OPTVDblAttr::X) << endl;
    cout << "z = " << model.GetVarByName("z").Get(OPTVDblAttr::X) << endl;
    cout << "c0 = " << model.GetConstr(0).Get(OPTVDblAttr::X) << endl;
    cout << "c1 = " << model.GetConstrByName("c1").Get(OPTVDblAttr::X) << endl;
} else {
    cout << "No feasible solution available!" << endl;
}
```

Compiling and Running

Several example files for the mathematical programming solver have been provided in `${OPTV_HOME}/examples/c++` along with the CMake file for compiling the examples on Linux.

To compile and run the examples, go to the `${OPTV_HOME}/examples` directory and execute the following:

```
mkdir build && cd build
cmake ..
make -j $(nproc)
```

After compilation, the example from this section can be run by calling:

```
./milp1
```

If a user already has a problem in the form of an `.mps` or `.lp` file, they can optimize and check the objective and status through:

```
./milp2 xxx.mps
```

4.1.2 Python API

This section will demonstrate the use of the OptVerse MP Solver's Python interface using the same example in the *previous section*. This example builds a model, solves it, and outputs the optimal solution.

```
"""
   Copyright (c) Huawei Technologies Co., Ltd. 2024. All rights reserved.
   Description: OptVerse python interface example - build and solve an MILP problem

   The mathematical formulation for the problem to be solved is as follows:
     min    - x - 14 y - 6 z
```

(continues on next page)

(continued from previous page)

```
    s.t.   2 x +    y +   z <= 3
                 3 y +   z <= 6
    where x is binary, y is integer, z is continuous
             0 <= x <= 1
             0 <= y
             0 <= z <= 3
"""

from optvpy import *

if __name__ == "__main__":
    # Create an environment and set log file
    env = OPTVEnv("milp1.log")
    env.Set(OPTVDblParam.TIME_LIMIT, 7200)

    # Create an empty model
    model = OPTVModel(env)
    model.Set(OPTVStrParam.RESULT_FILE, "milp1.sol")

    # Add variables to the model
    x = model.AddVar(0, 1, -1, OPTV_BINARY, "x")
    y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y")
    z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z")

    # Add constraint to the model
    c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0")
    c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1")

    model.Write('milp1.lp')
    # Optimize the model
    model.Optimize()

    # Output solution
    print(f"Best solution: {model.Get(OPTVDblAttr.OBJ_VAL)}")
    print(f"x = {x.Get(OPTVDblAttr.X)}")
    print(f"y = {model.GetVar(1).Get(OPTVDblAttr.X)}")
    print(f"z = {model.GetVarByName('z').Get(OPTVDblAttr.X)}")
```

Import the Module

To provide access to the Python interface, import the package `optvpy`.

```
from optvpy import *
```

Create the Environment

The first step is to create the environment object. The name of the log file is set by providing it to the constructor.

```
# Create an environment and set log file
env = OPTVEnv("milp1.log")
```

Configuring Parameters Before Model Construction

To edit solver parameters **before** calling the *OPTVModel* constructor, a user must set the parameter through the environment object. In this example, we demonstrate setting the solver time limit through the environment, prior to creation of the model object.

```
env.Set(OPTVDblParam.TIME_LIMIT, 7200)
```

Configuring Parameters After Model Construction

To edit solver parameters **after** calling the *OPTVModel* constructor, a user must set the parameter through the model object. In this example, we demonstrate setting the solution file through the model.

```
model.Set(OPTVStrParam.RESULT_FILE, "milp1.sol")
```

Creating the Model

After creating the environment, the next step is to create the model. Each OptVerse model is an optimization problem consisting of a set of variables, a set of constraints, and associated attributes (i.e. variable or constraint bounds, solution status, variable or constraint names, etc.).

Construct An Empty Model

If a user wishes to construct a model from scratch using API calls, they will first need to create an empty model.

```
# Create an empty model
model = OPTVModel(env)
```

Loading Model from File

If the `.mps`[1] or `.lp`[2] extension is passed as the second parameter, the problem file is read. This allows solving or modifying a pre-existing model through the API.

```
env = OPTVEnv("fileRead.log")
model = OPTVModel(env, fileName)
```

For more details regarding this mode, refer to the example `milp2.py`.

[1] For a description of the MPS format, see *File Formats*.
[2] For a description of the LP format, see *File Formats*.

Add Variables to Model

If an empty model was created in the previous step, variables and constraints **must** be added to the model before it can be solved. Alternatively, a model loaded from a file can be expanded through the addition of variables or constraints. The *OPTVModel.AddVar()* method allows adding a variable to a model object. The first and second parameters are the lower and upper bounds respectively. The third parameter is the objective coefficient. The fourth parameter is the variable type, one of: *OPTV_CONTINUOUS*, *OPTV_INTEGER*, or *OPTV_BINARY*. The fifth parameter is the name.

```
# Add variables to the model
x = model.AddVar(0, 1, -1, OPTV_BINARY, "x")
y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y")
z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z")
```

If all created variables are *OPTV_CONTINUOUS*, the model is an LP problem, and the program automatically invokes the linear programming algorithm to solve the problem. For details, see the sample code `lp1.py`.

Add Constraints to Model

After adding variables, a user may now add constraints to the model. In the OptVerse API, constraint expressions are written as double-sided. This means the user must provide both lower and upper bounds to the constraint object upon creation. The first parameter is the constraint expression of type *OPTVLinExpr*. The second parameter and third parameters are the lower and upper bounds respectively. The fourth parameter is the name of the constraint.

```
# Add constraint to the model
c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0")
c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1")
```

Model Optimization

Once the model is constructed and any necessary parameters have been set, the model can be solved by invoking *OPTVModel.Optimize()*. After optimization, the solution status and optimal solution are saved in the *OPTVModel* object.

```
# Optimize the model
model.Optimize()
```

Querying the Solution

To output the solution after optimization, the *OPTVModel.Get()* method can be used to query various attributes of the model. We obtain the best found objective value by using *OPTVDblAttr.OBJ_VAL*. Finally, we demonstrate three ways in which the value of a variable or constraint can be obtained.

```
# Output solution
print(f"Best solution: {model.Get(OPTVDblAttr.OBJ_VAL)}")
print(f"x = {x.Get(OPTVDblAttr.X)}")
print(f"y = {model.GetVar(1).Get(OPTVDblAttr.X)}")
print(f"z = {model.GetVarByName('z').Get(OPTVDblAttr.X)}")
```

Running Examples

Several example files have been provided in `${OPTV_HOME}/examples/python`.

To run the examples, go to the `${OPTV_HOME}/examples` directory and call the scripts through Python.

```
python3 milp1.py
```

If a user already has a problem in the form of an `.mps` or `.lp` file, they can optimize and check the objective and status through:

```
python3 milp2.py xxx.mps
```

4.1.3 Java API

This section will demonstrate the use of the OptVerse MP Solver's Java interface using the same example in the *previous section*. This example builds a model, solves it, and outputs the optimal solution.

```
import com.huaweicloud.optv.*;

public class Milp1 {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("Milp1.log");
            env.set(OPTVDblParam.TIME_LIMIT, 7200);


            OPTVModel model = new OPTVModel(env);
            model.set(OPTVStrParam.RESULT_FILE, "Milp1.sol");

            // add variables to the model
            OPTVVar x = model.addVar(0, 1, -1, OPTV.OPTV_BINARY, "x");
            OPTVVar y = model.addVar(0, OPTV.OPTV_INF, -14, OPTV.OPTV_INTEGER, "y");
            OPTVVar z = model.addVar(0, 3, -6, OPTV.OPTV_CONTINUOUS, "z");

            // add constraint to the model
            OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.add(OPTV.mul(2, x), y), z), -
↪OPTV.OPTV_INF, 3, "c0");
            OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(3, y), z), -OPTV.OPTV_INF,␣
↪6, "c1");

            model.write("Milp1.lp");
            model.optimize();

            if (model.get(OPTVIntAttr.SOL_COUNT) > 0) {
                System.out.println("Best solution: " + model.get(OPTVDblAttr.OBJ_VAL));

                System.out.println("x = " + x.get(OPTVDblAttr.X));
                System.out.println("y = " + model.getVar(1).get(OPTVDblAttr.X));
                System.out.println("z = " + model.getVarByName("z").get(OPTVDblAttr.X));
            } else {
                System.out.println("No feasible solution available!");
            }
```

(continues on next page)

(continued from previous page)

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Import the Module

To provide access to the Java interface, import the package `com.huaweicloud.optv`.

```
import com.huaweicloud.optv.*;
```

Create the Environment

The first step is to create the environment object. The name of the log file is set by providing it to the constructor.

```
OPTVEnv env = new OPTVEnv("Milp1.log");
```

Configuring Parameters Before Model Construction

To edit solver parameters **before** calling the *OPTVModel* constructor, a user must set the parameter through the environment object. In this example, we demonstrate setting the solver time limit through the environment, prior to creation of the model object.

```
env.set(OPTVDblParam.TIME_LIMIT, 7200);
```

Configuring Parameters After Model Construction

To edit solver parameters **after** calling the *OPTVModel* constructor, a user must set the parameter through the model object. In this example, we demonstrate setting the solution file through the model.

```
model.set(OPTVStrParam.RESULT_FILE, "Milp1.sol");
```

Creating the Model

After creating the environment, the next step is to create the model. Each OptVerse model is an optimization problem consisting of a set of variables, a set of constraints, and associated attributes (i.e. variable or constraint bounds, solution status, variable or constraint names, etc.).

Construct An Empty Model

If a user wishes to construct a model from scratch using API calls, they will first need to create an empty model.

```
OPTVModel model = new OPTVModel(env);
```

Loading Model from File

If the `.mps`[1] or `.lp`[2] extension is passed as the second parameter, the problem file is read. This allows solving or modifying a pre-existing model through the API.

```
OPTVEnv env = new OPTVEnv("fileRead.log")
OPTVModel model = new OPTVModel(env, fileName)
```

For more details regarding this mode, refer to the example `Milp2.java`

Add Variables to Model

If an empty model was created in the previous step, variables and constraints **must** be added to the model before it can be solved. Alternatively, a model loaded from a file can be expanded through the addition of variables or constraints. The *OPTVModel.addVar()* method allows adding a variable to a model object. The first and second parameters are the lower and upper bounds respectively. The third parameter is the objective coefficient. The fourth parameter is the variable type, one of: *OPTV_CONTINUOUS*, *OPTV_INTEGER*, or *OPTV_BINARY*. The fifth parameter is the name.

```
// add variables to the model
OPTVVar x = model.addVar(0, 1, -1, OPTV.OPTV_BINARY, "x");
OPTVVar y = model.addVar(0, OPTV.OPTV_INF, -14, OPTV.OPTV_INTEGER, "y");
OPTVVar z = model.addVar(0, 3, -6, OPTV.OPTV_CONTINUOUS, "z");
```

If all created variables are *OPTV_CONTINUOUS*, the model is an LP problem, and the program automatically invokes the linear programming algorithm to solve the problem. For details, see the sample code `Lp1.java`.

Add Constraints to Model

After adding variables, a user may now add constraints to the model. In the OptVerse API, constraint expressions are written as double-sided. This means the user must provide both lower and upper bounds to the constraint object upon creation. The first parameter is the constraint expression of type *OPTVLinExpr*. The second parameter and third parameters are the lower and upper bounds respectively. The fourth parameter is the name of the constraint.

```
// add constraint to the model
OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.add(OPTV.mul(2, x), y), z), -OPTV.OPTV_INF,
↪ 3, "c0");
OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(3, y), z), -OPTV.OPTV_INF, 6, "c1");
```

[1] For a description of the MPS format, see *File Formats*.
[2] For a description of the LP format, see *File Formats*.

Model Optimization

Once the model is constructed and any necessary parameters have been set, the model can be solved by invoking *OPTVModel.optimize()*. After optimization, the solution status and optimal solution are saved in the *OPTVModel* object.

```
model.optimize();
```

Querying the Solution

To output the solution after optimization, the *OPTVModel.get()* method can be used to query various attributes of the model. We obtain the best found objective value by using *OPTVDblAttr.OBJ_VAL*. Finally, we demonstrate three ways in which the value of a variable or constraint can be obtained.

```
System.out.println("Best solution: " + model.get(OPTVDblAttr.OBJ_VAL));

System.out.println("x = " + x.get(OPTVDblAttr.X));
System.out.println("y = " + model.getVar(1).get(OPTVDblAttr.X));
System.out.println("z = " + model.getVarByName("z").get(OPTVDblAttr.X));
```

Compiling and Running

Several example files have been provided in `${OPTV_HOME}/examples/java`.

To compile and run the examples, go to the `${OPTV_HOME}/examples` directory and execute the following:

```
javac -cp .:${OPTV_HOME}/lib/optv-java-sdk.jar Milp1.java
```

After compilation, the examples can be called through Java:

```
java -cp .:${OPTV_HOME}/lib/optv-java-sdk.jar Milp1
```

If a user already has a problem in the form of an `.mps` or `.lp` file, they can optimize and check the objective and status through:

```
java -cp .:${OPTV_HOME}/lib/optv-java-sdk.jar Milp2
```

4.1.4 Feasibility Relaxation

This section provides the user with a simple example demonstrating how an infeasible problem can be relaxed such that the problem becomes feasible.

The mathematical formulation for the problem to be solved is as follows:

$$\begin{aligned} \text{minimize} \quad & x + y \\ \text{subject to} \quad & 2x + y \quad \leq 2 \\ & -3x + 2y \quad \leq -3 \\ & x - 2y \quad \leq -2 \\ & 0 \quad \leq x, y \\ & x, y \in \mathbb{R}. \end{aligned} \tag{4.1}$$

C++

```cpp
#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("feasibility-relax.log");
        OPTVModel model(env);
        model.Set(OPTVStrParam::RESULT_FILE, "feasibility-relax.sol");

        // Add variables to the model, all variables are continuous
        OPTVVar x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x");
        OPTVVar y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y");

        // Add constraint to the model
        OPTVConstr c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0");
        OPTVConstr c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1");
        OPTVConstr c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2");

        model.FeasRelax();

        cout << "L1 violation: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;
        cout << "c2 upper bound = " << c2.Get(OPTVDblAttr::UB) << endl;
    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Python

```python
from optvpy import *

if __name__ == "__main__":
    try:
        env = OPTVEnv("feasibility-relax.log")
        model = OPTVModel(env)
        model.Set(OPTVStrParam.RESULT_FILE, "feasibility-relax.sol")
        obj = 0.0

        # Add variables to the model, all variables are continuous
        x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x")
        y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y")

        # Add constraint to the model
        c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0")
        c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1")
```

(continues on next page)

(continued from previous page)

```
        c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2")

        obj = model.FeasRelax()

        print(f"L1 violation: {model.OBJ_VAL}")
        print(f"c2 upper bound = {c2.UB}")
    except OPTVException as e:
        print(f"{e.GetErrorCode()}: {e.GetMessage()}")
    except Exception:
        print("Unknown exception")
```

Java

```
import com.huaweicloud.optv.*;

public class FeasibilityRelax {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("FeasibilityRelax.log");
            OPTVModel model = new OPTVModel(env);
            model.set(OPTVStrParam.RESULT_FILE, "FeasibilityRelax.sol");

            // add variables to the model, all variables are continuous
            OPTVVar x = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "x");
            OPTVVar y = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "y");

            // add constraint to the model
            OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.mul(2, x), y), -OPTV.OPTV_INF,␣
↪2, "c0");
            OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(-3, x), OPTV.mul(2, y)), -
↪OPTV.OPTV_INF, -3, "c1");
            OPTVConstr c2 = model.addConstr(OPTV.sub(x, OPTV.mul(2, y)), -OPTV.OPTV_INF,␣
↪-2, "c2");

            model.feasRelax();

            System.out.println("L1 violation: " + model.get(OPTVDblAttr.OBJ_VAL));
            System.out.println("c2 upper bound = " + c2.get(OPTVDblAttr.UB));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Import the Library

To provide access to the OptVerse interface, we must import the library for the desired language.

C++

```
#include "optv_c++.h"
```

Python

```
from optvpy import *
```

Java

```
import com.huaweicloud.optv.*;
```

Creating the Model

To create a model, an environment must be instantiated and provided to the *OPTVModel* constructor.

C++

```
OPTVEnv env("feasibility-relax.log");
OPTVModel model(env);
```

Python

```
env = OPTVEnv("feasibility-relax.log")
model = OPTVModel(env)
```

Java

```
OPTVEnv env = new OPTVEnv("FeasibilityRelax.log");
OPTVModel model = new OPTVModel(env);
```

Adding Variables and Constraints

C++

```
// Add variables to the model, all variables are continuous
OPTVVar x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x");
OPTVVar y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y");

// Add constraint to the model
OPTVConstr c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0");
OPTVConstr c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1");
OPTVConstr c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2");
```

Python

```
# Add variables to the model, all variables are continuous
x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x")
y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y")

# Add constraint to the model
c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0")
c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1")
c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2")
```

Java

```
// add variables to the model, all variables are continuous
OPTVVar x = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "x");
OPTVVar y = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "y");

// add constraint to the model
OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.mul(2, x), y), -OPTV.OPTV_INF, 2, "c0");
OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(-3, x), OPTV.mul(2, y)), -OPTV.OPTV_
↪INF, -3, "c1");
OPTVConstr c2 = model.addConstr(OPTV.sub(x, OPTV.mul(2, y)), -OPTV.OPTV_INF, -2, "c2");
```

Perform the Feasibility Relaxation

The LP feasibility relaxation is performed by invoking the *OPTVModel::FeasRelax()* function.

C++

```
model.FeasRelax();
```

Python

```
obj = model.FeasRelax()
```

Java

```
model.feasRelax();
```

Output the Result

To output the results of the relaxation we can query the L^1 norm of the violation through *model.Get(OPTVDblAttr::OBJ_VAL)*. The suggested change to the bounds can be found through querying the constraint bounds, as in *c2.Get(OPTVDblAttr::UB)*.

C++

```
cout << "L1 violation: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;
cout << "c2 upper bound = " << c2.Get(OPTVDblAttr::UB) << endl;
```

Python

```
print(f"L1 violation: {model.OBJ_VAL}")
print(f"c2 upper bound = {c2.UB}")
```

Java

```
System.out.println("L1 violation: " + model.get(OPTVDblAttr.OBJ_VAL));
System.out.println("c2 upper bound = " + c2.get(OPTVDblAttr.UB));
```

4.1.5 Irreducible Infeasible Subsystem (IIS)

This section demonstrates how to use the OptVerse solver to perform an IIS evaluation through the OptVerse SDK. This example will build a model, compute the IIS and write the IIS to a file.

The mathematical formulation for the problem to be solved is as follows:

$$
\begin{array}{lrrrrl}
\text{minimize} & & x_1 & + x_2 + & x_3 & \\
\text{subject to} & 1 \leq & x_1 & - 4x_2 + & x_3 & \\
& & 3x_1 & - x_2 - & x_3 & \leq 2 \\
& 3 \leq & x_1 & + 2x_2 - & 3x_3 & \\
& & x_1 & - x_2 - & & \leq 5 \\
& & 5x_1 & + 3x_2 + & x_3 & \leq 20 \\
& & & 0 \leq x_1,\ x_2,\ x_3 & & \\
& & & x_1,\ x_2,\ x_3 \in \mathbb{R}. & &
\end{array}
\tag{4.2}
$$

C++

```cpp
#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("iis.log");
        OPTVModel model(env);

        // Add variables to the model, all variables are continuous
        OPTVVar x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
        OPTVVar x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
        OPTVVar x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

        // Add constraints to the model
        OPTVConstr c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
        OPTVConstr c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
        OPTVConstr c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
        OPTVConstr c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4");
        OPTVConstr c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");

        if (model.ComputeIIS()) {
            model.WriteIIS("iis.ilp");
        } else {
            cout << "IIS computation failed" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Python

```python
from optvpy import *

if __name__ == "__main__":
    try:
        env = OPTVEnv("iis.log")
        model = OPTVModel(env)

        # Add variables to the model, all variables are continuous
        x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1")
        x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2")
        x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3")

        # Add constraints to the model
        c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1")
        c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2")
        c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3")
        c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4")
        c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5")

        if model.ComputeIIS():
            model.WriteIIS("iis.ilp")
        else:
            print("IIS computation failed")
    except OPTVException as e:
        print(f"{e.GetErrorCode()}: {e.GetMessage()}")
    except Exception:
        print("Unknown exception")
```

Java

```java
import com.huaweicloud.optv.*;

public class IISExample {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("iis.log");
            OPTVModel model = new OPTVModel(env);

            // Add variables to the model, all variables are continuous
            OPTVVar x1 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
            OPTVVar x2 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
            OPTVVar x3 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

            // Add constraints to the model
            OPTVConstr c1 = model.addConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
            OPTVConstr c2 = model.addConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
            OPTVConstr c3 = model.addConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
            OPTVConstr c4 = model.addConstr(x1 - x2, -OPTV_INF, 5, "c4");
            OPTVConstr c5 = model.addConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");
```

(continues on next page)

(continued from previous page)

```
            if (model.computeIIS()) {
                model.writeIIS("iis.ilp");
            } else {
                System.out.println("IIS computation failed");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Import the Library

To provide access to the OptVerse interface, we must import the library for the desired language.

C++

```
#include "optv_c++.h"
```

Python

```
from optvpy import *
```

Java

```
import com.huaweicloud.optv.*;
```

Creating the Model

To create a model, an environment must be instantiated and provided to the *OPTVModel* constructor.

C++

```
OPTVEnv env("iis.log");
OPTVModel model(env);
```

Python

```
env = OPTVEnv("iis.log")
model = OPTVModel(env)
```

Java

```
OPTVEnv env = new OPTVEnv("iis.log");
OPTVModel model = new OPTVModel(env);
```

Adding Variables and Constraints

C++

```
// Add variables to the model, all variables are continuous
OPTVVar x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

// Add constraints to the model
OPTVConstr c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
OPTVConstr c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
OPTVConstr c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
OPTVConstr c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4");
OPTVConstr c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");
```

Python

```
# Add variables to the model, all variables are continuous
x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3")

# Add constraints to the model
c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1")
c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2")
c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3")
c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4")
c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5")
```

Java

```
// Add variables to the model, all variables are continuous
OPTVVar x1 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

// Add constraints to the model
OPTVConstr c1 = model.addConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
OPTVConstr c2 = model.addConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
OPTVConstr c3 = model.addConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
OPTVConstr c4 = model.addConstr(x1 - x2, -OPTV_INF, 5, "c4");
OPTVConstr c5 = model.addConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");
```

Perform the IIS Evaluation

The IIS evaluation is performed by invoking the *OPTVModel::ComputeIIS()* function. This function will return a boolean value indicating whether the IIS was computed successfully or not.

C++

```
if (model.ComputeIIS()) {
    model.WriteIIS("iis.ilp");
} else {
    cout << "IIS computation failed" << endl;
}
```

Python

```
if model.ComputeIIS():
    model.WriteIIS("iis.ilp")
else:
    print("IIS computation failed")
```

Java

```
if (model.computeIIS()) {
    model.writeIIS("iis.ilp");
} else {
    System.out.println("IIS computation failed");
}
```

In this example the IIS is bounded by the c_1, c_2, c_3 constraints and the variables x_1, x_2, x_3 involved in it, since $c_1 + (-c2) + c3 \implies 2 \leq -x_1 - x_2 - x_3$. However, because the variables involved are all greater than or equal to 0, a contradiction arises, as they cannot all be true at the same time.

4.1.6 LP Sensitivity Analysis

This section will demonstrate the use of LP Sensitivity Analysis with a simple example. This example builds a model, solves it, and outputs the sensitivity analysis information.

The mathematical formulation for the problem to be solved is as follows:

$$\begin{aligned} \text{minimize} \quad & -x_1 - 2x_2 - 3x_3 \\ \text{subject to} \quad & -x_1 + x_2 + x_3 \leq 20 \\ & x_1 - 3x_2 + x_3 \leq 30 \\ & x_1 \leq 40, x_2 \geq 0, x_3 \geq 0 \\ & x_1, x_2, x_3 \in \mathbb{R}. \end{aligned} \tag{4.3}$$

C++

```
/*
* Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
* Description: OptVerse c++ interface example - LP sensitivity analysis functionalities
*/

#include <iomanip>

#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("lp-sensitivity-analysis.log");
        OPTVModel model(env);
        model.Set(OPTVStrParam::RESULT_FILE, "lp-sensitivity-analysis.sol");

        // Construct the model
        OPTVVar x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1");
        OPTVVar x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2");
        OPTVVar x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3");
        std::vector<OPTVVar> x {x1, x2, x3};

        model.AddConstr(-x1 + x2 +x3, -OPTV_INF, 20, "c1");
        model.AddConstr(x1 - 3*x2 +x3, -OPTV_INF, 30, "c2");

        // Optimize the model
        model.Optimize();
        if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
            std::vector<int> rows{0, 1};
            std::vector<int> cols{0, 1, 2};
            cout << "-----------------Display---------------" << endl;

            cout << "Optimal Solution:" << endl;
            cout << std::setprecision(2) << std::scientific;
            for (auto j: cols) {
                cout << "x"<< (j+1) << " = " << x[j].Get(OPTVDblAttr::X) << endl;
```

(continues on next page)

(continued from previous page)

```
            }

            // Sensitivity analysis on variables

            // Reduced cost: how much the objective will change if we increase the␣
↪simple bound constraints by 1
            cout << "Reduced Cost:" << endl;
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : " << model.GetVar(j).Get(OPTVDblAttr::DUAL) <
↪< endl;
            }


            // Objective coefficient sensitivity info, within which the current optimal␣
↪basis would remain optimal
            cout << "objective SA:" << endl;
            std::vector<double> objLower{};
            std::vector<double> objUpper{};
            bool ok = model.GetObjSA(objLower, objUpper, cols);
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : [" << objLower[j] <<", " << objUpper[j] << "]
↪" << endl;
            }

            // Lower bound sensitivity info, within which the current optimal basis␣
↪would remain optimal
            cout << "Lower bound SA:" << endl;
            std::vector<double> lbLower{};
            std::vector<double> lbUpper{};
            ok = model.GetVarLbSA(lbLower, lbUpper, cols);
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : [" << lbLower[j] <<", " << lbUpper[j] << "]" <
↪< endl;
            }

            // Upper bound sensitivity info, within which the current optimal basis␣
↪would remain optimal
            cout << "Upper bound SA:" << endl;
            std::vector<double> ubLower{};
            std::vector<double> ubUpper{};
            ok = model.GetVarUbSA(ubLower, ubUpper, cols);
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : [" << ubLower[j] <<", " << ubUpper[j] << "]" <
↪< endl;
            }

            // Sensitivity analysis on constraints

            // Shadow price: how much the objective will change if we increase the right-
↪hand side of a structural constraint by 1
            cout << "Shadow Price:" << endl;
            for (auto i : rows) {
```

(continues on next page)

(continued from previous page)

```
                cout << "c"<< (i+1) << " : " << model.GetConstr(i).
↪Get(OPTVDblAttr::DUAL) << endl;
            }

            // LHS sensitivity info, within which the current optimal basis would remain␣
↪optimal
            cout << "Constraint left-hand-side SA:" << endl;
            std::vector<double> lhsLower{};
            std::vector<double> lhsUpper{};
            ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows);
            for (auto i : rows) {
                cout << "c"<< (i+1) << " : [" << lhsLower[i] <<", " << lhsUpper[i] << "]
↪" << endl;
            }

            // RHS sensitivity info, within which the current optimal basis would remain␣
↪optimal
            cout << "Constraint right-hand-side SA:" << endl;
            std::vector<double> rhsLower{};
            std::vector<double> rhsUpper{};
            ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows);
            for (auto i : rows) {
                cout << "c"<< (i+1) << " : [" << rhsLower[i] <<", " << rhsUpper[i] << "]
↪" << endl;
            }
        } else {
            cout << std::defaultfloat << "No feasible solution available!" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Python

```
from __future__ import print_function
from optvpy import *

def print(*args):
    """ Print globally in scientific notation with a given precision """
    __builtins__.print(*("%.15e" % arg if isinstance(arg, float) else arg for arg in␣
↪args))

if __name__ == "__main__":
    env = OPTVEnv("lp-sensitivity-analysis.log")
    model = OPTVModel(env)
```

(continues on next page)

(continued from previous page)

```
    model.Set(OPTVStrParam.RESULT_FILE, "lp-sensitivity-analysis.sol")

    # Construct the model
    x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
    x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
    x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
    x = [x1, x2, x3]
    model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
    model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")

    # Optimize the model
    model.Optimize()
    if model.Get(OPTVIntAttr.SOL_COUNT) > 0:
        rows = [0, 1]
        cols = [0, 1, 2]

        print("-----------------Display---------------")

        print("Optimal Solution:")
        for j in range(3):
            print("x%s = %s" % (j+1, x[j].Get(OPTVDblAttr.X)))

        # Sensitivity analysis on variables

        # Reduced cost: how much the objective will change if we increase the simple␣
↪bound constraints by 1
        print("Reduced Cost:")
        for i in cols:
            print("x%s : %s" % (i+1, model.GetVar(i).Get(OPTVDblAttr.DUAL)))

        # Objective coefficient sensitivity info, within which the current optimal basis␣
↪would remain optimal
        print("objective SA:")
        objLower = []
        objUpper = []
        ok = model.GetObjSA(objLower, objUpper, cols)
        for i in cols:
            print("x%s : [%s, %s]" % (i+1, objLower[i], objUpper[i]))

        # Lower bound sensitivity info, within which the current optimal basis would␣
↪remain optimal
        print("Lower bound SA:")
        lbLower = []
        lbUpper = []
        ok = model.GetVarLbSA(lbLower, lbUpper, cols)
        for i in cols:
            print("x%s : [%s, %s]" % (i+1, lbLower[i], lbUpper[i]))

        # Upper bound sensitivity info, within which the current optimal basis would␣
↪remain optimal
        print("Upper bound SA:")
        ubLower = []
```

(continues on next page)

(continued from previous page)

```
        ubUpper = []
        ok = model.GetVarUbSA(ubLower, ubUpper, cols)
        for i in cols:
            print("x%s : [%s, %s]" % (i+1, ubLower[i], ubUpper[i]))

        # Sensitivity analysis on constraints

        # Shadow price: how much the objective will change if we increase the right-hand␣
↪side of a structural constraint by 1
        print("Shadow Price:")
        for i in rows:
            print("c%s : %s" % (i+1, model.GetConstr(i).Get(OPTVDblAttr.DUAL)))

        # LHS sensitivity info, within which the current optimal basis would remain␣
↪optimal
        print("Constraint left-hand-side SA:")
        lhsLower = []
        lhsUpper = []
        ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows)
        for i in rows:
            print("c%s : [%s, %s]" % (i+1, lhsLower[i], lhsUpper[i]))

        # RHS sensitivity info, within which the current optimal basis would remain␣
↪optimal
        print("Constraint right-hand-side SA:")
        rhsLower = []
        rhsUpper = []
        ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows)
        for i in rows:
            print("c%s : [%s, %s]" % (i+1, rhsLower[i], rhsUpper[i]))
    else:
        print("No feasible solution available!")
```

Java

```
import com.huaweicloud.optv.*;
import java.util.*;

public class LpSensitivityAnalysis {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("LpSensitivityAnalysis.log");
            OPTVModel model = new OPTVModel(env);
            model.set(OPTVStrParam.RESULT_FILE, "LpSensitivityAnalysis.sol");

            // construct the model
            OPTVVar x1 = model.addVar(-OPTV.OPTV_INF, 40, -1, OPTV.OPTV_CONTINUOUS, "x1
↪");
            OPTVVar x2 = model.addVar(0, OPTV.OPTV_INF, -2, OPTV.OPTV_CONTINUOUS, "x2");
            OPTVVar x3 = model.addVar(0, OPTV.OPTV_INF, -3, OPTV.OPTV_CONTINUOUS, "x3");
```

(continues on next page)

(continued from previous page)

```
            OPTVVar[] x = {x1, x2, x3};
            // -x1 + x2 +x3
            model.addConstr(OPTV.add(OPTV.add(OPTV.sub(x1), x2), x3), -OPTV.OPTV_INF, 20,
↪ "c1");
            // x1 - 3*x2 +x3
            model.addConstr(OPTV.add(OPTV.sub(x1, OPTV.mul(3, x2)), x3), -OPTV.OPTV_INF,␣
↪30, "c2");

            // optimize the model
            model.optimize();
            model.write("LpSensitivityAnalysis.lp");

            if (model.get(OPTVIntAttr.SOL_COUNT) > 0) {
                int[] rows = {0, 1};
                int[] cols = {0, 1, 2};

                System.out.println("-----------------Display---------------");

                System.out.println("Optimal Solution:");
                for (int j: cols) {
                    System.out.printf("x%d = %.2e\n", j+1, x[j].get(OPTVDblAttr.X));
                }

                // // sensitivity analysis on variables

                // reduced cost: how much the objective will change if we increase the␣
↪simple bound constraints by 1
                System.out.println("Reduced Cost:");
                for (int j : cols) {
                    System.out.printf("x%d : %.2e\n", j+1, model.getVar(j).
↪get(OPTVDblAttr.DUAL));
                }

                // objective coefficient sensitivity info, within which the current␣
↪optimal basis would remain optimal
                System.out.println("objective SA:");
                OPTVModel.SensitivityBounds objBounds = model.getObjSA(cols);
                for (int j : cols) {
                    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, objBounds.down[j],␣
↪objBounds.up[j]);
                }

                // lower bound sensitivity info, within which the current optimal basis␣
↪would remain optimal
                System.out.println("Lower bound SA:");
                OPTVModel.SensitvityBounds lbBounds = model.getVarLbSA(cols);
                for (int j : cols) {
                    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, lbBounds.down[j],␣
↪lbBounds.up[j]);
                }

                // upper bound sensitivity info, within which the current optimal basis␣
```

(continues on next page)

(continued from previous page)

```
↪would remain optimal
                System.out.println("Upper bound SA:");
                OPTVModel.SensitvityBounds ubBound = model.getVarUbSA(cols);
                for (int j : cols) {
                    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, ubBound.down[j],␣
↪ubBound.up[j]);
                }

                // // sensitivity analysis on constraints

                // shadow price: how much the objective will change if we increase the␣
↪right-hand side of a structural constraint by 1
                System.out.println("Shadow Price:");
                for (int i : rows) {
                    System.out.printf("c%d : %.2e\n", i+1, model.getConstr(i).
↪get(OPTVDblAttr.DUAL));
                }

                // LHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
                System.out.println("Constraint left-hand-side SA:");
                OPTVModel.SensitvityBounds lhsBounds = model.getConstrLbSA(rows);
                for (int i : rows) {
                    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, lhsBounds.down[i],␣
↪lhsBounds.up[i]);
                }

                // RHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
                System.out.println("Constraint right-hand-side SA:");
                OPTVModel.SensitvityBounds rhsBounds = model.getConstrUbSA(rows);
                for (int i : rows) {
                    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, rhsBounds.down[i],␣
↪rhsBounds.up[i]);
                }
            } else {
                System.out.println("No feasible solution available!");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Import the Library

To provide access to the OptVerse interface, we must import the library for the desired language.

C++

```
#include "optv_c++.h"
```

Python

```
from optvpy import *
```

Java

```
import com.huaweicloud.optv.*;
```

Creating the Model

To create a model an environment must be instantiated and provided to the *OPTVModel* constructor.

C++

```
OPTVEnv env("lp-sensitivity-analysis.log");
OPTVModel model(env);
```

Python

```
env = OPTVEnv("lp-sensitivity-analysis.log")
model = OPTVModel(env)
model.Set(OPTVStrParam.RESULT_FILE, "lp-sensitivity-analysis.sol")
```

Java

```
OPTVEnv env = new OPTVEnv("LpSensitivityAnalysis.log");
OPTVModel model = new OPTVModel(env);
```

Adding Variables and Constraints

C++

```
// Construct the model
OPTVVar x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3");
std::vector<OPTVVar> x {x1, x2, x3};

model.AddConstr(-x1 + x2 +x3, -OPTV_INF, 20, "c1");
model.AddConstr(x1 - 3*x2 +x3, -OPTV_INF, 30, "c2");
```

Python

```
# Construct the model
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
x = [x1, x2, x3]
model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")
```

Java

```
// construct the model
OPTVVar x1 = model.addVar(-OPTV.OPTV_INF, 40, -1, OPTV.OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.addVar(0, OPTV.OPTV_INF, -2, OPTV.OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.addVar(0, OPTV.OPTV_INF, -3, OPTV.OPTV_CONTINUOUS, "x3");
OPTVVar[] x = {x1, x2, x3};
// -x1 + x2 +x3
model.addConstr(OPTV.add(OPTV.add(OPTV.sub(x1), x2), x3), -OPTV.OPTV_INF, 20, "c1");
// x1 - 3*x2 +x3
model.addConstr(OPTV.add(OPTV.sub(x1, OPTV.mul(3, x2)), x3), -OPTV.OPTV_INF, 30, "c2");
```

Model Optimization

Once the model is constructed and any necessary parameters have been set the model can be solved by invoking *OPTVModel::Optimize()*.

C++

```
model.Optimize();
```

Python

```
model.Optimize()
```

Java

```
model.optimize();
```

Output Model Solution

C++

```
cout << "Optimal Solution:" << endl;
cout << std::setprecision(2) << std::scientific;
for (auto j: cols) {
    cout << "x"<< (j+1) << " = " << x[j].Get(OPTVDblAttr::X) << endl;
}
```

Python

```
print("Optimal Solution:")
for j in range(3):
    print("x%s = %s" % (j+1, x[j].Get(OPTVDblAttr.X)))
```

Java

```
System.out.println("Optimal Solution:");
for (int j: cols) {
    System.out.printf("x%d = %.2e\n", j+1, x[j].get(OPTVDblAttr.X));
}
```

Reduced Costs

The reduced costs indicate the sensitivity of the solution value to the bounds of the variable. Specifically, they correspond to the change in the target value for each unit increase in the bounds of the variable.

C++

```
cout << "Optimal Solution:" << endl;
cout << std::setprecision(2) << std::scientific;
for (auto j: cols) {
    cout << "x"<< (j+1) << " = " << x[j].Get(OPTVDblAttr::X) << endl;
}
```

Python

```
print("Optimal Solution:")
for j in range(3):
    print("x%s = %s" % (j+1, x[j].Get(OPTVDblAttr.X)))
```

Java

```
System.out.println("Optimal Solution:");
for (int j: cols) {
    System.out.printf("x%d = %.2e\n", j+1, x[j].get(OPTVDblAttr.X));
}
```

Objective Function SA

For the specified variables, as set through the `cols` parameter, the sensitivity analysis will return an interval between `objLower` and `objUpper`. If the corresponding objective coefficients are within this interval, the optimal basis of the model will remain unchanged.

C++

```
// Objective coefficient sensitivity info, within which the current optimal basis would␣
↪remain optimal
cout << "objective SA:" << endl;
std::vector<double> objLower{};
std::vector<double> objUpper{};
bool ok = model.GetObjSA(objLower, objUpper, cols);
for (auto j : cols) {
    cout << "x"<< (j+1) << " : [" << objLower[j] <<", " << objUpper[j] << "]" << endl;
}
```

Python

```
# Objective coefficient sensitivity info, within which the current optimal basis would␣
↪remain optimal
print("objective SA:")
objLower = []
objUpper = []
ok = model.GetObjSA(objLower, objUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, objLower[i], objUpper[i]))
```

Java

```
// objective coefficient sensitivity info, within which the current optimal basis would␣
↪remain optimal
System.out.println("objective SA:");
OPTVModel.SensitivityBounds objBounds = model.getObjSA(cols);
for (int j : cols) {
    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, objBounds.down[j], objBounds.up[j]);
}
```

Variable Bounds SA

For the specified variables, as set through the `cols` parameter, the sensitivity analysis will return an interval between `lbLower` and `lbUpper` for lower bounds (resp., `ubLower` and `ubUpper` for the upper bounds). If the corresponding lower/upper bounds are within this interval, the optimal basis of the model will remain unchanged.

C++

```
// Lower bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
cout << "Lower bound SA:" << endl;
std::vector<double> lbLower{};
std::vector<double> lbUpper{};
ok = model.GetVarLbSA(lbLower, lbUpper, cols);
for (auto j : cols) {
    cout << "x"<< (j+1) << " : [" << lbLower[j] <<", " << lbUpper[j] << "]" << endl;
}
```

Python

```
# Lower bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
print("Lower bound SA:")
lbLower = []
lbUpper = []
ok = model.GetVarLbSA(lbLower, lbUpper, cols)
```

(continues on next page)

(continued from previous page)

```
for i in cols:
    print("x%s : [%s, %s]" % (i+1, lbLower[i], lbUpper[i]))
```

Java

```
// lower bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
System.out.println("Lower bound SA:");
OPTVModel.SensitvityBounds lbBounds = model.getVarLbSA(cols);
for (int j : cols) {
    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, lbBounds.down[j], lbBounds.up[j]);
}
```

C++

```
// Upper bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
cout << "Upper bound SA:" << endl;
std::vector<double> ubLower{};
std::vector<double> ubUpper{};
ok = model.GetVarUbSA(ubLower, ubUpper, cols);
for (auto j : cols) {
    cout << "x"<< (j+1) << " : [" << ubLower[j] <<", " << ubUpper[j] << "]" << endl;
}
```

Python

```
# Upper bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
print("Upper bound SA:")
ubLower = []
ubUpper = []
ok = model.GetVarUbSA(ubLower, ubUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, ubLower[i], ubUpper[i]))
```

Java

```
// upper bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
System.out.println("Upper bound SA:");
OPTVModel.SensitvityBounds ubBound = model.getVarUbSA(cols);
for (int j : cols) {
    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, ubBound.down[j], ubBound.up[j]);
}
```

Shadow Price

The shadow prices reflect the sensitivity of the target value to the value on the right side of the constraint, specifically, they correspond to the change in the target value for each unit increase in the value on the right side of the constraint.

C++

```
// Shadow price: how much the objective will change if we increase the right-hand side␣
↪of a structural constraint by 1
cout << "Shadow Price:" << endl;
for (auto i : rows) {
    cout << "c"<< (i+1) << " : " << model.GetConstr(i).Get(OPTVDblAttr::DUAL) << endl;
}
```

Python

```
# Shadow price: how much the objective will change if we increase the right-hand side of␣
↪a structural constraint by 1
print("Shadow Price:")
for i in rows:
    print("c%s : %s" % (i+1, model.GetConstr(i).Get(OPTVDblAttr.DUAL)))
```

Java

```
// shadow price: how much the objective will change if we increase the right-hand side␣
↪of a structural constraint by 1
System.out.println("Shadow Price:");
for (int i : rows) {
    System.out.printf("c%d : %.2e\n", i+1, model.getConstr(i).get(OPTVDblAttr.DUAL));
}
```

Constraint Bounds SA

For the specified constraints, as set through the `rows` parameter, the sensitivity analysis will return an interval between `lhsLower` and `lhsUpper` for the left-hand-side (resp., `rhsLower` and `rhsUpper` for the right-hand-side). If the corresponding lower/upper bounds are within this interval, the optimal basis of the model will remain unchanged.

C++

```
// LHS sensitivity info, within which the current optimal basis would remain optimal
cout << "Constraint left-hand-side SA:" << endl;
std::vector<double> lhsLower{};
std::vector<double> lhsUpper{};
ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows);
for (auto i : rows) {
    cout << "c"<< (i+1) << " : [" << lhsLower[i] <<", " << lhsUpper[i] << "]" << endl;
}
```

Python

```
# LHS sensitivity info, within which the current optimal basis would remain optimal
print("Constraint left-hand-side SA:")
lhsLower = []
lhsUpper = []
ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, lhsLower[i], lhsUpper[i]))
```

Java

```
System.out.println("Constraint left-hand-side SA:");
OPTVModel.SensitvityBounds lhsBounds = model.getConstrLbSA(rows);
for (int i : rows) {
    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, lhsBounds.down[i], lhsBounds.up[i]);
}
```

C++

```
// RHS sensitivity info, within which the current optimal basis would remain optimal
cout << "Constraint right-hand-side SA:" << endl;
std::vector<double> rhsLower{};
std::vector<double> rhsUpper{};
ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows);
for (auto i : rows) {
    cout << "c"<< (i+1) << " : [" << rhsLower[i] <<", " << rhsUpper[i] << "]" << endl;
}
```

Python

```
# RHS sensitivity info, within which the current optimal basis would remain optimal
print("Constraint right-hand-side SA:")
rhsLower = []
rhsUpper = []
ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, rhsLower[i], rhsUpper[i]))
```

Java

```
System.out.println("Constraint right-hand-side SA:");
OPTVModel.SensitvityBounds rhsBounds = model.getConstrUbSA(rows);
for (int i : rows) {
    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, rhsBounds.down[i], rhsBounds.up[i]);
```

4.2 OptVerse CP Solver Quick Start

This chapter provides several basic examples demonstrating how to use the various language interfaces of the OptVerse CP Solver to model and solve constraint programming problems.

4.2.1 C++ CP API

This section presents a simple C++ example to demonstrate how to write a program using the OptVerse CP Solver's C++ API.

This example illustrates how to:

- Set solver parameters
- Construct a model
- Write the constructed problem to file
- Optimize the model
- Print solution information

The mathematical formulation for the problem to be solved is as follows:

$$
\begin{aligned}
\text{minimize } & b_1 + b_2 + b_3 + b_4 + 2x_1 + 2x_2 \\
\text{subject to } & -x_1 + 4x_2 + 8 \leq 15 \\
& 3b_1 - b_2 + x_1 = [3,4] \cup [6,10] \cup [12,14] \\
& b_1 \vee b_2 \vee b_4 \\
& \neg b_1 \wedge b_3 \\
& b_2 + b_3 \geq 1 \\
& b_2 + b_4 \leq 1 \\
& b_2 + b_4 = 1 \\
& b_1 \rightarrow b_4 \\
\text{where } & b_1, b_2, b_3, b_4 \in [0,1] \\
& x_1 \in [2,5] \\
& x_2 \in [1,2] \cup [5,7] \cup [15,15]
\end{aligned}
$$

```
/*
 * Copyright (c) Huawei Technologies Co., Ltd. 2025. All rights reserved.
 * Description: OptVerse c++ interface example - build and solve a CP-SAT problem
 */
#include "optv_c++.h"
using namespace std;
```

(continues on next page)

(continued from previous page)

```
int main()
{
    try {
        OPTVEnv env;
        // Before model construction: set parameters through environment
        env.Set(OPTVDblParam::CP_TIME_LIMIT, 7200);

        OPTVCPModel model(env);
        // After model construction: set parameters through model
        model.Set(OPTVIntParam::CP_THREADS, 12);

        // Add variables to the model
        OPTVCPVar b1 = model.NewBoolVar("b1");
        OPTVCPVar b2 = model.NewBoolVar("b2");
        OPTVCPVar b3 = model.NewBoolVar("b3");
        OPTVCPVar b4 = model.NewBoolVar("b4");
        OPTVCPVar x1 = model.NewIntVar(2, 5, "x1");
        // domain: [1, 2], [5, 7], [15, 15]
        OPTVCPVar x2 = model.NewIntVarFromDomain({1, 2, 5, 7, 15, 15}, "x2");
        OPTVCPVar c1 = model.NewConstant(11);

        // Add constraint to the model
        OPTVCPConstr lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_
↪INF, 15);
        // domain: [3, 4], [6, 10], [12, 14]
        std::vector<int64_t> domain = {3, 4, 6, 10, 12, 14};
        OPTVCPConstr lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain);

        OPTVCPConstr boolOr = model.AddBoolOr({b1, b2, b4});
        OPTVCPConstr boolAnd = model.AddBoolAnd({~b1, b3});
        OPTVCPConstr boolALO = model.AddAtLeastOne({b2, b3});
        OPTVCPConstr boolAMO = model.AddAtMostOne({b2, b4});
        OPTVCPConstr boolEO = model.AddExactlyOne({b2, b4});
        OPTVCPConstr boolImpl = model.AddImplication({b1, b4});

        boolOr.OnlyEnforceIf({b3, b4});

        model.Write("cp.fzn");
        OPTVCPSolver solver;
        auto status = solver.Solve(model);

        switch (status) {
            case OPTV_CP_UNKNOWN:
                cout << "Model is unknown." << endl;
                break;
            case OPTV_CP_MODEL_INVALID:
                cout << "Model is invalid." << endl;
                break;
            case OPTV_CP_FEASIBLE:
                cout << "Model is feasible." << endl;
                break;
```

(continues on next page)

(continued from previous page)

```
            case OPTV_CP_INFEASIBLE:
                cout << "Model is infeasible." << endl;
                break;
            case OPTV_CP_OPTIMAL: {
                std::vector<OPTVCPVar> vars{b1, b2, b3, b4, x1, x2};
                const auto& result = solver.GetValues(vars);
                cout << "Optimal solution found. Best solution: " << endl;
                for (int i = 0; i < result.size(); ++i) {
                    cout << "  " << vars[i].GetName() << " = " << result[i] << endl;
                }

                double objVal = solver.GetObjectiveValue();
                break;
            }
            case OPTV_CP_USER_INTERRUPT:
                cout << "Solver was interrupted by the user." << endl;
                break;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Include the Header

To provide access to the C++ interface, include the header file `optv_c++.h`.

```
#include "optv_c++.h"
```

Create the Environment

The first step is to create the environment object.

```
OPTVEnv env;
```

Configuring Parameters Before Model Construction

To edit solver parameters **before** calling the *OPTVCPModel* constructor, a user must set the parameter through the environment object. In this example, we demonstrate setting the solver time limit through the environment, prior to creation of the model object.

```
env.Set(OPTVDblParam::CP_TIME_LIMIT, 7200);
```

Configuring Parameters After Model Construction

To edit solver parameters **after** calling the *OPTVCPModel* constructor, a user must set the parameter through the model object. In this example, we demonstrate setting the number of threads through the model.

```
model.Set(OPTVIntParam::CP_THREADS, 12);
```

Creating the Model

After creating the environment, the next step is to create the model. Each OptVerse model is an optimization problem consisting of a set of variables, a set of constraints, and associated attributes (i.e., variable or constraint bounds, solution status, variable or constraint names, etc.).

Construct An Empty Model

If a user wishes to construct a model from scratch using API calls, they will first need to create an empty model.

```
OPTVCPModel model(env);
```

Loading Model from File

If the `.mps`[1], `.lp`[2], or `.fzn`[3] extension is passed as the second parameter, the problem file is read. This allows solving or modifying a pre-existing model through the API.

```
OPTVEnv env;
OPTVCPModel(env, fileName);
```

Add Variables to Model

If an empty model was created in the previous step, variables and constraints **must** be added to the model before it can be solved. Alternatively, a model loaded from a file can be expanded through the addition of variables or constraints.

The following methods can be used to add variables to the model:

- *OPTVCPModel::NewIntVar()*
- *OPTVCPModel::NewIntVarFromDomain()*
- *OPTVCPModel::NewBoolVar()*
- *OPTVCPModel::NewConstant()*

```
OPTVCPVar b1 = model.NewBoolVar("b1");
OPTVCPVar b2 = model.NewBoolVar("b2");
OPTVCPVar b3 = model.NewBoolVar("b3");
OPTVCPVar b4 = model.NewBoolVar("b4");
OPTVCPVar x1 = model.NewIntVar(2, 5, "x1");
// domain: [1, 2], [5, 7], [15, 15]
```

(continues on next page)

[1] For a description of the MPS format, see *File Formats*.
[2] For a description of the LP format, see *File Formats*.
[3] For a description of the FZN format, see *File Formats*.

(continued from previous page)

```
OPTVCPVar x2 = model.NewIntVarFromDomain({1, 2, 5, 7, 15, 15}, "x2");
OPTVCPVar c1 = model.NewConstant(11);
```

Add Constraints to Model

After adding variables, a user may now add constraints to the model. The OptVerse CP API provides several types of constraints:

- *Linear*;
- *Boolean Or*;
- *Boolean And*;
- *Boolean Xor*;
- *Implication*;
- *At Least One*;
- *At Most One*;
- *Exactly One*;
- *Allowed Assignments*;
- *Forbidden Assignments*;
- *Interval*;
- *No Overlap*;
- *No Overlap 2D*;
- *Cumulative*;
- *All Different*;
- *Element*;
- *Circuit*;
- *Routes*;
- *Max Equality*;
- *Min Equality*;
- *Abs Equality*;
- *Multiplication Equality*;
- *Division Equality*;
- *Modulo Equality*;
- *Inverse*.

```
OPTVCPConstr lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF, 15);
// domain: [3, 4], [6, 10], [12, 14]
std::vector<int64_t> domain = {3, 4, 6, 10, 12, 14};
OPTVCPConstr lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain);
```

(continues on next page)

(continued from previous page)

```
OPTVCPConstr boolOr = model.AddBoolOr({b1, b2, b4});
OPTVCPConstr boolAnd = model.AddBoolAnd({~b1, b3});
OPTVCPConstr boolALO = model.AddAtLeastOne({b2, b3});
OPTVCPConstr boolAMO = model.AddAtMostOne({b2, b4});
OPTVCPConstr boolEO = model.AddExactlyOne({b2, b4});
OPTVCPConstr boolImpl = model.AddImplication({b1, b4});

boolOr.OnlyEnforceIf({b3, b4});
```

Model Optimization

Once the model is constructed and any necessary parameters have been set, the model can be solved by invoking *OPTVCPSolver::Solve()*. After optimization, the solution status and optimal solution are saved in the *OPTVCPSolver* object.

```
OPTVCPSolver solver;
auto status = solver.Solve(model);
```

Querying the Solution

To output the solution after optimization, the *OPTVCPSolver* can be used to get data from the solved model. First, the function *OPTVCPSolver::Solve()* returns the status, which is used to check if a feasible solution has been found. We obtain the best found objective value by using the *OPTVCPSolver::GetObjectiveValue()* method. The value of a variable can be obtained with the *OPTVCPSolver::GetSolution()*, *OPTVCPSolver::GetValue()* or *OPTVCPSolver::GetValues()* methods.

```
switch (status) {
    case OPTV_CP_UNKNOWN:
        cout << "Model is unknown." << endl;
        break;
    case OPTV_CP_MODEL_INVALID:
        cout << "Model is invalid." << endl;
        break;
    case OPTV_CP_FEASIBLE:
        cout << "Model is feasible." << endl;
        break;
    case OPTV_CP_INFEASIBLE:
        cout << "Model is infeasible." << endl;
        break;
    case OPTV_CP_OPTIMAL: {
        std::vector<OPTVCPVar> vars{b1, b2, b3, b4, x1, x2};
        const auto& result = solver.GetValues(vars);
        cout << "Optimal solution found. Best solution: " << endl;
        for (int i = 0; i < result.size(); ++i) {
            cout << "  " << vars[i].GetName() << " = " << result[i] << endl;
        }

        double objVal = solver.GetObjectiveValue();
        break;
    }
```

(continues on next page)

(continued from previous page)

```
    case OPTV_CP_USER_INTERRUPT:
        cout << "Solver was interrupted by the user." << endl;
        break;
}
```

Compiling and Running

Several example files for the constraint programming solver have been provided in `${OPTV_HOME}/examples/c++` along with the CMake file for compiling the examples on Linux.

To compile and run the examples, go to the `${OPTV_HOME}/examples` directory and execute the following:

```
mkdir build && cd build
cmake ..
make -j $(nproc)
```

After compilation, the example from this section can be run by calling:

```
./cp
```

4.2.2 Python API

This section presents a simple Python example demonstrating how to use the OptVerse CP Solver's Python API.

This example illustrates how to:

- Set solver parameters
- Construct a model
- Write the constructed problem to file
- Optimize the model
- Print solution information

The mathematical formulation for the problem to be solved is as follows:

$$
\begin{aligned}
\text{minimize } & b_1 + b_2 + b_3 + b_4 + 2x_1 + 2x_2 \\
\text{subject to } & -x_1 + 4x_2 + 8 \leq 15 \\
& 3b_1 - b_2 + x_1 = [3,4] \cup [6,10] \cup [12,14] \\
& b_1 \vee b_2 \vee b_4 \\
& \neg b_1 \wedge b_3 \\
& b_2 + b_3 \geq 1 \\
& b_2 + b_4 \leq 1 \\
& b_2 + b_4 = 1 \\
& b_1 \rightarrow b_4 \\
\text{where } & b_1, b_2, b_3, b_4 \in [0,1] \\
& x_1 \in [2,5] \\
& x_2 \in [1,2] \cup [5,7] \cup [15,15]
\end{aligned}
$$

```
"""
   Copyright (c) Huawei Technologies Co., Ltd. 2025. All rights reserved.
   Description: OptVerse python interface example - build and solve a CP-SAT problem
"""

from optvpy import *

if __name__ == "__main__":
    # Create an environment
    env = OPTVEnv()
    env.Set(OPTVDblParam.CP_TIME_LIMIT, 7200)

    # Create an empty model
    model = OPTVCPModel(env)
    # After model construction: set parameters through model
    model.Set(OPTVIntParam.CP_THREADS, 12)

    # Add variables to the model
    b1 = model.NewBoolVar("b1")
    b2 = model.NewBoolVar("b2")
    b3 = model.NewBoolVar("b3")
    b4 = model.NewBoolVar("b4")
    x1 = model.NewIntVar(2, 5, "x1")
    # domain: [1, 2], [5, 7], [15, 15]
    x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
    c1 = model.NewConstant(11)

    # Add constraint to the model
    lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3 <= 15)
    # domain: [3, 4], [6, 10], [12, 14]
    domain = [3, 4, 6, 10, 12, 14]
    lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain)

    boolOr = model.AddBoolOr([b1, b2, b4])
    boolAnd = model.AddBoolAnd([~b1, b3])
    boolALO = model.AddAtLeastOne([b2, b3])
    boolAMO = model.AddAtMostOne([b2, b4])
    boolEO = model.AddExactlyOne([b2, b4])
    boolImpl = model.AddImplication([b1, b4])

    boolOr.OnlyEnforceIf(b3, b4)

    model.Write("cp.fzn")
    solver = OPTVCPSolver()
    status = solver.Solve(model)

    if status == OPTV_CP_UNKNOWN:
        print("Model is unknown.")
    elif status == OPTV_CP_MODEL_INVALID:
        print("Model is invalid.")
    elif status == OPTV_CP_FEASIBLE:
        print("Model is feasible.")
    elif status == OPTV_CP_INFEASIBLE:
```

(continues on next page)

(continued from previous page)

```
        print("Model is infeasible.")
    elif status == OPTV_CP_OPTIMAL:
        objVal = solver.GetObjectiveValue()
        print("Optimal solution found. Best objective value: ", objVal)
        vars = [b1, b2, b3, b4, x1, x2]
        result = solver.GetValues(vars)
        print("Optimal solution found. Best solution: ")
        for i in range(len(result)):
            print(vars[i].GetName(), " ", result[i])
    elif status == OPTV_CP_USER_INTERRUPT:
        print("Solver was interrupted by the user.")
```

Import the Module

To provide access to the Python interface, import the package `optvpy`.

```
from optvpy import *
```

Create the Environment

The first step is to create the environment object.

```
# Create an environment
env = OPTVEnv()
```

Configuring Parameters Before Model Construction

To edit solver parameters **before** calling the *OPTVCPModel* constructor, a user must set the parameter through the environment object. In this example, we demonstrate setting the solver time limit through the environment, prior to creation of the model object.

```
env.Set(OPTVDblParam.CP_TIME_LIMIT, 7200)
```

Configuring Parameters After Model Construction

To edit solver parameters **after** calling the *OPTVCPModel* constructor, a user must set the parameter through the model object. In this example, we demonstrate setting the number of threads through the model.

```
model.Set(OPTVIntParam.CP_THREADS, 12)
```

Creating the Model

After creating the environment, the next step is to create the model. Each OptVerse model is an optimization problem consisting of a set of variables, a set of constraints, and associated attributes (i.e., variable or constraint bounds, solution status, variable or constraint names, etc.).

Construct An Empty Model

If a user wishes to construct a model from scratch using API calls, they will first need to create an empty model.

```
# Create an empty model
model = OPTVCPModel(env)
```

Loading Model from File

If the `.mps`[1], `.lp`[2], or `.fzn`[3] extension is passed as the second parameter, the problem file is read. This allows solving or modifying a pre-existing model through the API.

```
env = OPTVEnv()
model = OPTVCPModel(env, fileName)
```

Add Variables to Model

If an empty model was created in the previous step, variables and constraints **must** be added to the model before it can be solved. Alternatively, a model loaded from a file can be expanded through the addition of variables or constraints.

The following methods can be used to add variables to the model:

- *OPTVCPModel.NewIntVar()*
- *OPTVCPModel.NewIntVarFromDomain()*
- *OPTVCPModel.NewBoolVar()*
- *OPTVCPModel.NewConstant()*

```
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
# domain: [1, 2], [5, 7], [15, 15]
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)
```

[1] For a description of the MPS format, see *File Formats*.
[2] For a description of the LP format, see *File Formats*.
[3] For a description of the FZN format, see *File Formats*.

Add Constraints to Model

After adding variables, a user may now add constraints to the model. The OptVerse CP API provides several types of constraints:

- *Linear*;
- *Boolean Or*;
- *Boolean And*;
- *Boolean Xor*;
- *Implication*;
- *At Least One*;
- *At Most One*;
- *Exactly One*;
- *Allowed Assignments*;
- *Forbidden Assignments*;
- *Interval*;
- *No Overlap*;
- *Cumulative*;
- *All Different*.

```
lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3 <= 15)
# domain: [3, 4], [6, 10], [12, 14]
domain = [3, 4, 6, 10, 12, 14]
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain)

boolOr = model.AddBoolOr([b1, b2, b4])
boolAnd = model.AddBoolAnd([~b1, b3])
boolALO = model.AddAtLeastOne([b2, b3])
boolAMO = model.AddAtMostOne([b2, b4])
boolEO = model.AddExactlyOne([b2, b4])
boolImpl = model.AddImplication([b1, b4])

boolOr.OnlyEnforceIf(b3, b4)
```

Model Optimization

Once the model is constructed and any necessary parameters have been set, the model can be solved by invoking *OPTVCPSolver.Solve()*. After optimization, the solution status and optimal solution are saved in the *OPTVCPSolver* object.

```
solver = OPTVCPSolver()
status = solver.Solve(model)
```

Querying the Solution

To output the solution after optimization, the *OPTVCPSolver* can be used to get data from the solved model. First, the function *OPTVCPSolver.Solve()* returns the status, which is used to check if a feasible solution has been found. We obtain the best found objective value by using the *OPTVCPSolver.GetObjectiveValue()* method. The value of a variable can be obtained with the *OPTVCPSolver.GetSolution()*, *OPTVCPSolver.GetValue()* or *OPTVCPSolver.GetValues()* methods.

```
if status == OPTV_CP_UNKNOWN:
    print("Model is unknown.")
elif status == OPTV_CP_MODEL_INVALID:
    print("Model is invalid.")
elif status == OPTV_CP_FEASIBLE:
    print("Model is feasible.")
elif status == OPTV_CP_INFEASIBLE:
    print("Model is infeasible.")
elif status == OPTV_CP_OPTIMAL:
    objVal = solver.GetObjectiveValue()
    print("Optimal solution found. Best objective value: ", objVal)
    vars = [b1, b2, b3, b4, x1, x2]
    result = solver.GetValues(vars)
    print("Optimal solution found. Best solution: ")
    for i in range(len(result)):
        print(vars[i].GetName(), " ", result[i])
elif status == OPTV_CP_USER_INTERRUPT:
    print("Solver was interrupted by the user.")
```

Running Examples

Several example files have been provided in `${OPTV_HOME}/examples/python`.

To run the examples, go to the `${OPTV_HOME}/examples` directory and call the scripts through Python.

```
python3 cp.py
```

CHAPTER

FIVE

FILE FORMATS

This chapter provides descriptions for the various supported file formats in OptVerse.

OptVerse provides support for the following types of model files:

- `.mps`
- `.qps`[1]
- `.lp`
- `.nl`[2]
- `.fzn`[3]

Additionally, OptVerse allows compression of the following types (except for `.nl` type):

- `.gz`
- `.z`
- `.Z`

5.1 MPS File Format Description

MPS file format is one of the most common file formats used by operations optimization solvers. The format is written in columnar format and is case sensitive. Each row of data represents a record. Records are classified into two types: indicator records and data records. A record contains spaces, which are often used as separators. According to the different ways of writing, it is often divided into **Fixed** format and **Free** format, in which the Fixed format field must be written in a fixed range column, while the Free format restriction is looser. Spaces are often used to separate fields. The row and column names in a fixed format are usually eight characters. Spaces are also considered as characters. Variable names can contain spaces. The free format has no character limit, but the variable name should preferably not exceed 255 characters and cannot contain spaces.

Indicator records are used to identify data segments. Each indicator record is written from the first column. There are five required and four optional indicator types.

[1] alias for `.mps`, sometimes used for QP, QCP, and QCQP problems.

[2] a file format for presenting and archiving both linear and nonlinear mathematical programming problems. Initially, this format has been invented for connecting solvers to AMPL, a popular modelling language. It contains a complete and unambiguous problem description of both linear and nonlinear problems (usually faster to read and write than `.mps` or `.lp`), and they introduce no new rounding errors. Visit AMPL MP library for details.

[3] `.fzn` format supported only for CP problems.

Record Name	Required	Purpose
NAME	Yes	Specify the problem name; unlike other flag records, the name record contains data.
OBJSENSE	No	Optionally, specify the problem as maximization or minimization. If omitted, will default to minimization.
ROWS	Yes	Specify the name and meaning of each constraint.
COLUMNS	Yes	Specify the names allocated to each variable (column) and the non-zero constraint coefficients corresponding to that variable.
RHS	Yes	Specify the name of the right-hand vector and the value for each constraint (row).
RANGES	No	Specify constraints that are within an interval between two values; it also specifies the interval endpoints.
BOUNDS	No	Specify constraints that each variable (column) must adhere to. If omitted, bounds will be set as $[0, \infty]$.
QUADOBJ/QMATRI	No	Optionally, specify quadratic terms for the objective function.
QCMATRIX	No	Optionally, specify quadratic terms for the quadratic constraints.
ENDATA	Yes	Indicates the end of data; always the last entry in an MPS file.

5.1.1 Example MPS File

The following example of a mixed-integer problem illustrates the meanings and usage of each indicator and data record.

Formulation

$$\min \; x_1 + 2x_5 - x_8$$

subject to:

$$\begin{aligned} 2.5 \leq 3x_1 + x_2 - 2x_4 - x_5 - x_8, \\ 2x_2 + 1.1x_3 \leq 2.1, \\ x_3 + x_6 = 4.0, \\ 1.8 \leq 2.8x_4 - 1.2x_7 \leq 5.0, \\ 3.0 \leq 5.6x_1 + x_5 + 1.9x_8 \leq 15.0, \end{aligned}$$

where

$$\begin{aligned} 2.5 \leq x_1 \\ 0 \leq x_2 \quad &\leq 4.1 \\ 0 \leq x_3 \\ 0 \leq x_4 \\ 0.5 \leq x_5 \quad &\leq 4.0 \\ 0 \leq x_6 \\ 0 \leq x_7 \\ 0 \leq x_8 \quad &\leq 4.3 \end{aligned}$$

and

$$\begin{aligned} x_1, x_2 &\in \mathbb{R} \\ x_3, x_4 &\in \{0, 1\}. \end{aligned}$$

MPS Representation

```
NAME          EXAMPLE
ROWS
 N  OBJ
 G  ROW01
 L  ROW02
 E  ROW03
 G  ROW04
 L  ROW05
COLUMNS
    COL01     OBJ                1.0
    COL01     ROW01              3.0   ROW05              5.6
    COL02     ROW01              1.0   ROW02              2.0
*
*  Mark COL03 and COL04 as integer variables.
*
    INT1      'MARKER'                 'INTORG'
    COL03     ROW02              1.1   ROW03              1.0
    COL04     ROW01             -2.0   ROW04              2.8
    INT1END   'MARKER'                 'INTEND'
*
    COL05     OBJ                2.0
    COL05     ROW01             -1.0   ROW05              1.0
    COL06     ROW03              1.0
    COL07     ROW04             -1.2
    COL08     OBJ               -1.0
    COL08     ROW01             -1.0   ROW05              1.9
RHS
    RHS1      ROW01              2.5
    RHS1      ROW02              2.1
    RHS1      ROW03              4.0
    RHS1      ROW04              1.8
    RHS1      ROW05             15.0
RANGES
    RNG1      ROW04              3.2
    RNG1      ROW05             12.0
BOUNDS
 LO BND1      COL01              2.5
 UP BND1      COL02              4.1
 LO BND1      COL05              0.5
 UP BND1      COL05              4.0
 UP BND1      COL08              4.3
ENDATA
```

Note

If the first character of a line is an asterisk (*), the line is treated as a comment line.

5.1.2 Data Record Format

Divided into Fixed format and Free format, the Fixed format imposes extremely strict requirements on the positions where parameters appear.

Fixed Format

For Fixed format, if the character offset value, Inumchar, is defined as 8, then the positions of the data records correspond to:

Field	Field 1	Field 2	Field 3	Field 4	Field 5	Field 6
Content	2-3	5-12	15-22	25-36	40-47	50-61

If the character offset value is not equal to 8, the positions of the data records will also change:

Parameter Record	Start Position	End Position
Field 1	2	3
Field 2	5	5 + Inumchar - 1
Field 3	Third column after Field 2	Start position + Inumchar -1
Field 4	Third column after Field 3	Start position + 11
Field 5	Fourth column after Field 4	Start position + Inumchar -1
Field 6	Fifth column after Field 5	Start position + 11

Note

1. In Fields 2, 3, and 5, the strings should not contain spaces. Any spaces will be removed.
2. Fields 4 and 6 are data fields, and the numerical values must be within 12 characters. Columns outside of these six fields must be spaces, otherwise, they cannot be recognized.

Free Format

For Free format, its basic form is also written in 6 fields, with slight differences:

1. Free format does not require fixed positions; it can be placed anywhere except the second column. Different fields are separated by spaces.
2. However, the order of the fields must be the same as in the Fixed format. Records with more than 6 fields will only read the first 6 fields, and the rest will not be processed further.
3. Fields that do not start from the first column and use `$$` will be considered as comments, and the subsequent parts of that line will not be processed.
4. Columns 72-80 are automatically ignored.

5.1.3 NAME Section

The first indicator record in an MPS file is the problem name field, representing the name of the model. It corresponds to the following character positions:

Field 1	Field 2	Field 3	Field 4
NAME	Space	Model Name	BINARY, FREE, or blank

For Fixed format, NAME must be written in columns 1-4, modelName must be written within the range $[5+\text{Inumchar}+2, 5+2\cdot\text{Inumchar}+1]$, and Field 4 must be written within the range $[5+2\cdot\text{Inumchar}+4, 5+2\cdot\text{Inumchar}+15]$.

For the modelName in Field 3, it is optional for Fixed format but mandatory for Free format. If the file is binary, for Fixed format, Field 4 must specify `BINARY`; otherwise, it can be left blank.

5.1.4 OBJSENSE Section

This indicator, not standard in MPS format, is often used as an extension format. It is an optional indicator record and cannot be included in fixed format. If omitted, the problem is assumed to be a minimization problem by default. In OptVerse code, the following two sections can be used to extend the MPS standard: OBJSENSE and OBJNAME. These sections may appear after the NAME section. OBJSENSE is used to set the meaning of the objective function, while OBJNAME is used to select the objective function from the available lines in the file. If these sections do not appear in the MPS file, the problem is assumed to be a minimization problem, and the objective function is the first available line encountered in the ROWS section. If these options are used, they must appear in order after the NAME section as the first and second sections. The value of OBJSENSE can be `MAX` or `MIN`.

For example:

```
NAME          example.mps
OBJSENSE
  MAX
OBJNAME
  rowname
```

5.1.5 ROWS Section

The header field ROWS indicates the beginning of this parameter segment, and writing should start from the first column. Parameter records are written on separate lines, with each line specifying the name and meaning of each constraint in the problem. Each record occupies one line, formatted as follows:

Field 1	Field 2
Constraint Indicator	Constraint Name

Field 1 must be in columns 2-3, and there must be at least one space between Field 1 and Field 2. The constraint indicator represents the type of constraint and consists of a single letter used to specify the meaning of each row. Acceptable values are:

1. E indicates equality constraint
2. L indicates less than or equal to constraint
3. G indicates greater than or equal to constraint

4. N indicates free constraint (for objective function)

Field 2 contains a character identifier (maximum length of 255 characters), and Fields 3-6 should not be blank in the ROWS section. In an MPS file, the objective function is also treated as a constraint, included in the ROWS section with type `N`. If multiple free constraints are defined, only the first one is considered as the objective function, and the others will be discarded.

Multiple Objectives

Multiple objectives can be specified by providing multiple `N` constraints. For the multiple objectives to be recognized, the following parameters must be provided, space separated, on the same line:

1. Priority
2. Weight (may be signed)
3. Absolute tolerance
4. Relative tolerance

All objectives must be linear for multi-objective optimization in OptVerse.

As an illustrative example, the following specifies two objectives, with the first having a priority of `2` and both having a weight of `1`.

```
N  OBJ0 2 1 0 0
N  OBJ1 1 1 0 0
```

5.1.6 COLUMNS Section

The COLUMNS section is used to define the names of all columns (variables). In the COLUMNS section, all columns of the constraint matrix are specified using their names and all non-zero elements. The data record format is as follows:

Field 1	Field 2	Field 3	Field 4	Field 5	Field 6
Blank	Column Name	Constraint 1 Name	Coefficient in Constraint 1 (Non-zero)	Constraint 2 Name	Coefficient in Constraint 2 (Non-zero)

If Field 2 is blank, it means that the record has the same column name as the previous record. Field 3 represents the non-zero coefficients in the constraint or objective function; Fields 5 and 6 have the same meaning as Fields 3 and 4 and are optional. The constraint names appearing in the records do not need to follow the order of appearance in the ROWS section.

If 'MARKER' 'INTORG' and 'MARKER' 'INTEND' appear, they respectively indicate the start and end of integer variables, and the columns between the keywords are all integer variables.

5.1.7 RHS Section

In the RHS section, the non-zero right-hand side values of the constraints are specified. The data record format is as follows:

Field 1	Field 2	Field 3	Field 4	Field 5	Field 6
Blank	RHS Name	Constraint 1 Name	RHS Value in Constraint 1 (Non-zero)	Constraint 2 Name	RHS Value in Constraint 2 (Non-zero)

Here:

1. The first column should be empty.
2. The second column represents the RHS name, which can be chosen arbitrarily. If it is blank, it will be the same as the previous record name.
3. Each RHS must have a unique name.
4. The fourth column represents the RHS value corresponding to the respective constraint.
5. The fifth and sixth columns have the same meaning as the third and fourth columns, respectively, and are optional fields.

Additionally, it is possible to declare the objective offset of the model in the RHS section of an MPS format file. To do this, use the negative value of the offset and declare it as the RHS of the objective function. For example, the following line declares the objective offset as 3.1415:

```
rhs       obj      -3.1415
```

5.1.8 RANGES Section

In the RANGES section, ranges values that are applied to the right-hand side (RHS) of constraints may be specified. The data record format is as follows:

Field 1	Field 2	Field 3	Field 4	Field 5	Field 6
Blank	Right Hand Range ID	Constraint 1 Name	Constraint 1 Range	Constraint 2 Name (Optional)	Constraint 2 Range (Optional)

The effect of specifying right-hand ranges depends on the meaning of the specified rows and whether the range has a positive or negative coefficient. For a given row, *rhs* represents the right-hand side value, while *range* represents the corresponding range value.

The interpretation of range values is as follows:

Row Type	Range Value Sign	Generated RHS Upper Bound	Generated RHS Lower Bound
G	+ or -	rhs + |range|	rhs
L	+ or -	rhs	rhs - |range|
E	+	rhs + range	rhs
E	-	rhs	rhs + range

5.1.9 BOUNDS Section

In the BOUNDS section, boundary values for variables may be specified. The data record format is as follows:

Field 1	Field 2	Field 3	Field 4	Field 5	Field 6
Bound Type	BOUNDS Name	Column Name	Column Bound	Blank	Blank

Note

1. Field 1: The type of bound, starting from the second column on the left.
2. OptVerse supports bound type values:
   - LO: Lower bound
   - LI: Lower bound, integer
   - UP: Upper bound
   - UI: Upper bound, integer
   - FX: Fixed value (upper and lower bounds are the same)
   - FR: Free variable (lower bound is $-\infty$ and upper bound is $+\infty$)
   - MI: Negative infinity (lower bound is $-\infty$)
   - PL: Positive infinity (upper bound is $+\infty$)
   - BV: Binary variable, integer (0 or 1)
3. OptVerse does not currently support bound type values: - SC: Semi-continuous variable upper bound
4. Fields 5 and 6 are not used in the BOUNDS section.
5. If the bound type is not LO, UP, FX, or UI, the value of Field 4 is ignored.
6. The order of variables in the bounds does not need to match the order in COLUMNS.

5.1.10 QUADOBJ/QMATRIX Section

The QUADOBJ section is an optional section for defining the quadratic terms in the objective function. If no section is provided, the problem is treated as an (MI)LP problem.

Field 1	Field 2	Field 3	Field 4	Field 5	Field 6
Blank	Variable 1 Name	Variable 2 Name	Coefficient	Blank	Blank

Each line of this section represents a non-zero value in the upper triangular portion of the Q matrix. There is an implicit factor of one-half for each value.

For example:

```
QUADOBJ
    X1        X1        2.0
    X1        X2        4.0
    X2        X2        6.0
```

represents the following Q matrix:

$$Q = \frac{1}{2}\begin{bmatrix} 2 & 4 \\ 4 & 6 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$$

QMATRIX is an alternative to QUADOBJ in which the full matrix is provided, as opposed to QUADOBJ where only the triangular portion was provided. For example, the above QUADOBJ section written as a QMATRIX section is the following:

```
QMATRIX
    X1        X1        2.0
    X1        X2        4.0
    X2        X1        4.0
    X2        X2        6.0
```

5.1.11 QCMATRIX Section

The QCMATRIX section is an extension to the MPS file format which allows for encoding quadratically constrained problems into the MPS file format. Each constraint with quadratic terms will require its own QCMATRIX section. For each constraint, the name of the constraint must be provided with the QCMATRIX keyword. The matrix is provided in a format similar to QMATRIX, where the full matrix must be provided. There is no implied division by two.

```
QCMATRIX c1
    X1        X1        2.0
    X1        X2        4.0
    X2        X1        4.0
    X2        X2        6.0

QCMATRIX c3
    X1        X1        1.0
```

5.2 LP File Format Description

The LP format, widely used in academia and industry, has the advantage of being more readable than the MPS format. Unlike the column-based format of MPS files, LP files describe problems using row expressions and do not have a fixed writing range or length. However, from the perspectives of processing performance and readability, various solvers have standardized the maximum byte length of each line. The OptVerse solver reads a buffer of 16 kB. It is advised to keep lines shorter than this 16 kB limit, although lines longer than this will still be parsed correctly.

Below, we provide a standard LP file:

```
\ENCODING-ISO-8859-1

\Problem name:Maximize
Maximize
 obj: x1 + 2 x2 + 3 x3 + x4
Subject To
 r1: - x1 + x2 + x3 + 10 x4 <= 20
 r2: x1 - 3 x2 + x3 <= 30
 r3: x2 - 3.5 x4 = 0
Bounds
```

(continues on next page)

(continued from previous page)

```
 0 <= x1 <= 40
 2 <= x4 <= 3
Generals
 x4
Binaries
 x3
End
```

`Problem`, `Maximize`, `Subject To`, `Bounds`, `Generals`, `Binaries`, `General Constraints`, and `End` are the indicator words for each section. The ordering of these sections matters and should adhere to the order specified in this document.

Each section indicator word must be written on a separate line and is not case-sensitive. Below, we provide detailed descriptions of each part of the file.

5.2.1 LP Format BNF Specification

This page provides the formal syntax specification for the OptVerse solver in Backus-Naur form.

⟨LP File⟩ ::=[⟨Model Name⟩]
⟨Objective Section⟩
⟨Constraint Section⟩
[⟨Bounds Section⟩]
[⟨Var Type Section⟩]
[⟨SOS Section⟩]
[⟨GenCon Section⟩]
“END”

⟨Model Name⟩ ::=(“PROBLEM”|“PROB”)⟨IDENT⟩

⟨Objective Section⟩ ::=⟨Objective Sense⟩([⟨Objective⟩]|⟨Multi Objective⟩+)

⟨Objective Sense⟩ ::=⟨Min Sense⟩|⟨Max Sense⟩

⟨Min Sense⟩ ::=(“MINIMIZE”|“MINIMUM”|“MIN”)

⟨Max Sense⟩ ::=(“MAXIMIZE”|“MAXIMUM”|“MAX”)

⟨Objective⟩ ::=[⟨IDENT⟩‘:’][⟨Linear Term⟩|⟨Quadratic Region⟩]+

⟨Multi Objective⟩ ::=[⟨IDENT⟩‘:’]⟨Multi Params⟩[⟨Linear Objective⟩]

⟨Constraint Section⟩ ::=(“SUBJECT TO”|“SUBJECT”|“ST”)[⟨Constraint⟩|⟨Indicator Constraint⟩]+

⟨Bounds Section⟩ ::=(“BOUNDS”|“BOUND”)[⟨Bound⟩]+

⟨Var Type Section⟩ ::=(⟨Binary Section⟩|⟨General Section⟩)+

⟨Binary Section⟩ ::=(“BINARIES”|“BINARY”|“BIN”)[⟨IDENT⟩]+

⟨General Section⟩ ::=(“GENERALS”|“GENERAL”|“GEN”)[⟨IDENT⟩]+

⟨Constraint⟩ ::=[⟨IDENT⟩‘:’][⟨Linear Term⟩|⟨Quadratic Region⟩] + ⟨SENSE⟩⟨Signed Number⟩

⟨Linear Term⟩ ::=[‘+’|‘-’] + [⟨Signed Number⟩]⟨IDENT⟩

⟨Quadratic Region⟩ ::=‘[’⟨Quadratic Term⟩ + ‘]’

⟨Quadratic Term⟩ ::=[‘+’|‘-’] + [⟨Signed Number⟩](⟨IDENT⟩“^ 2”|⟨IDENT⟩‘*’⟨IDENT⟩)

⟨SOS Section⟩ ::=(“SOS”)⟨SOS Constraint⟩+

⟨SOS Constraint⟩ ::=[⟨IDENT⟩‘:’]‘S’[‘1’|‘2’]“::”⟨SOS Weight⟩ + \n

⟨SOS Weight⟩ ::=⟨IDENT⟩‘:’⟨Signed Number⟩

⟨GenCon Section⟩ ::=⟨GenCon Indicator⟩[⟨General Constraint⟩]+

⟨GenCon Indicator⟩ ::=(“G.C.”|“GENCON”|“GENERAL CONSTRAINT”|“GENERAL CONSTRAINTS”)

⟨General Constraint⟩ ::=[⟨IDENT⟩‘:’]⟨IDENT⟩‘=’⟨GENCON TYPE⟩⟨GenCon Var List⟩

⟨GenCon Var List⟩ ::=‘(’⟨IDENT⟩[‘,’⟨IDENT⟩] + ‘)’

⟨Multi Params⟩ ::=“Priority=”⟨DIGIT⟩ + “Weight=”[‘+’|‘-’]⟨NUMBER⟩“AbsTol=”⟨NUMBER⟩“RelTol=”⟨NUMBER⟩

⟨Indicator Constraint⟩ ::=[⟨IDENT⟩‘:’]⟨IDENT⟩‘=’(‘0’|‘1’)“->”⟨Constraint⟩

⟨Bound⟩ ::=⟨Num or Inf⟩⟨SENSE⟩⟨IDENT⟩⟨SENSE⟩⟨Num or Inf⟩
|⟨Num or Inf⟩⟨SENSE⟩⟨IDENT⟩
|⟨IDENT⟩⟨SENSE⟩⟨Num or Inf⟩
|⟨IDENT⟩"FREE"
⟨SENSE⟩ ::='<'['=']
|'>'['=']
|'='['=']
⟨Num or Inf⟩ ::=⟨NUMBER⟩
|['+'|'-'](INFINITY"|"INF")
⟨Signed Number⟩ ::=['+'|'-'] + ⟨NUMBER⟩

Table 1: BNF Notation Symbols

Symbol	Description
⟨A⟩	Non-terminal rule
⟨A⟩ := ...	⟨A⟩ is to be replaced by '...'
[⟨A⟩]	⟨A⟩, optionally
'A'	Terminal character
"A"	Terminal string
(⟨A⟩)	⟨A⟩
(⟨A⟩|⟨B⟩)	⟨A⟩ or ⟨B⟩
[⟨A⟩|⟨B⟩]	⟨A⟩ or ⟨B⟩, optionally
⟨A⟩+	One or more of ⟨A⟩
[⟨A⟩]+	One or more of ⟨A⟩, optionally (essentially *)

5.2.2 Comments in LP Format

Content following a backslash (\) is considered a comment and will be ignored by the solver without parsing. Similarly, blank lines in the file will also be ignored. For example:

```
\ENCODING-ISO-8859-1

\Problem name:Maximize
```

5.2.3 Keywords in LP Format

Keywords in LP format can be in one of three forms:

1. All uppercase (e.g. `PROBLEM`)
2. All lowercase (e.g. `problem`)
3. First letter capitalized (e.g. `Problem`)

Keyword (Indicator)	Alternative Keywords	Meaning
`problem`	`prob`	Problem name
`minimize`	`min`, `minimum`	Minimization problem
`maximize`	`max`, `maximum`	Maximization problem
`subject to`	`s.t.`, `such that`, `st`	Subject to constraints
`bounds`	`bound`	Bounds of expressions
`binaries`	`bin`, `binary`	Binary variables (0-1)
`generals`	`gen`, `general`, `int`, `integer`, `integers`	Integer variables
`sos`		SOS constraints
`general constraints`	`gencon`, `gencons`, `general constraint` `g.c.`	General constraints
`and`		And general constraint type
`or`		Or general constraint type
`inf`	`infinity`	Infinity
`free`		Free variables
`end`		End

The minimization (maximization) keyword is an optional first valid field in the format. If omitted, the solver defaults to a minimization problem. The `end` keyword marks the end of the problem description. Anything after `end` will not be parsed.

5.2.4 LP Format Sections

Before introducing sections, we first introduce two concepts: **tokens** and **expressions**. A **token** is the smallest unit that can be processed at once and cannot be split by inserting any characters, such as spaces or newline characters. We consider numbers, strings, keywords, operators, and relational symbols as tokens. An **expression** is a group of tokens formed by combining tokens with spaces. We only support using spaces as token separators in token groups. For example, in the expression `10 x1 + x2 + 2000`, `10` and `2000` are number tokens, variable names `x1` and `x2` are string tokens, and tokens are separated by spaces. The expression `10,x1,+,x2,+,2000` is not a token group; it is just a string.

Strings are treated as an alphanumeric string of characters, which may additionally contain special characters. Strings must not begin with any of the following symbols: `*`, `+`, `< >`, `=`, `(`, `)`, `^`, `:`, `/`, `,`, or `..` Strings must not begin with a number, nor can they be a keyword.

It is not a variable name because variable names, constraint names, and objective function names need to meet the following criteria: they must be different from each other, their names must not exceed 255 characters in length, they cannot start with numbers or operators `+`, `-`, `*`, `^`, and relational symbols `<`, `>`, `=`, and they cannot be keywords. It is not recommended to use the `E` +number format for variable names, such as `E9` or `e9`, as this will be interpreted as a number. It is recommended to use a combination of characters and numbers, such as `var001`.

Problem Name Section [Optional]

Users may provide a name for their model through this section. If a name is not provided, the solver will assume the name `optv` for the model name.

Objective Function Section

The objective function can start with the `maximize` or `minimize` keyword. Its format is:

```
minimize
-x1 + x2 + -x3 + 0.5 x1 + 100 + x4 + 20
```

In the above format:

1. `minimize` appears alone on one line.
2. Variables can be directly connected with the minus sign (-), such as `x1` and `x3`.
3. Variables can appear multiple times, and we aggregate the coefficients, in this example, the coefficient of `x1` is `-1 + 0.5`.
4. Offsets can appear in the middle or at the end of the expression, such as `100` and `20`. Multiple offsets are summed.

In actual usage, the objective function can have a name and can be written over multiple lines, but attention should be paid to some details, such as:

```
minimize
obj: -x1 + x2 + -x3 + 0.5 x1
+ 100 + x4 + x5 + 1.5 + 2.0
x6
```

The objective function name needs to be followed by a colon (:) as a separator; when line breaks occur, tokens are the smallest unit. If no objective function name is provided, `obj` will be inferred as the objective name and a warning will be displayed.

Multiple Objectives

The OptVerse solver supports multi-objective problems as well. Multi-objective models must have linear objectives, with no quadratic terms. An empty objective is permitted for multi-objective problems. Each objective in a multi-objective model must have a header that includes the following information:

1. [Optional] name of objective, with colon to indicate end of name
2. Priority
3. Weight (may be signed)
4. Absolute tolerance
5. Relative tolerance

The format for these parameters is as follows, with an illustrative example of specifying multiple objectives:

```
obj1: Priority=2 Weight=1.0 AbsTol=1.0e-6 RelTol=0.0
 x0 + x1 + 2
obj2: Priority=1 Weight=1.0 AbsTol=1.0e-6 RelTol=0.0
 3 x0 + 2 x1 - 3
```

Quadratic Objectives

Objective functions may optionally contain quadratic terms. A quadratic region of the objective is enclosed by brackets `[ <quad terms> ]` and should be followed by `/ 2`, where the spaces are optional. If `/ 2` is not provided, this will still be inferred and a warning will be displayed. Division by values other than `2` will result in a syntax error. If a user wishes to avoid this division, they will have to multiply the terms in the file before reading in the `.lp` file.

There are two ways to express a quadratic term:

1. `x^2`, or `x ^ 2`
2. `x*x`, or `x * x`

either forms are accepted, with the spacing around the `*` and `^` symbols being optional. Cross-terms are indicated with `*` whereas diagonals may use either `*` or `^`. A coefficient of `1.0` is inferred if not provided for quadratic terms.

For efficiency, it is best to order your terms such that they are increasing in the lowest index variable, e.g:

```
minimize
obj: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x2^2 + x1 * x2 + x3^2 + x1 * x3 + x2 * x3 ] / 2
```

should be written as:

```
minimize
obj: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x1 * x2 + x1 * x3 + x2^2 + x2 * x3 + x3^2 ] / 2
```

Constraints Section

The keyword `subject to` for specifying constraints must also be written on a separate line. Each new constraint must start on a new line. Constraints can optionally provide constraint names, followed by a colon after the constraint name. Constraint names must follow the same rules as variable names. If a constraint name is not provided, one will be inferred as `R_#` where `#` is the total number of rows when adding.

Constraints are defined as linear expressions composed of variables, constants, and operators, followed by a relational symbol and a coefficient on the right-hand side. Using one of the following relational symbols can intuitively indicate the meaning of the constraint: >=, <=, or =. For example, here is a named constraint:

```
subject to:
depts01: -x1 + 0.5 x2 <= 40
```

However, it is not allowed to have expressions on both sides of the relational symbol, such as:

```
subject to:
depts02: -x1 + 0.5 x2 <= 40 + x1
```

And double-sided relational symbols, such as:

```
subject to:
depts03: 0 <= -x1 + 0.5 x2 <= 40 + x1 <= 40
```

Quadratic Constraints

Just like the objective function expression, constraint expressions can also be written over multiple lines, but consistency is required.

Constraint expressions may also optionally contain quadratic terms. A quadratic region of the constraint is enclosed by brackets `[ <quad terms> ]`. Unlike objectives, they should **not** be followed by `/ 2`. There are two ways to express a quadratic term:

1. `x^2`, or `x ^ 2`
2. `x*x`, or `x * x`

either forms are accepted, with the spacing around the `*` and `^` symbols being optional. Cross-terms are indicated with `*` whereas diagonals may use either `*` or `^`. A coefficient of `1.0` is inferred if not provided for quadratic terms.

For efficiency, it is best to order your terms such that they are increasing in the lowest index variable, e.g:

```
subject to:
c1: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x2^2 + x1 * x2 + x3^2 + x1 * x3 + x2 * x3 ]
```

should be written as:

```
subject to:
c1: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x1 * x2 + x1 * x3 + x2^2 + x2 * x3 + x3^2 ]
```

Indicator Constraints

A constraint may be represented as an indicator constraint. An indicator constraint allows conditionally including a constraint in the problem if some binary condition is met. An indicator constraint is formatted starting with an optional name, followed (in order) by a binary variable, equals sign, a value of 0 or 1, an arrow "->", and finally an unlabelled linear constraint.

For example:

```
ind0: x = 0 -> y + z <= 20
```

Bounds Section [Optional]

The `bounds` section is optional. If it exists, it indicates that variable bounds provided need to be used for the variables. Depending on the number of bounds provided, each expression in a line can be divided into single bounds or double bounds. Single bounds can be `x >= 10` or `x < 10`, while double bounds can be `10 <= x <= 15`. Neither type of bound allows for multi-line writing. If variable bounds are not provided, the solver's internal values will be used by default. If a single bound is defined for a variable, the appropriate default bound will be used as the second bound.

There are several guidelines to follow:

1. Fixed bound expressions are not allowed; that is, they cannot be written as `x1 = 20` but should be uniformly written as `20 <= x1 <= 20`.
2. Supports `+(-)inf`, such as `x1 > -inf`.
3. Supports `free`, such as `x1 free`.

Generals and Binaries Sections [Optional]

The `generals` and `binaries` sections in an LP file are used to indicate variables that must have integer values in feasible solutions. These two sections have similar formats, so we introduce them together in one section; in practice, their presence or absence does not affect each other. Variables registered in these two sections will have default bounds defined. For variables registered in the `generals` section, the default range is 0 to 10^{20}. For variables registered in the `binaries` section, the default bounds are 0 and 1. Variables are registered by listing them in the corresponding section, such as:

```
Generals
x4 x5 x6 x7
x8 x9

Binaries
x1 x2 x3 x4
```

As shown, different variables are separated by spaces, and multi-line writing is allowed. Also, variables can be registered in different sections. For example, for variable `x4`, the final variable type is determined by the smallest feasible range.

Variable designation can appear in any order or multiple times. If a variable is designated more than once, the final assignment will be respected.

SOS Constraints Section [Optional]

The `sos` section allows users to specify an SOS constraint for the model. Each SOS constraint must start on a new line, and must end with a newline. SOS constraints may optionally have a name, followed by a colon after the SOS constraint name. If a constraint name is not provided, one will be inferred as `SOS_#` where `#` is the number of SOS constraints. SOS names must be followed by an SOS type signifier (one of `S1` or `S2`) followed by two colons. Finally, a space-separated list of variables and weights are provided for the SOS constraints.

An example SOS section is provided for illustration:

```
SOS
   sos1: S1::   x : 1.0    y : 2.0
   sos2: S2::   x : 1.0    z : 2.0
```

General Constraints Section [Optional]

Table 2: Supported General Constraints

Constraint	Keyword	Description
And	`AND`	Resultant variable is non-zero if all supplied variables are non-zero.
Or	`OR`	Resultant variable is non-zero if any of the supplied variables are non-zero.
Max	`MAX`	Resultant variable is equal to maximum of supplied values and variables.
Min	`MIN`	Resultant variable is equal to minimum of supplied values and variables.

The `general constraints` section allows users to specify general constraints in the model, similarly to the `subject to` section. Each new general constraint must start on a new line. General constraints can optionally provide names, followed by a colon after the constraint name. Constraint names must follow the same rules as variable names. If a constraint name is not provided, one will be inferred as `GC_#` where `#` is the total number of general constraints when adding.

The general constraints in .lp files follow the same general specification. They are started by an optional name delimited by a `:`. After the constraint name, the user provides a variable name that represents the "resultant" followed by an equals sign. After the equality, the user must provide a general constraint type specifier (e.g. `AND`, or `OR`). Finally, the user must supply a comma separated list of variables which specify the constraint behaviour.

Consider the following examples:

```
General Constraints
    and1: r1 = AND ( x1, x2, x3 )
    and2: r2 = AND ( x3, x1 )
    or1:  r3 = OR ( x1, x2, x3 )
    or2:  r4 = OR ( x3, x1 )
    max1: r5 = MAX ( x1, x2, 10.0 )
    min1: r6 = MIN ( x1, x2, 10.0 )
```

Note

At present, our solver only supports LP and QP problems, and does not support nonlinear fields, semi-continuous, or semi-integer types.

5.3 FZN File Format Description

FlatZinc models consist of variable declarations and constraint definitions as well as a definition of the objective function if the problem is an optimization problem.

Below, we provide a standard FZN file:

```
var bool: BOOL0;
var bool: BOOL1;
var bool: BOOL2;
var 0..10: INT0;
var 0..20: INT1;
var {0, 2, 5, 8}: INT2;
array [1..3] of var 1..20: x;
var 0..1000: solution;
constraint array_bool_and([BOOL1, BOOL2], true);
constraint array_bool_or([BOOL0, BOOL1], true);
constraint int_lin_ne_imp([-2, 1, 1],[INT0, INT1, INT2], 5, BOOL0);
constraint int_lin_ge([-2, 2], [x[1], x[3]], 15);
constraint int_lin_eq([1, 1, 1, -1], [x[1], x[2], x[3], solution], 0);
solve maximize solution;    % objective: maximize(x[1] + x[2] + x[3])
```

An FZN problem consists of three sections: variables, constraints, and objective. The ordering of these sections matters and should adhere to the order specified in this document.

A FlatZinc model consists of multiple items each of which has a semicolon `;` at its end. FlatZinc supports comments. A comment starting with a `%` indicates that the rest of the line is a comment.

FlatZinc uses the UTF-8 character set. Non-ASCII characters can only appear in string literals.

FlatZinc syntax is case sensitive (`foo` and `Foo` are different names). Identifiers start with a letter (`[A-Za-z]`) and are followed by any sequence of letters, digits, or underscores (`[A-Za-z0-9_]`). Additionally, identifiers of variable or parameter names may start with an underscore. Identifiers that correspond to the names of predicates, predicate parameters and annotations cannot have leading underscores.

The following keywords are reserved and cannot be used as identifiers:

`annotation`, `any`, `array`, `bool`, `case`, `constraint`, `diff`, `div`, `else`, `elseif`, `endif`, `enum`, `false`, `float`, `function`, `if`, `in`, `include`, `int`, `intersect`, `let`, `list`, `maximize`, `minimize`, `mod`, `not`, `of`, `satisfy`, `subset`, `superset`, `output`, `par`, `predicate`, `record`, `set`, `solve`, `string`, `symdiff`, `test`, `then`, `true`, `tuple`, `union`, `type`, `var`, `where`, `xor`.

Note that some of these keywords are not used in FlatZinc. They are reserved because they are keywords in Zinc and MiniZinc.

FlatZinc syntax is insensitive to whitespace. FlatZinc has no predefined maximum line length.

5.3.1 FZN Format Sections

Variables Section

The `Variables` section allows users to specify variables and arrays of variables for the model. To specify a variable, the keyword `var` is used. The user should also specify the type of the variable, its domain (for non-binary) and the name of the variable. For example, a boolean variable can be specified as follows: `var bool: b1;`. To specify the integer variable, its domain also needs to be specified. This can be done either by an interval: `var 0..10: x1;` or by a set of values: `var {0, 2, 3, 7, 8}: x2;`. Also, the user can specify arrays. For this, the keyword `array` is used, as well as square brackets with indices of the array elements. For example: `array [1..3] of var bool: b;` or `array [1..3] of var 1..20: x;`.

Constraints Section

The `Constraints` section allows users to specify constraints for the model. Current OptVerse CP solver supports the following types of constraints:

Linear:

- bool_lin_eq(array [int] of int: as, array [int] of var bool: bs, var int: c)
- bool_lin_le(array [int] of int: as, array [int] of var bool: bs, int: c)
- bool2int(var bool: a, var int: b)
- int_plus(var int: a, var int: b, var int: c)
- set_in(var int: x, var set of int: S)
- set_in_reif(var int: x, var set of int: S, var bool: r)
- bool_not(var bool: a, var bool: b)

Boolean Or:

- array_bool_or(array [int] of var bool: as, var bool: r)
- bool_clause(array [int] of var bool: as, array [int] of var bool: bs)
- bool_or(var bool: a, var bool: b, var bool: r)

Boolean And:

- array_bool_and(array [int] of var bool: as, var bool: r)
- bool_and(var bool: a, var bool: b, var bool: r)

Boolean Xor:

- array_bool_xor(array [int] of var bool: as)

- bool_xor(var bool: a, var bool: b, var bool: r)

Table:

- table_bool(array [int] of var bool: x, array [int, int] of bool: t)
- table_int(array [int] of var int: x, array [int, int] of int: t)

No Overlap:

- disjunctive_strict(array [int] of var int: s, array [int] of var int: d)

No Overlap 2D:

- fzn_diffn(array[int] of var int: x, array[int] of var int: y, array[int] of var int: dx, array[int] of var int: dy)
- fzn_diffn_nonstrict(array[int] of var int: x, array[int] of var int: y, array[int] of var int: dx, array[int] of var int: dy)

Cumulative:

- disjunctive(array [int] of var int: s, array [int] of var int: d)
- cumulative(array [int] of var int: s, array [int] of int: d, array [int] of int: r, int: b)
- var_cumulative(array [int] of var int: s, array [int] of int: d, array [int] of int: r, var int: b)

All Different:

- all_different_int(array [int] of var int: x)

Element:

- array_bool_element(var int: b, array [int] of bool: as, var bool: c)
- array_var_bool_element(var int: b, array [int] of var bool: as, var bool: c)
- array_var_int_element(var int: b, array [int] of var int: as, var int: c)
- array_int_element(var int: b, array [int] of int: as, var int: c)

Circuit:

- circuit(array[int] of var int: x, int: base)
- subcircuit(array[int] of var int: x, int: base)

Max Equality:

- int_max(var int: a, var int: b, var int: c)
- maximum_int(var int: m, array [int] of var int: x)
- array_int_maximum(var int: m, array[int] of var int: x)

Min Equality:

- int_min(var int: a, var int: b, var int: c)
- minimum_int(var int: m, array [int] of var int: x)
- array_int_minimum(var int: m, array[int] of var int: x)

Abs Equality:

- int_abs(var int: a, var int: b)

Multiplication Equality:

- int_times(var int: a, var int: b, var int: c)

Division Equality:

- int_div(var int: a, var int: b, var int: c)

Modulo Equality:

- int_mod(var int: a, var int: b, var int: c)

Inverse:

- fzn_inverse(array [int] of var int: f, array [int] of var int: invf)

Next group of *Linear* constraints supports usage of `_imp` or `_reif` suffixes:

- bool_eq(var bool: a, var bool: b)
- bool_ge(var bool: a, var bool: b)
- bool_gt(var bool: a, var bool: b)
- bool_le(var bool: a, var bool: b)
- bool_lt(var bool: a, var bool: b)
- bool_ne(var bool: a, var bool: b)
- int_eq(var int: a, var int: b)
- int_ge(var int: a, var int: b)
- int_gt(var int: a, var int: b)
- int_le(var int: a, var int: b)
- int_lt(var int: a, var int: b)
- int_ne(var int: a, var int: b)
- int_lin_eq(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_ge(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_gt(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_le(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_lt(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_ne(array [int] of int: as, array [int] of var int: bs, int: c)

When using suffixes, it is necessary to use an additional literal, for example `bool_eq_reif(var bool: a, var bool: b, var bool: r);`. Suffix `_reif` denotes that variable `r` is equivalent to the constraint value. Suffix `_imp` adds an enforcement literal to the constraint.

Objective Function Section

The objective function starts with the `solve` keyword. Depending on the objective function, next keyword should be `satisfy` for satisfaction problems, `minimize` or `maximize` for optimization problems. For minimization or maximization problems, the objective variable should be specified. Its format is:

```
solve maximize INT0;
```

CHAPTER

SIX

ATTRIBUTES REFERENCES

6.1 OptVerse MP Attributes

6.1.1 OPTVModel Attributes

These are model-level attributes, meaning they pertain to the model as a whole rather than to any specific variable or constraint within it. To retrieve the value of such an attribute, call the `Get()` method directly on the model object. For attributes that users are allowed to modify, corresponding `Set()` methods can be used.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 1: OptVerse MP Model Attributes

Name	Description
NUM_INT_VARS	Number of integer variables
NUM_VARS	Total number of variables
NUM_CONSTRS	Total number of constraints
NUM_LIN_CONSTRS	Number of linear constraints
NUM_QUAD_CONSTRS	Number of quadratic constraints
NUM_CONES	Number of cones
SOL_COUNT	Number of solutions stored
OBJ_SENSE	Objective sense (minimization, maximization)
STATUS	Optimization status
OBJ_BOUND	Best bound on the objective
OBJ_VAL	Primal solution objective value
DUAL_OBJ_VAL	Dual objective value
OBJ_OFFSET	Objective offset constant
POOL_OBJ_VAL	The Nth best objective value stored in the solution pool
MIP_GAP	Relative optimality gap for mixed integer programming model
RUN_TIME	Elapsed time for optimization
FINGERPRINT	Model fingerprint
NAME	Model name

NUM_INT_VARS

Type: int

Modifiable: No

The number of integer variables. This includes both binary and integer variables.

NUM_VARS

Type: int

Modifiable: No

The total number of variables.

NUM_CONSTRS

Type: int

Modifiable: No

The total number of constraints.

NUM_LIN_CONSTRS

Type: int

Modifiable: No

The number of linear constraints.

NUM_QUAD_CONSTRS

Type: int

Modifiable: No

The number of quadratic constraints.

NUM_CONES

Type: int

Modifiable: No

Number of cones in the model.

SOL_COUNT

Type: int

Modifiable: No

The number of solutions stored. For LP problems, this will be 1 if an optimal solution is found, otherwise it will be zero. For MIP problems, this attribute indicates the number of solutions found (optimal + sub-optimal).

OBJ_SENSE

Type: int

Modifiable: Yes

Objective sense for optimization. Value may be -1 (maximization) or 1 (minimization).

See *OPTVSense*.

STATUS

Type: int

Modifiable: No

Optimization status.

Name	Value	Description
OPTV_OPTIMAL	1	Optimal solution of model has been found
OPTV_INFEASIBLE	2	Model proven to be infeasible
OPTV_INF_OR_UNBD	3	Model proven to be infeasible or unbounded
OPTV_UNBOUNDED	4	Model proven to be unbounded
OPTV_TIME_LIMIT	5	Optimization time exceeded limit set by `TIME_LIMIT`
OPTV_MEM_LIMIT	6	Optimization memory exceeded limit set by `MEM_LIMIT`
OPTV_INTERRUPTED	7	Optimization terminated by user (ex. through `Ctrl`+`C`) or callback
OPTV_UNKNOWN	0	Unknown model status

OBJ_BOUND

Type: double

Modifiable: No

Best bound on the optimal objective. This attribute only has meaning for MIP problems.

OBJ_VAL

Type: double

Modifiable: No

Primal solution objective value.

DUAL_OBJ_VAL

Type: double

Modifiable: No

Dual objective value for the model.

OBJ_OFFSET

Type: double

Modifiable: Yes

Objective offset value.

POOL_OBJ_VAL

Type: double

Modifiable: No

OBJ_VAL for the Nth solution, as set by *SOLUTION_NUMBER*. This attribute only has meaning for MIP problems.

MIP_GAP

Type: double

Modifiable: No

Final relative MIP optimality gap, refer to *GAP* for definitions. This attribute only has meaning for MIP problems.

RUN_TIME

Type: double

Modifiable: No

Runtime for the most recent optimization.

FINGERPRINT

Model fingerprint

Type: long

Modifiable: No

A hash value computed using internal model data and attributes. Models that differ in any meaningful way will differ in their fingerprints. This includes the order that variables or constraints are added to the model.

NAME

Type: str

Modifiable: Yes

Model name.

6.1.2 OPTVVar Attributes

These are variable attributes, meaning that they are associated with specific variables in the model. To retrieve the value of such an attribute, call the `Get()` method directly on the variable object. For attributes that users are allowed to modify, corresponding `Set()` methods can be used.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 2: OptVerse MP Var Attributes

Name	Description
BASIS	Basis status for variable
BRANCH_PRIORITY	MIP variable branching priority
PROHIBITED	Variable is excluded from presolve
LB	Variable lower bound
UB	Variable upper bound
OBJ	Variable linear coefficient in objective
START	MIP warm-start value
X	Variable primal solution value
XN	Variable primal Nth solution value
DUAL	Variable dual solution value
NAME	Variable name
VAR_TYPE	Variable type
IIS_INTEGRAL	Integer feasibility indicator in IIS
IIS_LB	Lower bound indicator in IIS
IIS_UB	Upper bound indicator in IIS

BASIS

Type: int

Modifiable: Yes

Variable status in the current basis.

Name	Value	Description
OPTV_BASIC	0	Basic
OPTV_NONBASIC_LOWER	-1	Non-basic and at lower bound
OPTV_NONBASIC_UPPER	-2	Non-basic and at upper bound
OPTV_SUPERBASIC	-3	Super basic

BRANCH_PRIORITY

Type: int

Modifiable: Yes

Default Value: 0

The numerical value assigned to this attribute guides the selection of fractional variables for branching purposes throughout the MIP search. Higher-valued variables are given precedence over those with lower values. Ties are resolved by resorting to the usual variable selection criteria for branching. The default value is zero. This attribute only affects MIP problems.

PROHIBITED

Type: int

Modifiable: Yes

Specifies if a variable may change during LP presolve. Value may be 0 (allowed) or 1 (prohibited). This attribute has no effect for MIP problems.

LB

Type: double

Modifiable: Yes

Lower bound of variable.

Note

For binary variables, only values of 0 or 1 are allowed. If a user tries to submit a value that is not within 10^{-6} of these values, they will be prevented and a warning will be displayed.

UB

Type: double

Modifiable: Yes

Upper bound of variable.

> **Note**
>
> For binary variables, only values of `0` or `1` are allowed. If a user tries to submit a value that is not within 10^{-6} of these values, they will be prevented and a warning will be displayed.

OBJ

Type: double

Modifiable: Yes

Variable linear coefficient in objective.

START

Type: double

Modifiable: Yes

MIP warm-start value. Default initialized to *OPTV_UNDF*. This attribute only has meaning for MIP problems.

X

Type: double

Modifiable: No

Variable primal solution value.

XN

Type: double

Modifiable: No

Variable primal solution value.

This attribute is only applicable to MIP problems. Similar to the attribute *X*, when the Nth MIP solution is specified using *SOLUTION_NUMBER*, `XN` returns the primal solution value of the variable.

DUAL

Type: double

Modifiable: No

Dual solution value for variable.

NAME

Type: str

Modifiable: Yes

Variable name.

VAR_TYPE

Type: char

Variable type.

Modifiable: Yes

Table 3: OptVerse MILP Variable Types

Name	Value	Description
OPTV_CONTINUOUS	`'C'`	Continuous
OPTV_INTEGER	`'I'`	Integer
OPTV_BINARY	`'B'`	Binary

Continuous variables can be any floating point number between the bounds. Integer variables can be any integer number between the bounds. Binary variables can be either 0 or 1.

IIS_INTEGRAL

Type: int

Modifiable: Yes

Integer feasibility indicator for the variable in the current IIS. A value of `1` indicates that the variable is infeasible in the IIS due to an integrality constraint. A value of `0` indicates that the integrality of the variable is not the cause of the infeasibility.

Note

This attribute is only meaningful when the variable's type is integer (VAR_TYPE is `'I'` or `'B'`).

IIS_LB

Type: int

Modifiable: Yes

Lower bound feasibility indicator for the variable in the current IIS. A value of `1` indicates that the variable's lower bound is part of the IIS. A value of `0` indicates that the lower bound is not required for infeasibility.

> **Note**
>
> This attribute can be set to `1` to force the variable's lower bound to be included in the IIS, or set to `0` to exclude it. Modifying this attribute may cause the IIS to become invalid.

IIS_UB

Type: int

Modifiable: Yes

Upper bound feasibility indicator for the variable in the current IIS. A value of `1` indicates that the variable's upper bound is part of the IIS. A value of `0` indicates that the upper bound is not required for infeasibility.

> **Note**
>
> This attribute can be set to `1` to force the variable's upper bound to be included in the IIS, or set to `0` to exclude it. Modifying this attribute may cause the IIS to become infeasible.

6.1.3 OPTVConstr Attributes

These are constraint attributes, meaning that they are associated with specific constraints in the model. To retrieve the value of such an attribute, call the `Get()` method directly on the constraint object. For attributes that users are allowed to modify, corresponding `Set()` methods can be used.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 4: OptVerse MP Constraint Attributes

Name	Description
IIS_CONSTR	Indicates whether or not a constraint belongs to a computed irreducible inconsistent subsystem (IIS)
BASIS	Basis status for constraint
PROHIBITED	Constraint is excluded from presolve
LB	Constraint lower bound
UB	Constraint upper bound
X	Constraint primal solution value
XN	Constraint primal Nth solution value
DUAL	Constraint dual solution value
NAME	Constraint name

IIS_CONSTR

Type: bool

Modifiable: No

Indicates whether or not a constraint belongs to a computed irreducible inconsistent subsystem (IIS). If no IIS has been computed, this attribute will be unavailable.

BASIS

Type: int

Modifiable: Yes

Constraint status in the current basis.

Name	Value	Description
OPTV_BASIC	0	Basic
OPTV_NONBASIC_LOWER	-1	Non-basic and at lower bound
OPTV_NONBASIC_UPPER	-2	Non-basic and at upper bound
OPTV_SUPERBASIC	-3	Super basic

PROHIBITED

Type: int

Modifiable: Yes

Specifies if a constraint may change during LP presolve. Value may be 0 (allowed) or 1 (prohibited). This attribute has no effect for MIP problems.

LB

Type: double

Modifiable: Yes

Lower bound of constraint.

UB

Type: double

Modifiable: Yes

Upper bound of constraint.

X

Type: double

Modifiable: No

Constraint primal solution value.

XN

Type: double

Modifiable: No

Constraint primal solution value.

This attribute is only applicable to MIP problems. Similar to the attribute *X*, when the Nth MIP solution is specified using *SOLUTION_NUMBER*, `XN` returns the primal solution value of the constraint.

DUAL

Type: double

Modifiable: No

Dual solution value for constraint.

NAME

Type: str

Modifiable: Yes

Constraint name.

6.1.4 OPTVQConstr Attributes

These are quadratic constraint attributes, meaning that they are associated with specific quadratic constraints in the model. To retrieve the value of such an attribute, call the `Get()` method directly on the constraint object. For attributes that users are allowed to modify, corresponding `Set()` methods can be used.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 5: OptVerse MP Quadratic Constraint Attributes

Name	Description
PROHIBITED	Constraint is excluded from presolve
X	Constraint primal solution value
DUAL	Constraint dual solution value
NAME	Constraint name
QCSENSE	Constraint sense
QCRHS	Constraint right-hand-side value

DUAL

Type: double

Modifiable: No

Dual solution value for quadratic constraint. Note this value might be unavailable due to model geometry at the optimal solution.

NAME

Type: str

Modifiable: Yes

Quadratic constraint name.

PROHIBITED

Type: int

Modifiable: Yes

Specifies if a constraint may change during QCP presolve. Value may be 0 (allowed) or 1 (prohibited). This attribute has no effect for MIP problems.

QCRHS

Type: double

Modifiable: Yes

Quadratic constraint right-hand side. Note that a RHS of 1e20 can lead to numerical issues, especially on quality quadratic constraints.

QCSENSE

Type: char

Modifiable: Yes

Quadratic constraint sense ('<', '>', or '='), while '<' and '>' refer to mathematical constraint senses less-than-or-equal and greater-than-or-equal respectively.

X

Type: double

Modifiable: No

Quadratic constraint primal solution value.

6.1.5 OPTVSOS Attributes

These are SOS attributes, meaning that they are associated with specific special-ordered set constraints in the model. To retrieve the value of such an attribute, call the `Get()` method directly on the OPTVSOS object. For attributes that users are allowed to modify, corresponding `Set()` methods can be used.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 6: OptVerse MP SOS Attributes

Name	Description
SOS_TYPE	SOS type $\in$ {1,2}
NAME	SOS constraint name

SOS_TYPE

Type: int

Modifiable: No

SOS Type. May be 1 or 2.

NAME

Type: str

Modifiable: Yes

SOS constraint name.

6.1.6 OPTVGenConstr Attributes

These are general constraint attributes, meaning that they are associated with specific general constraints in the model. To retrieve the value of such an attribute, call the `Get()` method directly on the general constraint object. For attributes that users are allowed to modify, corresponding `Set()` methods can be used.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 7: OptVerse MP General Constraint Attributes

Name	Description
GENCONSTR_TYPE	General constraint type

GENCONSTR_TYPE

Type: int

Modifiable: No

General constraint type.

Table 8: OptVerse General Constraint Types

Name	Value	Description
OPTV_GENCONSTR_AND	0	Resultant variable is non-zero if all supplied variables are non-zero.
OPTV_GENCONSTR_OR	1	Resultant variable is non-zero if any of the supplied variables are non-zero.
OPTV_GENCONSTR_INDICATOR	2	Binary variable is non-zero if and only if the associated logical condition is satisfied.

6.1.7 Multi-Objective Attributes

These are the attributes for setting and querying multiple objectives.

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 9: OptVerse MP Multi-Objective Attributes

Name	Description
NUM_OBJ	Number of multi-objectives
OBJ_N	Objective coefficient of the variable in the Nth objective
OBJ_N_VAL	Value of the Nth objective
OBJ_N_WEIGHT	Weight of the Nth objective (for weighted combination of multiple objectives on the same level)
OBJ_N_OFFSET	Offset value of the multi-objective
OBJ_N_REL_TOL	Relative tolerance of the multi-objective
OBJ_N_ABS_TOL	Absolute tolerance of the multi-objective
OBJ_N_NAME	Multi-objective name

NUM_OBJ

Type: int

Modifiable: Yes

Determines number of objectives in the problem. Adds/removes multi-objectives from the problem depending on the current number of multi-objectives in the problem. Setting this attribute to 0 converts the problem into a feasibility problem.

OBJ_N

Type: double

Modifiable: Yes

Variable linear coefficient in the multi-objective determined by the *OBJ_NUMBER* parameter.

OBJ_N_VAL

Type: double

Modifiable: No

Value of the primal solution determined by *SOLUTION_NUMBER* on the multi-objective determined by *OBJ_NUMBER*.

OBJ_N_WEIGHT

Type: double

Modifiable: Yes

Determines the weight of the multi-objective determined by the *OBJ_NUMBER* parameter. Allows to get or set the weight of the multi-objective.

The sign of the value of this attribute determines the sense of the multi-objective:

positive weight means that the multi-objective has the sense equal to *OPTVSense::MINIMIZE*;

negative weight means that the multi-objective has the sense equal to *OPTVSense::MAXIMIZE*.

OBJ_N_OFFSET

Type: double

Modifiable: Yes

Determines the offset of the multi-objective determined by the *OBJ_NUMBER* parameter. Allows to get or set the offset of the multi-objective.

OBJ_N_REL_TOL

Type: double

Modifiable: Yes

Determines the relative tolerance of the multi-objective determined by the *OBJ_NUMBER* parameter. Allows to get or set the relative tolerance of the multi-objective.

OBJ_N_ABS_TOL

Type: double

Modifiable: Yes

Determines the absolute tolerance of the multi-objective determined by the *OBJ_NUMBER* parameter. Allows to get or set the absolute tolerance of the multi-objective.

OBJ_N_NAME

Type: str

Modifiable: Yes

Name of the multi-objective determined by the *OBJ_NUMBER* parameter.

6.1.8 Solution Quality Attributes

For examples of how to query or modify attributes, refer to *Attribute Examples*.

Table 10: OptVerse MP Solution Quality Attributes

Name	Description
MAX_VIO	Maximum of all violations
BOUND_VIO	Maximum variable bound violation
CONSTR_VIO	Maximum constraint slack bound violation
INT_VIO	Maximum integrality violation
BOUND_VIO_INDEX	Index of variable with the largest unscaled bound violation
CONSTR_VIO_INDEX	Index of constraint with the largest slack bound violation
INT_VIO_INDEX	Index of variable with the largest integrality violation
BOUND_VIO_SUM	Sum of variable bound violations
CONSTR_VIO_SUM	Sum of constraint slack bound violations
INT_VIO_SUM	Sum of integrality violations

MAX_VIO

Type: double

Modifiable: No

Maximum of all violations.

BOUND_VIO

Type: double

Modifiable: No

Maximum variable bound violation.

CONSTR_VIO

Type: double

Modifiable: No

Maximum constraint slack bound violation.

INT_VIO

Type: double

Modifiable: No

Maximum integrality violation.

BOUND_VIO_INDEX

Type: int

Modifiable: No

Index of variable with the largest unscaled bound violation.

CONSTR_VIO_INDEX

Type: int

Modifiable: No

Index of constraint with the largest unscaled slack bound violation.

INT_VIO_INDEX

Type: int

Modifiable: No

Index of variable with the largest integrality violation.

BOUND_VIO_SUM

Type: double

Modifiable: No

Sum of variable bound violations.

CONSTR_VIO_SUM

Type: double

Modifiable: No

Sum of constraint slack bound violations.

INT_VIO_SUM

Type: double

Modifiable: No

Sum of integrality violations.

6.2 Attribute Examples

OptVerse MP's attribute interface is intentionally orthogonal: a small set of consistent routines lets you access and manipulate a large number of attributes. In particular:

- Attribute names and their semantics are identical across all supported programming language APIs—only minor language-specific syntax (or "decoration") varies.
- To query or modify an attribute, simply identify the attribute type corresponding to its data type (e.g., double, integer, etc.) in your chosen programming language.

The data types and their corresponding attribute types are as follows:

Data Type	Attribute Type
`int`	OPTVIntAttr
`double`	OPTVDblAttr
`long`	OPTVLongAttr
`string`	OPTVStrAttr
`char`	OPTVCharAttr
`bool`	OPTVBoolAttr

The following shows how to query this attribute's value for a single variable across the different APIs:

Language	Attribute Query Example
C++	*var.Get(OPTVDblAttr::LB)*
java	*var.get(OPTVDblAttr.LB)*
python	*var.Get(OPTVDblAttr.LB)* or var.LB

Note that attribute changes are not applied immediately. They only take effect when you update or optimize the model (using *OPTVModel::Update()* or *OPTVModel::Optimize()* in C++).

CHAPTER

SEVEN

PARAMETERS REFERENCES

7.1 MIP Parameters References

7.1.1 Termination

These parameters affect the termination of the algorithms. If the algorithm exceeds any of these limits, it will terminate and report a non-optimal termination status. Note that the algorithm won't necessarily stop the moment it hits the specified limit. The termination check may occur well after the limit has been exceeded.

Table 1: Termination Parameters

Name	Description
TIME_LIMIT	Maximum time (s).
MEM_LIMIT	Maximum memory (MB) to be used by the solver.
SUBMIP_NODES	Limit the number of nodes explored during MIP heuristics.
START_NODE_LIMIT	Limit the number of MIP branch-and-bound nodes.

TIME_LIMIT

Type: double

Maximum time (s).

Default Value: 10^{20}

Minimum Value: 0

Maximum Value: 10^{20}

If this limit is exceeded, the solver will stop and return a status of *OPTV_TIME_LIMIT* to indicate this threshold was exceeded.

The polling for time limit is restricted by the per iteration cost. For costly optimizations it is recommended to use an external time limit, such as through `timeout`.

MEM_LIMIT

Type: int

Maximum memory (MB) to be used by the solver.

Default Value: 2147483647

Minimum Value: 100

Maximum Value: 2147483647

This parameter will limit the solver from exceeding a specified amount of RAM during optimization. If this limit is exceeded, the solver will stop and return a status of *OPTV_MEM_LIMIT* to indicate this threshold was exceeded.

SUBMIP_NODES

Type: int

Limit the number of MIP nodes explored during heuristics.

Default Value: 500

Minimum Value: 0

Maximum Value: 2147483647

This parameter limits the number of nodes explored by MIP-based heuristics. Exploring more nodes can produce better solutions, but it generally takes longer.

Note

This parameter will only impact MIP problems.

START_NODE_LIMIT

Type: int

Limit the number of MIP branch-and-bound nodes.

Default Value: -1

Minimum Value: -3

Maximum Value: 2147483647

This parameter gives control over the number of branch-and-bound nodes that will be explored during an MIP partial start. By default, the solver will automatically determine a maximum number of nodes during MIP start. A value of -2 will disable partial MIP starts, instead only checking full MIP starts. A value of -3 will entirely disable MIP starts.

Note

This parameter will only impact MIP problems.

7.1.2 Tolerances

These parameters control the allowable feasibility or optimality violations.

Table 2: Tolerance Parameters

Name	Description
FEAS_TOL	Feasibility tolerance of primal constraints.
INT_TOL	Feasibility tolerance of integer variables.
MARKOWITZ_TOL	Threshold pivoting tolerance.
GAP	Relative MIP optimality gap.

FEAS_TOL

Type: double

Feasibility tolerance of primal constraints.

Default Value: 10^{-6}

Minimum Value: 10^{-9}

Maximum Value: 0.01

A smaller value will result in smaller constraint violations, but may lead to an increase in the number of iterations.

INT_TOL

Type: double

Feasibility tolerance of integer variables.

Default Value: 10^{-6}

Minimum Value: 10^{-9}

Maximum Value: 0.01

A smaller value may result in smaller integrality violations, but might affect performance. A larger value rarely impacts runtime performance.

MARKOWITZ_TOL

Type: double

Threshold pivoting tolerance for simplex factorization.

Default Value: 0.1

Minimum Value: 10^{-4}

Maximum Value: 0.999

A larger value indicates more numerical stability, at the potential cost of runtime performance.

GAP

Type: double

Relative MIP optimality gap.

Default Value: 0.0001

Minimum Value: 0

Maximum Value: 10^{100}

If g is the gap value, f_P is the primal objective and f_D is the dual objective, then $g = \frac{|f_P - f_D|}{|f_P|}$.

If the relative gap between lower and upper bounds on the objective is less than the gap value, the solver will terminate with a status of *OPTV_OPTIMAL*.

7.1.3 Logging

These parameters control the logging information.

Table 3: Logging Parameters

Name	Description
OUTPUT_FLAG	Disable logging entirely.

OUTPUT_FLAG

Type: int

Control OptVerse output.

Default Value: 0

Minimum Value: 0

Maximum Value: 1

When set to 0, this flag completely suppresses the output of OptVerse. Output to both console and to file will be suppressed.

7.1.4 I/O

These parameters can be used to read or write information.

Table 4: I/O Parameters

Name	Description
LOG_FILE	Path to log file.
PARAM_FILE	Path to parameter file.
REPLAY_FILE	Path to output recording file.
RESULT_FILE	Path to output solution file.
SOL_FORMAT	Output solution format.

LOG_FILE

Type: str

Path to log file.

Default Value: `"./optv.log"`

If *OUTPUT_FLAG* is set to 0, this parameter will be ignored.

Note

No log file is generated by default when building a model through the SDK.

When the command line tool is used, the default value of this parameter is `"./optv.log"`.

PARAM_FILE

Type: str

Path to parameter file.

Default Value: `""`

Provide a parameter file to the solver. By default, no parameter file will be read.

REPLAY_FILE

Type: str

Path to output replay file.

Default Value: `""`

If this is an empty string, no recording file will be produced. Otherwise, a recording file will be generated at the path provided.

Note

This parameter **must** be set prior to construction of an `OPTVModel` object, and may only be set through an `OPTVEnv` object.

RESULT_FILE

Type: str

Path to output solution file.

Default Value: `"./<input_file>.sol"`

Note

If a model is built through the SDK, this will default to no output and **must** be set to produce a solution file.

When using the command line tool, the default path for the result file is determined by the name of the input file.

SOL_FORMAT

Type: int

Output solution format.

Default Value: 0

Minimum Value: 0

Maximum Value: 1

This parameter allows specifying whether the output solution should exclude or include zero values. If a value of 0 is provided, the output will be sparse (only non-zero values). If the value is 1, the output will be dense (include zero values).

7.1.5 LP

These parameters control the operation of the LP algorithms.

Table 5: LP Parameters

Name	Description
CROSSOVER	IPM crossover strategy.
METHOD	LP Solving method (e.g. simplex, IPM, etc.).
OPTIMAL_BASIS	Perform basis cleanup for optimal solutions.

CROSSOVER

Type: int

IPM crossover strategy.

Default Value: 2

Minimum Value: 0

Maximum Value: 4

IPM crossover will produce a basic solution from the barrier solution. Crossover is composed of three steps:

1) Primal Push - Primal variables are pushed to their bounds.

2) Dual Push - Dual variables are pushed to their bounds.

3) Cleanup - Simplex removes any infeasibilities. Can be primal or dual.

The ordering of the first two steps, and the cleanup method used is determined by this parameter.

Value	First Push	Second Push	Cleanup
0	Disabled	Disabled	Disabled
1	Dual	Primal	Primal
2 (Default)	Dual	Primal	Dual
3	Primal	Dual	Primal
4	Primal	Dual	Dual

METHOD

Type: int

LP Solving method.

Default Value: -1

Minimum Value: -1

Maximum Value: 4

This parameter allows specifying the optimization method for LP problems.

Value	Method
-1	Auto
0	Primal simplex
1	Dual simplex
2	Interior point method (IPM)
3	Deterministic IPM
4	Concurrent Solver

Note

This parameter affects both the solution of a LP problem, and the first LP relaxation of an MIP problem.

OPTIMAL_BASIS

Type: int

Whether or not an optimal basis is required when the solver finishes.

Default Value: 1 (On)

Minimum Value: 0 (Off)

Maximum Value: 1 (On)

This advanced parameter allows the user to skip the basis cleanup that occurs after a feasible solution is found. Turning this option off is useful if a user is not concerned whether the feasible solution obtained is basic. If this setting is turned off, the basis cleanup will not be performed.

Warning

Basic status will likely not be correct if this parameter is turned off. This means that if a user wishes to obtain the value of a variable (*var.Get(OPTVDblAttr::X)*) or constraint (*constr.Get(OPTVDblAttr::X)*), this setting should be **on** (the default). Additionally, this will likely interfere with the warm-start behaviour. Users should only turn this setting off if they are only interested in knowing if a problem is feasible.

7.1.6 MIP

These parameters control the operation of the MIP algorithms.

Table 6: OptVerse MIP Parameters

Name	Description
BRANCH_DIR	Preferred MIP branching direction.
VAR_BRANCH	Variable branching method.
MIP_FOCUS	MIP solver focus.
HEURISTICS	Ratio of expected MIP heuristic overhead.
POOL_SOLUTIONS	Number of MIP solutions to store in the pool.
SOLUTION_NUMBER	Select an MIP solution.
PRESOLVE_LEVEL	Strength of presolve routines.
SEED	PRNG initial seed value.
THREADS	Number of threads.

BRANCH_DIR

Type: int

Preferred MIP branching direction.

Default Value: 0

Minimum Value: -1

Maximum Value: 1

This parameter controls which child node is explored first in the branch-and-cut search. By default this will be chosen automatically. A value of -1 will explore downwards first, while a value of +1 will explore upwards first.

VAR_BRANCH

Type: int

Variable branching method.

Default Value: -1

Minimum Value: -1

Maximum Value: 2

This parameter controls the variable branch selection strategy. The values of this parameter correspond to the following meanings:

Value	Description
-1	Auto
0	Reliability branching
1	Strong branching
2	Pseudo cost branching

MIP_FOCUS

Type: int

MIP solver focus.

Default Value: 0

Minimum Value: 0

Maximum Value: 3

This parameter controls the priority of the MIP solver. By default, the MIP will take a balanced approach to solving problems. The values of this parameter correspond to the following meanings:

Value	Description
0	Balanced.
1	Prioritize finding a feasible solution quickly.
2	Prioritize proving the optimality.
3	Prioritize improving the lower bound.

HEURISTICS

Type: double

Ratio of expected MIP heuristic overhead during solving.

Default Value: 0.05

Minimum Value: 0

Maximum Value: 1

This parameter controls how long the MIP solver spends in heuristics. The value is provided as a fraction of the total time, so the default value of 0.05 corresponds to 5% of total time. A larger value may allow the MIP solver to find more or better feasible solutions. Likewise, a larger value will lead to slower progress in improving the best bound.

POOL_SOLUTIONS

Type: int

Number of MIP solutions to store in the pool.

Default Value: 10

Minimum Value: 1

Maximum Value: 1000000

This parameter will set a maximum limit for the number of MIP solutions to be stored in the pool.

SOLUTION_NUMBER

Type: int

Select an MIP solution.

Default Value: 0

Minimum Value: 0

Maximum Value: 1000000

This parameter allows the user to specify which solution to query when checking *XN* or *POOL_OBJ_VAL*. A value of 0 will query the best solution, as if querying *X* or *OBJ_VAL*.

Note

This parameter will only impact MIP problems.

PRESOLVE_LEVEL

Type: int

Strength of presolve routines.

Default Value: -1

Minimum Value: -1

Maximum Value: 2

This parameter allows the user to control the strength of the presolve routines.

Value	Strength
-1	Auto
0	Off
1	Conservative
2	Aggressive

SEED

Type: int

PRNG initial seed value.

Default Value: 0

Minimum Value: 0

Maximum Value: 2147483647

The value provided to this parameter will be used as an initial value for the pseudo-random number generation.

THREADS

Type: int

Number of threads for use globally.

Default Value: Half the maximum number of threads available on the machine.

Minimum Value: 1

Maximum Value: The number of virtual processors available.

7.1.7 MIP Cut

These parameters affect the generation of MIP cutting planes.

Table 7: OptVerse MIP Cut Parameters

Name	Description
CUT_PASSES	Maximum number of root separation rounds.
CUTS	MIP cuts level.

CUT_PASSES

Type: int

Maximum number of root separation rounds.

Default Value: 2147483647

Minimum Value: 0

Maximum Value: 2147483647

This parameter limits the number of cutting plane passes during root cut generation.

CUTS

Type: int

MIP cuts level.

Default Value: -1

Minimum Value: -1

Maximum Value: 3

This parameter controls the aggressiveness of the cut generation globally. The default value of -1 will result in the solver automatically controlling this parameter. A value of 0 will turn off cuts, a value of 1 for light cutting, a value of 2 for moderate cutting and a value of 3 for aggressive cutting.

7.1.8 Multiple Objectives

These parameters can be relevant when working with multiple objectives.

Table 8: Multiple Objectives Parameters

Name	Description
OBJ_NUMBER	Id of multi-objective to interact with.

OBJ_NUMBER

Type: int

Select the multi-objective to interact with.

Default Value: 0

Minimum Value: 0

Maximum Value: 2147483647

This parameter allows the user to specify which multi-objective to query when interacting with multi-objective attributes.

7.2 Parameter Examples

OptVerse parameters are designed to be orthogonal: a small set of consistent routines lets you access and manipulate a large number of parameters. In particular:

- Parameter names and their semantics are identical across all supported programming language APIs—only minor language-specific syntax (or "decoration") varies.
- To set a parameter, simply identify the parameter type corresponding to its data type (e.g., double, integer, etc.) in your chosen programming language.

The data types and their corresponding parameter types are as follows:

Data Type	Parameter Type
`int`	OPTVIntParam
`double`	OPTVDblParam
`string`	OPTVStrParam

The following shows how to set this parameter using the different language APIs of MILP Solver.

Language	Parameter Set Example
C++	*`model.Set(OPTVIntParam::THREADS, 10)`*
java	*`model.set(OPTVIntParam.THREADS, 10)`*
python	*`model.Set(OPTVIntParam.THREADS, 10)`* or model.Threads = 10

Note that in MILP Solver parameter changes are not applied immediately. They only take effect when you next update or optimize the model (using *`OPTVModel::Update()`* or *`OPTVModel::Optimize()`* in C++).

Similarly, the following shows how to set CP-specific parameters using different language APIs in CP mode.

Language	Parameter Set Example
C++	*model.Set(OPTVIntParam::CP_THREADS, 10)*
python	*model.Set(OPTVIntParam.CP_THREADS, 10)*

7.3 CP Parameters References

7.3.1 Termination

These parameters affect the termination of the algorithms. If the algorithm exceeds any of these limits, it will terminate and report a non-optimal termination status (see the Status Code section for further details). Note that the algorithm won't necessarily stop the moment it hits the specified limit. The termination check may occur well after the limit has been exceeded.

Table 9: Termination Parameters

Name	Description
CP_TIME_LIMIT	Maximum time (s).
CP_DETER_TIME_LIMIT	CP deterministic time limitation.

CP_TIME_LIMIT

Type: double

Maximum time (s).

Default Value: $2^{63} - 1$

Minimum Value: 0

Maximum Value: $2^{63} - 1$

CP_DETER_TIME_LIMIT

Type: double

CP deterministic time limitation.

Default Value: $2^{63} - 1$

Minimum Value: 0

Maximum Value: $2^{63} - 1$

7.3.2 Tolerances

These parameters control the allowable feasibility or optimality violations.

Table 10: Tolerance Parameters

Name	Description
CP_GAP	CP relative gap.
CP_ABS_GAP	CP absolute gap.

CP_GAP

Type: double

CP relative gap.

Default Value: 0

Minimum Value: 0

Maximum Value: 10^{100}

If g is the gap value, f_P is the primal objective and f_D is the dual objective, then $g = \frac{|f_P - f_D|}{|f_P|}$.

If the relative gap between lower and upper bounds on the objective is less than the `gap` value, the solver will terminate with a status of *OPTV_CP_OPTIMAL*.

CP_ABS_GAP

Type: double

CP absolute gap.

Default Value: 10^{-6}

Minimum Value: 0

Maximum Value: 10^{100}

7.3.3 Algorithm

Algorithmic control parameters for the OptVerse CP Solver.

Table 11: CP Algorithm Parameters

Name	Description
CP_THREADS	Number of threads.
CP_SOLVER_SEED	Seed of the CP solver.
CP_DETERMINISTIC_MODE	Whether to solve with the deterministic mode.
CP_PRESOLVE	Whether CP presolve is enabled.
CP_LP_LEVERAGE_LEVEL	Lp leverage level in the CP solver.
CP_HEURISTIC_MODE	Whether to solve with the heuristic mode.
CP_CORE_BASE_OPT	Whether to optimize with core based methods.

CP_THREADS

Type: int

Number of threads.

Default Value: 1

Minimum Value: 1

Maximum Value: The number of virtual processors available.

CP_SOLVER_SEED

Type: int

Seed of the CP solver.

Default Value: 1

Minimum Value: 0

Maximum Value: $2^{63} - 1$

The value provided to this parameter will be used as an initial value for the pseudo-random number generation.

CP_DETERMINISTIC_MODE

Type: int

Whether to solve with the deterministic mode.

Default Value: 1

Minimum Value: 0

Maximum Value: 1

CP_PRESOLVE

Type: int

Whether CP presolve is enabled.

Default Value: 1

Minimum Value: 0

Maximum Value: 1

CP_LP_LEVERAGE_LEVEL

Type: int

LP leverage level in the CP solver.

Default Value: 1

Minimum Value: 0

Maximum Value: $2^{31} - 1$

CP_HEURISTIC_MODE

Type: int

Whether to solve with the heuristic mode.

Default Value: 0

Minimum Value: 0

Maximum Value: 1

CP_CORE_BASE_OPT

Type: int

Whether to optimize with core based methods.

Default Value: 0

Minimum Value: 0

Maximum Value: 1

CHAPTER

EIGHT

API REFERENCES

8.1 OPTVEnv

The OptVerse environment class handles parameter configuration for both the MP and CP solvers in OptVerse. It also loads the license file defined by the `OPTV_LICENSE_FILE` environment variable, and thus must be initialized before use.

8.1.1 C++ API Reference Manual

class **OPTVEnv**

OPTVEnv()

OPTVEnv::**OPTVEnv**()

Returns
Environment object with default settings and no log file specified.

Note

By default, the log file name `optv.log` will be used.

OPTVEnv::**OPTVEnv**(const char *logFileName)

Parameters
logFileName – Log file path, represented by a C-style string.

Returns
Environment object with default settings and log file specified.

OPTVEnv::**OPTVEnv**(const std::string &logFileName)

Parameters
logFileName – Log file path, represented by a string.

Returns
Environment object with default settings and log file specified.

OPTVEnv::**OPTVEnv**(const *OPTVEnv* &xenv)

Parameters
xenv – OptVerse environment to be copied.

Returns
Copy of OPTVEnv object `xenv`.

OPTVEnv::**OPTVEnv**(const *OPTVEnv* *xenv)

Parameters
xenv – Pointer to OptVerse environment to be copied.

Returns
Copy of OPTVEnv object at `xenv`.

Get()

Obtain a solver parameter value.

See *OptVerse MP Parameters*.

OPTVEnv::**Get**(*OPTVIntParam* param)

Parameters
param – Integer type parameter.

Returns
Parameter value.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVEnv::**Get**(*OPTVDblParam* param)

Parameters
param – Double type parameter.

Returns
Parameter value.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVEnv::**Get**(*OPTVStrParam* param)

Parameters
param – String type parameter.

Returns
Parameter value.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

Set()

Set a solver parameter for the environment. These settings will applied as default to any models subsequently constructed. After model construction however, this does not impact the parameters in the model. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Parameters*.

OPTVEnv::**Set**(*OPTVIntParam* param, int value)

Parameters

- **param** – Integer type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVEnv::**Set**(*OPTVDblParam* param, double value)

Parameters

- **param** – Double type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVEnv::**Set**(*OPTVStrParam* param, const std::string &value)

Parameters

- **param** – String type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

ResetParams()

Reset all parameters to default values.

OPTVEnv::**ResetParams**()

Returns
None.

8.1.2 Java API References

class OPTVEnv

OPTVEnv()

OPTVEnv.**OPTVEnv**()

Returns
Environment object with default settings and no log file specified.

Return type
OPTVEnv

Note

By default, the log file name `optv.log` will be used.

OPTVEnv.**OPTVEnv**(String *logFileName*)

Parameters

- **logFileName** – Log file path, represented by a C-style string.

Returns
Environment object with default settings and log file specified.

Return type
OPTVEnv

OPTVEnv.**OPTVEnv**(OPTVEnv *xenv*)

Parameters

- **xenv** – OptVerse environment to be copied.

Returns
Copy of OPTVEnv object `xenv`.

Return type
OPTVEnv

get()

Obtain a solver parameter value.

See *OptVerse MP Parameters*.

OPTVEnv.**get**(OPTVIntParam *param*)

Parameters

- **param** – Integer type parameter.

Returns
Parameter value.

Return type
int

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVEnv.**get**(OPTVDblParam *param*)

Parameters

- **param** – Double type parameter.

Returns
Parameter value.

Return type
double

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVEnv.**get**(OPTVStrParam *param*)

Parameters

- **param** – String type parameter.

Returns
Parameter value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

set()

Set a solver parameter for the environment. These settings will applied as default to any models subsequently constructed. After model construction however, this does not impact the parameters in the model. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Parameters*.

OPTVEnv.**set**(OPTVIntParam *param*, int *value*)

Parameters

- **param** – Integer type parameter.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVEnv.**set**(OPTVDblParam *param*, double *value*)

Parameters

- **param** – Double type parameter.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVEnv.**set**(OPTVStrParam *param*, String *value*)

Parameters

- **param** – String type parameter.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

8.1.3 Python API Reference Manual

class OPTVEnv

OPTVEnv()

OPTVEnv.**OPTVEnv**()

Returns
Environment object with default settings and no log file specified.

Return type
OPTVEnv

Note

By default, the log file name `optv.log` will be used.

OPTVEnv.**OPTVEnv**(*logFileName*)

Parameters
logFileName (*str*) – Path to log file.

Returns
Environment object with default settings and log file specified.

Return type
OPTVEnv

OPTVEnv.**OPTVEnv**(*xenv*)

Parameters
xenv (OPTVEnv) – OptVerse environment to be copied.

Returns
Copy of OPTVEnv object `xenv`.

Return type
OPTVEnv

__getattr__()

OPTVEnv.**__getattr__**(*param*)

Obtain a solver parameter value using attribute-style access.

This allows accessing parameters using dot notation, e.g., `env.TIME_LIMIT` instead of `env.Get(OPTVDblParam.TIME_LIMIT)`.

Parameters
param – Parameter name as an attribute (e.g., `TIME_LIMIT`, `THREADS`).

Returns
Parameter value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 1234.0)
print(env.TIME_LIMIT)  # 1234.0
```

__setattr__()

OPTVEnv.**__setattr__**(*param*, *value*)

Set a solver parameter using attribute-style access.

This allows setting parameters using dot notation, e.g., `env.TIME_LIMIT = 7200.0` instead of `env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)`.

Parameters

- **param** – Parameter name as an attribute (e.g., TIME_LIMIT, THREADS).
- **value** – Value to be set.

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
env = OPTVEnv()

env.TIME_LIMIT = 1234.0
env.THREADS = 2
print(env.TIME_LIMIT)  # 1234.0
print(env.THREADS)     # 2
```

Get()

Obtain a solver parameter value.

See *OptVerse MP Parameters*.

OPTVEnv.**Get**(*param*)

Parameters
param (OPTVIntParam/OPTVDblParam/OPTVStrParam) – Parameter to query.

Returns
Parameter value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)
print(env.Get(OPTVDblParam.TIME_LIMIT)) # 7200.0
```

Set()

Set a solver parameter for the environment. These settings will applied as default to any models subsequently constructed. After model construction however, this does not impact the parameters in the model. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Parameters*.

OPTVEnv.**Set**(*param*, *value*)

Parameters

- **param** (OPTVIntParam / OPTVDblParam / OPTVStrParam) – Parameter to set.
- **value** (*int* / *float* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)
env.Set(OPTVIntParam.THREADS, 2)
env.Set(OPTVStrParam.LOG_FILE, 'optv_new.log')

print(env.Get(OPTVDblParam.TIME_LIMIT)) # 7200.0
print(env.Get(OPTVIntParam.THREADS))    # 2
print(env.Get(OPTVStrParam.LOG_FILE))   # optv_new.log
```

ResetParams()

Reset all parameters to default values.

OPTVEnv.**ResetParams**()

Returns
None.

Return type
None

Example

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)
env.Set(OPTVIntParam.THREADS, 1)
env.Set(OPTVStrParam.LOG_FILE, 'optv_new.log')

# Reset all configured parameters
env.ResetParams()

print(env.Get(OPTVDblParam.TIME_LIMIT)) # 1e+20
print(env.Get(OPTVIntParam.THREADS))    # 1
print(env.Get(OPTVStrParam.LOG_FILE))   # ""
```

8.2 OptVerse MP Solver API References

8.2.1 C++ API Reference Manual

OPTVVar

class **OPTVVar**

OptVerse variable object. Used to access variable data from a model. Variables should not be created through their constructor. Instead, a user should add a variable to a model using the *OPTVModel::AddVar()* or *OPTVModel::AddVars()* interfaces.

Get()

Query the value of a variable attribute.

See *OptVerse MP Attributes*.

OPTVVar::**Get**(*OPTVIntAttr* attr)

Parameters
attr – Integer type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

OPTVVar::**Get**(*OPTVCharAttr* attr)

Parameters
attr – Character type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

OPTVVar::**Get**(*OPTVDblAttr* attr)

Parameters
attr – Double type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

OPTVVar::**Get**(*OPTVStrAttr* attr)

Parameters
attr – String type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

Set()

Set the value of a variable attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVVar::**Set**(*OPTVIntAttr* attr, int value)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

OPTVVar::**Set**(*OPTVCharAttr* attr, char value)

Parameters

- **attr** – Character type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

OPTVVar::**Set**(*OPTVDblAttr* attr, double value)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

OPTVVar::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for variables is provided.

Index()

Returns the index of the variable in a model.

OPTVVar::**Index**() const

Returns
Index of variable.

Return values

- **-1** – Variable is not in model.
- **-2** – Variable has been removed from a model.
- **>=0** – Variable index in model.

SameAs()

Determines if two variables are actually the same underlying object.

bool *OPTVVar*::**SameAs**(const *OPTVVar* &v2) const

Parameters
v2 – Variable to check sameness against.

Returns
Variable sameness.

Return values

- **true** – Variables are the same.
- **false** – Variables are not the same.

OPTVColumn

class **OPTVColumn**

OptVerse Column object. Used to add variables to the model column-wise. This enables users to add a new variable while also adding it to existing constraint objects.

Consider the following example, illustrating use of the column class. Here we will partially construct a model and then append another variable. This example recreates the *milp1* example presented in the *Quick Start* section.

```
#include "optv_c++.h"

OPTVEnv env;
OPTVModel model(env);

OPTVVar x = model.AddVar(0, 1, -1, OPTV_BINARY, "x");
OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");

OPTVConstr c0 = model.AddConstr(2 * x + y, -OPTV_INF, 3, "c0");
OPTVConstr c1 = model.AddConstr(3 * y, -OPTV_INF, 6, "c1");
```

At this stage, we have constructed the following model:

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

```
OPTVColumn col;

col.AddTerm(1, c0);
col.AddTerm(1, c1);

OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, col, "z");
```

By adding `col` to the model, we have now constructed the following matrix:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

AddTerm()

Add term to the end of the column.

OPTVColumn::**AddTerm**(double coef, const *OPTVConstr* &constr)

This function will add a single term into the column.

Parameters

- **coef** – Coefficient.
- **constr** – Constraint.

Returns

None.

AddTerms()

Add terms to the end of the column.

OPTVColumn::**AddTerms**(const double *coefs, const *OPTVConstr* *constrs, int cnt)

This function will add the terms to the column.

Parameters

- **coefs** – C-style array of coefficient values.
- **constrs** – C-style array of constraints.
- **cnt** – Number of elements in the C-array.

Returns
None.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – `constrs` or `coefs` is nullptr or `cnt` is negative.

Clear()

Clears the inner containers, such that *OPTVColumn::Size()* will return `0`.

OPTVColumn::**Clear**()

Returns
None.

GetCoeff()

Get the coefficient for a constraint at a given index in the column.

OPTVColumn::**GetCoef**(int i) const

Parameters
`i` – Index of the constraint in the column.

Returns
Corresponding coefficient in the constraint at a given index, `i`, in the column.

Raises OPTVErrCode::INDEX_OUT_OF_RANGE
If $i \notin [0, \texttt{this->Size()}]$.

GetConstr()

Get constraint at a specified index in the column.

OPTVColumn::**GetConstr**(int i) const

Parameters
`i` – Index of the constraint in the column.

Returns
Constraint at position `i`.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If `i` $\notin [0,$ `this->Size()`$]$.

Remove()

Remove term(s) from the column.

OPTVColumn::**Remove**(int i)

Remove the term at a specified index from the column.

Parameters
i – Index of term to be removed.

Returns
None.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If `i` $\notin [0,$ `this->Size()`$]$.

OPTVColumn::**Remove**(const *OPTVConstr* &constr)

Remove any instance of a constraint from the column.

Parameters
constr – Constraint to be removed.

Returns
None.

Throws
OPTVErrCode::*NOT_IN_MODEL* – If constraint, `constr`, is not in a model.

Size()

Get the size of the column.

OPTVColumn::**Size**() const

This value corresponds to the number of elements in the column, regardless of whether the element is repeated. For example, if the same constraint is added multiple times, the size will keep growing.

Returns
Size of the column.

OPTVLinExpr

class **OPTVLinExpr**

The *OPTVLinExpr* class is used to construct linear constraints and objective functions. An object of this type is constructed from variables and constant terms. The expressions are accessed as constructed. That is, an expression `1 * x + 2 * y + 3 * x` will be accessed as:

Index	Coef	Variable
0	1	x
1	2	y
2	3	x

AddTerms()

Add terms to the end of the linear expression.

OPTVLinExpr::**AddTerms**(const double *coefs, const *OPTVVar* *vars, int cnt)

This function will add the terms $\text{coefs}_i * \text{vars}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coefs** – C-style array of coefficient values.
- **vars** – C-style array of variables.
- **cnt** – Number of elements in the C-array.

Returns
None.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – `vars` or `coefs` is nullptr or `cnt` is negative.

Clear()

Clears the inner "terms" container, such that *OPTVLinExpr::Size()* will return `0`. Additionally, resets the expression such that it is numerically equivalent to `0`.

OPTVLinExpr::**Clear**()

Returns
None.

GetCoef()

Get the coefficient for a variable at a given index in the expression.

OPTVLinExpr::**GetCoef**(int i) const

Parameters
`i` – Index of the variable in the expression.

Returns
Coefficient of variable at position `i`.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If $\texttt{i} \notin [0, \texttt{this->Size()}]$.

GetVar()

Get variable at a specified index in the expression.

OPTVLinExpr::**GetVar**(int i) const

Parameters
i – Index of the variable in the expression.

Returns
Variable at position i.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If i $\notin [0,$ this->Size()$]$.

GetConstant()

Get the constant term from the expression.

OPTVLinExpr::**GetConstant**() const

Returns
Constant value in expression.

GetValue()

Get the evaluated value of the linear expression.

OPTVLinExpr::**GetValue**() const

Returns
Expression value according to the primal solution.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Value queried for solution is unavailable.

Remove()

Remove term(s) from the expression.

OPTVLinExpr::**Remove**(int i)

Remove the term at a specified index from the expression.

Parameters
i – Index of term to be removed.

Returns
None.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If i $\notin [0,$ this->Size()$]$.

OPTVLinExpr::**Remove**(const *OPTVVar* &v)

Remove any instance of a variable from the expression.

Parameters
v – Variable to be removed.

Returns
Whether at least one term removed.

Return values

- **true** – One or more terms was removed.
- **false** – No terms were removed.

Throws
OPTVErrCode::*NOT_IN_MODEL* – If variable is not in a model.

SameAs()

Determines if two linear expressions are actually the same underlying expression.

OPTVLinExpr::**SameAs**(const *OPTVLinExpr* &e2) const

Parameters
e2 – Linear expression to check sameness against.

Returns
Expression sameness.

Return values

- **true** – Expressions are the same.
- **false** – Expressions are not the same.

Size()

Returns the size of the expression.

OPTVLinExpr::**Size**() const

This value corresponds to the number of elements in the expression, regardless of whether the element is repeated. For example, the expression `x+y` has `Size = 2`. The expression `x+y+x` has `Size = 3`.

Returns
Size of the linear expression.

OPTVConstr

class **OPTVConstr**

OptVerse constraint object. Used to access constraint data from a model. Constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVModel::AddConstr()* or *OPTVModel::AddConstrs()* interfaces for linear constraints. For quadratic constraints, the *OPTVModel::AddQConstr()* interface is provided.

Get()

Query the value of a constraint attribute.

See *OptVerse MP Attributes*.

OPTVConstr::**Get**(*OPTVIntAttr* attr)

Parameters
attr – Integer type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVConstr::**Get**(*OPTVDblAttr* attr)

Parameters
attr – Double type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVConstr::**Get**(*OPTVStrAttr* attr)

Parameters
attr – String type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Set()

Set the value of a constraint attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVConstr::**Set**(*OPTVIntAttr* attr, int value)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVConstr::**Set**(*OPTVDblAttr* attr, double value)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVConstr::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Index()

Return the index of the constraint in its model.

OPTVConstr::**Index**() const

Returns
Index of constraint.

Return values

- **-1** – Constraint is not in model.
- **-2** – Constraint has been removed from a model.
- **>=0** – Constraint index in model.

SameAs()

Determines if two constraints are actually the same underlying object.

OPTVConstr::**SameAs**(const *OPTVConstr* &c2) const

Parameters
c2 – Constraint to check sameness against.

Returns
Constraint sameness.

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

OPTVQuadExpr

class **OPTVQuadExpr**

The *OPTVQuadExpr* class is used to construct quadratic constraints and objective functions. An object of this type is constructed from variables and constant terms. Quadratic expressions are composed of three types of terms: quadratic, linear, and constant. The quadratic terms are stored in containers that hold the first variable, second variable, and coefficient. The linear component is accessed through the internal linear expression object. Indexing in a quadratic expression refers strictly to the quadratic components, with the linear expression being treated independently.

For example, the quadratic expression `1*x*x + 2*x*y + 3*x*y + 4*y*y + 5*x + 6*y + 7*x` has the following description:

Index	Coef	Var1	Var2
0	1	x	x
1	2	x	y
2	3	x	y
3	4	y	y

To access the linear components of the same expression, use the *OPTVQuadExpr::GetLinExpr()* interface. This will return a *OPTVLinExpr* object that represents the linear component of the expression. This linear expression can be accessed as normal, for example:

Index	Coef	Variable
0	5	x
1	6	y
2	7	x

AddConstant()

Add to the constant term.

OPTVQuadExpr`::AddConstant(double c)`

This function will add to the constant term, not override.

Parameters

c – Constant value to add to expression.

Returns

None.

AddTerm()

Add a single linear term to the end of the quadratic expression.

OPTVQuadExpr::**AddTerm**(double coeff, const *OPTVVar* &var)

This function will add the terms coeff $*$ var to the end of the expression.

Parameters

- **coeff** – Coefficient values.
- **var** – Variable to add to expression.

Returns

None.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `var` is nullptr.

Add a single quadratic term to the end of the quadratic expression.

OPTVQuadExpr::**AddTerm**(double coeff, const *OPTVVar* &var1, const *OPTVVar* &var2)

This function will add the terms coeff $*$ var1 $*$ var2 to the end of the expression.

Parameters

- **coeff** – Coefficient values.
- **var1** – First variable in term.
- **var2** – Second variable in term.

Returns

None.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `var1` or `var2` is nullptr.

AddTerms()

Add linear terms to the end of the quadratic expression.

OPTVQuadExpr::**AddTerms**(const double *coeff, const *OPTVVar* *var, int cnt)

This function will add the terms $\text{coeff}_i * \text{var}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coeff** – C-style array of coefficient values.
- **var** – C-style array of variables.
- **cnt** – Number of elements in the C-array.

Returns

None.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `var` or `coeff` is nullptr or `cnt` is negative.

Add quadratic terms to the end of the quadratic expression.

OPTVQuadExpr::**AddTerms**(const double *coeff, const *OPTVVar* *var1, const *OPTVVar* *var2, int cnt)

This function will add the terms $\text{coeff}_i * \text{var1}_i * \text{var2}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coeff** – C-style array of coefficient values.
- **var1** – C-style array of first variables.
- **var2** – C-style array of second variables.
- **cnt** – Number of elements in the C-array.

Returns
None.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – `var1`, `var2`, or `coeff` is nullptr or `cnt` is negative.

Clear()

Clears the quadratic expression.

OPTVQuadExpr::**Clear**()

Clears the inner containers, such that *OPTVQuadExpr::Size()* will return `0`. Additionally, this function clears the linear component of the expression, and resets the constant to `0`.

Returns
None.

GetCoeff()

Get the coefficient for a term at a given index in the quadratic component of the expression.

OPTVQuadExpr::**GetCoeff**(int i) const

Parameters
`i` – Index of the term in the expression.

Returns
Coefficient of term at position `i`.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – If `i` $\notin [0,$ `this->Size()`$]$.

GetConstant()

Get the constant term from the expression.

OPTVQuadExpr::**GetConstant**() const

Returns
Constant value in expression.

GetLinExpr()

Get the linear components of the expression.

OPTVQuadExpr::**GetLinExpr**() const

Returns
Linear component of expression.

GetValue()

Get the evaluated value of the quadratic expression.

OPTVQuadExpr::**GetValue**() const

Returns
Expression value according to the primal solution.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Value queried for solution is unavailable.

GetVar1()

Get the first variable of a quadratic term at a specified index in the expression.

OPTVQuadExpr::**GetVar1**(int i) const

This function will return the first variable in a quadratic term for an expression. For example, an expression `x*x + x*y + y*y` has the following prescription:

Index	Var1	Var2
0	x	x
1	x	y
2	y	y

Parameters
i – Index of the quadratic term in the expression.

Returns
First variable at position `i`.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – If `i` $\notin [0,$ `this->Size()`$]$.

GetVar2()

Get the second variable of a quadratic term at a specified index in the expression. Refer to *OPTVQuadExpr::GetVar1()* for a description of indexing.

OPTVQuadExpr::**GetVar2**(int i) const

This function will return the second variable in a quadratic term for an expression.

Parameters
i – Index of the quadratic term in the expression.

Returns
Second variable at position `i`.

Throws
OPTVErrCode`::`*DATA_NOT_AVAILABLE* – If `i` $\notin [0,$ `this->Size()`$]$.

Remove()

Remove term(s) from the quadratic terms.

OPTVQuadExpr`::`**Remove**(int i)

Remove the term at a specified index from the expression. This interface does not allow removing terms from the linear component.

Parameters
i – Index of term to be removed.

Returns
None.

Throws
OPTVErrCode`::`*DATA_NOT_AVAILABLE* – If `i` $\notin [0,$ `this->Size()`$]$.

Remove variable from the expression.

OPTVQuadExpr`::`**Remove**(const *OPTVVar* &v)

Remove any instance of a variable from the expression. This interface will remove any occurrence of the variable from the linear component of the expression.

Parameters
v – Variable to be removed.

Returns
Whether at least one term removed.

Return values

- **true** – One or more terms was removed.
- **false** – No terms were removed.

Throws
OPTVErrCode`::`*NOT_IN_MODEL* – If variable is not in a model.

SameAs()

Determines if two quadratic expressions are actually the same underlying expression.

OPTVQuadExpr`::`**SameAs**(const *OPTVQuadExpr* &e2) const

Parameters
e2 – Quadratic expression to check sameness against.

Returns
Quadratic expression sameness.

Return values

- **true** – Quadratic expressions are the same.

- **false** – Quadratic expressions are not the same.

Size()

Returns the size of the expression.

OPTVQuadExpr::**Size**() const

This value corresponds to the number of **quadratic** elements in the expression, regardless of whether the element is repeated. The expression `x*x + y*y + y` has `Size = 2`. The expression `x*x + x*x + y*y + y` has `Size = 3`.

Returns
Size of the quadratic expression.

OPTVQConstr

class **OPTVQConstr**

OptVerse quadratic constraint object. Used to access quadratic constraint data from a model. Quadratic constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVModel::AddQConstr()*.

Warning

Quadratic constraints must satisfy the condition that their matrix representation are positive semi-definite or OptVerse will throw a *OPTVErrCode::MATRIX_NOT_PSD* exception.

Get()

Query the value of a quadratic constraint attribute.

See *OptVerse MP Attributes*.

OPTVQConstr::**Get**(*OPTVIntAttr* attr) const

Parameters
attr – Integer type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVQConstr::**Get**(*OPTVCharAttr* attr) const

Parameters
attr – Char type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVQConstr::**Get**(*OPTVDblAttr* attr) const

Parameters
attr – Double type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVQConstr::**Get**(*OPTVStrAttr* attr) const

Parameters
attr – String type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Set()

Set the value of a quadratic constraint attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVQConstr::**Set**(*OPTVIntAttr* attr, int value)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVQConstr::**Set**(*OPTVCharAttr* attr, char value)

Parameters

- **attr** – Char type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVQConstr::**Set**(*OPTVDblAttr* attr, double value)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVQConstr::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Index()

Return the index of the quadratic constraint in its model.

OPTVQConstr::**Index**() const

Returns
Index of quadratic constraint.

Return values

- **-1** – Quadratic constraint is not in model.
- **-2** – Quadratic constraint has been removed from a model.
- **>=0** – Quadratic constraint index in model.

SameAs()

Determines if two quadratic constraints are actually the same underlying object.

OPTVQConstr::**SameAs**(const *OPTVQConstr* &c2) const

Parameters
c2 – The quadratic constraint to check sameness against.

Returns
The quadratic constraint sameness.

Return values

- **true** – The quadratic constraints are the same.
- **false** – The quadratic constraints are not the same.

OPTVAffineExpr

class **OPTVAffineExpr**

The *OPTVAffineExpr* class serves as the foundation for constructing conic constraints by encapsulating the affine transformation $Fx + g$. An *OPTVAffineExpr* object contains:

- **Linear coefficients**: The sparse representation of matrix F, storing non-zero coefficients and their corresponding variable indices in Coordinate (COO) sparse format. Affine expressions are constructed by adding rows of linear expressions (the COO storage is internal, users will provide data through SDK objects).
- **Model variables**: The sparse representation of the vector x. Users only need to provide the participating variables, through adding *linear expression* objects as rows.
- **Constant terms**: The offset vector g that shifts the linear combination.

When creating an *OPTVCone*, the *OPTVAffineExpr* provides the mathematical structure (the affine transformation), while the cone type parameter specifies the geometric constraint C that the expression must satisfy. This separation allows for flexible composition of conic constraints from simpler affine building blocks.

See *OPTVCone*.

AddRow()

Add a single row to the affine expression.

OPTVAffineExpr::**AddRow**(const *OPTVLinExpr* &expr)

Parameters
expr – Linear expression to add to the affine expression.

Returns
None.

AddRows()

Add multiple rows to the affine expression.

OPTVAffineExpr::**AddRows**(const *OPTVLinExpr* *expr, int count)

Parameters

- **expr** – Array of linear expressions to add to the affine expression. Each expression corresponds to a row in the affine expression.
- **count** – Number of rows to be added (length of expr array).

Returns
None.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – `count` is negative.

GetColumnIndices()

Returns the column indices of the nonzero entries in the affine expression. Each value corresponds to the column index for a nonzero entry. The affine expression is internally stored in COO (Coordinate) format, where each nonzero entry is identified by its row index, column index, and value.

OPTVAffineExpr::**GetColumnIndices**() const

Returns
A vector of integers, where each element corresponds to the column index of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr::GetRowIndices()* and *OPTVAffineExpr::GetValues()*.

GetOffset()

OPTVAffineExpr::**GetOffset**(int index) const

Parameters
index – Index of the row whose offset is to be queried.

Returns
A double corresponding to the offset for the row at `index`.

Throws
`std::out_of_range` – If `index` is larger than the number of rows in the affine expression.

GetOffsets()

OPTVAffineExpr::**GetOffsets**() const

Returns
A vector of doubles, where each element is the offset for a specific row added to the affine expression.

GetRowIndices()

Returns the row indices of the nonzero entries in the affine expression. Each value corresponds to the row index for a nonzero entry. The affine expression is internally stored in COO (Coordinate) format, where each nonzero entry is identified by its row index, column index, and value.

OPTVAffineExpr::**GetRowIndices**() const

Returns
A vector of integers, where each element corresponds to the row index of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr::GetColumnIndices()* and *OPTVAffineExpr::GetValues()*.

GetValues()

Returns the coefficient values of the nonzero entries in the affine expression. Each value corresponds to the coefficient stored at the position defined by the matching row and column index from *OPTVAffineExpr::GetRowIndices()* and *OPTVAffineExpr::GetColumnIndices()*.

OPTVAffineExpr::**GetValues**() const

Returns
A vector of doubles, where each element is the coefficient of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr::GetRowIndices()* and *OPTVAffineExpr::GetColumnIndices()*.

SetOffset()

Set the offset for a specific row, based on the index.

OPTVAffineExpr::**SetOffset**(int index, double value)

Parameters

- **index** – Index of the row whose offset is to be set.
- **value** – Value for the offset to set.

Throws
`std::out_of_range` – If `index` is larger than the number of rows in the affine expression.

Size()

Get the size of the affine expression.

OPTVAffineExpr::**Size**() const

This value corresponds to the number of elements in the affine expression, regardless of whether the element is repeated. For example, if the same linear expression is added multiple times, the size will keep growing.

Returns
Size of the affine expression.

OPTVCone

class **OPTVCone**

OptVerse cone object. Used to access conic constraint data from a model. Cones should not be created directly through their constructor. Instead, a user should add a cone constraint to a model using the *OPTVModel::AddCone()* interface.

A conic constraint is mathematically defined as:

$$Fx + g \in C$$

Where:

- F is an $m \times n$ coefficient matrix that linearly transforms the optimization variables,
- x is the n-dimensional vector of optimization variables,
- g is an m-dimensional constant offset vector (right-hand side),

- C is the cone that constrains the resulting affine expression.
- The expression $Fx + g$ produces an m-dimensional vector that must belong to cone C.

The matrix F is stored in Coordinate (COO) sparse format for memory efficiency, containing only non-zero entries as (row, column, value) triplets. Users will construct $Fx + g$ as an *affine expression*, by adding rows through *linear expressions*.

OptVerse supports the following cone types:

- *Second-order cone*
- *Rotated second-order cone*

See *OPTVAffineExpr* and *OPTVConeType*.

Get()

Query the value of a cone constraint attribute.

OPTVCone::**Get**(*OPTVStrAttr* attr)

Parameters
attr – String type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

GetType()

Get the type of the cone constraint.

See *OPTVConeType*.

OPTVCone::**GetType**() const

Returns
Type of the cone constraint.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – Cone constraint has been removed from a model.

Index()

Return the index of the cone constraint in its model.

OPTVCone::**Index**() const

Returns
Index of the cone constraint in the model.

Return values

- **-1** – Cone constraint is not part of any model.

- **-2** – Cone constraint has been removed from a model.
- **>=0** – Cone constraint index in model.

SameAs()

Determines if two cone constraints are actually the same underlying object.

OPTVCone::**SameAs**(const *OPTVCone* &c2) const

Parameters
c2 – Cone constraint to check sameness against.

Returns
Cone constraint sameness.

Return values

- **true** – Cone constraints are the same
- **false** – Cone constraints are not the same

Set()

Set the value of a cone constraint attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVCone::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

OPTVSOS

class **OPTVSOS**

OptVerse SOS constraint object. Used to access SOS constraint data from a model. SOS constraints should not be created through their constructor. Instead, a user should add an SOS constraint to a model using the *OPTVModel::AddSOS()*.

A special ordered set (SOS) is an additional way to specify integrality conditions in a model. In particular, a special ordered set is a way to restrict the number of nonzero solution values among a specified set of variables in a model.

There are various types of SOS:

- SOS Type 1 is a set of variables where at most one variable may be nonzero.
- SOS Type 2 is a set of variables where at most two variables may be nonzero. If two variables are nonzero, they must be adjacent in the set.

Get()

Query the value of an SOS constraint attribute.

See *OptVerse MP Attributes*.

OPTVSOS::**Get**(*OPTVIntAttr* attr)

Parameters
attr – Integer type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVSOS::**Get**(*OPTVStrAttr* attr)

Parameters
attr – String type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Set()

Set the value of an SOS constraint attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVSOS::**Set**(*OPTVIntAttr* attr, int value)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVSOS::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Index()

Return the index of the SOS constraint in its model.

OPTVSOS::**Index**() const

Returns
Index of SOS constraint.

Return values

- **-1** – SOS constraint is not in model.
- **-2** – SOS constraint has been removed from a model.
- **>=0** – SOS constraint index in model.

SameAs()

Determines if two SOS constraints are actually the same underlying object.

OPTVSOS::**SameAs**(const *OPTVSOS* &c2) const

Parameters
c2 – SOS constraint to check sameness against.

Returns
SOS constraint sameness.

Return values

- **true** – SOS constraints are the same.
- **false** – SOS constraints are not the same.

OPTVGenConstr

class **OPTVGenConstr**

OptVerse general constraint object. Used to access general constraint data from a model. General constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the `OPTVModel::AddGenConstrXxx`.

Table 1: Supported General Constraints

Constraint	Type	Interface	Description
And	*OPTV_GENCONSTR_A*	*OPTVModel::AddGenConst*	Resultant variable is non-zero if all supplied variables are non-zero.
Or	*OPTV_GENCONSTR_O*	*OPTVModel::AddGenConst*	Resultant variable is non-zero if any of the supplied variables are non-zero.
Indicator	*OPTV_GENCONSTR_I*	*OPTVModel::AddGenConst*	Binary variable is non-zero if and only if the associated logical condition is satisfied.

Get()

Query the value of a general constraint attribute.

See *OptVerse MP Attributes*.

OPTVGenConstr::**Get**(*OPTVIntAttr* attr)

Parameters
attr – Integer type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVGenConstr::**Get**(*OPTVDblAttr* attr)

Parameters
attr – Double type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVGenConstr::**Get**(*OPTVStrAttr* attr)

Parameters
attr – String type attribute.

Returns
Attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Set()

Set the value of a general constraint attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVGenConstr::**Set**(*OPTVDblAttr* attr, double value)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

OPTVGenConstr::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute for constraints is provided.

Index()

Return the index of the general constraint in its model.

OPTVGenConstr::**Index**() const

Returns
Index of constraint.

Return values

- **-1** – Constraint is not in model.
- **-2** – Constraint has been removed from a model.
- **>=0** – Constraint index in model.

SameAs()

Determines if two general constraints are actually the same underlying object.

OPTVGenConstr::**SameAs**(const *OPTVGenConstr* &c2) const

Parameters
c2 – Constraint to check sameness against.

Returns
Constraint sameness.

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

OPTVModel

class **OPTVModel**

OptVerse optimization model object.

The `OPTVModel` class is the central object in OptVerse for building and solving optimization models. It stores all variables, constraints, objective function, and solver settings.

A model has a **lazy update** behaviour: changes to the model (adding variables, constraints, modifying attributes, etc.) are not immediately reflected in the underlying solver. Call *OPTVModel::Update()* to synchronize the model state with the solver before optimizing.

Typical Model Lifecycle:

1. Create an *OPTVModel* using one of the constructors
2. Add variables using *OPTVModel::AddVar()* or *OPTVModel::AddVars()*
3. Add constraints using *OPTVModel::AddConstr()*, *OPTVModel::AddQConstr()*, or specialized constraint methods
4. Set the objective using *OPTVModel::SetObjective()* (or via *OPTVModel::AddVar()* / *OPTVModel::AddVars()* arguments)
5. (Optional) Configure solver parameters directly on the model
6. Call *OPTVModel::Update()* to synchronize
7. Call *OPTVModel::Optimize()* to solve

See *Quick Start* for a typical model construction example.

OPTVModel()

explicit *OPTVModel*::**OPTVModel**(const *OPTVEnv* *env)

Create an empty model associated with the given environment.

Parameters
env – Pointer to the OptVerse environment.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – env is nullptr.

---

explicit *OPTVModel*::**OPTVModel**(const *OPTVEnv* &env)

Create an empty model associated with the given environment.

Parameters
env – Reference to the OptVerse environment.

OPTVModel::**OPTVModel**(const *OPTVEnv* *env, const std::string &fileName)

Create a model by reading from a file.

Parameters

- **env** – Pointer to the OptVerse environment.

- **fileName** – Path to the model file (MPS, LP, etc.).

Throws

- *OPTVErrCode*::*INVALID_ARGUMENT* – env is nullptr.
- *OPTVException* – Failed to read the model from file.

OPTVModel::**OPTVModel**(const *OPTVEnv* &env, const std::string &fileName)

Create a model by reading from a file.

Parameters

- **env** – Reference to the OptVerse environment.
- **fileName** – Path to the model file (MPS, LP, etc.).

Throws

OPTVException – Failed to read the model from file.

OPTVModel::**OPTVModel**(const *OPTVEnv* *env, const std::string &content, const std::string &format)

Create a model by reading from a string.

Parameters

- **env** – Pointer to the OptVerse environment.
- **content** – Model content as a string.
- **format** – Model format (e.g., *"mps"*, *"lp"*).

Throws

- *OPTVErrCode*::*INVALID_ARGUMENT* – env is nullptr.
- *OPTVException* – Failed to read the model from string.

OPTVModel::**OPTVModel**(const *OPTVEnv* &env, const std::string &content, const std::string &format)

Create a model by reading from a string.

Parameters

- **env** – Reference to the OptVerse environment.
- **content** – Model content as a string.
- **format** – Model format (e.g., *"mps"*, *"lp"*).

Throws

OPTVException – Failed to read the model from string.

AddCone()

Add a single cone constraint to the model.

OPTVModel::**AddCone**(const *OPTVAffineExpr* &affine, *OPTVConeType* type, const std::string &name = "")

This function will add a single cone constraint to the model.

Parameters

- **affine** – The affine expression, $Fx + g$, used to specify the cone constraint.
- **type** – Type of cone constraint (e.g. second-order cone, rotated second-order cone).
- **name** – Cone constraint name.

Returns

Cone constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `type` is not one of *OPTVConeType::SOC* or *OPTVConeType::RSOC*. The provided affine expression is empty. If `type` is *OPTVConeType::RSOC* and the affine expression size is less than *3*.

AddConstr()

Add a single linear constraint to the model.

OPTVModel::**AddConstr**(const *OPTVLinExpr* &expr, double lhs, double rhs, const std::string &name = "")

This function will add a single constraint to the model. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel::AddConstrs()*) is preferred.

Parameters

- **expr** – Linear expression describing the constraint.
- **lhs** – Lower bound of constraint.
- **rhs** – Upper bound of constraint.
- **name** – Constraint name.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

AddConstrs()

Add multiple linear constraints to the model.

OPTVModel::**AddConstrs**(const *OPTVLinExpr* *expr, const double *lhs, const double *rhs, const std::string *names, int count)

This function will add multiple linear constraints to the model. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **expr** – Linear expressions describing the constraints.

- **lhs** – Constraint lower bounds.
- **rhs** – Constraint upper bounds.
- **names** – Constraint names.
- **count** – Number of constraints to be added.

Returns
Vector of constraint objects added to the model.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – `names` is empty, numeric value is `NaN`, or a constraint with a name in `names` already exists in the model.

AddQConstr()

Add a single quadratic constraint to the model.

OPTVModel::**AddQConstr**(const *OPTVQuadExpr* &expr, double lhs, double rhs, const std::string &name = "")

This function will add a single quadratic constraint to the model.

Parameters

- **expr** – Quadratic expression describing the constraint.
- **lhs** – Lower bound of constraint.
- **rhs** – Upper bound of constraint.
- **name** – Constraint name.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_ARGUMENT* – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

AddGenConstrAnd()

Add a single `AND` constraint to the model.

OPTVModel::**AddGenConstrAnd**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "")

This function will add a single `AND` constraint to the model.

Parameters

- **resultant** – The resultant binary variable of constraint.
- **vars** – C-array of variables that must be non-zero for resultant to be non-zero.
- **len** – Length of `vars` array.
- **name** – Constraint name.

Returns
General constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – name is empty, numeric value is NaN, or constraint with name already exists in the model.

AddGenConstrOr()

Add a single OR constraint to the model.

OPTVModel::**AddGenConstrOr**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "")

This function will add a single OR constraint to the model.

Parameters

- **resultant** – The resultant binary variable of constraint.
- **vars** – C-array of variables that must include at least one non-zero variable for resultant to be non-zero.
- **len** – Length of vars array.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – name is empty, numeric value is NaN, or constraint with name already exists in the model.

AddGenConstrIndicator()

Add a single INDICATOR constraint to the model.

OPTVModel::**AddGenConstrIndicator**(const *OPTVVar* &binVar, int binVal, const *OPTVLinExpr* &expr, char sense, double rhs, const std::string &name = "")

This function will add a single INDICATOR constraint to the model.

Parameters

- **binVar** – The binary indicator variable.
- **binVal** – The value of the indicator variable that would force the constraint to be satisfied (0 or 1).
- **expr** – Expression for the linear constraint triggered by the indicator.
- **sense** – Sense for the linear constraint. One of *OPTV_SENSE_LESS*, *OPTV_SENSE_EQUAL*, or *OPTV_SENSE_GREATER*.
- **rhs** – Right-hand side of the linear constraint.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – name is empty, numeric value is NaN, or constraint with name already exists in the model.

AddGenConstrAbs()

Add a single ABS constraint to the model.

An ABS constraint of the form $r = \text{abs}(x)$ requires that the variable r is equal to the absolute value of the variable x. This constraint enforces the relationship $r = |x|$, where $r \geq 0$.

OPTVModel::**AddGenConstrAbs**(const *OPTVVar* &resultant, const *OPTVVar* &inputVar, const std::string &name = "")

This function will add a single ABS constraint to the model.

Parameters

- **resultant** – The variable representing the absolute value of the input variable.
- **inputVar** – The variable for which the absolute value will be taken.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – name is empty, numeric value is NaN, or constraint with name already exists in the model.

AddGenConstrNorm()

Add a single NORM constraint to the model.

A NORM constraint of the form $r = \text{norm}\{x_1, \ldots, x_n\}$ requires that the variable r is equal to the vector norm of the argument vector $x_1, \ldots, x_n$.

OPTVModel::**AddGenConstrNorm**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, int normType, const std::string &name = "")

This function will add a single NORM constraint to the model.

Parameters

- **resultant** – The variable representing the vector norm of the other variables.
- **vars** – Variables on which the vector norm is computed.
- **len** – Length of vars array.
- **normType** – Select the norm type to use. Valid options are: -1 for the L-infinity norm (also denoted as L^∞ norm), 0 for the L^0 norm, 1 for the L^1 norm.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – name is empty, numeric value is NaN, or constraint with name already exists in the model.

AddGenConstrMax()

Add a single `MAX` constraint to the model.

A MAX constraint of the form $r = \max\{x_1, \ldots, x_n\}$ ensures that the variable r is equal to the maximum value among the argument vector $x_1, \ldots, x_n$.

OPTVModel`::`**`AddGenConstrMax`**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "", double constant = -OPTV_INF)

This function will add a single `MAX` constraint to the model.

Parameters

- **`resultant`** – The resultant variable whose value will be equal to the maximum of the other variables.
- **`vars`** – A C-style array of variables, from which the maximum value is taken.
- **`len`** – Length of `vars` array.
- **`name`** – Constraint name.
- **`constant`** – [**optional**] A constant value to be included in the list of arguments for the MAX operation.

Returns

General constraint object added to the model.

Throws

OPTVErrCode`::`*INVALID_ARGUMENT* – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

AddGenConstrMin()

Add a single `MIN` constraint to the model.

A MIN constraint of the form $r = \min\{x_1, \ldots, x_n\}$ ensures that the variable r is equal to the minimum value among the argument vector $x_1, \ldots, x_n$.

OPTVModel`::`**`AddGenConstrMin`**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "", double constant = OPTV_INF)

This function will add a single `MIN` constraint to the model.

Parameters

- **`resultant`** – The resultant variable whose value will be equal to the minimum of the other variables.
- **`vars`** – A C-style array of variables, from which the minimum value is taken.
- **`len`** – Length of `vars` array.
- **`name`** – Constraint name.
- **`constant`** – [**optional**] A constant value to be included in the list of arguments for the MIN operation.

Returns

General constraint object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

AddSOS()

Add an SOS constraint to the model.

OPTVModel::**AddSOS**(const *OPTVVar* *vars, const double *weights, int len, int type, const std::string &name = "")

Parameters

- **vars** – C-array of variables that participate in SOS.
- **weights** – C-array of weights of variables in SOS.
- **len** – Length of arrays `vars` and `weights`.
- **type** – SOS type. One of `1` or `2`.
- **name** – Name of the SOS constraint.

Returns

SOS constraint.

Throws

- *OPTVErrCode*::*INVALID_ARGUMENT* – `type` is not `1` or `2`.
- *OPTVErrCode*::*NOT_IN_MODEL* – A variable in the `vars` array is not part of the model.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – The model has an invalid index. This may happen if trying access a model after its environment has gone out of scope.

AddVar()

Add a single variable to the model.

OPTVModel::**AddVar**(double lb, double ub, double obj, char type, const std::string &name = "")

This function will add a single variable to the model. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel::AddVars()*) is preferred.

Parameters

- **lb** – Lower bound of variable.
- **ub** – Upper bound of variable.
- **obj** – Linear coefficient in objective function.
- **type** – Variable type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **name** – Variable name.

Returns

Variable object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – Unknown variable type, `name` is empty, a numeric value is `NaN`, or variable with `name` already exists in the model.

OPTVModel::**AddVar**(double lb, double ub, double obj, char type, const *OPTVColumn* &col, const std::string &name = "")

This function will add a single variable to the model, allowing to set coefficients for pre-existing constraints. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel::AddVars()*) is preferred.

Parameters

- **lb** – Lower bound of variable.
- **ub** – Upper bound of variable.
- **obj** – Linear coefficient in objective function.
- **type** – Variable type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** – Column object to set constraint coefficients.
- **name** – Variable name.

Returns

Variable object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – Unknown variable type, `name` is empty, a numeric value is `NaN`, or variable with `name` already exists in the model.

AddVars()

Add multiple variables to the model.

OPTVModel::**AddVars**(const double *lb, const double *ub, const double *obj, const char *type, const std::string *names, int count)

This function will add multiple variables to the model. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **lb** – Variable lower bounds.
- **ub** – Variable upper bounds.
- **obj** – Linear coefficients in objective function.
- **type** – Variable types. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **names** – Variable names.
- **count** – Number of variables to be added to the model.

Returns

Vector of variables added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – Unknown variable type, `names` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

OPTVModel::**AddVars**(const double *lb, const double *ub, const double *obj, const char *type, const *OPTVColumn* *col, const std::string *names, int count)

This function will add multiple variables to the model, allowing to set coefficients for pre-existing constraints. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **lb** – Variable lower bounds.
- **ub** – Variable upper bounds.
- **obj** – Linear coefficients in objective function.
- **type** – Variable types. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** – Column objects.
- **names** – Variable names.
- **count** – Number of variables to be added to the model.

Returns

Vector of variables added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – Unknown variable type, `names` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

CheckSolution()

Determine if a given solution is feasible to the model.

OPTVModel::**CheckSolution**(const std::vector<double> &solution, double tol = 1e-8) const

This function allows a user to check if an externally provided solution is feasible to the model. This function **does not** determine whether the solution stored internally is feasible.

Parameters

- **solution** – Vector containing *column* solution.
- **tol** – [**optional**, defaults to 10^{-8}] Feasibility and integrality tolerance.

Returns

Boolean value indicating feasibility of solution to the model.

Return values

- **false** – The solution is infeasible to the model.
- **true** – The solution is feasible to the model.

ComputeIIS()

Compute an irreducible inconsistent subsystem (IIS).

See *IIS Quick Start Guide* for more details on IIS.

OPTVModel::**ComputeIIS**()

Returns

Boolean value indicating success of IIS construction.

Return values

- **false** – IIS construction was unsuccessful.
- **true** – IIS construction was successful.

FeasRelax()

Perform a feasibility relaxation.

See *Feasibility Relaxation Quick Start Guide* for more details on feasibility relaxation.

OPTVModel::**FeasRelax**()

This function will perform a feasibility relaxation. Presently there is only support for the L^1 norm.

Returns
Status of feasibility relaxation.

Return values

- **0** – The problem is feasible.
- **-1** – Feasibility relaxation was unsuccessful.
- **>0** – Phase 1 objective value of minimal violation.

GetCoef()

Get the coefficient of a variable in a given constraint.

OPTVModel::**GetCoef**(const *OPTVConstr* &con, const *OPTVVar* &var) const

Parameters

- **con** – Constraint to query.
- **var** – The variable to be accessed.

Returns
Coefficient value of variable in constraint.

Throws

- *OPTVErrCode*::*INVALID_ARGUMENT* – Constraint or variable is not present in the model.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – No inner model data, or constraint linear expression is unavailable.

GetColumn()

Get a column object corresponding to a specified variable.

OPTVModel::**GetColumn**(const *OPTVVar* &v) const

Parameters
v – The variable to be accessed.

Returns
Column object that represents the variable.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Variable is not associated with a model.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – *Index* of the variable is larger than the number of variables in model. This may occur if the variable needs to be flushed to the model. This exception may also be thrown if the variable is from another model.

GetCone()

Get a specific cone constraint based on its position.

OPTVModel::**GetCone**(int i) const

Parameters
i – Index of the cone constraint in the model.

Returns
Cone constraint at index `i`.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – Index is larger than the number of cone constraints in the model.

GetCones()

Get all cone constraints in the model.

OPTVModel::**GetCones**() const

Returns
Vector containing all cone constraints in the model.

GetConstrByName()

Get a specific constraint based on its name.

OPTVModel::**GetConstrByName**(const std::string &name) const

Parameters
name – Name of constraint to be returned.

Returns
Constraint with name matching `name`.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Name is empty, or no constraint named `name` found in the model.

GetConstr()

Get a specific constraint based on its position.

OPTVModel::**GetConstr**(int i) const

Parameters
i – Index of the constraint in the model.

Returns
Constraint at index i.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – Index is larger than number of constraints in the model.

The index *i* in the function refers to the total index, not specific to the type of constraint. For example, if a user first adds a linear constraint and then a quadratic constraint, the linear constraint will have index *0* and the quadratic constraint will have index *1*.

GetConstrs()

Get all linear constraints in the model.

OPTVModel::**GetConstrs**() const

Returns
Array containing all linear constraints in the model.

GetGenConstr()

OPTVGenConstr OPTVModel::**GetGenConstr**(int type, int index) const

Get a single general constraint of the specified type.

Parameters

- **type** – The general constraint type (e.g., *OPTV_GENCONSTR_AND*, *OPTV_GENCONSTR_OR*).
- **index** – The index of the general constraint to retrieve.

Returns
The requested general constraint.

Raises OPTVErrCode.INVALID_ARGUMENT
Invalid constraint type.

Raises OPTVErrCode.INDEX_OUT_OF_RANGE
Index out of range.

GetGenConstrs()

std::vector<*OPTVGenConstr*> *OPTVModel*::**GetGenConstrs**(int type) const

Get all general constraints of the specified type.

Parameters
type – The general constraint type (e.g., *OPTV_GENCONSTR_AND*, *OPTV_GENCONSTR_OR*).

Returns
A vector of all general constraints of the specified type.

Raises OPTVErrCode.INVALID_ARGUMENT
Invalid constraint type.

GetGenConstrAnd()

Get the data associated with a specific general AND constraint.

OPTVModel::**GetGenConstrAnd**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars) const

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.

Throws
OPTVErrCode::*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetGenConstrOr()

Get the data associated with a specific general OR constraint.

OPTVModel::**GetGenConstrOr**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars) const

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.

Throws
OPTVErrCode::*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetGenConstrIndicator()

Get the data associated with a specific general `INDICATOR` constraint.

OPTVModel`::`**GetGenConstrIndicator**(const *OPTVGenConstr* &genConstr, *OPTVVar* &binVar, int &binVal, *OPTVLinExpr* &expr, char &sense, double &rhs) const

Parameters

- **genConstr** – General constraint to be queried.
- **binVar** – The binary indicator variable.
- **binVal** – The value of the indicator variable that would force the constraint to be satisfied (`0` or `1`).
- **expr** – Expression for the linear constraint triggered by the indicator.
- **sense** – Sense for the linear constraint. One of *OPTV_SENSE_LESS*, *OPTV_SENSE_EQUAL*, or *OPTV_SENSE_GREATER*.
- **rhs** – Right-hand side of the linear constraint.

Throws

OPTVErrCode`::`*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetGenConstrAbs()

Get the data associated with a specific general `ABS` constraint.

OPTVModel`::`**GetGenConstrAbs**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, *OPTVVar* &argument) const

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **argument** – Argument variable of general constraint.

Throws

OPTVErrCode`::`*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetGenConstrNorm()

Get the data associated with a specific general `NORM` constraint.

OPTVModel`::`**GetGenConstrNorm**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars, int &normType) const

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose values is used to calculate the norm in constraint.

- **normType** – The norm type (-1 for the L^{∞} norm, 0 for the L^{0} norm and 1 for the L^{1} norm).

Throws

OPTVErrCode::*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetGenConstrMin()

Get the data associated with a specific general `MIN` constraint.

OPTVModel::**GetGenConstrMin**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars, double &constant)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.
- **constant** – The additional constant operand of the constraint.

Throws

OPTVErrCode::*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetGenConstrMax()

Get the data associated with a specific general `MAX` constraint.

OPTVModel::**GetGenConstrMax**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars, double &constant)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.
- **constant** – The additional constant operand of the constraint.

Throws

OPTVErrCode::*NOT_IN_MODEL* – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

GetQConstr()

Get a specific quadratic constraint based on its position.

OPTVModel::**GetQConstr**(int i) const

Parameters

i – Index of the constraint in the model.

Returns

Constraint at index `i`.

Throws

OPTVErrCode::*INDEX_OUT_OF_RANGE* – Index is larger than number of constraints in the model.

The index `i` in the function refers to the total index, not specific to the type of constraint. For example, if a user first adds a linear constraint and then a quadratic constraint, the linear constraint will have index `0` and the quadratic constraint will have index `1`.

GetQConstrs()

Get all quadratic constraints in the model.

OPTVModel::**GetQConstrs**() const

Returns

Vector containing all quadratic constraints in the model.

GetSOS()

Get a specific SOS constraint of a given type based on its position.

OPTVModel::**GetSOS**(int i, int type) const

Parameters

- **i** – Index of the target SOS constraint in the model.
- **type** – SOS type. One of `1` or `2`.

Returns

SOS constraint at index `i`.

Throws

- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index `i` is larger than number of SOS constraints in the model.
- *OPTVErrCode*::*INVALID_ARGUMENT* – `type` is not `1` or `2`.

The index `i` in the function refers to the index, specific to the type of SOS constraint. For example, if a user first adds an SOS constraint of type 1 and then an SOS constraint of type 2, both will have an index of `0`.

GetSOSData()

OPTVModel::**GetSOSData**(const *OPTVSOS* &sos, std::vector<*OPTVVar*> &vars, std::vector<double> &weights) const

Get the internal data representing an SOS constraint.

Parameters

- **sos** – SOS constraint to query.
- **vars** – Empty vector of variables. If non-empty, this container will be cleared by the function.
- **weights** – Empty vector of weights. If non-empty, this container will be cleared by the function.

Returns
None.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – SOS constraint was defaulted constructed.
- *OPTVErrCode*::*NOT_IN_MODEL* – SOS constraint is a member of a different model or previously removed from the model.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of constraint is larger than model container. This may indicate that a flush command is needed.

GetSOSs()

Get all SOS constraints of a given type.

OPTVModel::**GetSOSs**(int type) const

Parameters
type – SOS type. One of 1 or 2.

Returns
vector containing all SOS of type `type` in model.

Throws

- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of the SOS constraint is larger than the number of SOS constraints of same type in the model. This may occur if the SOS constraints need to be flushed to the model.
- *OPTVErrCode*::*INVALID_ARGUMENT* – `type` is not 1 or 2.

GetObjective()

Get the objective linear expression.

OPTVModel::**GetObjective**() const

Returns
Linear expression object that represents objective function.

GetObjectiveN()

Get the linear expression of the specified multi-objective.

OPTVModel::**GetObjectiveN**(int objId) const

Parameters
objId – Id of the multi-objective to be accessed.

Returns
Linear expression object that represents the specified multi-objective function.

Throws
OPTVErrCode::*VALUE_OUT_OF_RANGE* – Id of the multi-objective is out of the range of the number of multi-objectives.

GetQObjective()

Get the objective function as a quadratic expression.

OPTVModel::**GetQObjective**() const

Returns
Quadratic expression object that represents objective function.

GetRow()

Get a linear expression corresponding to a specified constraint.

OPTVModel::**GetRow**(const *OPTVConstr* &c) const

Parameters
c – The constraint to be accessed.

Returns
Linear expression object that represents the constraint.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Constraint is not associated with a model.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – *Index* of the constraint is larger than the number of constraints in model. This may occur if the constraint needs to be flushed to the model. This exception may also be thrown if the constraint is from another model.

Get()

Parameters

Obtain a solver parameter value.

See *OptVerse MP Parameters*.

OPTVModel::**Get**(*OPTVIntParam* param) const

Parameters
param – Integer type parameter.

Returns
Parameter value.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVModel::**Get**(*OPTVDblParam* param) const

Parameters
param – Double type parameter.

Returns
Parameter value.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVModel::**Get**(*OPTVStrParam* param) const

Parameters
param – String type parameter.

Returns
Parameter value.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

Attributes

Obtain a model attribute value.

See *OptVerse MP Attributes*.

OPTVModel::**Get**(*OPTVIntAttr* attr) const

Parameters
attr – Integer type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

OPTVModel::**Get**(*OPTVDblAttr* attr) const

Parameters
attr – Double type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

OPTVModel::**Get**(*OPTVLongAttr* attr) const

Parameters
attr – Long type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

OPTVModel::**Get**(*OPTVStrAttr* attr) const

Parameters
attr – String type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

GetVarByName()

Get a specific variable based on its name.

OPTVModel::**GetVarByName**(const std::string &name) const

Parameters
name – Name of variable to be returned.

Returns
Variable with name matching `name`.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Name is empty, or no variable named `name` found in the model.

GetVar()

Get a specific variable based on its position.

OPTVModel::**GetVar**(int i) const

Parameters
i – Index of the variable in the model.

Returns
Variable at index `i`.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – Index is larger than number of variables in the model.

GetVars()

Get all variables in the model.

OPTVModel::**GetVars**() const

Returns
Vector containing all variables in the model.

Optimize()

Perform an optimization.

OPTVModel::**Optimize**()

Returns
None.

Throws
OPTVErrCode::*MATRIX_NOT_PSD* – A quadratic constraint or quadratic objective was provided with a matrix representation that is not positive semi-definite. This error will not occur if a quadratic objective is provided to problem with strictly linear constraints.

The optimization algorithm will depend on the solver parameters and model attributes. Detection of LP, QP, MIP, and network problems occurs automatically based on the variables' types.

Presolve()

Presolve the model.

OPTVModel::**Presolve**()

This function will return a new *OPTVModel* object that has been run through the presolve routine. The original model will also retain the presolve status.

Returns
Presolved copy of model.

Read()

Read model/basis data from file.

OPTVModel::**Read**(const std::string &fileName)

Generally speaking, this function serves for reading in basis information. There is additional support for reading model files; however, the preferred method for reading model files is through the constructor. See *File Formats*.

Table 2: Supported Read Types

.mps	.lp	.bas

Table 3: Supported Read Compression

.gz	.z	.Z

Parameters
fileName – File name, as C++ string.

Returns
None.

Throws
OPTVErrCode::*ERROR_FILE_READ* – File name or extension is empty. File is incorrectly formatted.

Remove()

Remove a variable from a model.

OPTVModel::**Remove**(const *OPTVVar* &var)

This function removes a variable from a model. The removal will shift the indices of the model.

For example, if a model, `model` was constructed with the following variables:

Variable	Index
x0	0
x1	1
x2	2
x3	3

Calling `model.Remove(x0)` will result in the following shift:

Variable	Index
x1	0
x2	1
x3	2

Parameters

var – Variable to be removed.

Returns

None.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – Variable is not part of any model.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Variable is part of a different model than the one removing.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of the variable is larger than the number of variables in the model. This may occur if the variable needs to be flushed to the model.

Remove a constraint from a model.

OPTVModel::**Remove**(const *OPTVConstr* &constr)

This function removes a constraint from a model. The removal will shift the indices of the model.

For example, if a model, `model` was constructed with the following constraints:

Constraint	Index
c0	0
c1	1
c2	2
c3	3

Calling `model.Remove(c0)` will result in the following shift:

Constraint	Index
c1	0
c2	1
c3	2

Parameters
constr – Constraint to be removed.

Returns
None.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – Constraint is not part of any model.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Constraint is part of a different model than the one removing.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of the constraint is larger than the number of constraints in the model. This may occur if the constraint needs to be flushed to the model.

OPTVModel::**Remove**(const *OPTVGenConstr* &genConstr)

This function removes a general constraint from a model. The removal will shift the indices of general constraints that match the type of the removed constraint in the model.

For example, if a model, `model` was constructed with the following general constraints:

Constraint	Type	Index
and0	AND	0
and1	AND	1
or0	OR	0
or1	OR	1

Calling `model.Remove(and0)` will result in the following shifting:

Constraint	Type	Index
and1	AND	0
or0	OR	0
or1	OR	1

Parameters
genConstr – General constraint to be removed.

Returns
None.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – General constraint is not part of any model.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – General constraint is part of a different model than the one removing.

- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of the general constraint is larger than the number of general constraints of same type in the model. This may occur if the general constraint needs to be flushed to the model.

OPTVModel::**Remove**(const *OPTVSOS* &sos)

This function removes an SOS constraint from a model. Unlike other constraint types, the removal will **not** shift the indices of SOS constraints that match the type of the removed constraint in the model.

For example, if a model, `model` was constructed with the following constraints:

Constraint	Type	Index
sos1_1	1	0
sos1_2	1	1
sos1_3	1	2
sos2_1	2	0

Calling `model.Remove(sos1_2)` will result in the following shifting:

Constraint	Type	Index
sos1_1	1	0
sos1_3	1	2
sos2_1	2	0

Parameters
sos – SOS constraint to be removed.

Returns
None.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – SOS constraint is not part of any model.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – SOS constraint is part of a different model than the one being removed from.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of the SOS constraint is larger than the number of SOS constraints of same type in the model. This may occur if the SOS constraints need to be flushed to the model.

OPTVModel::**Remove**(const *OPTVCone* &cone)

This function removes a cone constraint from a model. The removal will shift the indices of cone constraints.

Constraint	Type	Index
c0	SOC	0
c1	RSOC	1
c2	SOC	2
c3	RSOC	3

Calling `model.Remove(c0)` will result in the following shift:

Constraint	Type	Index
c1	RSOC	0
c2	SOC	1
c3	RSOC	2

Parameters
cone – Conic constraint to be removed.

Returns
None.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – Cone constraint is not part of any model.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Cone constraint is part of a different model than the one removing.
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* – Index of the cone constraint is larger than the number of cone constraints in the model. This may occur if the cone constraint needs to be flushed to the model.

Relax()

Perform an LP relaxation.

OPTVModel::**Relax**() const

This function will return a new *OPTVModel* object where all of the variables have been converted to *OPTV_CONTINUOUS*. Will create a copy of the model, leaving the original model unchanged.

Returns
LP relaxed copy of model.

Reset()

Reset model to an unsolved state.

OPTVModel::**Reset**(bool clearAll = false)

This function will clear the internal solution information. By default, this includes solution values, LP basis values, and the model status. The `clearAll` flag allows for clearing additional model attribute values (MIP warm start values, MIP branch priority, LP prohibited).

Parameters
clearAll – Clear additional information. (Default is `false`).

Returns
None.

GetObjSA()

LP sensitivity analysis of objective function.

See the *SA Quick Start Guide*.

OPTVModel::**GetObjSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

Parameters

- **down** – Smallest objective coefficient value at which the current optimal basis would remain optimal.
- **up** – Largest objective coefficient value at which the current optimal basis would remain optimal.
- **list** – Indices of variables for which to return objective coefficient analysis information.

Returns

Sensitivity information successfully populated.

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

GetVarLbSA()

LP sensitivity analysis of variable lower bounds.

See the *SA Quick Start Guide*.

OPTVModel::**GetVarLbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

Parameters

- **down** – Smallest lower bound value at which the current optimal basis would remain optimal.
- **up** – Largest lower bound value at which the current optimal basis would remain optimal.
- **list** – Indices of variables for which to return lower bound analysis information.

Returns

Sensitivity information successfully populated.

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

GetVarUbSA()

LP sensitivity analysis of variable upper bounds.

See the *SA Quick Start Guide*.

OPTVModel::**GetVarUbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

Parameters

- **down** – Smallest upper bound value at which the current optimal basis would remain optimal.
- **up** – Largest upper bound value at which the current optimal basis would remain optimal.
- **list** – Indices of variables for which to return upper bound analysis information.

Returns
Sensitivity information successfully populated.

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

GetConstrLbSA()

LP sensitivity analysis of constraint lower bounds.

See the *SA Quick Start Guide*.

OPTVModel::**GetConstrLbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

Parameters

- **down** – Smallest lower bound value at which the current optimal basis would remain optimal.
- **up** – Largest lower bound value at which the current optimal basis would remain optimal.
- **list** – Indices of constraints for which to return lower bound analysis information.

Returns
Sensitivity information successfully populated.

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

GetConstrUbSA()

LP sensitivity analysis of constraint upper bounds.

See the *SA Quick Start Guide*.

OPTVModel::**GetConstrUbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

Parameters

- **down** – Smallest upper bound value at which the current optimal basis would remain optimal.
- **up** – Largest upper bound value at which the current optimal basis would remain optimal.
- **list** – Indices of constraints for which to return upper bound analysis information.

Returns
Sensitivity information successfully populated.

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

SetCallback()

OPTVModel::**SetCallback**(*OPTVCallback* *callback)

This function connects a user defined callback to the model. Callbacks will be triggered at different stages of the optimization process based on the callback type (see *OPTVCBType*).

Parameters
callback – Pointer to a user derived callback class.

Returns
None.

SetCoef()

Set the coefficient of a variable in a given constraint.

OPTVModel::**SetCoef**(const *OPTVConstr* &con, const *OPTVVar* &var, double newValue)

Parameters

- **con** – Constraint to be modified.
- **var** – Variable whose coefficient we wish to modify.
- **newValue** – New coefficient value.

Returns
None.

Throws

- *OPTVErrCode*::*INVALID_ARGUMENT* – Constraint or variable is not present in the model, or `newValue` is `NaN`.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – No inner model data, or constraint linear expression is unavailable.

SetObjective()

Set a linear objective function.

OPTVModel::**SetObjective**(const *OPTVLinExpr* &expr, *OPTVSense* sense = *OPTVSense*::*MINIMIZE*)

This function clears the current objective (as opposed to adding to it). If no `sense` parameter is provided, the solver will default to *OPTVSense::MINIMIZE*.

Parameters

- **expr** – Linear objective function to be set.
- **sense** – [**optional**, defaults to *OPTVSense::MINIMIZE*] Minimize or maximize. See *OPTVSense*.

Returns

None.

Set a quadratic objective function.

OPTVModel::**SetObjective**(const *OPTVQuadExpr* &expr, *OPTVSense* sense = *OPTVSense*::*MINIMIZE*)

This function clears the current objective (as opposed to adding to it). If no `sense` parameter is provided, the solver will default to *OPTVSense::MINIMIZE*.

Parameters

- **expr** – Quadratic objective function to be set.
- **sense** – [**optional**, defaults to *OPTVSense::MINIMIZE*] Minimize or maximize. See *OPTVSense*.

Returns

None.

SetObjectiveN()

Set a linear multi-objective function.

OPTVModel::**SetObjectiveN**(int objId, const *OPTVLinExpr* &expr, const std::string &name, int priority, double weight = 1.0, double absTolerance = 0.0, double relTolerance = 0.0)

This function clears the current multi-objective (as opposed to adding to it). Sense of the multi-objective is determined by the sign of `weight` parameter.

Parameters

- **objId** – Id of the multi-objective to be set.
- **expr** – Objective function to be set.
- **name** – Name of the multi-objective to be set.
- **priority** – Priority of the multi-objective to be set (multi-objectives with greater priority are processed earlier).
- **weight** – [**optional**, defaults to `1.0`] Weight of the multi-objective to be set (used to determine the sense of the multi-objective and to process multiple multi-objectives with the same priority).

- **absTolerance** – [**optional**, defaults to `0.0`] Absolute tolerance of the multi-objective (determines maximal absolute violation of the multi-objective in the following optimization passes).
- **relTolerance** – [**optional**, defaults to `0.0`] Relative tolerance of the multi-objective (determines maximal relative violation of the multi-objective in the following optimization passes).

Returns
None.

Set()

Parameters

Set a solver parameter for the model. Once a model has been created, this interface provides the ability to control parameters specifically for this model. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Parameters*.

OPTVModel::**Set**(*OPTVIntParam* param, int value)

Parameters

- **param** – Integer type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVModel::**Set**(*OPTVDblParam* param, double value)

Parameters

- **param** – Double type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVModel::**Set**(*OPTVStrParam* param, const std::string &value)

Parameters

- **param** – String type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

Attributes

Set the value of a model attribute. Will not be reflected internally until *OPTVModel::Update()* is called.

See *OptVerse MP Attributes*.

OPTVModel::**Set**(*OPTVIntAttr* attr, int value)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

OPTVModel::**Set**(*OPTVDblAttr* attr, double value)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

OPTVModel::**Set**(*OPTVStrAttr* attr, const std::string &value)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_ATTRIBUTE* – An unknown attribute is provided.

Update()

Update the internal model.

OPTVModel::**Update**()

This function will perform a model update, or lazy-construction (if necessary). Calling this function is **required** before changes to the model will take effect. This function will be called, internally, whenever it is required that the internal model reflects the model in the SDK. For example, users **do not** need to call this before calling *OPTVModel::Optimize()*, since the function will be called internally.

Returns
None.

WriteIIS()

Write the computed IIS to file.

See *IIS Quick Start Guide* for more details on IIS.

OPTVModel::**WriteIIS**(const std::string &fileName)

Parameters
fileName – Name of `.ilp` file to be written. Must include `.ilp` extension (i.e `iis.ilp`).

Returns
None.

Throws
OPTVErrCode::*ERROR_FILE_WRITE* – Unsupported file type, or no IIS computed.

Note

This function will **not** compute an IIS. If the IIS was not previously computed, this function will throw an exception.

Write()

Write model/basis data to file.

OPTVModel::**Write**(const std::string &fileName)

See *File Formats*.

Table 4: Supported Write Types

`.mps`	`.lp`	`.bas`	`.ilp`

Table 5: Supported Write Compression

`.gz`	`.z`	`.Z`

Parameters
fileName – File name, as C++ string.

Returns
None.

Throws
OPTVErrCode::*ERROR_FILE_WRITE* – File name or extension is empty, or error.

OPTVTempConstr

class **OPTVTempConstr**

OptVerse temporary constraint object. The `OPTVTempConstr` class serves as a placeholder for intermediate results in constraint expressions formed with overloaded operators. This class does not contain any member functions. Instead, comparison operations (==, <=, >=) on expression and variable types (*OPTVVar*, *OPTVLinExpr*, *OPTVQuadExpr*) return `OPTVTempConstr` objects. These are typically consumed immediately by constraint-adding methods and are not intended for storage in user-defined variables.

The `OPTVTempConstr` object allows you to create several different types of constraints:

- **Linear Constraint**: A linear constraint is defined as an expression of the form `Expr1 sense Expr2`, where `Expr1` and `Expr2` can be *OPTVVar* objects, *OPTVLinExpr* objects, or numeric constants, and `sense` is one of the comparison operators: ==, <=, or >=. For example, both `x + y <= 1 + z` and `x + y == 5` are valid linear constraints.
- **Quadratic Constraint**: A quadratic constraint is defined as an expression of the form `Expr1 sense Expr2`, where `Expr1` and `Expr2` can be *OPTVVar* objects, *OPTVLinExpr* objects, *OPTVQuadExpr*, or numeric constants, and `sense` is one of the comparison operators: ==, <=, or >=. For example, both `x*y <= 1 + z` and `x*x + y*y <= z*z` are valid quadratic constraints. Note that at least one of `Expr1` or `Expr2` must be an *OPTVQuadExpr*; otherwise, the constraint is considered linear.

The following examples illustrate this:

```
model.AddConstr(x + y >= 1, "c0");

model.AddQConstr(x*x + y*y <= z, "c1");
```

OPTVCallback

class **OPTVCallback**

OptVerse callback object.

This class serves as an abstract base class, from which users may derive their own callback subclass. The function *OPTVCallback::RunCallback()* will be called periodically through solver execution. Depending on the callback type, various data may be queried throughout the optimization process.

A user may provide a callback to a given problem through the *OPTVModel::SetCallback()* function. Setting multiple callbacks is allowed.

Consider the following example, in which two user supplied callbacks are provided to a model. In this example, one callback is used to query solution information, while the other serves to provide MIP node information.

```
#include "optv_c++.h"
#include <iostream>

using namespace std;

class SDKMipSolutionCallback final : public OPTVCallback {
public:
    SDKMipSolutionCallback() : OPTVCallback(OPTVCBType::MIP_SOLUTION) {}
    ~SDKMipSolutionCallback() override = default;

    [[nodiscard]] int GetNTriggered() const { return nTriggered_; }
```

(continues on next page)

(continued from previous page)

```
    OPTVVar varToWatch;
protected:
    void RunCallback() final
    {
        ++nTriggered_;
        std::cout << "Callback 1: " << std::endl;
        cout << "Time: " << this->GetInfo(OPTVCBDblInfo::RUN_TIME);
        cout << ", dual bound: " << this->GetInfo(OPTVCBDblInfo::MIP_DUAL_BOUND);
        cout << ", new incumbent: " << this->GetInfo(OPTVCBDblInfo::MIP_INCUMBENT_OBJ);
        cout << ", new incumbent sol @ i: " << this->GetSolution(varToWatch) << " ";
        cout << "type: " << static_cast<int>(this->GetType()) << endl;
    }


private:
    int nTriggered_{0};
};

class SDKMipNodeCallback final : public OPTVCallback {
public:
    SDKMipNodeCallback() : OPTVCallback(OPTVCBType::MIP_NODE) {}
    ~SDKMipNodeCallback() override = default;

    [[nodiscard]] int GetNTriggered() const { return nTriggered_; }

    OPTVVar varToWatch;
protected:
    void RunCallback() final
    {
        ++nTriggered_;
        std::cout << "Callback 2: " << std::endl;
        cout << "n solution: " << this->GetInfo(OPTVCBIntInfo::MIP_NUM_SOLUTION) << ", ";
        cout << "Time: " << this->GetInfo(OPTVCBDblInfo::RUN_TIME);
        this->SetSolution(varToWatch, 0);
        std::cout << "type: " << static_cast<int>(this->GetType()) << std::endl;

        this->Abort();
    }


private:
    int nTriggered_{0};
};

int main()
{
    OPTVEnv env();
    OPTVModel model(env, "cb_demo.mps");

    model.Update();
    OPTVVar x1 = model.GetVarByName("x1");
```

(continues on next page)

(continued from previous page)

```
    SDKMipSolutionCallback sdkMipSolutionCallback;
    SDKMipNodeCallback sdkMipNodeCallback;
    sdkMipSolutionCallback.varToWatch = x1;
    sdkMipNodeCallback.varToWatch = x1;

    model.SetCallback(&sdkMipSolutionCallback);
    model.SetCallback(&sdkMipNodeCallback);

    model.Optimize();

    return 0;
}
```

Abort()

Interrupt the optimization.

OPTVCallback::**Abort**()

Returns
None.

GetInfo()

OPTVCallback::**GetInfo**(*OPTVCBIntInfo* info) const

Parameters
info – Integer type callback data to query.

Returns
Callback data value.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Queried data unavailable to callback type.
- *OPTVErrCode*::*INVALID_ARGUMENT* – Unknown callback data type was queried.

OPTVCallback::**GetInfo**(*OPTVCBDblInfo* info) const

Parameters
info – Double type callback data to query.

Returns
Callback data value.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Queried data unavailable to callback type.
- *OPTVErrCode*::*INVALID_ARGUMENT* – Unknown callback data type was queried.

OPTVCallback::**GetInfo**(*OPTVCBLongInfo* info) const

Parameters
info – Long long type callback data to query.

Returns
Callback data value.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Queried data unavailable to callback type.
- *OPTVErrCode*::*INVALID_ARGUMENT* – Unknown callback data type was queried.

OPTVCallback::**GetInfo**(*OPTVCBStrInfo* info) const

Parameters
info – String type callback data to query.

Returns
Callback data value.

Throws

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Queried data unavailable to callback type.
- *OPTVErrCode*::*INVALID_ARGUMENT* – Unknown callback data type was queried.

GetNodeRel()

OPTVCallback::**GetNodeRel**(const *OPTVVar* &var)

Only available if callback type is *OPTVCBType::MIP_NODE*.

Parameters
var – Variable to get node relaxation solution value for.

Returns
Node relaxation solution value for variable `var`.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – `var` is not part of a model. This may occur if `var` is removed, or if the model has not been flushed.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType::MIP_NODE*.

OPTVCallback::**GetNodeRel**()

Only available if callback type is *OPTVCBType::MIP_NODE*.

Returns
Vector containing node relaxation solution at current node for model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType::MIP_NODE*.

GetSolution()

OPTVCallback::**GetSolution**(const *OPTVVar* &var)

Only available if callback type is *OPTVCBType::MIP_SOLUTION*.

Parameters
var – Variable to get current incumbent solution data for.

Returns
Current incumbent solution value for variable `var`.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – `var` is not part of a model. This may occur if `var` is removed, or if the model has not been flushed.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType::MIP_SOLUTION*.

OPTVCallback::**GetSolution**()

Only available if callback type is *OPTVCBType::MIP_SOLUTION*.

Returns
Vector containing all current incumbent solution data for model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType::MIP_SOLUTION*.

GetType()

Returns the type of the callback. See *OPTVCBType*.

OPTVCallback::**GetType**() const

Returns
Type of the callback.

RunCallback()

This is a virtual function to be overridden when creating user-derived callback class. This function will be executed on callback steps to allow users to obtain callback information and perform other logic.

OPTVCallback::**RunCallback**()

Returns
None.

SetSolution()

OPTVCallback::**SetSolution**(const *OPTVVar* &var, double value)

Only available if callback type is *OPTVCBType::MIP_NODE*.

Parameters

- **var** – Variable whose solution we wish to set.
- **value** – Solution value to set.

Returns
None.

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – `var` is not part of a model. This may occur if `var` is removed, or if the model has not been flushed.
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType::MIP_NODE*.

OPTVCallback::**SetSolution**(const std::vector<double> &values)

Only available if callback type is *OPTVCBType::MIP_NODE*.

Parameters
values – Vector containing solution values to be set for entire model.

Returns
None.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType::MIP_NODE*.

UseSolution()

OPTVCallback::**UseSolution**()

After importing solution values using *OPTVCallback::SetSolution()*, you can optionally call `UseSolution` from within a *OPTVCBType::MIP_NODE* callback. This allows the solver to immediately use the imported values to generate a heuristic solution.

Returns
New candidate solution objective value, or *OPTV_INF* if the solution is infeasible or there is no new incumbent found.

Raises OPTVErrCode::DATA_NOT_AVAILABLE
callback type is not *OPTVCBType::MIP_NODE*.

OptVerse Callback Information

OPTVCBDblInfo

enum **OPTVCBDblInfo**

Table 6: OptVerse Double Callback Information

Name	Description
MIP_DUAL_BOUND	Current dual bound.
MIP_INCUMBENT_OBJ	New incumbent solution.
MIP_PRIMAL_BOUND	Current primal bound.
RUN_TIME	Elapsed time for optimization.
SIMPLEX_OBJ	Simplex objective value.

MIP_DUAL_BOUND

enumerator **MIP_DUAL_BOUND**

Current dual bound. Available in all callback types except *OPTVCBType::MESSAGE*.

MIP_INCUMBENT_OBJ

enumerator **MIP_INCUMBENT_OBJ**

Objective value of new incumbent solution. Available in only in callbacks of type *OPTVCBType::MIP_SOLUTION*.

MIP_PRIMAL_BOUND

enumerator **MIP_PRIMAL_BOUND**

Current primal bound. Available in all callback types except *OPTVCBType::MESSAGE*.

RUN_TIME

enumerator **RUN_TIME**

Elapsed optimization time. Available in all callback types.

SIMPLEX_OBJ

enumerator **SIMPLEX_OBJ**

Simplex objective value. Only available in callbacks of type *OPTVCBType::SIMPLEX*.

OPTVCBIntInfo

enum **OPTVCBIntInfo**

Table 7: OptVerse Integer Callback Information

Name	Description
MIP_NODE_STATUS	Node LP model status.
MIP_NUM_SOLUTION	Number of feasible solutions found.
SIMPLEX_NUM_ITERS	Simplex iteration count.

MIP_NODE_STATUS

enumerator **MIP_NODE_STATUS**

Node LP model status. Only available in *OPTVCBType::MIP_NODE*

MIP_NUM_SOLUTION

enumerator **MIP_NUM_SOLUTION**

Number of feasible solutions found. Available in all callback types except *OPTVCBType::MESSAGE*.

SIMPLEX_NUM_ITERS

enumerator **SIMPLEX_NUM_ITERS**

Simplex iteration count. Only available in callbacks of type *OPTVCBType::SIMPLEX*.

OPTVCBLongInfo

enum **OPTVCBLongInfo**

Table 8: OptVerse Long Callback Information

Name	Description
MIP_NUM_NODE	Number of nodes explored.
MIP_NUM_OPENNODE	Number of open nodes left.

MIP_NUM_NODE

enumerator **MIP_NUM_NODE**

Number of nodes explored. Available in all callback types except *OPTVCBType::MESSAGE*.

MIP_NUM_OPENNODE

enumerator **MIP_NUM_OPENNODE**

Number of open nodes left. Only available in callbacks of type *OPTVCBType::MIP*.

OPTVCBStrInfo

enum **OPTVCBStrInfo**

Table 9: OptVerse String Callback Information

Name	Description
MESSAGE	Next log message.

MESSAGE

enumerator **MESSAGE**

The next log message to be printed. Only available in callbacks of type *OPTVCBType::MESSAGE*.

OPTVCBType

OptVerse distinguishes the functionality of different callbacks through the `OPTVCBType` enumerator. Each type is triggered under a specific context. A callback type is assigned upon instantiation of the callback class and may not be changed.

enum **OPTVCBType**

enumerator **MESSAGE**

Triggered prior to printing a log message.

enumerator **MIP**

Triggered at various points during MIP evaluation.

enumerator **MIP_NODE**

Triggered after each node LP evaluation.

enumerator **MIP_SOLUTION**

Triggered after each non-incumbent solution is found.

enumerator **SIMPLEX**

Triggered during simplex optimization.

OPTVException

class **OPTVException**

OptVerse exception class.

This class represents exceptions thrown by the OptVerse solver. It inherits from `std::exception` and provides access to both an error code (*OPTVErrCode*) and a descriptive message.

Use *OPTVException::GetErrorCode()* to determine the specific error type, and *OPTVException::GetMessage()* to get a human-readable description.

OPTVException::**GetMessage**() const

Get error message from exception.

Returns
Exception error message.

OPTVException::**GetErrorCode**() const

Get error code from exception.

Returns
Exception error code.

For more details on *OPTVErrCode* refer to *OptVerse Error Code*.

OPTVErrCode

`OPTVErrCode` is used to generate a specific *OPTVException* object and provide the corresponding error details.

enum class **OPTVErrCode**

OptVerse error code.

enumerator **ERROR**

Generic error message, occasionally thrown by internal solver calls.

enumerator **WARNING**

Generic warning message, occasionally thrown by internal solver calls.

enumerator **UNKNOWN**

Encountered unknown error.

enumerator **INVALID_ARGUMENT**

A value provided to a function did not meet the specification.

For example: if we provide an undefined variable type when setting *OPTVCharAttr::VAR_TYPE*, or dividing a linear expression by zero.

enumerator **UNKNOWN_ATTRIBUTE**

An attribute was queried or set for an object of which it does not belong.

For example: *OPTVConstr::Get(OPTVIntAttr::NUM_VARS)* will throw this error, since *OPTVIntAttr::NUM_VARS* does not correspond to an attribute of the constraints.

enumerator **DATA_NOT_AVAILABLE**

An attribute was queried when the solution is not available.

For example: *OPTVVar::Get(OPTVDblAttr::X)* before calling *OPTVModel::Optimize()*.

enumerator **INDEX_OUT_OF_RANGE**

The index used to access an object is outside the range of allowed values.

For example: If a model has 2 variables and a user calls *model.GetVar(10)*.

enumerator **UNKNOWN_PARAMETER**

A parameter was queried or set that is not defined.

enumerator **VALUE_OUT_OF_RANGE**

A value for a function parameter is outside of its allowed value (ex. *OPTVSense*).

enumerator **ERROR_FILE_READ**

An error occurred while reading a file.

enumerator **ERROR_FILE_WRITE**

An error occurred while writing a file.

enumerator **LICENSE_CHECK_FAILED**

A license file could not be detected. License files are set using the environment variable, OPTV_LICENSE_FILE.

See the *Installation Guide*.

enumerator **INVALID_MODEL_INPUT**

Attempted to solve a model with invalid input.

enumerator **NOT_IMPLEMENTED**

Attempted to use functionality which is not implemented.

enumerator **NOT_IN_MODEL**

A variable or constraint unassociated with a model was used.

enumerator **MATRIX_NOT_PSD**

A quadratic expression whose matrix representation is not positive semi-definite was provided to the QCQP solver.

This may be the result of either a quadratic constraint or quadratic objective in a QCQP problem. This error will not occur if a quadratic objective is provided to a problem strictly with linear constraints.

OPTVSense

enum class **OPTVSense**

Objective sense. The default value is 1, which indicates minimization.

enumerator **MINIMIZE**

Minimize problem (+1)

enumerator **MAXIMIZE**

Maximize problem (-1)

OPTVConeType

enum class **OPTVConeType**

Cone type, for cone constraints.

The cone type parameter specifies the geometric constraint, C, that the *affine expression* must satisfy in cone constraints.

enumerator **SOC**

Second-order cones

A second-order cone, C_s, is defined as:

$$C_s^n = \left\{ x \in \mathbb{R}^n \mid x_1 \geq \sqrt{x_2^2 + x_3^2 + \cdots + x_n^2} \right\}.$$

enumerator **RSOC**

Rotated second-order cone.

A rotated second-order cone, C_r, is defined as:

$$C_r^n = \left\{ x \in \mathbb{R}^n \mid 2x_1 x_2 \geq x_3^2 + \cdots + x_n^2,\ x_1, x_2 \geq 0 \right\}.$$

See *OPTVCone*.

OptVerse Constants

OPTV_INF

Value

`1.0e+100`

Description

Infinity value.

OPTV_UNDF

Value

`1.0e+101`

Description

Undefined value.

Variable Types

OPTV_BINARY

Value

`B`

Description

Binary variable type.

`OPTV_INTEGER`

Value

`I`

Description

Normal integer variable type.

`OPTV_CONTINUOUS`

Value

`C`

Description

Continuous variable type.

Model Statuses

`OPTV_UNKNOWN`

Value

`0`

Description

Unknown model status (Default).

`OPTV_OPTIMAL`

Value

`1`

Description

Model solved to optimality. An optimal solution is available for querying.

`OPTV_INFEASIBLE`

Value

`2`

Description

Model proven to be infeasible.

`OPTV_INF_OR_UNBD`

Value

3

Description

Model proven to be infeasible or unbounded.

`OPTV_UNBOUNDED`

Value

4

Description

Model proven to be unbounded.

`OPTV_TIME_LIMIT`

Value

5

Description

Optimization time exceeded limit set by *OPTVDblParam::TIME_LIMIT*.

`OPTV_MEM_LIMIT`

Value

6

Description

Optimization memory exceeded limit set by *OPTVIntParam::MEM_LIMIT*.

`OPTV_INTERRUPTED`

Value

7

Description

Optimization terminated by user (ex. through `Ctrl+C`).

Variable and Constraint Basis Statuses

`OPTV_BASIC`

Value

0

Description

Variable or constraint is basic.

`OPTV_NONBASIC_LOWER`

Value

-1

Description

Variable or constraint is non-basic at its lower bound.

`OPTV_NONBASIC_UPPER`

Value

-2

Description

Variable or constraint is non-basic at its upper bound.

`OPTV_SUPERBASIC`

Value

-3

Description

Variable or constraint is superbasic.

General Constraint Types

`OPTV_GENCONSTR_AND`

Value

0

Description

Resultant variable is non-zero if all supplied variables are non-zero.

`OPTV_GENCONSTR_OR`

Value

1

Description

Resultant variable is non-zero if any of the supplied variables are non-zero.

OPTV_GENCONSTR_INDICATOR

Value

2

Description

Binary variable is non-zero if and only if the associated logical condition is satisfied.

OPTV_GENCONSTR_ABS

Value

3

Description

Absolute value constraint: `x == Abs(y)`.

OPTV_GENCONSTR_NORM

Value

4

Description

Euclidean norm constraint: `x == Norm(y1, y2, ...)`.

OPTV_GENCONSTR_MIN

Value

5

Description

Minimum constraint: `x == Min(y1, y2, ...)`.

OPTV_GENCONSTR_MAX

Value

6

Description

Maximum constraint: `x == Max(y1, y2, ...)`.

Constraint Sense

OPTV_SENSE_LESS

Value

<

Description

Less than or equal to.

`OPTV_SENSE_EQUAL`

Value

=

Description

Equal to.

`OPTV_SENSE_GREATER`

Value

>

Description

Greater than or equal to.

`OPTV_SENSE_NOT_EQUAL`

Value

!

Description

Not equal to.

CP Infinity

`OPTV_CP_INF`

Value

`int64_t max`

Description

Infinity value for CP solver.

8.2.2 Java API Reference Manual

OPTVVar

`class OPTVVar`

OptVerse variable object. Used to access variable data from a model. Variables should not be created through their constructor. Instead, a user should add a variable to a model using the *`OPTVModel.addVar()`* or *`OPTVModel.addVars()`* interfaces.

---

get()

Query the value of a variable attribute.

See *OptVerse MP Attributes*.

OPTVVar.**get**(OPTVIntAttr *attr*)

Parameters

- **attr** – Integer type attribute.

Returns
Attribute value.

Return type
int

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

OPTVVar.**get**(OPTVCharAttr *attr*)

Parameters

- **attr** – Character type attribute.

Returns
Attribute value.

Return type
char

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

OPTVVar.**get**(OPTVDblAttr *attr*)

Parameters

- **attr** – Double type attribute.

Returns
Attribute value.

Return type
double

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

OPTVVar.**get**(OPTVStrAttr *attr*)

Parameters

- **attr** – String type attribute.

Returns
Attribute value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

set()

Set the value of a variable attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVVar.**set**(OPTVIntAttr *attr*, int *value*)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

OPTVVar.**set**(OPTVCharAttr *attr*, char *value*)

Parameters

- **attr** – Character type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

OPTVVar.**set**(OPTVDblAttr *attr*, double *value*)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

OPTVVar.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

index()

Returns the index of the variable in a model.

OPTVVar.**index**()

Returns
Index of variable.

Return type
int

Return values

- **-1** – Variable is not in model.
- **-2** – Variable has been removed from a model.
- **>=0** – Variable index in model.

sameAs()

Determines if two variables are actually the same underlying object.

OPTVVar.**sameAs**(OPTVVar *v2*)

Parameters

- **v2** – Variable to check sameness against.

Returns
Variable sameness.

Return type
boolean

Return values

- **true** – Variables are the same.
- **false** – Variables are not the same.

OPTVColumn

class OPTVColumn

OptVerse Column object. Used to add variables to the model column-wise. This enables users to add a new variable while also adding it to existing constraint objects.

Consider the following example, illustrating use of the column class. Here we will partially construct a model and then append another variable. This example recreates the *milp1* example presented in the *Quick Start* section.

```
import com.huaweicloud.optv.*;

OPTVEnv env;
OPTVModel model(env);

OPTVVar x = model.addVar(0, 1, -1, OPTV_BINARY, "x");
OPTVVar y = model.addVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");

OPTVConstr c0 = model.addConstr(2 * x + y, -OPTV_INF, 3, "c0");
OPTVConstr c1 = model.addConstr(3 * y, -OPTV_INF, 6, "c1");
```

At this stage, we have constructed the following model:

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

```
OPTVColumn col;

col.addTerm(1, c0);
col.addTerm(1, c1);

OPTVVar z = model.addVar(0, 3, -6, OPTV_CONTINUOUS, col, "z");
```

By adding `col` to the model, we have now constructed the following matrix:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

AddTerm()

Add term to the end of the column.

OPTVColumn.**addTerm**(double *coef*, OPTVConstr *constr*)

This function will add a single term into the column.

Parameters

- **coef** – Coefficient.
- **constr** – Constraint.

Returns

None.

Return type

void

AddTerms()

Add terms to the end of the column.

OPTVColumn.**addTerms**(double[] *coefs*, OPTVConstr[] *constrs*, int *cnt*)

This function will add the terms to the column.

Parameters

- **coefs** – Array of coefficient values.
- **constrs** – Array of constraints.
- **cnt** – Number of elements in the C-array.

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `constrs` or `coefs` is null or `cnt` is negative.

Clear()

Clears the inner containers, such that *OPTVColumn.size()* will return `0`.

OPTVColumn.**clear**()

Returns
None.

Return type
void

GetCoeff()

Get the coefficient for a constraint at a given index in the column.

OPTVColumn.**getCoef**(int *i*)

Parameters

- **i** – Index of the constraint in the column.

Returns
Corresponding coefficient in the constraint at a given index, `i`, in the column.

Return type
double

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

GetConstr()

Get constraint at a specified index in the column.

OPTVColumn.**getConstr**(int *i*)

Parameters

- **i** – Index of the constraint in the column.

Returns
Constraint at position `i`.

Return type
OPTVConstr

Throw OPTVErrCode.INDEX_OUT_OF_RANGE
If `i` $\notin [0,$ `this->Size()`$]$.

Remove()

Remove term(s) from the column.

OPTVColumn.**remove**(int *i*)

Remove the term at a specified index from the column.

Parameters

- **i** – Index of term to be removed.

Returns
None.

Return type
void

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

OPTVColumn.**remove**(OPTVConstr *constr*)

Remove any instance of a constraint from the column.

Parameters

- **constr** – Constraint to be removed.

Returns
Whether at least one term removed.

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – If constraint, `constr`, is not in a model.

Size()

Get the size of the column.

OPTVColumn.**size**()

This value corresponds to the number of elements in the column, regardless of whether the element is repeated. For example, if the same constraint is added multiple times, the size will keep growing.

Returns
Size of the column.

Return type
int

OPTVLinExpr

class OPTVLinExpr

The *OPTVLinExpr* class is used to construct linear constraints and objective functions. An object of this type is construct from variables and constant terms. The expressions are accessed as constructed. That is, an expression `1 * x + 2 * y + 3 * x` will be accessed as:

Index	Coef	Variable
0	1	x
1	2	y
2	3	x

---

addTerms()

Add terms to the end of the linear expression.

OPTVLinExpr.**addTerms**(double[] *coefs*, OPTVVar[] *vars*, int *cnt*)

This function will add the terms $\text{coefs}_i * \text{vars}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coefs** – Array of coefficient values.
- **vars** – Array of variables.
- **cnt** – Number of elements in the C-array.

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `vars` or `coefs` is nullptr or `cnt` is negative.

clear()

Clears the inner "terms" container, such that *OPTVLinExpr.size()* will return `0`. Additionally, resets the expression such that it is numerically equivalent to `0`.

OPTVLinExpr.**clear**()

Returns
None.

Return type
void

getCoef()

Get the coefficient for a variable at a given index in the expression.

OPTVLinExpr.**getCoef**(int *i*)

Parameters

- **i** – Index of the variable in the expression.

Returns
Coefficient of variable at position `i`.

Return type
double

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

getVar()

Get variable at a specified index in the expression.

OPTVLinExpr.**getVar**(int *i*)

Parameters

- **i** – Index of the variable in the expression.

Returns
Variable at position `i`.

Return type
OPTVVar

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

getConstant()

Get the constant term from the expression.

OPTVLinExpr.**getConstant**()

Returns
Constant value in expression.

Return type
double

getValue()

Get the evaluated value of the linear expression.

OPTVLinExpr.**getValue**()

Returns
Expression value according to the primal solution.

Return type
double

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – Value queried for solution is unavailable.

remove()

Remove term(s) from the expression.

OPTVLinExpr.**remove**(int *i*)

Remove the term at a specified index from the expression.

Parameters

- **i** – Index of term to be removed.

Returns
None.

Return type
void

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

OPTVLinExpr.**remove**(OPTVVar *v*)

Remove any instance of a variable from the expression.

Parameters

- **v** – Variable to be removed.

Returns
Whether at least one term removed.

Return type
boolean

Return values

- **true** – One or more terms was removed.
- **false** – No terms were removed.

Throws
OPTVErrCode.NOT_IN_MODEL – If variable is not in a model.

sameAs()

Determines if two linear expressions are actually the same underlying expression.

OPTVLinExpr.**sameAs**(OPTVLinExpr *e2*)

Parameters

- **e2** – Linear expression to check sameness against.

Returns
Expression sameness.

Return type
boolean

Return values

- **true** – Expressions are the same.
- **false** – Expressions are not the same.

size()

Returns the size of the expression.

OPTVLinExpr.**size**()

This value corresponds to the number of elements in the expression, regardless of whether the element is repeated. For example, the expression `x+y` has `Size = 2`. The expression `x+y+x` has `Size = 3`.

Returns
Size of the linear expression.

Return type
int

OPTVConstr

class OPTVConstr

OptVerse constraint object. Used to access constraint data from a model. Constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVModel.addConstr()* or *OPTVModel.addConstrs()* interfaces for linear constraints. For quadratic constraints, the *OPTVModel.addQConstr()* interface is provided.

get()

Query the value of a constraint attribute.

See *OptVerse MP Attributes*.

OPTVConstr.**get**(OPTVIntAttr *attr*)

Parameters

- **attr** – Integer type attribute.

Returns
Attribute value.

Return type
int

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVConstr.**get**(OPTVDblAttr *attr*)

Parameters

- **attr** – Double type attribute.

Returns
Attribute value.

Return type
double

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVConstr.**get**(OPTVStrAttr *attr*)

Parameters

- **attr** – String type attribute.

Returns
Attribute value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

set()

Set the value of a constraint attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVConstr.**set**(OPTVIntAttr *attr*, int *value*)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVConstr.**set**(OPTVDblAttr *attr*, double *value*)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVConstr.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

index()

Return the index of the constraint in its model.

OPTVConstr.**index**()

Returns
Index of constraint.

Return values

- **-1** – Constraint is not in model.
- **-2** – Constraint has been removed from a model.
- **>=0** – Constraint index in model.

sameAs()

Determines if two constraints are actually the same underlying object.

OPTVConstr.**sameAs**(OPTVConstr *c2*)

Parameters

- **c2** – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
boolean

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

OPTVQuadExpr

class OPTVQuadExpr

The *OPTVQuadExpr* class is used to construct quadratic constraints and objective functions. An object of this type is constructed from variables and constant terms. Quadratic expressions are composed of three types of terms: quadratic, linear, and constant. The quadratic terms are stored in containers that hold the first variable, second variable, and coefficient. The linear component is accessed through the internal linear expression object. Indexing in a quadratic expression refers strictly to the quadratic components, with the linear expression being treated independently.

For example, the quadratic expression `1*x*x + 2*x*y + 3*x*y + 4*y*y + 5*x + 6*y + 7*x` has the following description:

Index	Coeff	Var1	Var2
0	1	x	x
1	2	x	y
2	3	x	y
3	4	y	y

To access the linear components of the same expression, use the *OPTVQuadExpr.getLinExpr()* interface. This will allow accessing the linear component as normal, for example:

Index	Coef	Variable
0	5	x
1	6	y
2	7	x

addConstant()

Add to the constant term.

OPTVQuadExpr.**addConstant**(double *c*)

This function will add to the constant term, not override.

Parameters

- **c** – Constant value to add to expression.

Returns
None.

Return type
void

addTerm()

Add a single linear term to the end of the quadratic expression.

OPTVQuadExpr.**addTerm**(double *coeff*, OPTVVar *var*)

This function will add the terms coeff ∗ var to the end of the expression.

Parameters

- **coeff** – Coefficient values.
- **var** – Variable to add to expression.

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `var` is nullptr.

Add a single quadratic term to the end of the quadratic expression.

OPTVQuadExpr.**addTerm**(double *coeff*, OPTVVar *var1*, OPTVVar *var2*)

This function will add the terms coeff ∗ var1 ∗ var2 to the end of the expression.

Parameters

- **coeff** – Coefficient values.
- **var1** – First variable in term.
- **var2** – Second variable in term.

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `var1` or `var2` is nullptr.

addTerms()

Add linear terms to the end of the quadratic expression.

OPTVQuadExpr.**addTerms**(double[] *coeff*, OPTVVar[] *var*, int *cnt*)

This function will add the terms $\text{coeff}_i * \text{var}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coeff** – C-style array of coefficient values.
- **var** – C-style array of variables.
- **cnt** – Number of elements in the C-array.

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `var` or `coeff` is empty or `cnt` is negative.

Add quadratic terms to the end of the quadratic expression.

OPTVQuadExpr.**addTerms**(double[] *coeff*, OPTVVar[] *var1*, OPTVVar[] *var2*, int *cnt*)

This function will add the terms $\text{coeff}_i * \text{var1}_i * \text{var2}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coeff** – C-style array of coefficient values.
- **var1** – C-style array of first variables.
- **var2** – C-style array of second variables.
- **cnt** – Number of elements in the C-array.

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `var1`, `var2`, or `coeff` is empty or `cnt` is negative.

clear()

Clears the quadratic expression.

OPTVQuadExpr.**clear**()

Clears the inner containers, such that *OPTVQuadExpr.size()* will return `0`. Additionally, this function clears the linear component of the expression, and resets the constant to `0`.

Returns
None.

Return type
void

getCoeff()

Get the coefficient for a term at a given index in the quadratic component of the expression.

OPTVQuadExpr.**getCoeff**(int *i*)

Parameters

- **i** – Index of the term in the expression.

Returns
Coefficient of term at position i.

Return type
double

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – If i $\notin [0,$ this->Size()$]$.

getConstant()

Get the constant term from the expression.

OPTVQuadExpr.**getConstant**()

Returns
Constant value in expression.

Return type
double

getLinExpr()

Get the linear components of the expression.

OPTVQuadExpr.**getLinExpr**()

Returns
Linear component of expression.

Return type
OPTVLinExpr

getValue()

Get the evaluated value of the quadratic expression.

OPTVQuadExpr.**getValue**()

Returns
Expression value according to the primal solution.

Return type
double

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – Value queried for solution is unavailable.

getVar1()

Get the first variable of a quadratic term at a specified index in the expression.

OPTVQuadExpr.**getVar1**(int *i*)

This function will return the first variable in a quadratic term for an expression. For example, an expression `x*x + x*y + y*y` has the following prescription:

Index	Var1	Var2
0	x	x
1	x	y
2	y	y

Parameters

- **i** – Index of the quadratic term in the expression.

Returns
First variable at position `i`.

Return type
OPTVVar

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – If `i` $\notin [0,$ `this->Size()`$]$.

getVar2()

Get the second variable of a quadratic term at a specified index in the expression. Refer to *OPTVQuadExpr.getVar1()* for a description of indexing.

OPTVQuadExpr.**getVar2**(int *i*)

This function will return the second variable in a quadratic term for an expression.

Parameters

- **i** – Index of the quadratic term in the expression.

Returns
Second variable at position `i`.

Return type
OPTVVar

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – If `i` $\notin [0,$ `this->Size()`$]$.

remove()

Remove term(s) from the quadratic terms.

OPTVQuadExpr.**remove**(int *i*)

Remove the term at a specified index from the expression. This interface does not allow removing terms from the linear component.

Parameters

- **i** – Index of term to be removed.

Returns
None.

Return type
void

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – If $\texttt{i} \notin [0, \texttt{this->Size()}]$.

Remove variable from the expression.

OPTVQuadExpr.**remove**(OPTVVar *v*)

Remove any instance of a variable from the expression. This interface will remove any occurrence of the variable from the linear component of the expression.

Parameters

- **v** – Variable to be removed.

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – If variable is not in a model.

sameAs()

Determines if two quadratic expressions are actually the same underlying expression.

OPTVQuadExpr.**sameAs**(OPTVQuadExpr *e2*)

Parameters

- **e2** – Quadratic expression to check sameness against.

Returns
Quadratic expression sameness.

Return type
boolean

Return values

- **true** – Quadratic expressions are the same.
- **false** – Quadratic expressions are not the same.

size()

Returns the size of the expression.

OPTVQuadExpr.**size**()

This value corresponds to the number of **quadratic** elements in the expression, regardless of whether the element is repeated. The expression `x*x + y*y + y` has `Size = 2`. The expression `x*x + x*x + y*y + y` has `Size = 3`.

Returns
Size of the quadratic expression.

Return type
int

OPTVQConstr

class OPTVQConstr

OptVerse quadratic constraint object. Used to access quadratic constraint data from a model. Quadratic constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVModel.addQConstr()*.

Warning

Quadratic constraints must satisfy the condition that their matrix representation are positive semi-definite or OptVerse will throw a *OPTVErrCode.MATRIX_NOT_PSD* exception.

---

get()

Query the value of a quadratic constraint attribute.

See *OptVerse MP Attributes*.

OPTVQConstr.**get(OPTVIntAttr attr) const**()

Parameters

- **attr** – Integer type attribute.

Returns
Attribute value.

Return type
int

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVQConstr.**get(OPTVCharAttr attr) const**()

Parameters

- **attr** – Char type attribute.

Returns
Attribute value.

Return type
char

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVQConstr.**get(OPTVDblAttr attr) const**()

Parameters

- **attr** – Double type attribute.

Returns
Attribute value.

Return type
double

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVQConstr.**get(OPTVStrAttr attr) const**()

Parameters

- **attr** – String type attribute.

Returns
Attribute value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

set()

Set the value of a quadratic constraint attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVQConstr.**set**(OPTVIntAttr *attr*, int *value*)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVQConstr.**set**(OPTVCharAttr *attr*, char *value*)

Parameters

- **attr** – Char type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVQConstr.**set**(OPTVDblAttr *attr*, double *value*)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVQConstr.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

index()

Return the index of the quadratic constraint in its model.

OPTVQConstr.**index**()

Returns
Index of quadratic constraint.

Return type
int

Return values

- **-1** – Quadratic constraint is not in model.
- **-2** – Quadratic constraint has been removed from a model.
- **>=0** – Quadratic constraint index in model.

sameAs()

Determines if two quadratic constraints are actually the same underlying object.

OPTVQConstr.**sameAs**(OPTVConstr *c2*)

Parameters

- **c2** – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
boolean

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

OPTVAffineExpr

class OPTVAffineExpr

The *OPTVAffineExpr* class serves as the foundation for constructing conic constraints by encapsulating the affine transformation $Fx + g$. An *OPTVAffineExpr* object contains:

- **Linear coefficients**: The sparse representation of matrix F, storing non-zero coefficients and their corresponding variable indices in Coordinate (COO) sparse format. Affine expressions are constructed by adding rows of linear expressions (the COO storage is internal, users will provide data through SDK objects).
- **Model variables**: The sparse representation of the vector x. Users only need to provide the participating variables, through adding *linear expression* objects as rows.
- **Constant terms**: The offset vector g that shifts the linear combination.

When creating an *OPTVCone*, the *OPTVAffineExpr* provides the mathematical structure (the affine transformation), while the cone type parameter specifies the geometric constraint C that the expression must satisfy. This separation allows for flexible composition of conic constraints from simpler affine building blocks.

See *OPTVCone*.

---

addRow()

Add a single row to the affine expression.

OPTVAffineExpr.**addRow**(OPTVLinExpr *expr*)

Parameters

- **expr** – Linear expression to add to the affine expression.

Returns
None.

Return type
void

addRows()

Add multiple rows to the affine expression.

OPTVAffineExpr.**addRows**(OPTVLinExpr[] *expr*, int *count*)

Parameters

- **expr** – Array of linear expressions to add to the affine expression. Each expression corresponds to a row in the affine expression.
- **count** – Number of rows to be added (length of expr array).

Returns
None.

Return type
void

Throws
OPTVErrCode.INVALID_ARGUMENT – `count` is negative.

getColumnIndices()

Returns the column indices of the nonzero entries in the affine expression. Each value corresponds to the column index for a nonzero entry. The affine expression is internally stored in COO (Coordinate) format, where each nonzero entry is identified by its row index, column index, and value.

OPTVAffineExpr.**getColumnIndices**()

Returns
A vector of integers, where each element corresponds to the column index of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr.getRowIndices()* and *OPTVAffineExpr.getValues()*.

Return type
int[]

getOffset()

OPTVAffineExpr.**getOffset**(int *index*)

Parameters

- **index** – Index of the row whose offset is to be queried.

Returns
A double corresponding to the offset for the row at `index`.

Return type
int

Throws
`java.lang.IndexOutOfBoundsException` – If `index` is larger than the number of rows in the affine expression.

getOffsets()

OPTVAffineExpr.**getOffsets**()

Returns
A vector of doubles, where each element is the offset for a specific row added to the affine expression.

Return type
double[]

getRowIndices()

Returns the row indices of the nonzero entries in the affine expression. Each value corresponds to the row index for a nonzero entry. The affine expression is internally stored in COO (Coordinate) format, where each nonzero entry is identified by its row index, column index, and value.

OPTVAffineExpr.**getRowIndices**()

Returns
A vector of integers, where each element corresponds to the row index of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr.getColumnIndices()* and *OPTVAffineExpr.getValues()*.

Return type
int[]

getValues()

Returns the coefficient values of the nonzero entries in the affine expression. Each value corresponds to the coefficient stored at the position defined by the matching row and column index from *OPTVAffineExpr.getRowIndices()* and *OPTVAffineExpr.getColumnIndices()*.

OPTVAffineExpr.**getValues**()

Returns
A vector of doubles, where each element is the coefficient of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr.getRowIndices()* and *OPTVAffineExpr.getColumnIndices()*.

Return type
double[]

setOffset()

Set the offset for a specific row, based on the index.

OPTVAffineExpr.**setOffset**(int *index*, double *value*)

Parameters

- **index** – Index of the row whose offset is to be set.
- **value** – Value for the offset to set.

Return type
void

Throws
`java.lang.IndexOutOfBoundsException` – If `index` is larger than the number of rows in the affine expression.

size()

Get the size of the affine expression.

OPTVAffineExpr.**size**()

This value corresponds to the number of elements in the affine expression, regardless of whether the element is repeated. For example, if the same linear expression is added multiple times, the size will keep growing.

Returns
Size of the affine expression.

Return type
int

OPTVCone

class OPTVCone

OptVerse cone object. Used to access conic constraint data from a model. Cones should not be created directly through their constructor. Instead, a user should add a cone constraint to a model using the *OPTVModel.addCone()* interface.

A conic constraint is mathematically defined as:

$$Fx + g \in C$$

Where:

- F is an $m \times n$ coefficient matrix that linearly transforms the optimization variables,
- x is the n-dimensional vector of optimization variables,

- g is an m-dimensional constant offset vector (right-hand side),
- C is the cone that constrains the resulting affine expression.
- The expression $Fx + g$ produces an m-dimensional vector that must belong to cone C.

The matrix F is stored in Coordinate (COO) sparse format for memory efficiency, containing only non-zero entries as (row, column, value) triplets. Users will construct $Fx + g$ as an *affine expression*, by adding rows through *linear expressions*.

OptVerse supports the following cone types:

- *Second-order cone*
- *Rotated second-order cone*

See *OPTVAffineExpr* and *OPTVConeType*.

---

get()

Query the value of a cone constraint attribute.

OPTVCone.**get**(OPTVStrAttr *attr*)

Parameters

- **attr** – String type attribute.

Returns
Attribute value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

getType()

Get the type of the cone constraint.

See *OPTVConeType*.

OPTVCone.**getType**()

Returns
Type of the cone constraint.

Return type
OPTVConeType

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – Cone constraint has been removed from a model.

index()

Return the index of the cone constraint in its model.

OPTVCone.**index**()

Returns
Index of the cone constraint in the model.

Return type
int

Return values

- **-1** – Cone constraint is not part of any model.
- **-2** – Cone constraint has been removed from a model.
- **>=0** – Cone constraint index in model.

sameAs()

Determines if two cone constraints are actually the same underlying object.

OPTVCone.**sameAs**(OPTVCone *c2*)

Parameters

- **c2** – Cone constraint to check sameness against.

Returns
Cone constraint sameness.

Return type
boolean

Return values

- **true** – Cone constraints are the same
- **false** – Cone constraints are not the same

Set()

Set the value of a cone constraint attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVCone.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVSOS

class OPTVSOS

OptVerse SOS constraint object. Used to access SOS constraint data from a model. SOS constraints should not be created through their constructor. Instead, a user should add an SOS constraint to a model using the *OPTVModel.addSOS()*.

A special ordered set (SOS) is an additional way to specify integrality conditions in a model. In particular, a special ordered set is a way to restrict the number of nonzero solution values among a specified set of variables in a model.

There are various types of SOS:

- SOS Type 1 is a set of variables where at most one variable may be nonzero.
- SOS Type 2 is a set of variables where at most two variables may be nonzero. If two variables are nonzero, they must be adjacent in the set.

---

get()

Query the value of an SOS constraint attribute.

See *OptVerse MP Attributes*.

OPTVSOS.**get**(OPTVIntAttr *attr*)

Parameters
- **attr** – Integer type attribute.

Returns
Attribute value.

Return type
int

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVSOS.**get**(OPTVStrAttr *attr*)

Parameters
- **attr** – String type attribute.

Returns
Attribute value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

set()

Set the value of an SOS constraint attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVSOS.**set**(OPTVIntAttr *attr*, int *value*)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVSOS.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

index()

Return the index of the SOS constraint in its model.

OPTVSOS.**index**()

Returns
Index of SOS constraint.

Return type
int

Return values

- **-1** – SOS constraint is not in model.
- **-2** – SOS constraint has been removed from a model.
- **>=0** – SOS constraint index in model.

sameAs()

Determines if two SOS constraints are actually the same underlying object.

OPTVSOS.**sameAs**(OPTVSOS *c2*)

Parameters

- **c2** – SOS constraint to check sameness against.

Returns
SOS constraint sameness.

Return type
boolean

Return values

- **true** – SOS constraints are the same.
- **false** – SOS constraints are not the same.

OPTVGenConstr

class OPTVGenConstr

OptVerse general constraint object. Used to access general constraint data from a model. General constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the `OPTVModel.addGenConstrXxx`.

Table 10: Supported General Constraints

Constraint	Type	Interface	Description
And	*OPTV_GENCONSTR_A*	*OPTVModel.addGenConstrAnd()*	Resultant variable is non-zero if all supplied variables are non-zero.
Or	*OPTV_GENCONSTR_O*	*OPTVModel.addGenConstrOr()*	Resultant variable is non-zero if any of the supplied variables are non-zero.
Indicator	*OPTV_GENCONSTR_I*	*OPTVModel.addGenConstrIndicator*	Binary variable is non-zero if and only if the associated logical condition is satisfied.

get()

Query the value of a general constraint attribute.

See *OptVerse MP Attributes*.

OPTVGenConstr.**get**(OPTVIntAttr *attr*)

Parameters

- **attr** – Integer type attribute.

Returns
Attribute value.

Return type
int

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVGenConstr.**get**(OPTVDblAttr *attr*)

Parameters

- **attr** – Double type attribute.

Returns
Attribute value.

Return type
double

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVGenConstr.**get**(OPTVStrAttr *attr*)

Parameters

- **attr** – String type attribute.

Returns
Attribute value.

Return type
String

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

set()

Set the value of a general constraint attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVGenConstr.**set**(OPTVDblAttr *attr*, int *value*)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

OPTVGenConstr.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

index()

Return the index of the general constraint in its model.

OPTVGenConstr.**index**()

Returns
Index of constraint.

Return type
int

Return values

- **-1** – Constraint is not in model.
- **-2** – Constraint has been removed from a model.
- **>=0** – Constraint index in model.

sameAs()

Determines if two general constraints are actually the same underlying object.

OPTVGenConstr.**sameAs**(OPTVGenConstr *c2*)

Parameters

- **c2** – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
boolean

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

OPTVModel

`class OPTVModel`

OPTVModel()

`OPTVModel`(OPTVEnv *env*)

Create an empty model associated with the given environment.

Parameters

- **env** – OptVerse environment for the model.

`OPTVModel`(OPTVEnv *env*, String *fileName*)

Create a model by reading from a file.

Parameters

- **env** – OptVerse environment for the model.
- **fileName** – Path to the model file (MPS, LP, etc.).

addCone()

Add a single cone constraint to the model.

OPTVModel.`addCone`(OPTVAffineExpr *affine*, OPTVConeType *type*, String *name* = *""*)

This function will add a single cone constraint to the model.

Parameters

- **affine** – The affine expression, $Fx + g$, used to specify the cone constraint.
- **type** – Type of cone constraint (e.g. second-order cone, rotated second-order cone).
- **name** – Cone constraint name.

Returns

Cone constraint object added to the model.

Return type

OPTVCone

Throws

OPTVErrCode.INVALID_ARGUMENT – `type` is not one of *OPTVConeType.SOC* or *OPTVConeType.RSOC*. The provided affine expression is empty. If `type` is *OPTVConeType.RSOC* and the affine expression size is less than *3*.

addConstr()

Add a single linear constraint to the model.

OPTVModel.**addConstr**(OPTVLinExpr *expr*, double *lhs*, double *rhs*, String *name*="")

This function will add a single constraint to the model. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel.addConstrs()*) is preferred.

Parameters

- **expr** – Linear expression describing the constraint.
- **lhs** – Lower bound of constraint.
- **rhs** – Upper bound of constraint.
- **name** – Constraint name.

Returns
Constraint object added to the model.

Return type
OPTVConstr

Throws
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

addConstrs()

Add multiple linear constraints to the model.

OPTVModel.**addConstrs**(OPTVLinExpr[] *expr*, double[] *lhs*, double[] *rhs*, String[] *names*, int *count*)

This function will add multiple linear constraints to the model. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **expr** – Linear expressions describing the constraints.
- **lhs** – Constraint lower bounds.
- **rhs** – Constraint upper bounds.
- **names** – Constraint names.
- **count** – Number of constraints to be added.

Returns
Array of constraint objects added to the model.

Return type
OPTVConstr[]

Throws
OPTVErrCode.INVALID_ARGUMENT – `names` is empty, numeric value is `NaN`, or a constraint with a name in `names` already exists in the model.

addQConstr()

Add a single quadratic constraint to the model.

OPTVModel.**addQConstr**(OPTVQuadExpr *expr*, double *lhs*, double *rhs*, String *name*="")

This function will add a single quadratic constraint to the model.

Parameters

- **expr** – Quadratic expression describing the constraint.
- **lhs** – Lower bound of constraint.
- **rhs** – Upper bound of constraint.
- **name** – Constraint name.

Returns

Constraint object added to the model.

Return type

OPTVQConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

addGenConstrAnd()

Add a single `AND` constraint to the model.

OPTVModel.**addGenConstrAnd**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = "")

This function will add a single `AND` constraint to the model.

Parameters

- **resultant** – The resultant binary variable of constraint.
- **vars** – Array of variables that must be non-zero for resultant to be non-zero.
- **len** – Length of `vars` array.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

addGenConstrOr()

Add a single OR constraint to the model.

OPTVModel.**addGenConstrOr**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = *""*)

This function will add a single OR constraint to the model.

Parameters

- **resultant** – The resultant binary variable of constraint.
- **vars** – Array of variables that must include at least one non-zero variable for resultant to be non-zero.
- **len** – Length of vars array.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

addGenConstrIndicator()

Add a single INDICATOR constraint to the model.

OPTVModel.**addGenConstrIndicator**(OPTVVar *binVar*, int *binVal*, OPTVLinExpr *expr*, char *sense*, double *rhs*, String *name* = *""*)

This function will add a single INDICATOR constraint to the model.

Parameters

- **binVar** – The binary indicator variable.
- **binVal** – The value of the indicator variable that would force the constraint to be satisfied (0 or 1).
- **expr** – Expression for the linear constraint triggered by the indicator.
- **sense** – Sense for the linear constraint. One of *OPTV_SENSE_LESS*, *OPTV_SENSE_EQUAL*, or *OPTV_SENSE_GREATER*.
- **rhs** – Right-hand side of the linear constraint.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

addGenConstrAbs()

Add a single ABS constraint to the model.

An ABS constraint of the form $r = \text{abs}(x)$ requires that the variable r is equal to the absolute value of the variable x. This constraint enforces the relationship $r = |x|$, where $r \geq 0$.

OPTVModel.**addGenConstrAbs**(OPTVVar *resultant*, OPTVVar *inputVar*, String *name*)

This function will add a single ABS constraint to the model.

Parameters

- **resultant** – The variable representing the absolute value of the input variable.
- **inputVar** – The variable for which the absolute value will be taken.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

AddGenConstrNorm()

Add a single NORM constraint to the model.

A NORM constraint of the form $r = \text{norm}\{x_1, \ldots, x_n\}$ requires that the variable r is equal to the vector norm of the argument vector $x_1, \ldots, x_n$.

OPTVModel.**addGenConstrNorm**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, int *normType*, String *name*)

This function will add a single NORM constraint to the model.

Parameters

- **resultant** – The variable representing the vector norm of the other variables.
- **vars** – Variables on which the vector norm is computed.
- **len** – Length of vars array.
- **normType** – Select the norm type to use. Valid options are: -1 for the L-infinity norm (also denoted as L^∞ norm), 0 for the L^0 norm, 1 for the L^1 norm.
- **name** – Constraint name.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

addGenConstrMax()

Add a single `MAX` constraint to the model.

A MAX constraint of the form $r = \max\{x_1, \ldots, x_n\}$ ensures that the variable r is equal to the maximum value among the argument vector $x_1, \ldots, x_n$.

OPTVModel.**addGenConstrMax**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = *""*, double *constant* = *-OPTV_INF*)

This function will add a single `MAX` constraint to the model.

Parameters

- **resultant** – The resultant variable whose value will be equal to the maximum of the other variables.
- **vars** – An array of variables from which the maximum value is taken.
- **len** – Length of `vars` array.
- **name** – Constraint name.
- **constant** – [**optional**] A constant value to be included in the list of arguments for the MAX operation.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Throws

OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

addGenConstrMin()

Add a single `MIN` constraint to the model.

A MIN constraint of the form $r = \min\{x_1, \ldots, x_n\}$ ensures that the variable r is equal to the minimum value among the argument vector $x_1, \ldots, x_n$.

OPTVModel.**addGenConstrMin**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = *""*, double *constant* = *OPTV_INF*)

This function will add a single `MIN` constraint to the model.

Parameters

- **resultant** – The resultant variable whose value will be equal to the minimum of the other variables.
- **vars** – An array of variables from which the minimum value is taken.
- **len** – Length of `vars` array.
- **name** – Constraint name.
- **constant** – [**optional**] A constant value to be included in the list of arguments for the MIN operation.

Returns

General constraint object added to the model.

Return type
OPTVGenConstr

Throws
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

addSOS()

Add an SOS constraint to the model.

OPTVModel.**addSOS**(OPTVVar[] *vars*, double[] *weights*, int *len*, int *type*, String *name*=*""*)

Parameters

- **vars** – Array of variables that participate in SOS.
- **weights** – Array of weights of variables in SOS.
- **len** – Length of arrays `vars` and `weights`.
- **type** – SOS type. One of `1` or `2`.
- **name** – Name of the SOS constraint.

Returns
SOS constraint.

Return type
OPTVSOS

Throws

- *OPTVErrCode.INVALID_ARGUMENT* – `type` is not `1` or `2`.
- *OPTVErrCode.NOT_IN_MODEL* – A variable in the `vars` array is not part of the model.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – The model has an invalid index. This may happen if trying access a model after its environment has gone out of scope.

addVar()

Add a single variable to the model.

OPTVModel.**addVar**(double *lb*, double *ub*, double *obj*, OPTVCharAttr *type*, String *name* = *""*)

This function will add a single variable to the model. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel.addVars()*) is preferred.

Parameters

- **lb** – Lower bound of variable.
- **ub** – Upper bound of variable.
- **obj** – Linear coefficient in objective function.
- **type** – Variable type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **name** – Variable name.

Returns
Variable object added to the model.

Return type
OPTVVar

Throws
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `name` is empty, a numeric value is `NaN`, or variable with `name` already exists in the model.

OPTVModel.**addVar**(double *lb*, double *ub*, double *obj*, char *type*, OPTVColumn *col*, String *name* = *""*)

This function will add a single variable to the model, allowing to set coefficients for pre-existing constraints. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel.addVars()*) is preferred.

Parameters

- **lb** – Lower bound of variable.
- **ub** – Upper bound of variable.
- **obj** – Linear coefficient in objective function.
- **type** – Variable type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** – Column object to set constraint coefficients.
- **name** – Variable name.

Returns
Variable object added to the model.

Return type
OPTVVar

Throws
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `name` is empty, a numeric value is `NaN`, or variable with `name` already exists in the model.

addVars()

Add multiple variables to the model.

OPTVModel.**addVars**(double[] *lb*, double[] *ub*, double[] *obj*, char[] *type*, String[] *names*, int *count*)

This function will add multiple variables to the model. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **lb** – Variable lower bounds.
- **ub** – Variable upper bounds.
- **obj** – Linear coefficients in objective function.
- **type** – Variable types. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **names** – Variable names.
- **count** – Number of variables to be added to the model.

Returns
Array of variables added to the model.

Return type
OPTVVar[]

Throws
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `names` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

OPTVModel.**addVars**(double[] *lb*, double[] *ub*, double[] *obj*, char[] *type*, OPTVColumn[] *col*, String[] *names*, int *count*)

This function will add multiple variables to the model, allowing to set coefficients for pre-existing constraints. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **lb** – Variable lower bounds.
- **ub** – Variable upper bounds.
- **obj** – Linear coefficients in objective function.
- **type** – Variable types. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** – Column objects.
- **names** – Variable names.
- **count** – Number of variables to be added to the model.

Returns
Array of variables added to the model.

Return type
OPTVVar[]

Throws
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `names` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

checkSolution()

Determine if a given solution is feasible to the model.

OPTVModel.**checkSolution**(double[] *solution*, double *tol=1e-8*)

This function allows a user to check if an externally provided solution is feasible to the model. This function **does not** determine whether the solution stored internally is feasible.

Parameters

- **solution** – Vector containing *column* solution.
- **tol** – [**optional**, defaults to 10^{-8}] Feasibility and integrality tolerance.

Returns
Boolean value indicating feasibility of solution to the model.

Return type
boolean

Return values

- **false** – The solution is infeasible to the model.

- **true** – The solution is feasible to the model.

computeIIS()

Compute an irreducible inconsistent subsystem (IIS).

See *IIS Quick Start Guide* for more details on IIS.

OPTVModel.**computeIIS**()

Returns
Boolean value indicating success of IIS construction.

Return type
boolean

Return values

- **false** – IIS construction was unsuccessful.
- **true** – IIS construction was successful.

feasRelax()

Perform a feasibility relaxation.

See *Feasibility Relaxation Quick Start Guide* for more details on feasibility relaxation.

OPTVModel.**feasRelax**()

This function will perform a feasibility relaxation. Presently there is only support for the L^1 norm.

Returns
Status of feasibility relaxation.

Return type
int

Return values

- **0** – The problem is feasible.
- **-1** – Feasibility relaxation was unsuccessful.
- **>0** – Phase 1 objective value of minimal violation.

getCoef()

Get the coefficient of a variable in a given constraint.

OPTVModel.**getCoef**(OPTVConstr *con*, OPTVVar *var*)

Parameters

- **con** – Constraint to query.
- **var** – The variable to be accessed.

Returns
Coefficient value of variable in constraint.

Return type
double

Throws

- *OPTVErrCode.INVALID_ARGUMENT* – Constraint or variable is not present in the model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – No inner model data, or constraint linear expression is unavailable.

GetColumn()

Get a column object corresponding to a specified variable.

OPTVModel.**getColumn**(OPTVVar *v*)

Parameters

- **v** – The variable to be accessed.

Returns
Column object that represents the variable.

Return type
OPTVColumn

Throw OPTVErrCode.DATA_NOT_AVAILABLE
Variable is not associated with a model.

Throw OPTVErrCode.INDEX_OUT_OF_RANGE
Index of the variable is larger than the number of variables in model. This may occur if the variable needs to be flushed to the model. This exception may also be thrown if the variable is from another model.

getConstrByName()

Get a specific constraint based on its name.

OPTVModel.**getConstrByName**(String *name*)

Parameters

- **name** – Name of constraint to be returned.

Returns
Constraint with name matching `name`.

Return type
OPTVConstr

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – Name is empty, or no constraint named `name` found in the model.

getCone()

Get a specific cone constraint based on its position.

OPTVModel.**getCone**(int *i*)

Parameters

- **i** – Index of the cone constraint in the model.

Returns
Cone constraint at index i.

Return type
OPTVCone

Throw OPTVErrCode.INDEX_OUT_OF_RANGE
Index is larger than the number of cone constraints in the model.

getCones()

Get all cone constraints in the model.

OPTVModel.**getCones**()

Returns
Array containing all cone constraints in the model.

Return type
OPTVCone[]

getConstr()

Get a specific constraint based on its position.

OPTVModel.**getConstr**(int *i*)

Parameters

- **i** – Index of the constraint in the model.

Returns
Constraint at index i.

Return type
OPTVConstr

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than number of constraints in the model.

getConstrs()

Get all linear constraints in the model.

OPTVModel.**getConstrs**()

Returns
Array containing all linear constraints in the model.

Return type
OPTVConstr[]

getGenConstrAnd()

Get the data associated with a specific general AND constraint.

OPTVModel.**getGenConstrAnd**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.

Returns
Number of variables in vars array.

Return type
int

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getGenConstrOr()

Get the data associated with a specific general OR constraint.

OPTVModel.**getGenConstrOr**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.

Returns
Number of variables in vars array.

Return type
int

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getGenConstrIndicator()

Get the data associated with a specific general `INDICATOR` constraint.

OPTVModel.**getGenConstrIndicator**(OPTVGenConstr *genConstr*, OPTVVar *binVar*, int *binVal*, OPTVLinExpr *expr*, char *sense*, double *rhs*)

Parameters

- **genConstr** – General constraint to be queried.
- **binVar** – The binary indicator variable.
- **binVal** – The value of the indicator variable that would force the constraint to be satisfied (`0` or `1`).
- **expr** – Expression for the linear constraint triggered by the indicator.
- **sense** – Sense for the linear constraint. One of *OPTV_SENSE_LESS*, *OPTV_SENSE_EQUAL*, or *OPTV_SENSE_GREATER*.
- **rhs** – Right-hand side of the linear constraint.

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getGenConstrAbs()

Get the data associated with a specific general `ABS` constraint.

OPTVModel.**getGenConstrAnd**(OPTVGenConstr *genConstr*, OPTVVar *resvar*, OPTVVar *argvar*)

Parameters

- **genConstr** – General constraint to be queried.
- **resvar** – Resultant variable of general constraint.
- **argvar** – Argument variable of general constraint.

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getGenConstrNorm()

Get the data associated with a specific general `NORM` constraint.

OPTVModel.**getGenConstrNorm**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*, int[] *normType*)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose values is used to calculate the norm in constraint.
- **normType** – The norm type (-1 for the L^{∞} norm, 0 for the L^{0} norm and 1 for the L^{1} norm).

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getGenConstrMin()

Get the data associated with a specific general `MIN` constraint.

OPTVModel.**getGenConstrMin**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*, double[] *constant*)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.
- **constant** – The additional constant operand of the constraint.

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getGenConstrMax()

Get the data associated with a specific general `MAX` constraint.

OPTVModel.**getGenConstrMax**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*, double[] *constant*)

Parameters

- **genConstr** – General constraint to be queried.
- **resultant** – Resultant variable of general constraint.
- **vars** – List of variables whose value is checked by constraint.
- **constant** – The additional constant operand of the constraint.

Returns
None.

Return type
void

Throws
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

getQConstr()

Get a specific quadratic constraint based on its position.

OPTVModel.**getQConstr(int i) const**()

Parameters

- **i** – Index of the constraint in the model.

Returns
Constraint at index `i`.

Return type
OPTVQConstr

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than number of constraints in the model.

The index `i` in the function refers to the total index, not specific to the type of constraint. For example, if a user first adds a linear constraint and then a quadratic constraint, the linear constraint will have index `0` and the quadratic constraint will have index `1`.

getQConstrs()

Get all quadratic constraints in the model.

OPTVModel.**getQConstrs() const**()

Returns
Vector containing all quadratic constraints in the model.

Return type
OPTVQConstr[]

getSOS()

Get a specific SOS constraint of a given type based on its position.

OPTVModel.**getSOS(int i, int type) const**()

Parameters

- **i** – Index of the target SOS constraint in the model.
- **type** – SOS type. One of `1` or `2`.

Returns
SOS constraint at index `i`.

Return type
OPTVSOS

Throws

- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index `i` is larger than number of SOS constraints in the model.
- *OPTVErrCode.INVALID_ARGUMENT* – `type` is not `1` or `2`.

The index `i` in the function refers to the index, specific to the type of SOS constraint. For example, if a user first adds an SOS constraint of type 1 and then an SOS constraint of type 2, both will have an index of `0`.

getSOSData()

OPTVModel.**getSOSData**(OPTVSOS *sos*, OPTVVar[] *vars*, double[] *weights*)

Get the internal data representing an SOS constraint.

Parameters

- **sos** – SOS constraint to query.
- **vars** – Empty vector of variables. If non-empty, this container will be cleared by the function.
- **weights** – Empty vector of weights. If non-empty, this container will be cleared by the function.

Returns
None.

Return type
void

Throws

- *OPTVErrCode.DATA_NOT_AVAILABLE* – SOS constraint was defaulted constructed.
- *OPTVErrCode.NOT_IN_MODEL* – SOS constraint is a member of a different model or previously removed from the model.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of constraint is larger than model container. This may indicate that a flush command is needed.

getSOSs()

Get all SOS constraints of a given type.

OPTVModel.**getSOSs(int type) const**()

Parameters

- **type** – SOS type. One of `1` or `2`.

Returns
Array of all SOS constraints of type `type`.

Return type
OPTVSOS[]

Throws

- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the SOS constraint is larger than the number of SOS constraints of same type in the model. This may occur if the SOS constraints need to be flushed to the model.
- *OPTVErrCode.INVALID_ARGUMENT* – `type` is not `1` or `2`.

getObjective()

Get the objective linear expression.

OPTVModel.**getObjective**()

Returns
Linear expression object that represents objective function.

Return type
OPTVLinExpr

getObjectiveN()

Get the linear expression of the specified multi-objective.

OPTVModel.**getObjectiveN**(int *objId*)

Parameters

- **objId** – Id of the multi-objective to be accessed.

Returns
Linear expression object that represents the specified multi-objective function.

Return type
OPTVLinExpr

Throws
OPTVErrCode.VALUE_OUT_OF_RANGE – Id of the multi-objective is out of the range of the number of multi-objectives.

getQObjective()

Get the objective function as a quadratic expression.

OPTVModel.**getQObjective() const**()

Returns
Quadratic expression object that represents objective function.

Return type
OPTVQuadExpr

getRow()

Get a linear expression corresponding to a specified constraint.

OPTVModel.**getRow**(OPTVConstr *c*)

Parameters

- **c** – The constraint to be accessed.

Returns
Linear expression object that represents the constraint.

Return type
OPTVLinExpr

Throws

- *OPTVErrCode.DATA_NOT_AVAILABLE* – Constraint is not associated with a model.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – *Index* of the constraint is larger than the number of constraints in model. This may occur if the constraint needs to be flushed to the model. This exception may also be thrown if the constraint is from another model.

get()

Parameters

Obtain a solver parameter value.

See *OptVerse parameters*.

OPTVModel.**get**(OPTVIntParam *param*)

Parameters

- **param** – Integer type parameter.

Returns
Parameter value.

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVModel.**get**(OPTVDblParam *param*)

Parameters

- **param** – Double type parameter.

Returns
Parameter value.

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVModel.**get**(OPTVStrParam *param*)

Parameters

- **param** – String type parameter.

Returns
Parameter value.

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Attributes

Obtain a model attribute value.

See *OptVerse MP Attributes*.

OPTVModel.**get**(OPTVIntAttr *attr*)

Parameters

- **attr** – Integer type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVModel.**get**(OPTVDblAttr *attr*)

Parameters

- **attr** – Double type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVModel.**get**(OPTVLongAttr *attr*)

Parameters

- **attr** – Long type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVModel.**get**(OPTVStrAttr *attr*)

Parameters

- **attr** – String type attribute.

Returns
Model attribute value.

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

getVarByName()

Get a specific variable based on its name.

OPTVModel.**getVarByName**(String *name*)

Parameters

- **name** – Name of variable to be returned.

Returns
Variable with name matching `name`.

Return type
OPTVVar

Throws
OPTVErrCode.DATA_NOT_AVAILABLE – Name is empty, or no variable named `name` found in the model.

getVar()

Get a specific variable based on its position.

OPTVModel.**getVar**(int *i*)

Parameters

- **i** – Index of the variable in the model.

Returns
Variable at index `i`.

Return type
OPTVVar

Throws
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than number of variables in the model.

getVars()

Get all variables in the model.

OPTVModel.**getVars**()

Returns
Array containing all variables in the model.

Return type
OPTVVar[]

optimize()

Perform an optimization.

OPTVModel.**optimize**()

Returns
None.

Return type
void

Throws
OPTVErrCode.MATRIX_NOT_PSD – A quadratic constraint or quadratic objective was provided with a matrix representation that is not positive semi-definite. This error will not occur if a quadratic objective is provided to problem with strictly linear constraints.

The optimization algorithm will depend on the solver parameters and model attributes. Detection of LP, QP, MIP, and network problems occurs automatically based on the variables' types.

presolve()

Presolve the model.

OPTVModel.**presolve**()

This function will return a new *OPTVModel* object that has been run through the presolve routine. The original model will also retain the presolve status.

Returns
Presolved copy of model.

Return type
OPTVModel

read()

Read model/basis data from file.

OPTVModel.**read**(String *fileName*)

Generally speaking, this function serves for reading in basis information. There is additional support for reading model files; however, the preferred method for reading model files is through the constructor. See *File Formats*.

Table 11: Supported Read Types

.mps	.lp	.bas

Table 12: Supported Read Compression

.gz	.z	.Z

Parameters

- **fileName** – File name, as C++ string.

Returns
None.

Return type
void

Throws
OPTVErrCode.ERROR_FILE_READ – File name or extension is empty. File is incorrectly formatted.

relax()

Perform an LP relaxation.

OPTVModel.**relax**()

This function will return a new *OPTVModel* object where all of the variables have been converted to *OPTV_CONTINUOUS*. Will create a copy of the model, leaving the original model unchanged.

Returns
LP relaxed copy of model.

Return type
OPTVModel

remove()

Remove a variable from a model.

OPTVModel.**remove**(OPTVVar *var*)

This function removes a variable from a model. The removal will shift the indices of the model.

For example, if a model, `model` was constructed with the following variables:

Variable	Index
x0	0
x1	1
x2	2
x3	3

Calling `model.remove(x0)` will result in the following shift:

Variable	Index
x1	0
x2	1
x3	2

Parameters

- **var** – Variable to be removed.

Returns
None.

Return type
void

Throws

- *OPTVErrCode.NOT_IN_MODEL* – Variable is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Variable is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the variable is larger than the number of variables in the model. This may occur if the variable needs to be flushed to the model.

OPTVModel.**remove**(OPTVConstr *constr*)

This function removes a constraint from a model. The removal will shift the indices of the model.

For example, if a model, `model` was constructed with the following constraints:

Constraint	Index
c0	0
c1	1
c2	2
c3	3

Calling `model.remove(c0)` will result in the following shift:

Constraint	Index
c1	0
c2	1
c3	2

Parameters

- **constr** – Constraint to be removed.

Returns
None.

Return type
void

Throws

- *OPTVErrCode.NOT_IN_MODEL* – Constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Constraint is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the constraint is larger than the number of constraints in the model. This may occur if the constraint needs to be flushed to the model.

OPTVModel.**remove**(OPTVGenConstr *genConstr*)

This function removes a general constraint from a model. The removal will shift the indices of general constraints that match the type of the removed constraint in the model.

For example, if a model, `model` was constructed with the following general constraints:

Constraint	Type	Index
and0	AND	0
and1	AND	1
or0	OR	0
or1	OR	1

Calling `model.remove(and0)` will result in the following shifting:

Constraint	Type	Index
and1	AND	0
or0	OR	0
or1	OR	1

Parameters

- **genConstr** – General constraint to be removed.

Returns
None.

Return type
void

Throws

- *OPTVErrCode.NOT_IN_MODEL* – Constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Constraint is part of a different model than the one removing.

- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the general constraint is larger than the number of general constraints of same type in the model. This may occur if the general constraint needs to be flushed to the model.

OPTVModel.**remove**(OPTVSOS *sos*)

This function removes an SOS constraint from a model. Unlike other constraint types, the removal will **not** shift the indices of SOS constraints that match the type of the removed constraint in the model.

For example, if a model, `model` was constructed with the following constraints:

Constraint	Type	Index
sos1_1	1	0
sos1_2	1	1
sos1_3	1	2
sos2_1	2	0

Calling `model.remove(sos1_2)` will result in the following shifting:

Constraint	Type	Index
sos1_1	1	0
sos1_3	1	2
sos2_1	2	0

Parameters

- **sos** – SOS constraint to be removed.

Returns
None.

Return type
void

Throws

- *OPTVErrCode.NOT_IN_MODEL* – SOS constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – SOS constraint is part of a different model than the one being removed from.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the SOS constraint is larger than the number of SOS constraints of same type in the model. This may occur if the SOS constraints need to be flushed to the model.

OPTVModel.**remove**(OPTVCone *cone*)

This function removes a cone constraint from a model. The removal will shift the indices of cone constraints.

Constraint	Type	Index
c0	SOC	0
c1	RSOC	1
c2	SOC	2
c3	RSOC	3

Calling `model.remove(c0)` will result in the following shift:

Constraint	Type	Index
c1	RSOC	0
c2	SOC	1
c3	RSOC	2

Parameters

- **cone** – Conic constraint to be removed.

Returns
None.

Throws

- *OPTVErrCode.NOT_IN_MODEL* – Cone constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Cone constraint is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the cone constraint is larger than the number of cone constraints in the model. This may occur if the cone constraint needs to be flushed to the model.

reset()

Reset model to an unsolved state.

OPTVModel.**reset**(boolean *clearAll=false*)

This function will clear the internal solution information. By default, this includes solution values, LP basis values, and the model status. The `clearAll` flag allows for clearing additional model attribute values (MIP warm start values, MIP branch priority, LP prohibited).

Parameters

- **clearAll** – Clear additional information. (Default is `false`).

Returns
None.

Return type
void

getObjSA()

LP sensitivity analysis of objective function.

See the *SA Quick Start Guide*.

OPTVModel.**getObjSA**(double[] *down*, double[] *up*, int[] *list*)

Parameters

- **down** – Smallest objective coefficient value at which the current optimal basis would remain optimal.

- **up** – Largest objective coefficient value at which the current optimal basis would remain optimal.
- **list** – Indices of variables for which to return objective coefficient analysis information.

Returns
Sensitivity information successfully populated.

Return type
boolean

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

getVarLbSA()

LP sensitivity analysis of variable lower bounds.

See the *SA Quick Start Guide*.

OPTVModel.**getVarLbSA**(double[] *down*, double[] *up*, int[] *list*)

Parameters

- **down** – Smallest lower bound value at which the current optimal basis would remain optimal.
- **up** – Largest lower bound value at which the current optimal basis would remain optimal.
- **list** – Indices of variables for which to return lower bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
boolean

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

getVarUbSA()

LP sensitivity analysis of variable upper bounds.

See the *SA Quick Start Guide*.

OPTVModel.**getVarUbSA**(double[] *down*, double[] *up*, int[] *list*)

Parameters

- **down** – Smallest upper bound value at which the current optimal basis would remain optimal.
- **up** – Largest upper bound value at which the current optimal basis would remain optimal.
- **list** – Indices of variables for which to return upper bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
boolean

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

getConstrLbSA()

LP sensitivity analysis of constraint lower bounds.

See the *SA Quick Start Guide*.

OPTVModel.**getConstrLbSA**(double[] *down*, double[] *up*, int[] *list*)

Parameters

- **down** – Smallest lower bound value at which the current optimal basis would remain optimal.
- **up** – Largest lower bound value at which the current optimal basis would remain optimal.
- **list** – Indices of constraints for which to return lower bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
boolean

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

getConstrUbSA()

LP sensitivity analysis of constraint upper bounds.

See the *SA Quick Start Guide*.

OPTVModel.**getConstrUbSA**(double[] *down*, double[] *up*, int[] *list*)

Parameters

- **down** – Smallest upper bound value at which the current optimal basis would remain optimal.
- **up** – Largest upper bound value at which the current optimal basis would remain optimal.
- **list** – Indices of constraints for which to return upper bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
boolean

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

setCoef()

Set the coefficient of a variable in a given constraint.

OPTVModel.**setCoef**(OPTVConstr *con*, OPTVVar *var*, double *newValue*)

Parameters

- **con** – Constraint to be modified.
- **var** – Variable whose coefficient we wish to modify.
- **newValue** – New coefficient value.

Returns
None.

Return type
void

Throws

- *OPTVErrCode.INVALID_ARGUMENT* – Constraint or variable is not present in the model, or `newValue` is `NaN`.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – No inner model data, or constraint linear expression is unavailable.

setObjective()

Set a linear objective function.

OPTVModel.**setObjective**(OPTVLinExpr *expr*, OPTVSense *sense=OPTVSense.MINIMIZE*)

This function clears the current objective (as opposed to adding to it). If no `sense` parameter is provide, the solver will default to *OPTVSense.MINIMIZE*.

Parameters

- **expr** – Objective function to be set.
- **sense** – [**optional**, defaults to *OPTVSense.MINIMIZE*] Minimize or maximize. See *OPTVSense*.

Returns
None.

Return type
void

Set a quadratic objective function.

OPTVModel.**setObjective**(OPTVQuadExpr *expr*, OPTVSense *sense=OPTVSense.MINIMIZE*)

This function clears the current objective (as opposed to adding to it). If no `sense` parameter is provide, the solver will default to *OPTVSense.MINIMIZE*.

Parameters

- **expr** – Objective function to be set.
- **sense** – [**optional**, defaults to *OPTVSense.MINIMIZE*] Minimize or maximize. See *OPTVSense*.

Returns
None.

Return type
void

setObjectiveN()

Set a linear multi-objective function.

OPTVModel.**setObjectiveN**(int *objId*, OPTVLinExpr *expr*, String *name*, int *priority*, double *weight = 1.0*, double *absTolerance = 0.0*, double *relTolerance = 0.0*)

This function clears the current multi-objective (as opposed to adding to it). Sense of the multi-objective is determined by the sign of `weight` parameter.

Parameters

- **objId** – Id of the multi-objective to be set.
- **expr** – Objective function to be set.
- **name** – Name of the multi-objective to be set.
- **priority** – Priority of the multi-objective to be set (multi-objectives with greater priority are processed earlier).
- **weight** – [**optional**, defaults to `1.0`] Weight of the multi-objective to be set (used to determine the sense of the multi-objective and to process multiple multi-objectives with the same priority).
- **absTolerance** – [**optional**, defaults to `0.0`] Absolute tolerance of the multi-objective (determines maximal absolute violation of the multi-objective in the following optimization passes).
- **relTolerance** – [**optional**, defaults to `0.0`] Relative tolerance of the multi-objective (determines maximal relative violation of the multi-objective in the following optimization passes).

Returns
None.

Return type
void

set()

Parameters

Set a solver parameter for the model. Once a model has been created, this interface provides the ability to control parameters specifically for this model. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse parameters*.

OPTVModel.**set**(OPTVIntParam *param*, int *value*)

Parameters

- **param** – Integer type parameter.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVModel.**set**(OPTVDblParam *param*, double *value*)

Parameters

- **param** – Double type parameter.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

OPTVModel.**set**(OPTVStrParam *param*, String *value*)

Parameters

- **param** – String type parameter.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Attributes

Set the value of a model attribute. Will not be reflected internally until *OPTVModel.update()* is called.

See *OptVerse MP Attributes*.

OPTVModel.**set**(OPTVIntAttr *attr*, int *value*)

Parameters

- **attr** – Integer type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVModel.**set**(OPTVDblAttr *attr*, double *value*)

Parameters

- **attr** – Double type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVModel.**set**(OPTVStrAttr *attr*, String *value*)

Parameters

- **attr** – String type attribute.
- **value** – Value to be set.

Returns
None.

Return type
void

Throws
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

update()

Update the internal model.

OPTVModel.**update**()

This function will perform a model update, or lazy-construction (if necessary). Calling this functions is **required** before changes to the model will take effect. This function will be called, internally, whenever it is required that the internal model reflects the model in the SDK. For example, users **do not** need to call this before calling *OPTVModel.optimize()*, since the function will be called internally.

Returns
None.

Return type
void

writeIIS()

Write the computed IIS to file.

See *IIS Quick Start Guide* for more details on IIS.

OPTVModel.**writeIIS**(String *fileName*)

Parameters

- **fileName** – Name of `.ilp` file to be written. Must include `.ilp` extension (i.e `iis.ilp`).

Returns
None.

Return type
void

Throws
OPTVErrCode.ERROR_FILE_WRITE – Unsupported file type, or no IIS computed.

Note

This function will **not** compute an IIS. If the IIS was not previously computed, this function will throw an exception.

write()

Write model/basis data to file.

OPTVModel.**write**(String *fileName*)

See *File Formats*.

Table 13: Supported Write Types

`.mps`	`.lp`	`.bas`	`.ilp`

Table 14: Supported Write Compression

`.gz`	`.z`	`.Z`

Parameters

- **fileName** – File name, as C++ string.

Returns
None.

Return type
void

Throws
OPTVErrCode.ERROR_FILE_WRITE – File name or extension is empty, or error.

OPTVErrCode

`OPTVErrCode` is used to generate a specific *`OPTVException`* object and provide the corresponding error details.

class OPTVErrCode

OptVerse error code.

`OPTVErrCode`.**ERROR**

Generic error message, occasionally thrown by internal solver calls.

`OPTVErrCode`.**WARNING**

Generic warning message, occasionally thrown by internal solver calls.

`OPTVErrCode`.**UNKNOWN**

Encountered unknown error.

`OPTVErrCode`.**INVALID_ARGUMENT**

A value provided to a function did not meet the specification.

For example: if we provide an undefined variable type when setting *OPTVCharAttr.VAR_TYPE*, or dividing a linear expression by zero.

`OPTVErrCode`.**UNKNOWN_ATTRIBUTE**

An attribute was queried or set for an object of which it does not belong.

For example: *`OPTVConstr.get(OPTVIntAttr.NUM_VARS)`* will throw this error, since the *OPTVIntAttr.NUM_VARS* corresponds to an attribute of the constraints.

`OPTVErrCode`.**DATA_NOT_AVAILABLE**

An attribute was queried when the solution is not available.

For example: *`OPTVVar.get(OPTVDblAttr.X)`* before calling *`OPTVModel.optimize()`*.

`OPTVErrCode`.**INDEX_OUT_OF_RANGE**

The index used to access an object is outside the range of allowed values.

For example: If a model has 2 variables and a user calls *`model.getVar(10)`*.

`OPTVErrCode`.**UNKNOWN_PARAMETER**

A parameter was queried or set that is not defined.

`OPTVErrCode`.**VALUE_OUT_OF_RANGE**

A value for a function parameter is outside of it's allowed value (ex. *`OPTVSense`*).

`OPTVErrCode`.**ERROR_FILE_READ**

An error occurred while reading a file.

`OPTVErrCode`.**ERROR_FILE_WRITE**

An error occurred while writing a file.

`OPTVErrCode`.**LICENSE_CHECK_FAILED**

A license file could not be detected. License files are set using the environment variable, `OPTV_LICENSE_FILE`.

See the *Installation Guide*.

`OPTVErrCode`.**INVALID_MODEL_INPUT**

Attempted to solve a model with invalid input.

OPTVErrCode.**NOT_IMPLEMENTED**

Attempted to use functionality which is not implemented.

OPTVErrCode.**NOT_IN_MODEL**

A variable or constraint unassociated with a model was used.

OPTVErrCode.**MATRIX_NOT_PSD**

A quadratic expression whose matrix representation is not positive semi-definite was provided to the QCQP solver.

This may be the result of either a quadratic constraint or quadratic objective in a QCQP problem. This error will not occur if a quadratic objective is provided to a problem strictly with linear constraints.

OPTVException

class OPTVException

OptVerse exception class.

This class represents exceptions thrown by the OptVerse solver. It provides access to both an error code (*OPTVErrCode*) and a descriptive message.

Use *OPTVException.getErrorCode()* to determine the specific error type, and *OPTVException.getMessage()* to get a human-readable description.

OPTVException.**getMessage**()

Get error message from exception.

Returns
Exception error message.

Return type
String

OPTVException.**getErrorCode**()

Get error code from exception.

Returns
Exception error code.

Return type
OPTVErrCode

For more details on *OPTVErrCode* refer to *OptVerse Error Code*.

OPTVSense

class OPTVSense

Objective sense. The default value is 1, which indicates minimization.

OPTVSense.**MINIMIZE**

Minimize problem (+1)

Type
int

Value
1

OPTVSense.**MAXIMIZE**

Maximize problem (-1)

Type
int

Value
-1

OPTVConeType

class OPTVConeType

Cone type, for cone constraints.

The cone type parameter specifies the geometric constraint, C, that the *affine expression* must satisfy in cone constraints.

OPTVConeType.**SOC**

Second-order cones

A second-order cone, C_s, is defined as:

$$C_s^n = \left\{ x \in \mathbb{R}^n \mid x_1 \geq \sqrt{x_2^2 + x_3^2 + \cdots + x_n^2} \right\}.$$

OPTVConeType.**RSOC**

Rotated second-order cone.

A rotated second-order cone, C_r, is defined as:

$$C_r^n = \left\{ x \in \mathbb{R}^n \mid 2x_1 x_2 \geq x_3^2 + \cdots + x_n^2,\ x_1, x_2 \geq 0 \right\}.$$

See *OPTVCone*.

OptVerse Constants

OPTV_INF

Infinity value.

Type
float

Value
`1.0e+100`

OPTV_UNDF

Undefined value.

Type
float

Value
`1.0e+101`

Variable Types

OPTV_BINARY

Binary variable type.

Type
char

Value
'B'

OPTV_INTEGER

Normal integer variable type.

Type
char

Value
'I'

OPTV_CONTINUOUS

Continuous variable type.

Type
char

Value
'C'

Model Statuses

OPTV_UNKNOWN

Unknown model status (Default).

Type
int

Value
0

OPTV_OPTIMAL

Model solved to optimality. An optimal solution is available for querying.

Type
int

Value
1

OPTV_INFEASIBLE

Model proven to be infeasible.

Type
int

Value
2

OPTV_INF_OR_UNBD

Model proven to be infeasible or unbounded.

Type
int

Value
3

OPTV_UNBOUNDED

Model proven to be unbounded.

Type
int

Value
4

OPTV_TIME_LIMIT

Optimization time exceeded limit set by *OPTVDblParam.TIME_LIMIT*.

Type
int

Value
5

OPTV_MEM_LIMIT

Optimization memory exceeded limit set by *OPTVIntParam.MEM_LIMIT*.

Type
int

Value
6

OPTV_INTERRUPTED

Optimization terminated by user (ex. through `Ctrl`+`C`).

Type
int

Value
7

Variable and Constraint Basis Statuses

OPTV_BASIC

Variable or constraint is basic.

Type
int

Value
0

OPTV_NONBASIC_LOWER

Variable or constraint is non-basic at its lower bound.

Type
int

Value
-1

OPTV_NONBASIC_UPPER

Variable or constraint is non-basic at its upper bound.

Type
int

Value
-2

OPTV_SUPERBASIC

Variable or constraint is superbasic.

Type
int

Value
-3

General Constraint Types

OPTV_GENCONSTR_AND

Resultant variable is non-zero if all supplied variables are non-zero.

Type
int

Value
0

OPTV_GENCONSTR_OR

Resultant variable is non-zero if any of the supplied variables are non-zero.

Type
int

Value
1

OPTV_GENCONSTR_INDICATOR

Binary variable is non-zero if and only if the associated logical condition is satisfied.

Type
int

Value
2

Constraint Sense

OPTV_SENSE_LESS

Less than or equal to.

Type
char

Value
<

OPTV_SENSE_EQUAL

Equal to.

Type
char

Value
=

OPTV_SENSE_GREATER

Greater than or equal to.

Type
char

Value
>

8.2.3 Python API Reference Manual

OPTVVar

class OPTVVar

OptVerse variable object. Used to access variable data from a model. Variables should not be created through their constructor. Instead, a user should add a variable to a model using the *OPTVModel.AddVar()* or *OPTVModel.AddVars()* interfaces.

Get()

Query the value of a variable attribute.

See *OptVerse MP Attributes*.

OPTVVar.**Get**(*attr*)

Parameters
attr (OPTVIntAttr / OPTVDblAttr / OPTVCharAttr / OPTVStrAttr) – OptVerse variable attribute to be queried.

Returns
Attribute value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

Example

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, "x")
model.Update()
# query the name of x
result = x.Get(OPTVStrAttr.NAME)
print(result) # x
```

Set()

Set the value of a variable attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

OPTVVar.**Set**(*attr*, *value*)

Parameters

- **attr** (OPTVIntAttr / OPTVCharAttr / OPTVDblAttr / OPTVStrAttr) – OptVerse variable attribute to be set.
- **value** (*int* / *float* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

Example

```
x = model.AddVar(0, OPTV_INF, 0, OPTV_CONTINUOUS, "x")
x.Set(OPTVDblAttr.Ub, 100)
```

Index()

Returns the index of the variable in a model.

`OPTVVar.Index()`

Returns
Index of variable.

Return type
int

Return values

- **-1** – Variable is not in model.
- **-2** – Variable has been removed from a model.
- **>=0** – Variable index in model.

Example

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, "x")
y = model.AddVar(0, 1, 0, OPTV_BINARY, "y")
x_ind = x.Index()
y_ind = y.Index()
print(x_ind, y_ind) # 0, 1
```

SameAs()

Determines if two variables are actually the same underlying object.

`OPTVVar.SameAs(v2)`

Parameters
v2 – Variable to check sameness against.

Returns
Variable sameness.

Return type
bool

Return values

- **true** – Variables are the same.
- **false** – Variables are not the same.

Example

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, "x")
y = model.AddVar(0, 1, 0, OPTV_BINARY, "y")
z = x
print(x.SameAs(z)) # True
print(x.SameAs(y)) # False
```

OPTVMVar

class OPTVMVar

OptVerse matrix variable object. An OPTVMVar is a `numpy.ndarray` of OPTVVar. A user should create these objects using the *OPTVModel.AddMVar()*, rather than using the default constructor to instantiate them.

Copy()

Create a copy of this OPTVMVar object.

`OPTVMVar.`**`Copy`**`()`

Returns
A copy of the OPTVMVar object.

Return type
OPTVMVar

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
copy_matrix_vars = matrix_vars.Copy()
```

Diagonal()

Create a new OPTVMVar object with variables on the specified diagonal of this OPTVMVar object.

`OPTVMVar.`**`Diagonal`**`(`*offset=0*, *axis1=0*, *axis2=1*`)`

Parameters

- **offset** (*int*) – Optional parameter. The offset between the diagonal and the main diagonal. Values > 0 mean above the main diagonal, and values < 0 below it.
- **axis1** (*int*) – Optional parameter. The axis that will serve as the primary axis of the 2-D sub-OPTVMVar for extracting the diagonal. Defaults to 0. A user needs to consider this argument only for OPTVMVar object with more than 2 dimensions.
- **axis2** (*int*) – Optional parameter. The axis that will serve as the second axis of the 2-D sub-OPTVMVar for extracting the diagonal. Defaults to 1. A user only needs to take this argument into account when dealing with OPTVMVar objects of more than two dimensions.

Returns
A new OPTVMVar with variables on the requested diagonal.

Return type
OPTVMVar

Example

```
matrix_vars = model.AddMVar((8, 8), name="x")
diagonal_main = matrix_vars.Diagonal()   # the main diagonal of matrix_
↪vars
diagonal_sub1 = matrix_vars.Diagonal(1)  # the OPTVVar above the main␣
```

(continues on next page)

(continued from previous page)

```
↪diagonal
diagonal_sub1 = matrix_vars.Diagonal(-2) # the OPTVVar below the main␣
↪diagonal
```

FromList()

Convert a list of OPTVVar objects into an OPTVMVar object.

`OPTVMVar.`**`FromList`**`(`*varlist*`)`

Parameters
varlist (*list*) – A list of OPTVVar objects.

Returns
A new OPTVMVar object corresponding to the input list.

Return type
OPTVMVar

Example

```
x0 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x0")
x1 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x1")
x2 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x2")
x3 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x3")
matrix_1d = OPTVMMVar.FromList([x0, x1, x2, x3])     # 1-D OPTVMVar
matrix_2d = OPTVMMVar.FromList([[x0, x1], [x2, x3]]) # 2-D OPTVMVar
```

FromVar()

Convert an OPTVVar object into a 0-dimensional OPTVMVar object.

`OPTVMVar.`**`FromVar`**`(`*var*`)`

Parameters
var (OPTVVar) – An OPTVVar object.

Returns
A 0-dimensional OPTVMVar object.

Return type
OPTVMVar

Example

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, name="x")
matrix_1d = OPTVMMVar.FromVar(x)
```

Get()

Query the value of an attribute for an OPTVMVar object.

See *OptVerse MP Attributes*.

`OPTVMVar.Get(attr)`

Parameters
attr (OPTVIntAttr / OPTVDblAttr / OPTVCharAttr / OPTVStrAttr) – OptVerse variable attribute to retrieve.

Returns
A `numpy.ndarray` containing the current values of the requested variable attribute.

Return type
`numpy.ndarray`

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Get(OPTVCharAttr.VAR_TYPE))
```

Set()

Set the value of the OPTVMVar attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

`OPTVMVar.Set(attr, newvalue)`

Parameters

- **attr** (OPTVIntAttr / OPTVCharAttr / OPTVDblAttr / OPTVStrAttr) – OptVerse variable attribute to be set.
- **newvalue** (*int* / *float* / *str*) – Value to be set. The shape must match that of the OPTVMVar object. Alternatively, a user can provide a scalar set argument, which will be automatically promoted to the correct shape.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for variables is provided.

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
matrix_vars.Set(OPTVIntAttr.BASIS, 1)
```

Item()

For an OPTVMVar containing a single element, it returns a copy of that element as an OPTVVar object. Invoking this method on an OPTVMVar object containing multiple elements will result in a ValueError.

`OPTVMVar.Item()`

Returns
A copy of the OPTVVar object.

Return type
OPTVVar

Raises
ValueError – The OPTVMVar object contains multiple elements.

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
matrix_var = matrix_vars[0, 1] # A 0-dimensional OPTVMVar containing a␣
↪single OPTVVar object
var = matrix_vars[0, 1].Item() # The OPTVVar object that resides within
```

Reshape()

Return a new OPTVMVar object with the same variables of this OPTVMVar object, but with a new shape.

`OPTVMVar.Reshape(shape)`

Parameters
shape – An int, or a tuple of int.

Returns
A new OPTVMVar object with the given shape.

Return type
OPTVMVar

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
new_matrix_vars = matrix_vars.Reshape(4) # 1-D
```

ToList()

Convert the OPTVMVar object to a list of OPTVVar objects.

`OPTVMVar.ToList()`

Returns
List of variables contained in this OPTVMVar.

Return type
list[*OPTVVar*]

Example

```
matrix_vars = model.AddMVar(3, name="x")
vars_list = matrix_vars.ToList()
print(vars_list) # [<optv.Var x_0> <optv.Var x_1> <optv.Var x_2>]
```

Transpose()

Create a transposed OPTVMVar object by reversing the order of the original OPTVMVar's axes.

`OPTVMVar.Transpose()`

Returns
A transposed copy of the OPTVMVar object.

Return type
OPTVMVar

Example

```
matrix_vars = model.AddMVar((3, 1), name="x")
matrix_vars_T = matrix_vars.Transpose()
print(matrix_vars_T.Shape) # (1, 3)
matrix_vars = model.AddMVar((2, 3), name="x")
matrix_vars_T = matrix_vars.Transpose()
print(matrix_vars_T.Shape) # (3, 2)
```

Ndim

The number of dimensions in this OPTVMVar object.

`OPTVMVar.Ndim`

Returns
The dimensions of OPTVMVar object.

Return type
int

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Ndim) # "2"
matrix_vars = model.AddMVar((2), name="x")
print(matrix_vars.Ndim) # "1"
```

Shape

The shape of this OPTVMVar object.

`OPTVMVar.Shape`

Returns
A tuple of int representing the shape of this OPTVMVar object.

Return type
tuple[int, ...]

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Shape) # (2, 2)
matrix_vars = model.AddMVar(3, name="x")
print(matrix_vars.Shape) # (3,)
```

Size

The total number of elements in this OPTVMVar object.

`OPTVMVar.Size`

Returns
The number of elements in OPTVMVar.

Return type
int

Example

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Size) # 4
matrix_vars = model.AddMVar(3, name="x")
print(matrix_vars.Size) # 3
```

T

Equivalent attribute for transpose.

`OPTVMVar.T`

Returns
A transposed copy of the OPTVMVar object.

Return type
OPTVMVar

Example

```
matrix_vars = model.AddMVar((3, 1), name="x")
matrix_vars_T = matrix_vars.T
print(matrix_vars_T.Shape) # (1, 3)
matrix_vars = model.AddMVar((2, 3), name="x")
```

(continues on next page)

(continued from previous page)

```
matrix_vars_T = matrix_vars.T
print(matrix_vars_T.Shape) # (3, 2)
```

OPTVColumn

class OPTVColumn

OptVerse Column object. Used to add variables to the model column-wise. This enables users to add a new variable while also adding it to existing constraint objects.

Consider the following example, illustrating use of the column class. Here we will partially construct a model and then append another variable. This example recreates the *milp1* example presented in the *Quick Start* section.

```
from optvpy import *

env = OPTVEnv()
model = OPTVModel(env)

x = model.AddVar(0, 1, -1, OPTV_BINARY, "x")
y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y")

c0 = model.AddConstr(2 * x + y, -OPTV_INF, 3, "c0")
c1 = model.AddConstr(3 * y, -OPTV_INF, 6, "c1")
```

At this stage, we have constructed the following model:

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

```
col = OPTVColumn()

col.addTerm(1, c0)
col.addTerm(1, c1)

z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, col, "z");
```

By adding `col` to the model, we have now constructed the following matrix:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

AddTerm()

Add term to the end of the column.

OPTVColumn.**AddTerm**(*coef*, *constr*)

This function will add a single term into the column.

Parameters

- **coef** (*float*) – Coefficient.
- **constr** (OPTVConstr) – Constraint.

Returns
None.

Return type
None

Example

```
col = OPTVColumn()

col.AddTerm(1.0, x)
col.AddTerm(3.0, y)
col.AddTerm(2.0, z)
```

AddTerms()

Add terms to the end of the column.

OPTVColumn.**AddTerms**(*coefs*, *constrs*)

This function will add the terms to the column.

Parameters

- **coefs** (*list[float]*) – List of coefficient values.
- **constrs** (*list[OPTVConstr]*) – List of constraints.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_ARGUMENT – constrs or coefs is null.

Example

```
constrs = self.model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
```

Clear()

Clears the inner containers, such that *OPTVColumn.Size()* will return **0**.

OPTVColumn.**Clear**()

Returns
None.

Return type
None

Example

```
model.Update()

constrs = model.GetConstrs()
coefs = [1.0, 2.0, 3.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3

col.Clear()
print(col.Size()) # 0
```

GetCoef()

Get the coefficient for a constraint at a given index in the column.

OPTVColumn.**GetCoef**(*i*)

Parameters
i (*int*) – Index of the constraint in the column.

Returns
Corresponding coefficient in the constraint at a given index, **i**, in the column.

Return type
float

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If i $\notin [0,$ this->Size()$]$.

Example

```
col = OPTVColumn()

col.AddTerm(1.0, x)
col.AddTerm(3.0, y)
col.AddTerm(2.0, z)

print(col.GetCoef(0)) # 1.0
```

GetConstr()

Get constraint at a specified index in the column.

`OPTVColumn.GetConstr(i)`

Parameters
`i` (*int*) – Index of the constraint in the column.

Returns
Constraint at position `i`.

Return type
OPTVConstr

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

Example

```
constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)

col.GetConstr(0)
```

Remove()

Remove term(s) from the column.

`OPTVColumn.Remove(i)`

Remove the term at a specified index from the column.

Parameters
`i` (*int*) – Index of term to be removed.

Returns
None.

Return type
None

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

Example

```
model.Update()

constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
```

(continues on next page)

(continued from previous page)

```
col.AddTerms(coefs, constrs)
print(col.Size()) # 3

col.Remove(1)
print(col.Size()) # 2
```

OPTVColumn.**Remove**(*constr*)

Remove any instance of a constraint from the column.

Parameters
constr (OPTVConstr) – Constraint to be removed.

Returns
Whether at least one term removed.

Returns
None.

Return type
None

Raises
OPTVErrCode.NOT_IN_MODEL – If constraint, constr, is not in a model.

Example

```
model.Update()

constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3

col.Remove(constrs[1])
print(col.Size()) # 2
```

Size()

Get the size of the column.

OPTVColumn.**Size**()

This value corresponds to the number of elements in the column, regardless of whether the element is repeated. For example, if the same constraint is added multiple times, the size will keep growing.

Returns
Size of the column.

Return type
int

Example

```
model.Update()

constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3
```

OPTVLinExpr

class OPTVLinExpr

The *OPTVLinExpr* class is used to construct linear constraints and objective functions. An object of this type is construct from variables and constant terms. The expressions are accessed as constructed. That is, an expression `1 * x + 2 * y + 3 * x` will be accessed as:

Index	Coef	Variable
0	1	x
1	2	y
2	3	x

AddTerms()

Add terms to the end of the linear expression.

`OPTVLinExpr.`**`AddTerms`**`(`*coefs*, *vars*, *cnt*`)`

Parameters

- **coefs** (*list[float]*) – List of coefficient values.
- **vars** (*list[OPTVVar]*) – List of variables.
- **cnt** (*int*) – Number of elements in the list.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_ARGUMENT – `vars` or `coefs` is `None` or `cnt` is negative.

Example

```
expr = x + 1
expr.AddTerms([2.0, 3.0], [y, z], 2)
model.Update()
print(expr) # x + 2*y + 3*z + 1
```

Clear()

Clears the inner "terms" container, such that *OPTVLinExpr.Size()* will return `0`. Additionally, resets the expression such that it is numerically equivalent to `0`.

OPTVLinExpr.**Clear**()

Returns
None.

Return type
None

Example

```
expr = x + 1
expr.Clear()
print(expr)      # 0
print(expr.Size) # 0
```

GetCoef()

Get the coefficient for a variable at a given index in the expression.

OPTVLinExpr.**GetCoef**(*i*)

Parameters
i (*int*) – Index of the variable in the expression.

Returns
Coefficient of variable at position `i`.

Return type
float

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

Example

```
expr = 2*x + 3*y + z + 1
result = expr.GetCoef(1)
print(result) # 3
```

GetVar()

Get variable at a specified index in the expression.

OPTVLinExpr.**GetVar**(*i*)

Parameters
i (*int*) – Index of the variable in the expression.

Returns
Variable at position `i`.

Return type
OPTVVar

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0,$ `this->Size()`$]$.

Example

```
expr = 2*x + 3*y + 1
result = expr.GetVar(0)
print(result) # x
```

GetConstant()

Get the constant term from the expression.

`OPTVLinExpr.GetConstant()`

Returns
Constant value in expression.

Return type
float

Example

```
expr = 2*x + 3*y + z + 6
result = expr.GetConstant()
print(result) # 6
```

GetValue()

Get the evaluated value of the linear expression.

`OPTVLinExpr.GetValue()`

Returns
Expression value according to the primal solution.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Value queried for solution is unavailable.

Example

```
model.Optimize()
obj = model.GetObjective()
result = obj.GetValue()
```

Remove()

Remove term(s) from the expression.

`OPTVLinExpr.Remove(i)`

Remove the term at a specified index from the expression.

Parameters
i (*int*) – Index of term to be removed.

Returns
None.

Return type
None

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0, \texttt{this->Size()}]$.

Example

```
expr = 2*x + 3*y + 1
expr.Remove(0)
print(expr) # 3*y + 1
```

`OPTVLinExpr.Remove(v)`

Remove any instance of a variable from the expression.

Parameters
v (OPTVVar) – Variable to be removed.

Returns
Whether at least one term removed.

Return type
bool

Return values

- **true** – One or more terms was removed.
- **false** – No terms were removed.

Raises
OPTVErrCode.NOT_IN_MODEL – If variable is not in a model.

Example

```
expr = 2*x + 3*y + 1
expr.Remove(y)
print(expr) # 2*x + 1
```

SameAs()

Determines if two linear expressions are actually the same underlying expression.

`OPTVLinExpr.SameAs(e2)`

Parameters
e2 (OPTVLinExpr) – Linear expression to check sameness against.

Returns
Expression sameness.

Return type
bool

Return values

- **true** – Expressions are the same.
- **false** – Expressions are not the same.

Example

```
expr1 = 2*x + 3*y + 1
expr2 = x + y
expr3 = 2*x + 3*y + 1
print(expr1.SameAs(expr3)) # True
print(expr1.SameAs(expr2)) # False
```

Size()

Returns the size of the expression.

`OPTVLinExpr.Size()`

This value corresponds to the number of elements in the expression, regardless of whether the element is repeated. For example, the expression `x+y` has `Size = 2`. The expression `x+y+x` has `Size = 3`.

Returns
Size of the linear expression.

Return type
int

Example

```
expr = 2*x + 3*y + 1
print(expr.Size) # 2
```

OPTVMLinExpr

class OPTVMLinExpr

OptVerse linear matrix expression object. A linear matrix expression emerges as a result of linear operators performed on an *OPTVMVar* object. A user generally builds linear matrix expressions using overloaded operators, such as `A @ x` or `x + 1` (x is an *OPTVMVar* object). An OPTVMLinExpr object containing empty expressions can be created using the *OPTVMLinExpr.Zeros()* method.

Copy()

Create a copy of this OPTVMLinExpr object.

OPTVMLinExpr.**Copy**()

Returns
A copy of the OPTVMLinExpr object.

Return type
OPTVMLinExpr

Example

```
expr = model.AddMVar(3, name="x") + 1
expr_copy = expr.Copy()
```

Clear()

Reset all expressions in this OPTVMLinExpr object to zero.

OPTVMLinExpr.**Clear**()

Returns
None.

Return type
None

Example

```
expr = model.AddMVar(3, name="x") + 1
expr.Clear()
# All three entries are set to the constant value of 0.0.
print(expr) # ['0.0' '0.0' '0.0']
```

GetValue()

Get the evaluated values of the linear expressions in this OPTVMLinExpr object.

`OPTVMLinExpr.GetValue()`

Returns
A `numpy.ndarray` containing the evaluated values of the linear expressions in this OPTVMLinExpr object.

Return type
`numpy.ndarray`

Example

```
expr = model.AddMVar(3, name="x") + 1
model.AddConstrs((expr[i].Item() <= 1 for i in range(3)), name='c')
model.Optimize()
values = expr.GetValue()
```

Item()

For an OPTVMLinExpr object containing a single element, it returns a copy of that element as an OPTVLinExpr object. Invoking this method on an OPTVMLinExpr object containing multiple elements will result in a ValueError.

`OPTVMLinExpr.Item()`

Returns
An OPTVLinExpr object.

Return type
OPTVLinExpr

Raises
ValueError – The OPTVMLinExpr object contains multiple elements.

Example

```
expr = model.AddMVar(3, name="x") + 1
expr_sub = expr[0, 1]       # A 0-D OPTVMLinExpr that includes one␣
↪OPTVLinExpr object within it
expr_le = expr[0, 1].Item() # A copy of the resident OPTVLinExpr object
```

Zeros()

Create an OPTVMLinExpr object filled with zeros of the given shape.

`OPTVMLinExpr.Zeros(shape)`

Parameters
shape (*int | tuple[int, ...]*) – The desired shape.

Returns
An OPTVMLinExpr object filled with zeros.

Return type
OPTVMLinExpr

Example

```
expr_zeros = OPTVMLinExpr.Zeros((2, 2)) # all elements in this␣
↪OPTVMLinExpr object are zero
```

Sum()

Sum the elements in the OPTVMLinExpr object.

`OPTVMLinExpr.Sum(axis=None)`

Parameters
axis (*int | None*) – Sum along the specified axis. If axis is None, compute the sum of all the elements.

Returns
An OPTVMLinExpr representing the sum.

Return type
OPTVMLinExpr

Example

```
expr = model.AddMVar((2, 3), name='x') + 1
sum_row = expr.Sum(axis=0) # sum the OPTVLinExpr for each row
sum_col = expr.Sum(axis=1) # sum the OPTVLinExpr for each column
sum_all = expr.Sum()       # sum all OPTVLinExpr objects, return a 0-D␣
↪OPTVMLinExpr object
```

Ndim

The number of dimensions in this OPTVMLinExpr object.

`OPTVMLinExpr.Ndim`

Returns
Number of dimensions in OPTVMLinExpr.

Return type
int

Example

```
expr1 = model.AddMVar((3,), name="x") + 1
print(expr1.Ndim) # 1
expr2 = model.AddMVar((1, 3), name='y') + 1
print(expr2.Ndim) # 2
```

Shape

The shape of this OPTVMLinExpr object.

`OPTVMLinExpr.Shape`

Returns
A tuple of int representing the shape of this OPTVMLinExpr object.

Return type
tuple[int, ...]

Example

```
expr1 = model.AddMVar((3,), name="x") + 1
print(expr1.Shape) # (3,)
expr2 = model.AddMVar((1, 3), name='y') + 1
print(expr2.Shape) # (1, 3)
```

Size

The total number of elements in this OPTVMLinExpr object.

`OPTVMLinExpr.Size`

Returns
The number of elements in OPTVMLinExpr.

Return type
int

Example

```
expr1 = model.AddMVar((3,), name="x") + 1
print(expr1.Size) # 3
expr2 = model.AddMVar((2, 3), name='y') + 1
print(expr2.Size) # 6
```

OPTVConstr

class OPTVConstr

OptVerse constraint object. Used to access constraint data from a model. Constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVModel.AddConstr()* or *OPTVModel.AddConstrs()* interfaces for linear constraints. For quadratic constraints, the *OPTVModel.AddQConstr()* interface is provided.

Get()

Query the value of a constraint attribute.

See *OptVerse MP Attributes*.

`OPTVConstr.Get(attr)`

Parameters
attr (OPTVIntAttr / OPTVDblAttr / OPTVStrAttr) – OptVerse constraint attribute to be queried.

Returns
Attribute value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")
model.Update()
print(c0.Get(OPTVIntAttr.PROHIBITED)) # 0
```

Set()

Set the value of a constraint attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

`OPTVConstr.Set(attr, value)`

Parameters

- **attr** (OPTVIntAttr / OPTVDblAttr / OPTVStrAttr) – OptVerse constraint attribute to be set which may be integer, double, or string.
- **value** (*int* / *float* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")

c0.Set(OPTVDblAttr.UB, 999)
print(c0.Get(OPTVDblAttr.UB)) # 3
model.Update()

print(c0.Get(OPTVDblAttr.UB)) # 999
```

Index()

Return the index of the constraint in its model.

`OPTVConstr.Index()`

Returns
Index of constraint.

Return type
int

Return values

- **-1** – Constraint is not in model.
- **-2** – Constraint has been removed from a model.
- **>=0** – Constraint index in model.

Example

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")
c1 = model.AddConstr(2*x+z, 2, 6, "c1")

print(c0.Index()) # 0
print(c1.Index()) # 1
```

SameAs()

Determines if two constraints are actually the same underlying object.

`OPTVConstr.SameAs(c2)`

Parameters
c2 (OPTVConstr) – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
bool

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

Example

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")
c1 = model.AddConstr(2*x+z, 2, 6, "c1")
copy_c0 = c0

print(c0.SameAs(copy_c0)) # True
print(c1.SameAs(c1))      # False
```

OPTVMConstr

class OPTVMConstr

OptVerse linear matrix constraint object. An OPTVMConstr acts like a `numpy.ndarray` of *OPTVConstr* objects. A user should create linear matrix constraint object using the *OPTVModel.AddMConstr()*, rather than using the default constructor to instantiate them.

FromList()

Generates an OPTVMConstr object from a list of constraints. The shape of the new OPTVMConstr object is directly determined by the input list. A list of *OPTVConstr* objects produces a one-dimensional OPTVMConstr object, a list of lists of *OPTVConstr* objects produces a two-dimensional OPTVMConstr object, etc.

OPTVMConstr.**FromList**(*constr_list*)

Parameters
constr_list (*list*) – list[OPTVConstr]

Returns
A new OPTVMConstr object corresponding to the input *OPTVConstr* objects.

Return type
OPTVMConstr

Example

```
constrs_list = model.GetConstrs()
matrix_constrs = OPTVMConstr.FromList(constrs_list)
```

Get()

Query the value of an attribute for an OPTVMConstr object.

OPTVMConstr.**Get**(*attrname*)

Parameters
attr (OPTVIntAttr / OPTVDblAttr / OPTVCharAttr / OPTVStrAttr) – OptVerse constraint attribute to be queried.

Returns
A `numpy.ndarray` containing the current values of the requested constraint attribute.

Return type
`numpy.ndarray`

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
x = model.AddMVar(3, ub=1, vtype=OPTV_BINARY, name=['x1', 'y1', 'z1'])
constrs = model.AddMConstr(np.array([[1, 2, 3], [1, 1, 0]]), x, [-OPTV_
↪INF, 1], [4, OPTV_INF], name='c')
result = constrs.Get(OPTVDblAttr.UB)
```

Set()

Set the value of the OPTVMConstr attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

`OPTVMConstr.Set(`*attrname*, *newvalue*`)`

Parameters

- **attrname** (OPTVIntAttr / OPTVDblAttr / OPTVCharAttr / OPTVStrAttr) – OptVerse constraint attribute to be set.
- **newvalue** (*int* / *float* / *str*) – Value to be set. The shape must match that of the OPTVMConstr object. Alternatively, a user can provide a scalar argument, which will be automatically promoted to the correct shape.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
x = model.AddMVar(3, ub=1, vtype=OPTV_BINARY, name=['x1', 'y1', 'z1'])
constrs =  model.AddMConstr(np.array([[1, 2, 3], [1, 1, 0]]), x, [-OPTV_
↪INF, 1], [4, OPTV_INF], name='c')
constrs.Set(OPTVDblAttr.LB, -10)
```

ToList()

Convert the OPTVMConstr object to a list of *OPTVConstr* objects.

`OPTVMConstr.ToList()`

Returns
List of constraints contained in this OPTVMConstr.

Return type
list[*OPTVConstr*]

Example

```
x = model.AddMVar(3, ub=1, vtype=OPTV_BINARY, name=['x1', 'y1', 'z1'])
constrs =  model.AddMConstr(np.array([[1, 2, 3], [1, 1, 0]]), x, [-OPTV_
↪INF, 1], [4, OPTV_INF], name='c')
constrs_list = constrs.ToList()
```

OPTVQuadExpr

class OPTVQuadExpr

The *OPTVQuadExpr* class is used to construct quadratic constraints and objective functions. An object of this type is constructed from variables and constant terms. Quadratic expressions are composed of three types of terms: quadratic, linear, and constant. The quadratic terms are stored in containers that hold the first variable, second variable, and coefficient. The linear component is accessed through the internal linear expression object. Indexing in a quadratic expression refers strictly to the quadratic components, with the linear expression being treated independently.

For example, the quadratic expression `1*x*x + 2*x*y + 3*x*y + 4*y*y + 5*x + 6*y + 7*x` has the following description:

Index	Coeff	Var1	Var2
0	1	x	x
1	2	x	y
2	3	x	y
3	4	y	y

To access the linear components of the same expression, use the *OPTVQuadExpr.GetLinExpr()* interface. This will allow accessing the linear component as normal (see *OPTVLinExpr*), i.e.

Index	Coef	Variable
0	5	x
1	6	y
2	7	x

AddConstant()

Add to the constant term.

OPTVQuadExpr.**AddConstant**(*c*)

This function will add to the constant term, not override.

Parameters
c (*float*) – Constant value to add to expression.

Returns
None.

Return type
None

Example

```
quad_expr = x*y + x
quad_expr.AddConstant(3)
print(quad_expr) # x*y + x + 3
```

AddTerm()

Add a single linear term to the end of the quadratic expression.

OPTVQuadExpr.**AddTerm**(*coeff*, *var*)

This function will add the terms coeff ∗ var to the end of the expression.

Parameters

- **coeff** (*float*) – Coefficient values.
- **var** (OPTVVar) – Variable to add to expression.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_ARGUMENT – var is nullptr.

Example

```
quad_expr = x*y + x
quad_expr.AddTerm(3, z)
print(quad_expr) # x*y + x + 3*z
```

Add a single quadratic term to the end of the quadratic expression.

OPTVQuadExpr.**AddTerm**(*coeff*, *var1*, *var2*)

This function will add the terms coeff ∗ var1 ∗ var2 to the end of the expression.

Parameters

- **coeff** (*float*) – Coefficient values.
- **var1** (OPTVVar) – First variable in term.
- **var2** (OPTVVar) – Second variable in term.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_ARGUMENT – var1 or var2 is nullptr.

Example

```
quad_expr = x*y + x
quad_expr.AddTerm(3, z, x)
print(quad_expr) # x*y + x + 3*z*x
```

AddTerms()

Add linear terms to the end of the quadratic expression.

`OPTVQuadExpr.`**`AddTerms`**(*coeff*, *var*, *cnt*)

This function will add the terms $\text{coeff}_i * \text{var}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coeff** (*list[float]*) – List of coefficient values.
- **var** (*list[OPTVVar]*) – List of variables.
- **cnt** (*int*) – Number of elements in the list.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_ARGUMENT – `var` or `coeff` is empty or `cnt` is negative.

Example

```
quad_expr = y*y
quad_expr.AddTerms([1, 2], [x, z], 2)
print(quad_expr) # y*y + x + 2*z
```

Add quadratic terms to the end of the quadratic expression.

`OPTVQuadExpr.`**`AddTerms`**(*coeff*, *var1*, *var2*, *cnt*)

This function will add the terms $\text{coeff}_i * \text{var1}_i * \text{var2}_i$, $\forall i \in [0, \text{cnt}]$ to the end of the expression.

Parameters

- **coeff** (*list[float]*) – List of coefficient values.
- **var1** (*list[OPTVVar]*) – List of first variables.
- **var2** (*list[OPTVVar]*) – List of second variables.
- **cnt** (*int*) – Number of elements in the list.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_ARGUMENT – `var1`, `var2`, or `coeff` is empty or `cnt` is negative.

Example

```
quad_expr = y*y
quad_expr.AddTerms([1, 2], [x, z], [x, z], 2)
print(quad_expr) # y*y + x*x + 2*z*z
```

Clear()

Clears the quadratic expression.

OPTVQuadExpr.**Clear**()

Clears the inner containers, such that *OPTVQuadExpr.Size()* will return `0`. Additionally, this function clears the linear component of the expression, and resets the constant to `0`.

Returns
None.

Return type
None

Example

```
quad_expr = x*y + z
quad_expr.Clear()
print(quad_expr.Size()) # 0
```

GetCoeff()

Get the coefficient for a term at a given index in the quadratic component of the expression.

OPTVQuadExpr.**GetCoeff**(*i*)

Parameters
i (*int*) – Index of the term in the expression.

Returns
Coefficient of term at position `i`.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – If `i` $\notin [0, \texttt{this->Size()}]$.

Example

```
quad_expr = x*y - 2*y*z
coeff1 = quad_expr.GetCoeff(0)
coeff2 = quad_expr.GetCoeff(1)
print(coeff1) # 1
print(coeff2) # -2
```

GetConstant()

Get the constant term from the expression.

OPTVQuadExpr.**GetConstant**()

Returns
Constant value in expression.

Return type
float

Example

```
quad_expr = x*y - 2*y*z + 3
constant = quad_expr.GetConstant()
print(constant) # 3
```

GetLinExpr()

Get the linear components of the expression.

`OPTVQuadExpr.GetLinExpr()`

Returns
Linear component of expression.

Return type
OPTVLinExpr

Example

```
quad_expr = x*y - 2*z + 3
lin_expr = quad_expr.GetLinExpr()
print(lin_expr) # -2*z
```

GetValue()

Get the evaluated value of the quadratic expression.

`OPTVQuadExpr.GetValue()`

Returns
Expression value according to the primal solution.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Value queried for solution is unavailable.

Example

```
model.Optimize()
result = model.GetQObjective()
print(result.GetValue())
```

GetVar1()

Get the first variable of a quadratic term at a specified index in the expression.

`OPTVQuadExpr.GetVar1(i)`

This function will return the first variable in a quadratic term for an expression. For example, an expression `x*x + x*y + y*y` has the following prescription:

Index	Var1	Var2
0	x	x
1	x	y
2	y	y

Parameters
i (*int*) – Index of the quadratic term in the expression.

Returns
First variable at position i.

Return type
OPTVVar

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – If i $\notin [0,$ this->Size()$]$.

Example

```
quad_expr = x1*z + *x2*y
print(quad_expr.GetVar1(1)) # x2
print(quad_expr.GetVar2(1)) # y
```

GetVar2()

Get the second variable of a quadratic term at a specified index in the expression. Refer to *OPTVQuadExpr.GetVar1()* for a description of indexing.

OPTVQuadExpr.**GetVar2**(*i*)

This function will return the second variable in a quadratic term for an expression.

Parameters
i (*int*) – Index of the quadratic term in the expression.

Returns
Second variable at position i.

Return type
OPTVVar

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – If i $\notin [0,$ this->Size()$]$.

Example

```
quad_expr = x1*z + *x2*y
print(quad_expr.GetVar1(1)) # x2
print(quad_expr.GetVar2(1)) # y
```

Remove()

Remove term(s) from the quadratic terms.

`OPTVQuadExpr.Remove(i)`

Remove the term at a specified index from the expression. This interface does not allow removing terms from the linear component.

Parameters
i (*int*) – Index of term to be removed.

Returns
None.

Return type
None

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – If i $\notin [0,$ this->Size()$]$.

Example

```
quad_expr = x*x + x*y + y*y
quad_expr.Remove(1)
print(quad_expr) # x*x + y*y
```

Remove variable from the expression.

`OPTVQuadExpr.Remove(v)`

Remove any instance of a variable from the expression. This interface will remove any occurrence of the variable from the linear component of the expression.

Parameters
v (OPTVVar) – Variable to be removed.

Returns
None.

Return type
None

Raises
OPTVErrCode.NOT_IN_MODEL – If variable is not in a model.

Example

```
quad_expr = x*x + x + y*y
quad_expr.Remove(x)
print(quad_expr) # y*y
```

SameAs()

Determines if two quadratic expressions are actually the same underlying expression.

`OPTVQuadExpr.SameAs(e2)`

Parameters
e2 (OPTVQuadExpr) – Quadratic expression to check sameness against.

Returns
Quadratic expression sameness.

Return type
bool

Return values

- **true** – Quadratic expressions are the same.
- **false** – Quadratic expressions are not the same.

Example

```
quad_expr = x*x + x + y*y
quad_expr1 = x*z + y + 1
quad_expr2 = quad_expr
print(quad_expr.SameAs(quad_expr1)) # False
print(quad_expr.SameAs(quad_expr2)) # True
```

Size()

Returns the size of the expression.

`OPTVQuadExpr.Size()`

This value corresponds to the number of **quadratic** elements in the expression, regardless of whether the element is repeated. The expression `x*x + y*y + y` has `Size = 2`. The expression `x*x + x*x + y*y + y` has `Size = 3`.

Returns
Size of the quadratic expression.

Return type
int

Example

```
quad_expr = x*x + x + y*y
print(quad_expr.Size()) # 2
```

OPTVQConstr

class OPTVQConstr

OptVerse quadratic constraint object. Used to access quadratic constraint data from a model. Quadratic constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVModel.AddQConstr()*.

Warning

Quadratic constraints must satisfy the condition that their matrix representation are positive semi-definite or OptVerse will throw a *OPTVErrCode.MATRIX_NOT_PSD* exception.

Get()

Query the value of a quadratic constraint attribute.

See *OptVerse MP Attributes*.

OPTVQConstr.Get(attr) const

Parameters
attr (OPTVIntAttr / OPTVCharAttr / OPTVDblAttr / OPTVStrAttr) – OptVerse quadratic constraint attribute to query.

Returns
Attribute value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
model.Update()
print(c0.Get(OPTVIntAttr.PROHIBITED)) # 0
```

Set()

Set the value of a quadratic constraint attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

OPTVQConstr.**Set**(*attr*, *value*)

Parameters

- **attr** (OPTVIntAttr / OPTVCharAttr / OPTVDblAttr / OPTVStrAttr) – OptVerse quadratic constraint attribute to be set.
- **value** (*int* / *float* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
c0.Set(OPTVStrAttr.NAME, "dummy")
model.Update()
print(c0.Get(OPTVStrAttr.NAME)) # dummy
```

Index()

Return the index of the quadratic constraint in its model.

`OPTVQConstr.Index()`

Returns
Index of quadratic constraint.

Return type
int

Return values

- **-1** – Quadratic constraint is not in model.
- **-2** – Quadratic constraint has been removed from a model.
- **>=0** – Quadratic constraint index in model.

Example

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
c1 = model.AddQConstr(3*y*y + z, -OPTV_INF, 6, "c1")
print(c0.Index()) # 0
print(c1.Index()) # 1
```

SameAs()

Determines if two quadratic constraints are actually the same underlying object.

`OPTVQConstr.SameAs(c2)`

Parameters
c2 (OPTVQConstr) – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
bool

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

Example

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
c1 = c0
invalid = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "Invalid")

c0.SameAs(c1)      # True
c0.SameAs(invalid) # False
```

OPTVAffineExpr

class OPTVAffineExpr

The *OPTVAffineExpr* class serves as the foundation for constructing conic constraints by encapsulating the affine transformation $Fx + g$. An *OPTVAffineExpr* object contains:

- **Linear coefficients**: The sparse representation of matrix F, storing non-zero coefficients and their corresponding variable indices in Coordinate (COO) sparse format. Affine expressions are constructed by adding rows of linear expressions (the COO storage is internal, users will provide data through SDK objects).
- **Model variables**: The sparse representation of the vector x. Users only need to provide the participating variables, through adding *linear expression* objects as rows.
- **Constant terms**: The offset vector g that shifts the linear combination.

When creating an *OPTVCone*, the *OPTVAffineExpr* provides the mathematical structure (the affine transformation), while the cone type parameter specifies the geometric constraint C that the expression must satisfy. This separation allows for flexible composition of conic constraints from simpler affine building blocks.

See *OPTVCone*.

AddRow()

Add a single row to the affine expression.

OPTVAffineExpr.**AddRow**(*expr*)

Parameters
expr (OPTVLinExpr) – Linear expression to add to the affine expression.

Returns
None.

Return type
None

AddRows()

Add multiple rows to the affine expression.

OPTVAffineExpr.**AddRows**(*expr*)

Parameters
expr (*list[OPTVLinExpr]*) – List of linear expressions to add to the affine expression. Each expression corresponds to a row in the affine expression.

Returns
None.

Return type
None

GetColumnIndices()

Returns the column indices of the nonzero entries in the affine expression. Each value corresponds to the column index for a nonzero entry. The affine expression is internally stored in COO (Coordinate) format, where each nonzero entry is identified by its row index, column index, and value.

OPTVAffineExpr.**GetColumnIndices**()

Returns
A vector of integers, where each element corresponds to the column index of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr.GetRowIndices()* and *OPTVAffineExpr.GetValues()*.

GetOffset()

OPTVAffineExpr.**GetOffset**(*index*)

Parameters
index (*int*) – Index of the row whose offset is to be queried.

Returns
A float corresponding to the offset for the row at `index`.

Return type
float

Raises
IndexError – If `index` is larger than the number of rows in the affine expression.

GetOffsets()

OPTVAffineExpr.**GetOffsets**()

Returns
A list of doubles, where each element is the offset for a specific row added to the affine expression.

Return type
list[float]

GetRowIndices()

Returns the row indices of the nonzero entries in the affine expression. Each value corresponds to the row index for a nonzero entry. The affine expression is internally stored in COO (Coordinate) format, where each nonzero entry is identified by its row index, column index, and value.

`OPTVAffineExpr.GetRowIndices()`

Returns
A list of integers, where each element corresponds to the row index of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr.GetColumnIndices()* and *OPTVAffineExpr.GetValues()*.

Return type
list[int]

GetValues()

Returns the coefficient values of the nonzero entries in the affine expression. Each value corresponds to the coefficient stored at the position defined by the matching row and column index from *OPTVAffineExpr.GetRowIndices()* and *OPTVAffineExpr.GetColumnIndices()*.

`OPTVAffineExpr.GetValues()`

Returns
A list of floats, where each element is the coefficient of a nonzero entry. The order of elements aligns with the order returned by *OPTVAffineExpr.GetRowIndices()* and *OPTVAffineExpr.GetColumnIndices()*.

Return type
list[float]

SetOffset()

Set the offset for a specific row, based on the index.

`OPTVAffineExpr.SetOffset(`*index*, *value*`)`

Parameters

- **index** (*int*) – Index of the row whose offset is to be set.
- **value** (*float*) – Value for the offset to set.

Returns
None.

Return type
None

Raises
IndexError – If `index` is larger than the number of rows in the affine expression.

Size()

Get the size of the affine expression.

`OPTVAffineExpr.Size()`

This value corresponds to the number of elements in the affine expression, regardless of whether the element is repeated. For example, if the same linear expression is added multiple times, the size will keep growing.

Returns
Size of the affine expression.

Return type
int

OPTVCone

class OPTVCone

OptVerse cone object. Used to access conic constraint data from a model. Cones should not be created directly through their constructor. Instead, a user should add a cone constraint to a model using the *OPTVModel.AddCone()* interface.

A conic constraint is mathematically defined as:

$$Fx + g \in C$$

Where:

- F is an $m \times n$ coefficient matrix that linearly transforms the optimization variables,
- x is the n-dimensional vector of optimization variables,
- g is an m-dimensional constant offset vector (right-hand side),
- C is the cone that constrains the resulting affine expression.
- The expression $Fx + g$ produces an m-dimensional vector that must belong to cone C.

The matrix F is stored in Coordinate (COO) sparse format for memory efficiency, containing only non-zero entries as (row, column, value) triplets. Users will construct $Fx + g$ as an *affine expression*, by adding rows through *linear expressions*.

OptVerse supports the following cone types:

- *Second-order cone*
- *Rotated second-order cone*

See *OPTVAffineExpr* and *OPTVConeType*.

Get()

Query the value of a cone constraint attribute.

`OPTVCone.Get(attr)`

Parameters
attr (OPTVStrAttr) – String type attribute.

Returns
Attribute value.

Return type
str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

GetType()

Get the type of the cone constraint.

See *OPTVConeType*.

`OPTVCone.GetType()`

Returns
Type of the cone constraint.

Return type
OPTVConeType

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – Cone constraint has been removed from a model.

Index()

Return the index of the cone constraint in its model.

`OPTVCone.Index()`

Returns
Index of the cone constraint in the model.

Return type
int

Return values

- **-1** – Cone constraint is not part of any model.
- **-2** – Cone constraint has been removed from a model.
- **>=0** – Cone constraint index in model.

SameAs()

Determines if two cone constraints are actually the same underlying object.

`OPTVCone.SameAs(c2)`

Parameters
c2 (OPTVCone) – Cone constraint to check sameness against.

Returns
Cone constraint sameness.

Return type
bool

Return values

- **true** – Cone constraints are the same
- **false** – Cone constraints are not the same

Set()

Set the value of a cone constraint attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

OPTVCone.**Set**(*attr*, *value*)

Parameters

- **attr** (OPTVStrAttr) – String type attribute.
- **value** (*str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

OPTVSOS

class OPTVSOS

OptVerse SOS constraint object. Used to access SOS constraint data from a model. SOS constraints should not be created through their constructor. Instead, a user should add an SOS constraint to a model using the *OPTVModel.AddSOS()*.

A special ordered set (SOS) is an additional way to specify integrality conditions in a model. In particular, a special ordered set is a way to restrict the number of nonzero solution values among a specified set of variables in a model.

There are various types of SOS:

- SOS Type 1 is a set of variables where at most one variable may be nonzero.
- SOS Type 2 is a set of variables where at most two variables may be nonzero. If two variables are nonzero, they must be adjacent in the set.

Get()

Query the value of an SOS constraint attribute.

See *OptVerse MP Attributes*.

OPTVSOS.**Get**(*attr*)

Parameters
attr (OPTVIntAttr / OPTVStrAttr) – OptVerse SOS constraint attribute to query.

Returns
Attribute value.

Return type
int|str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
# query the type of the SOS constraint
result = sos.Get(OPTVIntAttr.SOS_TYPE)
```

Set()

Set the value of an SOS constraint attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

OPTVSOS.**Set**(*attr*, *value*)

Parameters

- **attr** (OPTVIntAttr / OPTVStrAttr) – OptVerse SOS attribute to be set.
- **value** (*int* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
sos.Set(OPTVStrAttr.NAME, "dummy")
model.Update()
print(sos.Get(OPTVStrAttr.NAME)) # dummy
```

Index()

Return the index of the SOS constraint in its model.

OPTVSOS.**Index**()

Returns
Index of SOS constraint.

Return type
int

Return values

- **-1** – SOS constraint is not in model.
- **-2** – SOS constraint has been removed from a model.
- **>=0** – SOS constraint index in model.

Example

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
result = sos.Index()
```

SameAs()

Determines if two SOS constraints are actually the same underlying object.

OPTVSOS.**SameAs**(*c2*)

Parameters
c2 (OPTVSOS) – SOS constraint to check sameness against.

Returns
SOS constraint sameness.

Return type
bool

Return values

- **true** – SOS constraints are the same.
- **false** – SOS constraints are not the same.

Example

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
sos1 = sos
invalid = OPTVSOS()

sos.SameAs(sos1)    # True
sos.SameAs(invalid) # False
```

OPTVGenConstr

class OPTVGenConstr

OptVerse general constraint object. Used to access general constraint data from a model. General constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the `OPTVModel.AddGenConstrXxx`.

Table 15: Supported General Constraints

Constraint	Type	Interface	Description
And	*OPTV_GENCONSTR_A*	*OPTVModel.AddGenConstrAnd()*	Resultant variable is non-zero if all supplied variables are non-zero.
Or	*OPTV_GENCONSTR_O*	*OPTVModel.AddGenConstrOr()*	Resultant variable is non-zero if any of the supplied variables are non-zero.
Indicator	*OPTV_GENCONSTR_I*	*OPTVModel.AddGenConstrIndicator*	Binary variable is non-zero if and only if the associated logical condition is satisfied.

Get()

Query the value of a general constraint attribute.

See *OptVerse MP Attributes*.

`OPTVGenConstr.Get(attr)`

Parameters
attr (OPTVIntAttr/OPTVDblAttr/OPTVStrAttr) – OptVerse constraint attribute to be queried.

Returns
Attribute value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
# query the general constraint type
print(c0.Get(OPTVIntAttr.GENCONSTR_TYPE))
```

Set()

Set the value of a general constraint attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

OPTVGenConstr.**set**(*attr*, *value*)

Parameters

- **attr** (OPTVDblAttr / OPTVStrAttr) – OptVerse constraint attribute to be set.
- **value** (*float* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute for constraints is provided.

Example

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
c0.Set(OPTVStrAttr.NAME, "dummy")
model.Update()
print(c0.Get(OPTVStrAttr.NAME)) # dummy
```

Index()

Return the index of the general constraint in its model.

OPTVGenConstr.**Index**()

Returns
Index of constraint.

Return type
int

Return values

- **-1** – Constraint is not in model.
- **-2** – Constraint has been removed from a model.
- **>=0** – Constraint index in model.

Example

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
c0.Index()
```

SameAs()

Determines if two general constraints are actually the same underlying object.

`OPTVGenConstr.SameAs(c2)`

Parameters
c2 (OPTVGenConstr) – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
bool

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

Example

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
c1 = c0
invalid = model.AddGenConstrOr(x3, [x1, x2], "Or")

c0.SameAs(c1)      # True
c0.SameAs(invalid) # False
```

OPTVModel

class OPTVModel

OptVerse optimization model object.

The OPTVModel class is the central object in OptVerse for building and solving optimization models. It stores all variables, constraints, objective function, and solver settings.

A model has a **lazy update** behaviour: changes to the model (adding variables, constraints, modifying attributes, etc.) are not immediately reflected in the underlying solver. Call *OPTVModel.Update()* to synchronize the model state with the solver before optimizing.

OPTVModel()

`OPTVModel.OPTVModel()`

Create an empty model with an auto-managed environment.

`OPTVModel.OPTVModel(env)`

Create an empty model associated with the given environment.

Parameters
env (OPTVEnv) – OptVerse environment for the model.

Raises
TypeError – env is not an *OPTVEnv*.

OPTVModel.**OPTVModel**(*filename*)

Create a model by reading from a file, using an auto-managed environment.

Parameters
filename (*str*) – Path to the model file (MPS, LP, etc.).

Raises

- **TypeError** – `filename` is not a `str`.
- ***OPTVErrCode.ERROR_FILE_READ*** – Failed to read the model from file.

OPTVModel.**OPTVModel**(*env*, *filename*)

Create a model by reading from a file.

Parameters

- **env** (OPTVEnv) – OptVerse environment for the model.
- **filename** (*str*) – Path to the model file (MPS, LP, etc.).

Raises

- **TypeError** – Argument types do not match `(OPTVEnv, str)`.
- ***OPTVErrCode.ERROR_FILE_READ*** – Failed to read the model from file.

OPTVModel.**OPTVModel**(*env*, *content*, *format*)

Create a model by reading from a string.

Parameters

- **env** (OPTVEnv) – OptVerse environment for the model.
- **content** (*str*) – Model content as a string.
- **format** (*str*) – Model format (e.g., *"mps"*, *"lp"*).

Raises

- **TypeError** – Argument types do not match `(OPTVEnv, str, str)`.
- ***OPTVErrCode.ERROR_FILE_READ*** – Failed to read the model from string.

AddCone()

Add a single cone constraint to the model.

OPTVModel.**AddCone**(*affine*, *type*, *name=''*)

This function will add a single cone constraint to the model.

Parameters

- **affine** (OPTVAffineExpr) – The affine expression, $Fx + g$, used to specify the cone constraint.

- **type** (OPTVConeType) – Type of cone constraint (e.g. second-order cone, rotated second-order cone).
- **name** (*str*) – Cone constraint name.

Returns
Cone constraint object added to the model.

Return type
OPTVCone

Raises
OPTVErrCode.INVALID_ARGUMENT – `type` is not one of *OPTVConeType.SOC* or *OPTVConeType.RSOC*. The provided affine expression is empty. If `type` is *OPTVConeType.RSOC* and the affine expression size is less than *3*.

AddConstr()

Add a single linear constraint to the model.

OPTVModel.**AddConstr**(*expr*, *lhs*, *rhs*, *name=''*)

This function will add a single constraint to the model. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel.AddConstrs()*) is preferred.

Parameters

- **expr** (OPTVLinExpr) – Linear expression describing the constraint.
- **lhs** (*float*) – Lower bound of constraint.
- **rhs** (*float*) – Upper bound of constraint.
- **name** (*str*) – Constraint name.

Returns
Constraint object added to the model.

Return type
OPTVConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

Example

```
# add a single linexpr constraint: 1 <= x+y <= 10
model.AddConstr(x+y, 1, 10, "c0")
```

OPTVModel.**AddConstr**(*constr*, *name=''*)

This method accepts a *OPTVTempConstr* object as the primary argument and optionally a name for the constraint as the second argument. The *OPTVTempConstr* object represents a constraint formed by applying comparison operators (e.g., <=, >=, or ==) between expression objects and/or numeric values It serves as a unified interface for adding both linear and general constraints.

Parameters

- **constr** (OPTVTempConstr) – *OPTVTempConstr* object.

- **name** (*str*) – [**optional**] Constraint name. When no name is provided, the default name format `C_num` is used, with `num` being the current constraint count in the model.

Returns
Constraint object added to the model. This can be an *OPTVConstr*, or an *OPTVGenConstr*.

Return type
OPTVConstr | *OPTVGenConstr*

Raises
OPTVErrCode.INVALID_ARGUMENT – numeric value is `NaN`, or constraint with `name` already exists in the model.

Example

```
# add a single linexpr constraint
model.AddConstr(x + y <= 1, "c0")

# add a single general constraint
model.AddConstr(z == And_([x, y, w]), name="c1")
```

AddConstrs()

Add multiple linear constraints to the model.

`OPTVModel.`**`AddConstrs`**(*expr*, *lhs*, *rhs*, *names*, *count*)

This function will add multiple linear constraints to the model. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **expr** (*list[OPTVLinExpr]*) – Linear expressions describing the constraints.
- **lhs** (*list[float]*) – Constraint lower bounds.
- **rhs** (*list[float]*) – Constraint upper bounds.
- **names** (*list[str]*) – Constraint names.
- **count** (*int*) – [**optional**, defaults to length of parameters] Number of constraints to be added to the model. Historically, this was a required parameter. However, past v1.2.6 users are encouraged to omit this parameter. If the `count` parameter is omitted, it will be inferred from the size of the other parameters.

Returns
List of constraint objects added to the model.

Return type
list[*OPTVConstr*]

Raises
OPTVErrCode.INVALID_ARGUMENT – `names` is empty, numeric value is `NaN`, or a constraint with a name in `names` already exists in the model.

Example

```
# add multiple linear constraints
model.AddConstrs([x+y, x+z, 2*x], [1, 2, 3], [6, 7, 8], ["c0", "c1", "c2
↪"])
```

OPTVModel.**AddConstrs**(*constrs*, *name=''*)

Add multiple constraints to a model by means of a Python generator expression.

The method accepts a Python generator expression as its first argument, which iteratively provides values to construct OptVerse constraints. A new constraint is added to the model for each iteration.

This method provides support for adding linear constraints and general constraints to the model.

If you specify a name for the constraint, the generator expression must be wrapped in parentheses. This syntactic requirement is enforced by the Python language.

Parameters

- **constrs** (*generator*) – A generator expression that yields one constraint per iteration.
- **name** (*str*) – Constraint name. When no name is provided, the default name format C_num is used, with num being the current constraint count in the model.

Returns

List of constraint objects added to the model.

Return type

list[*OPTVConstr*] | list[*OPTVGenConstr*]

Raises

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

Example

```
model.AddConstrs((x[i,j] == 0 for i in range(4)
                          for j in range(4)
                          if i != j), name='c')

model.AddConstrs(z[i] == max_([x[i], y[i]]) for i in range(5))
```

AddQConstr()

Add a single quadratic constraint to the model.

OPTVModel.**AddQConstr**(*expr*, *lhs*, *rhs*, *name=''*)

This function will add a single quadratic constraint to the model.

Parameters

- **expr** (OPTVQuadExpr) – Quadratic expression describing the constraint.
- **lhs** (*float*) – Lower bound of constraint.
- **rhs** (*float*) – Upper bound of constraint.
- **name** (*str*) – Constraint name.

Returns

Constraint object added to the model.

Return type

OPTVQConstr

Raises

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

Example

```
# add a single quadratic constraint: x*x + 2*y +1 <= 100
model.AddQConstr(x*x+2*y+1, -OPTV_INF, 100, name="qconstr")
```

OPTVModel.**AddQConstr**(*constr*, *name=''*)

This method accepts a *OPTVTempConstr* object as the primary argument and optionally a name for the constraint as the second argument. The *OPTVTempConstr* object represents a constraint formed by applying comparison operators (e.g., <=, >=, or ==) between expression objects and/or numeric values.

Parameters

- **constr** (OPTVTempConstr) – *OPTVTempConstr* object.
- **name** (*str*) – Constraint name. When no name is provided, the default name format C_num is used, with num being the current constraint count in the model.

Returns

Constraint object added to the model.

Return type

OPTVQConstr

Raises

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

Example

```
# add a single quadratic constraint: x*x + 2*y +1 <= 100
model.AddQConstr(x*x + 2*y + 1 <= 100, name="qconstr")
```

AddGenConstrAnd()

Add a single AND constraint to the model.

OPTVModel.**AddGenConstrAnd**(*resultant*, *conVars*, *name=''*)

This function will add a single AND constraint to the model.

Parameters

- **resultant** (OPTVVar) – The resultant binary variable of constraint.
- **conVars** (*list[OPTVVar]*) – Array of variables that must be non-zero for resultant to be non-zero.
- **name** (*str*) – Constraint name.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

Example

```
model.AddGenConstrAnd(x3, [x1, x2], "and")
```

AddGenConstrOr()

Add a single OR constraint to the model.

OPTVModel.**AddGenConstrOr**(*resultant*, *conVars*, *name=''*)

This function will add a single OR constraint to the model.

Parameters

- **resultant** (OPTVVar) – The resultant binary variable of constraint.
- **conVars** (*list[OPTVVar]*) – Array of variables that must include at least one non-zero variable for resultant to be non-zero.
- **name** (*str*) – Constraint name.

Returns
General constraint object added to the model.

Return type
OPTVGenConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

Example

```
model.AddGenConstrOr(x3, [x1, x2], "or")
```

AddGenConstrIndicator()

Add a single INDICATOR constraint to the model.

OPTVModel.**AddGenConstrIndicator**(*binVar*, *binVal*, *expr*, *sense*, *rhs*, *name=''*)

This function will add a single INDICATOR constraint to the model.

Parameters

- **binVar** (OPTVVar) – The binary indicator variable.
- **binVal** (*float*) – The value of the indicator variable that would force the constraint to be satisfied (0 or 1).
- **expr** (OPTVLinExpr) – Expression for the linear constraint triggered by the indicator.
- **sense** (*str*) – Sense for the linear constraint. One of *OPTV_SENSE_LESS*, *OPTV_SENSE_EQUAL*, or *OPTV_SENSE_GREATER*.
- **rhs** (*float*) – Right-hand side of the linear constraint.
- **name** (*str*) – Constraint name.

Returns
General constraint object added to the model.

Return type
OPTVGenConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

Example

```
# x3 = 1 -> x2 + x1 + 1 <= 10
model.AddGenConstrIndicator(x3, 1, x2+x1+1, OPTV_SENSE_LESS, 10, name=
↪"indicator")
```

AddGenConstrAbs()

Add a single `ABS` constraint to the model.

An ABS constraint of the form $r = \text{abs}(x)$ requires that the variable r is equal to the absolute value of the variable x. This constraint enforces the relationship $r = |x|$, where $r \geq 0$.

`OPTVModel.`**`AddGenConstrAbs`**(*res_var*, *input_var*, *name=''*)

This function will add a single `Abs` constraint to the model.

Parameters

- **`res_var`** (`OPTVVar`) – The variable representing the absolute value of the input variable.
- **`input_var`** (`OPTVVar`) – The variable for which the absolute value will be taken.
- **`name`** (*`str`*) – Constraint name.

Returns
General constraint object added to the model.

Return type
OPTVGenConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

Example

```
model.AddGenConstrAbs(x1, x2, "abs")
```

AddGenConstrNorm()

Add a single `NORM` constraint to the model.

A NORM constraint of the form $r = \text{norm}\{x_1, \ldots, x_n\}$ requires that the variable r is equal to the vector norm of the argument vector $x_1, \ldots, x_n$.

`OPTVModel.`**`AddGenConstrNorm`**(*res_var*, *vars_list*, *norm_type*, *name=''*)

This function will add a single `NORM` constraint to the model.

Parameters

- **res_var** (OPTVVar) – The variable representing the vector norm of the other variables.
- **vars_list** (*list[OPTVVar]*) – Variables on which the vector norm is computed.
- **norm_type** (*int*) – Select the norm type to use. Valid options are: -1 for the L-infinity norm (also denoted as L^∞ norm), 0 for the L^0 norm, 1 for the L^1 norm.
- **name** (*str*) – Constraint name.

Returns
General constraint object added to the model.

Return type
OPTVGenConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

Example

```
model.AddGenConstrNorm(x1, [x2, x3, x4], 0, "norm")
```

AddGenConstrMax()

Add a single `MAX` constraint to the model.

A MAX constraint of the form $r = \max\{x_1, \ldots, x_n\}$ ensures that the variable r is equal to the maximum value among the argument vector $x_1, \ldots, x_n$.

`OPTVModel.`**`AddGenConstrMax`**(*resultant*, *var_list*, *name=''*, *constant=-OPTV_INF*)

This function will add a single `MAX` constraint to the model.

Parameters

- **resultant** (OPTVVar) – The resultant variable whose value will be equal to the maximum of the other variables.
- **var_list** (*list[OPTVVar]*) – An array of variables from which the maximum value is taken.
- **name** (*str*) – Constraint name.
- **constant** (*float*) – [**optional**] A constant value to be included in the list of arguments for the MAX operation.

Returns
General constraint object added to the model.

Return type
OPTVGenConstr

Raises
OPTVErrCode.INVALID_ARGUMENT – `name` is empty, numeric value is `NaN`, or constraint with `name` already exists in the model.

Example

```
# x1 = max([x2, x3, x4])
model.AddGenConstrMax(x1, [x2, x3, x4] name="max")
```

AddGenConstrMin()

Add a single MIN constraint to the model.

A MIN constraint of the form $r = \min\{x_1, \ldots, x_n\}$ ensures that the variable r is equal to the minimum value among the argument vector $x_1, \ldots, x_n$.

OPTVModel.**AddGenConstrMin**(*resultant*, *var_list*, *name=''*, *constant=OPTV_INF*)

This function will add a single MIN constraint to the model.

Parameters

- **resultant** (OPTVVar) – The resultant variable whose value will be equal to the minimum of the other variables.
- **var_list** (*list[OPTVVar]*) – An array of variables from which the minimum value is taken.
- **name** (*str*) – Constraint name.
- **constant** (*float*) – [**optional**] A constant value to be included in the list of arguments for the MIN operation.

Returns

General constraint object added to the model.

Return type

OPTVGenConstr

Raises

OPTVErrCode.INVALID_ARGUMENT – name is empty, numeric value is NaN, or constraint with name already exists in the model.

Example

```
# x1 = min([x2, x3, x4])
model.AddGenConstrMin(x1, [x2, x3, x4] name="min")
```

AddSOS()

Add an SOS constraint to the model.

OPTVModel.**AddSOS**(*vars*, *weights*, *type*, *name=''*)

Parameters

- **vars** (*list[OPTVVar]*) – Array of variables that participate in SOS.
- **weights** (*list[float]*) – Array of weights of variables in SOS.
- **type** (*int*) – SOS type. One of 1 or 2.
- **name** (*str*) – Name of the SOS constraint.

Returns

SOS constraint.

Return type

OPTVSOS

Raises

- *OPTVErrCode.INVALID_ARGUMENT* – `type` is not 1 or 2.
- *OPTVErrCode.NOT_IN_MODEL* – A variable in the `vars` array is not part of the model.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – The model has an invalid index. This may happen if trying access a model after its environment has gone out of scope.

Example

```
# At most one variable can be non-zero
model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos1")
# At most two adjacent variables can be non-zero
model.AddSOS([x, y, z], [1, 2, 3], 2, name="sos2")
```

AddVar()

Add a single variable to the model.

OPTVModel.**AddVar**(*lb*, *ub*, *obj*, *vtype*, *name=''*)

This function will add a single variable to the model. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel.AddVars()*) is preferred.

Parameters

- **lb** (*float*) – Lower bound of variable.
- **ub** (*float*) – Upper bound of variable.
- **obj** (*float*) – Linear coefficient in objective function.
- **vtype** (*str*) – Variable type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **name** (*str*) – Variable name.

Returns
Variable object added to the model.

Return type
OPTVVar

Raises
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `name` is empty, a numeric value is `NaN`, or variable with `name` already exists in the model.

Example

```
# add a binary variable
model.AddVar(0, 1, 0, OPTV_BINARY, name="x")
```

OPTVModel.**AddVar**(*lb=0.0*, *ub=OPTV_INF*, *obj=0.0*, *type=OPTV_CONTINUOUS*, *col=None*, *name=''*)

This function will add a single variable to the model. The col parameter is optional. If a *OPTVColumn* object is provided, it specifies the constraints in which the new variable participates and the corresponding coefficients; if None, the variable is not associated with any constraints. In most circumstances, the difference is negligible, but when adding many elements at once, the bulk form (*OPTVModel.AddVars()*) is preferred.

Parameters

- **lb** (*float*) – [**optional**] Lower bound of variable.

- **ub** (*float*) – [**optional**] Upper bound of variable.
- **obj** (*float*) – [**optional**] Linear coefficient in objective function.
- **type** (*str*) – [**optional**] Variable type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** (OPTVColumn) – [**optional**] Column object to set constraint coefficients.
- **name** (*str*) – [**optional**] Variable name. When no name is provided, the default name format `v_num` is used, with `num` being the current variable count in the model.

Returns
Variable object added to the model.

Return type
OPTVVar

Raises
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, a numeric value is `NaN`, or variable with `name` already exists in the model.

Example

```
model.AddVar()                                      # var name is "v_0"
model.AddVar(vtype=OPTV_BINARY, obj=1.0, name="y") # var name is "y"

# add a binary variable and set its coefficient for pre-existing␣
↪constraint
col = OPTVColumn()
col.AddTerms(3, c0)
model.AddVar(0, 1, 0, OPTV_BINARY, col, name="x")
```

AddMVar()

Add an OPTVMVar object to a model. An OPTVMVar acts like a `numpy.ndarray` of OPTVVar objects.

`OPTVModel.`**`AddMVar`**(*shape*, *lb=0.0*, *ub=OPTV_INF*, *obj=0.0*, *vtype=OPTV_CONTINUOUS*, *name=''*)

Parameters

- **shape** – An int, or tuple of int. The shape of the array.
- **lb** – Optional parameter. Lower bound(s) of variables.
- **ub** – Optional parameter. Upper bound(s) of variables.
- **obj** – Optional parameter. Linear coefficients in objective function.
- **vtype** – Optional parameter. Variables type. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **name** – Optional parameter. Variables name.

The values of the lb, ub, obj, and vtype parameters can either be scalars, lists, or ndarrays. Their shapes should match the shape of the new OPTVMVar object, or they should be broadcastable to the given shape.

The name parameter can either be a single string, used as a common base name that will be suffixed for each variable by its indices, or an ndarray of strings matching the shape of the new OPTVMVar object.

Returns
A new OPTVMVar object.

Return type
OPTVMVar

Example

```
# Add a 2-by-3 OPTVMVar
model.AddMVar((2, 3), name="x")
# Add a vector of three variables with non-default lower bounds
model.AddMVar((3), lb=[1, 2, 3], name="y")
```

AddVars()

Add multiple variables to the model.

OPTVModel.**AddVars**(*lb*, *ub*, *obj*, *vtype*, *names*, *count=None*)

This function will add multiple variables to the model. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **lb** (*list[float]*) – List of variable lower bounds.
- **ub** (*list[float]*) – List of variable upper bounds.
- **obj** (*list[float]*) – List of linear coefficients in objective function.
- **vtype** (*list[str]*) – List of variable types. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **names** (*list[str]*) – List of variable names.
- **count** (*int*) – [**optional**, defaults to length of parameters] Number of variables to be added to the model. Historically, this was a required parameter. However, past v1.2.6 users are encouraged to omit this parameter. If the `count` parameter is omitted, it will be inferred from the size of the other parameters.

Returns
List of variables added to the model.

Return type
list[*OPTVVar*]

Raises
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `name` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

Example

```
# add multiple variables
model.AddVars([0]*3, [1]*3, [0]*3, [OPTV_BINARY]*3, [col_x, col_y, col_
↪z], name=["x", "y", "z"])
```

OPTVModel.**AddVars**(*lb*, *ub*, *obj*, *vtype*, *col*, *names*, *count=None*)

This function will add multiple variables to the model, allowing to set coefficients for pre-existing constraints. In most circumstances, the difference is negligible, but when adding many elements at once, this form is preferred.

Parameters

- **lb** (*list[float]*) – List of variable lower bounds.
- **ub** (*list[float]*) – List of variable upper bounds.
- **obj** (*list[float]*) – List of linear coefficients in objective function.
- **type** (*list[str]*) – List of variable types. One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** (*list[OPTVColumn]*) – Column objects.
- **names** (*list[str]*) – List of variable names.
- **count** (*int*) – Number of variables to be added to the model.

Returns
List of variables added to the model.

Return type
list[*OPTVVar*]

Raises
OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `names` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

Example

```
# add multiple variables and set coefficients for pre-existing␣
↪constraints
col_x = OPTVColumn()
col_x.addTerm(1, c0)
col_y = OPTVColumn()
col_y.addTerm(1, c0)
col_z = OPTVColumn()
col_z.addTerm(1, c0)
model.AddVars([0]*3, [1]*3, [0]*3, [OPTV_BINARY]*3, [col_x, col_y, col_
↪z], name=["x", "y", "z"])
```

OPTVModel.**AddVars**(**indices*, *lb=0.0*, *ub=OPTV_INF*, *obj=0.0*, *vtype=OPTV_CONTINUOUS*, *col=None*, *names=''*, *count=None*)

This function will add multiple variables to the model, allowing to set coefficients for pre-existing constraints.

It returns a `OPTVTupleDict` object that holds the variables created. The keys in the `OPTVTupleDict` object correspond to the values in the `indices` arguments. Multiple argument forms are supported, as detailed in the following section.

The first arguments specify the indices that will serve as keys for accessing the variables in the returned `OPTVTupleDict`. In its simplest form, you provide one or more integer values, and the method creates a multi-dimensional array of variables, analogous to a dense array in traditional programming. For example, `x = model.AddVars(2, 2, name="x")` creates four variables, with names: `x_0_0, x_0_1, x_1_0, x_1_1`.

When lists of immutable objects are provided, variables are created for all combinations of their elements. For example, `x = model.AddVars([3, 7], ['a', 'b', 'c'], name="x")` creates six variables, with names: `x_3_a, x_7_a`, etc.

You can also provide a custom list of tuples as indices. For example, `x = model.addVars([(3,'a'), (3, 'b'), (7,'b'), (7,'c')], name="x")` creates variables accessible in the same way as in the previous example (e.g., `x_3_a, x_7_c`, etc.), but only for the specified combinations. This approach is commonly used to implement sparse indexing.

Note that while indices can be specified either as multiple lists or as a list of tuples, each element within an index must be a scalar value (such as an integer, float, or string). Composite values like tuples are not allowed as individual index components. The following example will raise an IndexError:

```
x = model.AddVars([((1, 3),'a'), (7,'a')], name="x")

y = model.AddVars([(1, 3), 7], ['a'], name="y")
```

Scalar values for keyword arguments like `lb` or `obj` apply uniformly to all variables. For example, `lb=1.0` sets a lower bound of 1.0 for every variable created. The `name` parameter behaves differently: a scalar value (e.g., `name="x"`) is interpreted as a prefix, and unique names are automatically generated (e.g., `x_0`, `x_1`, etc.). However, if the `indices` argument is a single list, you can provide a list of strings of the same length for the `name` parameter. Each string in the list will be used as the explicit name for the corresponding variable. For example, `name=["varA", "varB", "varC"]` assigns those exact names to the variables in order.

Parameters

- **indices** – Indices corresponding to the newly created variables.
- **lb** – [**optional**] Variable lower bound(s).
- **ub** – [**optional**] Variable upper bound(s).
- **obj** – [**optional**] Linear coefficient(s) in objective function.
- **vtype** – [**optional**] Variable type(s). One of *OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** – [**optional**] Column object(s).
- **name** – Variable name(s).
- **count** – [**optional**] Number of variables to be added to the model.

Returns

New `OPTVTupleDict` object that contains the new variables as values, using the provided indices as keys.

Return type

OPTVTupleDict

Raises

OPTVErrCode.INVALID_ARGUMENT – Unknown variable type, `names` is empty, numeric value is `NaN`, or a variable with a name in `names` already exists in the model.

Example

```
# 3-D array of binary variables
x = model.AddVars(3, 4, 5, vtype=OPTV_BINARY)

# variables index by tuplelist
l = OPTVTupleList([(1, 2), (1, 3), (2, 3)])
y = model.addVars(l, ub=[1, 2, 3])

# variables with predetermined names
z = model.AddVars(3, name=["a","b","c"])
```

AddMConstr()

Add a set of linear constraints to the model using matrix semantics. The added constraints are `lhs <= Ax <= rhs`. The A argument must be a `numpy.ndarray`.

`OPTVModel.`**`AddMConstr`**(*A*, *x*, *lhs*, *rhs*, *name=''*)

Parameters

- **A** (*numpy.ndarray*) – The linear constraint matrix. *A* must be a two-dimensional `numpy.ndarray`.
- **x** (*OPTVMVar|list[OPTVVar]|None*) – Decision variables. This argument can be an *OPTVMVar* object, a list of *OPTVVar* objects, or None (None uses all *OPTVVar* in the model). The length of the argument must correspond to the size of the second dimension of A.
- **lhs** (*numpy.ndarray*) – Left-hand side vector, a one-dimensional `numpy.ndarray`. The length of the argument must correspond to the size of the first dimension of A.
- **rhs** (*numpy.ndarray*) – Right-hand side vector, a one-dimensional `numpy.ndarray`. The length of the argument must correspond to the size of the first dimension of A.
- **name** (*numpy.ndarray|list[str]|str*) – Names of the new constraints. When a single string is provided, each constraint's index in the matrix will serve as its subscript. Any string list or one-dimensional `numpy.ndarray` input must align with the size of the first dimension of A.

Returns

A new OPTVMConstr object.

Return type

OPTVMConstr

Raises

- **TypeError** – `x` is not list, *OPTVMVar*, or None.
- **IndexError** – `x` has wrong number of dimensions.
- **ValueError** – `x` has wrong shape.

Example

```
vars = model.AddMVar(3, vtype=OPTV_BINARY, name=['x', 'y', 'z'])
model.Update()
obj = vars @ np.array([1, 1, 2])
A = np.array([[1, 2, 3], [1, 1, 0]])
constrs = model.AddMConstr(A, vars, [-OPTV_INF, 1], [4, OPTV_INF], name=
↪'c')
```

CheckSolution()

Determine if a given solution is feasible to the model.

`OPTVModel.`**`CheckSolution`**(*solution*, *tol=1e-8*)

This function allows a user to check if an externally provided solution is feasible to the model. This function **does not** determine whether the solution stored internally is feasible.

Parameters

- **`solution`** (*`list[float]`*) – Vector containing *column* solution.
- **`tol`** (*`float`*) – [**optional**, defaults to 10^{-8}] Feasibility and integrality tolerance.

Returns
Boolean value indicating feasibility of solution to the model.

Return type
bool

Return values

- **`false`** – The solution is infeasible to the model.
- **`true`** – The solution is feasible to the model.

Example

```
result = model.CheckSolution(solution)
```

ComputeIIS()

Compute an irreducible inconsistent subsystem (IIS).

See *IIS Quick Start Guide* for more details on IIS.

`OPTVModel.`**`ComputeIIS`**()

Returns
Boolean value indicating success of IIS construction.

Return type
bool

Return values

- **`false`** – IIS construction was unsuccessful.
- **`true`** – IIS construction was successful.

Example

```
if model.ComputeIIS():
    model.WriteIIS("iis.ilp")
else:
    print("IIS computation failed")
```

FeasRelax()

Perform a feasibility relaxation.

See *Feasibility Relaxation Quick Start Guide* for more details on feasibility relaxation.

`OPTVModel.FeasRelax()`

This function will perform a feasibility relaxation. Presently there is only support for the L^1 norm.

Returns
Status of feasibility relaxation.

Return type
int

Return values

- **0** – The problem is feasible.
- **-1** – Feasibility relaxation was unsuccessful.
- **>0** – Phase 1 objective value of minimal violation.

Example

```
result = model.FeasRelax()
```

GetCoef()

Get the coefficient of a variable in a given constraint.

`OPTVModel.GetCoef(con, var)`

Parameters

- **con** (OPTVConstr) – Constraint to query.
- **var** (OPTVVar) – The variable to be accessed.

Returns
Coefficient value of variable in constraint.

Return type
float

Raises

- ***OPTVErrCode.INVALID_ARGUMENT*** – Constraint or variable is not present in the model.
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – No inner model data, or constraint linear expression is unavailable.

Example

```
c0 = model.AddConstr(x+2*y, 1, 10, "c0")
# get the coefficient of y in c0
result = model.GetCoef(c0, y)
```

GetColumn()

Get a column object corresponding to a specified variable.

`OPTVModel.GetColumn(v)`

Parameters
v – The variable to be accessed.

Returns
Column object that represents the variable.

Return type
OPTVColumn

Raises

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – Variable is not associated with a model.
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – *Index* of the variable is larger than the number of variables in model. This may occur if the variable needs to be flushed to the model. This exception may also be thrown if the variable is from another model.

Example

```
result = model.GetColumn(x)
```

GetCone()

Get a specific cone constraint based on its position.

`OPTVModel.GetCone(i)`

Parameters
i – Index of the cone constraint in the model.

Returns
Cone constraint at index `i`.

Return type
OPTVCone

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than the number of cone constraints in the model.

GetCones()

Get all cone constraints in the model.

`OPTVModel.GetCones()`

Returns
List containing all cone constraints in the model.

Return type
list[*OPTVCone*]

GetConstrByName()

Get a specific constraint based on its position.

`OPTVModel.GetConstrByName(name)`

Parameters
name (*str*) – Name of constraint to be returned.

Returns
Constraint with name matching `name`.

Return type
OPTVConstr

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Name is empty, or no constraint named `name` found in the model.

Example

```
# get a constraint with the name c0
result = model.GetConstrByName("c0")
```

GetConstr()

Get a specific constraint based on its position.

`OPTVModel.GetConstr(i)`

Parameters
i (*int*) – Index of the constraint in the model.

Returns
Constraint at index `i`.

Return type
OPTVConstr

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than number of constraints in the model.

Example

```
result = model.GetConstr(1)
```

GetConstrs()

Get all linear constraints in the model.

`OPTVModel.GetConstrs()`

Returns
A list containing all constraints in the model.

Return type
list[*OPTVConstr*]

Example

```
result = model.GetConstrs()
c0 = result[0]
```

GetGenConstr()

Get a single general constraint of the specified type.

`OPTVModel.GetGenConstr(gcType, index)`

Parameters

- **gcType** (*int*) – The general constraint type (e.g., *OPTV_GENCONSTR_AND*, *OPTV_GENCONSTR_OR*).
- **index** (*int*) – The index of the general constraint to retrieve (0-based).

Returns
The requested general constraint.

Return type
OPTVGenConstr

Raises

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – Model has been freed.
- ***OPTVErrCode.INVALID_ARGUMENT*** – Invalid constraint type.
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – `index` is out of range, or the constraint has not been added to the model.

Example

```
c0 = model.AddGenConstrAnd(x1, [x2, x3, x4], "and0")
model.Update()
gen_constr = model.GetGenConstr(OPTV_GENCONSTR_AND, 0)
```

GetGenConstrs()

Get all general constraints of the specified type.

`OPTVModel.GetGenConstrs(gcType)`

Parameters
gcType (*int*) – The general constraint type (e.g., *OPTV_GENCONSTR_AND*, *OPTV_GENCONSTR_OR*).

Returns
List of all general constraints of the specified type.

Return type
list[*OPTVGenConstr*]

Raises

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – Model has been freed.
- ***OPTVErrCode.INVALID_ARGUMENT*** – Invalid constraint type.

Example

```
model.AddGenConstrAnd(x1, [x2, x3], "and0")
model.AddGenConstrAnd(x4, [x5, x6], "and1")
model.Update()
and_constrs = model.GetGenConstrs(OPTV_GENCONSTR_AND) # [and0, and1]
```

GetGenConstrAnd()

Get the data associated with a specific general AND constraint.

OPTVModel.**GetGenConstrAnd**(*genConstr*)

Parameters
genConstr (OPTVGenConstr) – General constraint to be queried.

Returns
Tuple containing resultant var in index 0 and constraint variables in index 1.

Return type
(*OPTVVar*, list[*OPTVVar*])

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
c0 = model.AddGenConstrAnd(x1, [x2, x3, x4], "and")
model.Update()
resvar, vars_list = model.GetGenConstrAnd(c0)
```

GetGenConstrOr()

Get the data associated with a specific general OR constraint.

OPTVModel.**GetGenConstrOr**(*genConstr*)

Parameters
genConstr (OPTVGenConstr) – General constraint to be queried.

Returns
Tuple containing resultant var in index 0 and constraint variables in index 1.

Return type
(*OPTVVar*, list[*OPTVVar*])

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "or")
model.Update()
resvar, vars_list = mdoel.GetGenConstrOr(c0)
```

GetGenConstrIndicator()

Get the data associated with a specific general `INDICATOR` constraint.

`OPTVModel.`**`GetGenConstrIndicator`**`(`*genConstr*`)`

Parameters
genConstr (`OPTVGenConstr`) – General constraint to be queried.

Returns
Tuple containing indicator variable in index `0`, indicator variable value constraint variables in index `1`, constraint expression in index `2`, constraint sense in index `3` and the right hand side value in index `4`.

Return type
(*OPTVVar*, float, *OPTVLinExpr*, str, float)

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
# x3 = 1 -> x2+x1+1 <= 10
c0 = model.AddGenConstrIndicator(x3, 1, x2+x1+1, OPTV_SENSE_LESS, 10,␣
↪name="indicator")
model.Update()
binVar, binVal, expr, sense, rhs = mdoel.GetGenConstrIndicator(c0)
```

GetGenConstrAbs()

Get the data associated with a specific general `ABS` constraint.

`OPTVModel.`**`GetGenConstrAbs`**`(`*genConstr*`)`

Parameters
genConstr (`OPTVGenConstr`) – General constraint to be queried.

Returns
Tuple containing resultant var in index `0` and constraint variable in index `1`.

Return type
(*OPTVVar*, *OPTVVar*)

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
c0 = model.AddGenConstrAbs(x1, x2, "abs")
model.Update()
resvar, consvar = model.GetGenConstrAbs(c0)
```

GetGenConstrNorm()

Get the data associated with a specific general `NORM` constraint.

`OPTVModel.`**`GetGenConstrNorm`**(*genConstr*)

Parameters
genConstr (`OPTVGenConstr`) – General constraint to be queried.

Returns
Tuple containing resultant var in index `0`, constraint variables in index `1` and norm type (-1 for the L^∞ norm, 0 for the L^0 norm and 1 for the L^1 norm) in index `2`.

Return type
(*OPTVVar*, list[*OPTVVar*], int)

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
c0 = model.AddGenConstrNorm(x1, [x2, x3, x4], 1, "norm")
model.Update()
resvar, consvars, norm_type = model.GetGenConstrNorm(c0)
```

GetGenConstrMin()

Get the data associated with a specific general `MIN` constraint.

`OPTVModel.`**`GetGenConstrMin`**(*genConstr*)

Parameters
genConstr (`OPTVGenConstr`) – General constraint to be queried.

Returns
Tuple containing resultant var in index `0`, constraint variables in index `1` and an additional constant operand in index `2`.

Return type
(*OPTVVar*, list[*OPTVVar*], float)

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
c0 = model.AddGenConstrMin(x1, [x2, x3, x4], "min")
model.Update()
resvar, vars_list, constant_oprend = model.GetGenConstrMin(c0)
```

GetGenConstrMax()

Get the data associated with a specific general `MAX` constraint.

`OPTVModel.GetGenConstrMax(`*genConstr*`)`

Parameters
genConstr (`OPTVGenConstr`) – General constraint to be queried.

Returns
Tuple containing resultant var in index `0`, constraint variables in index `1` and an additional constant operand in index `2`.

Return type
(*OPTVVar*, list[*OPTVVar*], float)

Raises
OPTVErrCode.NOT_IN_MODEL – General constraint is not in a model. This may occur if a constraint has been removed or if it was default constructed.

Example

```
c0 = model.AddGenConstrMax(x1, [x2, x3, x4], "max")
model.Update()
resvar, vars_list, constant_oprend = model.GetGenConstrMax(c0)
```

GetLinMatrix()

Get the linear constraint matrix of the model.

`OPTVModel.GetLinMatrix()`

This function returns the linear constraint matrix associated with a model, in CSR format.

Returns
CSR linear constraint matrix for model.

Return type
scipy.sparse.csr_matrix

Example

```
result = mdoel.GetLinMatrix()
```

GetObjective()

Get the objective linear expression.

`OPTVModel.GetObjective()`

Returns
Linear expression object that represents objective function.

Return type
OPTVLinExpr

Example

```
result = model.GetObjective()
```

GetObjectiveN()

Get the linear expression of the specified multi-objective.

OPTVModel.**GetObjectiveN**(*objId*)

Parameters
objId (*int*) – Id of the multi-objective to be accessed.

Returns
Linear expression object that represents the specified multi-objective function.

Return type
OPTVLinExpr

Raises
OPTVErrCode.VALUE_OUT_OF_RANGE – Id of the multi-objective is out of the range of the number of multi-objectives.

Example

```
result = model.GetObjectiveN(0)
```

GetQConstr()

Get a specific quadratic constraint based on its position.

OPTVModel.**GetQConstr**(*i*)

Parameters
i (*int*) – Index of the constraint in the model.

Returns
Constraint at index `i`.

Return type
OPTVQConstr

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than number of constraints in the model.

Example

```
result = model.GetQConstr(1)
```

The index `i` in the function refers to the total index, not specific to the type of constraint. For example, if a user first adds a linear constraint and then a quadratic constraint, the linear constraint will have index `0` and the quadratic constraint will have index 1.

GetQConstrs()

Get all quadratic constraints in the model.

`OPTVModel.GetQConstrs()`

Returns
List containing all quadratic constraints in the model.

Return type
list[*OPTVQConstr*]

Example

```
result = model.GetQConstrs()
qc0 = result[0]
```

GetSOS()

Get a specific SOS constraint of a given type based on its position.

`OPTVModel.GetSOS(i, type)`

Parameters

- **i** (*int*) – Index of the target SOS constraint in the model.
- **type** (*int*) – SOS type. One of 1 or 2.

Returns
SOS constraint at index `i`.

Return type
OPTVSOS

Raises

- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – Index `i` is larger than number of SOS constraints in the model.
- ***OPTVErrCode.INVALID_ARGUMENT*** – `type` is not 1 or 2.

Example

```
c0 = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos1")
result = model.GetSOS(0, 1)
```

The index `i` in the function refers to the index, specific to the type of SOS constraint. For example, if a user first adds an SOS constraint of type 1 and then an SOS constraint of type 2, both will have an index of `0`.

GetSOSData()

OPTVModel.**GetSOSData**(*sos*, *sosVars*, *weights*)

Get the internal data representing an SOS constraint.

Parameters

- **sos** (OPTVSOS) – SOS constraint to query.
- **sosVars** (*list[OPTVVar]*) – Empty vector of variables. If non-empty, this container will be cleared by the function.
- **weights** (*list[float]*) – Empty vector of weights. If non-empty, this container will be cleared by the function.

Returns

Tuple of variables in SOS and their weights.

Return type

Tuple[*OPTVVar*, float]

Raises

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – SOS constraint was defaulted constructed.
- ***OPTVErrCode.NOT_IN_MODEL*** – SOS constraint is a member of a different model or previously removed from the model.
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – Index of constraint is larger than model container. This may indicate that a flush command is needed.

Example

```
c0 = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos1")
(var, weight) = model.GetSOSData(c0, [x, y, z], [1, 2, 3])[0]
```

GetSOSs()

Get all SOS constraints of a given type.

OPTVModel.**GetSOSs**(*type*)

Parameters

type (*int*) – SOS type. One of 1 or 2.

Returns

List containing all SOS constraints of type `type`.

Return type

list[*OPTVSOS*]

Raises

- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – Index of the SOS constraint is larger than the number of SOS constraints of same type in the model. This may occur if the SOS constraints need to be flushed to the model.
- ***OPTVErrCode.INVALID_ARGUMENT*** – `type` is not 1 or 2.

Example

```
result = model.GetSOSs(1)
```

GetQObjective()

Get the objective function as a quadratic expression.

OPTVModel.**GetQObjective**()

Returns
Quadratic expression object that represents objective function.

Return type
OPTVQuadExpr

Example

```
result = model.GetQObjective()
```

GetRow()

Get a linear expression corresponding to a specified constraint.

OPTVModel.**GetRow**(*c*)

Parameters
c (OPTVConstr) – The constraint to be accessed.

Returns
Linear expression object that represents the constraint.

Return type
OPTVLinExpr

Raises

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – Constraint is not associated with a model.
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – *Index* of the constraint is larger than the number of constraints in model. This may occur if the constraint needs to be flushed to the model. This exception may also be thrown if the constraint is from another model.

Example

```
constrs = model.GetConstrs()
linexpr = model.GetRow(constrs[0])
```

Get()

Parameters

Obtain a solver parameter value.

See *OptVerse MP Parameters*.

`OPTVModel.Get(param)`

Parameters
param (OPTVIntParam/OPTVDblParam/OPTVStrParam) – Parameter to query.

Returns
Parameter value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
result1 = model.Get(OPTVIntParam.CUTS)
result2 = model.Get(OPTVDblParam.GAP)
result3 = model.Get(OPTVStrParam.LOG_FILE)
```

Attributes

Obtain a model attribute value.

See *OptVerse MP Attributes*.

`OPTVModel.Get(attr)`

Parameters
attr (OPTVIntAttr/OPTVDblAttr/OPTVLongAttr/OPTVStrAttr) – Model attribute to query.

Returns
Model attribute value.

Return type
int|float|str

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

Example

```
result1 = model.Get(OPTVIntAttr.NUM_CONSTRS)
result2 = model.Get(OPTVDblAttr.MIP_GAP)
result3 = model.Get(OPTVLongAttr.FINGERPRINT)
result4 = model.Get(OPTVStrAttr.OBJ_N_NAME)
```

GetVarByName()

Get a specific variable based on its name.

OPTVModel.**GetVarByName**(*name*)

Parameters
name (*str*) – Name of variable to be returned.

Returns
Variable with name matching name.

Return type
OPTVVar

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Name is empty, or no variable named name found in the model.

Example

```
result = model.GetVarByName("x")
```

GetVar()

Get a specific variable based on its position.

OPTVModel.**GetVar**(*i*)

Parameters
i (*int*) – Index of the variable in the model.

Returns
Variable at index i.

Return type
OPTVVar

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – Index is larger than number of variables in the model.

Example

```
result = model.GetVa(0)
```

GetVars()

Get all variables in the model.

OPTVModel.**GetVars**()

Returns
List containing all variables in the model.

Return type
list[*OPTVVar*]

Example

```
result = model.GetVars()
x = result[0]
```

Optimize()

Perform an optimization.

OPTVModel.**Optimize**()

Returns
None.

Return type
None

Raises
OPTVErrCode.MATRIX_NOT_PSD – A quadratic constraint or quadratic objective was provided with a matrix representation that is not positive semi-definite. This error will not occur if a quadratic objective is provided to problem with strictly linear constraints.

Example

```
model.Optimize()
```

The optimization algorithm will depend on the solver parameters and model attributes. Detection of LP, QP, MIP, and network problems occurs automatically based on the variables' types.

Presolve()

Presolve the model.

OPTVModel.**Presolve**()

This function will return a new *OPTVModel* object that has been run through the presolve routine. The original model will also retain the presolve status.

Returns
Presolved copy of model.

Return type
OPTVModel

Example

```
result = mdoel.Presolve()
```

Read()

Read model/basis data from file.

`OPTVModel.`**`Read`**`(`*fileName*`)`

Generally speaking, this function serves for reading in basis information. There is additional support for reading model files; however, the preferred method for reading model files is through the constructor. See *File Formats*.

Table 16: Supported Read Types

.mps	.lp	.bas

Table 17: Supported Read Compression

.gz	.z	.Z

Parameters
`fileName` (*str*) – File name, as C++ string.

Returns
None.

Return type
None

Raises
`OPTVErrCode.ERROR_FILE_READ` – File name or extension is empty. File is incorrectly formatted.

Example

```
model.Read("input.mps")
model.Read("input.gz")
```

Relax()

Perform an LP relaxation.

`OPTVModel.`**`Relax`**`()`

This function will return a new *`OPTVModel`* object where all of the variables have been converted to *`OPTV_CONTINUOUS`*. Will create a copy of the model, leaving the original model unchanged.

Returns
LP relaxed copy of model.

Return type
OPTVModel

Example

```
r = model.Relax()
```

Remove()

Remove a variable from a model.

`OPTVModel.Remove(var)`

This function removes a variable from a model. The removal will shift the indices of the model.

For example, if a model, `model` was constructed with the following variables:

Variable	Index
x0	0
x1	1
x2	2
x3	3

Calling `model.Remove(x0)` will result in the following shift:

Variable	Index
x1	0
x2	1
x3	2

Parameters
var (OPTVVar) – Variable to be removed.

Returns
None.

Return type
None

Raises

- *OPTVErrCode.NOT_IN_MODEL* – Variable is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Variable is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the variable is larger than the number of variables in the model. This may occur if the variable needs to be flushed to the model.

`OPTVModel.Remove(constr)`

This function removes a constraint from a model. The removal will shift the indices of the model.

For example, if a model, `model` was constructed with the following constraints:

Constraint	Index
c0	0
c1	1
c2	2
c3	3

Calling `model.Remove(c0)` will result in the following shift:

Constraint	Index
c1	0
c2	1
c3	2

Parameters
constr (OPTVConstr) – Constraint to be removed.

Returns
None.

Return type
None

Raises

- *OPTVErrCode.NOT_IN_MODEL* – Constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Constraint is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the constraint is larger than the number of constraints in the model. This may occur if the constraint needs to be flushed to the model.

`OPTVModel.Remove(genConstr)`

This function removes a general constraint from a model. The removal will shift the indices of general constraints that match the type of the removed constraint in the model.

For example, if a model, `model` was constructed with the following general constraints:

Constraint	Type	Index
and0	AND	0
and1	AND	1
or0	OR	0
or1	OR	1

Calling `model.Remove(and0)` will result in the following shifting:

Constraint	Type	Index
and1	AND	0
or0	OR	0
or1	OR	1

Parameters
genConstr (OPTVGenConstr) – General constraint to be removed.

Returns
None.

Return type
None

Raises

- *OPTVErrCode.NOT_IN_MODEL* – Constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Constraint is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the general constraint is larger than the number of general constraints of same type in the model. This may occur if the general constraint needs to be flushed to the model.

`OPTVModel.Remove(sos)`

This function removes an SOS constraint from a model. Unlike other constraint types, the removal will **not** shift the indices of SOS constraints that match the type of the removed constraint in the model.

For example, if a model, `model` was constructed with the following constraints:

Constraint	Type	Index
sos1_1	1	0
sos1_2	1	1
sos1_3	1	2
sos2_1	2	0

Calling `model.Remove(sos1_2)` will result in the following shifting:

Constraint	Type	Index
sos1_1	1	0
sos1_3	1	2
sos2_1	2	0

Parameters
sos (OPTVSOS) – SOS constraint to be removed.

Returns
None.

Return type
None

Raises

- *OPTVErrCode.NOT_IN_MODEL* – Constraint is not part of any model.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Constraint is part of a different model than the one removing.
- *OPTVErrCode.INDEX_OUT_OF_RANGE* – Index of the SOS constraint is larger than the number of SOS constraints of same type in the model. This may occur if the SOS constraints need to be flushed to the model.

`OPTVModel.`**`Remove`**`(`*cone*`)`

This function removes a cone constraint from a model. The removal will shift the indices of cone constraints.

Constraint	Type	Index
c0	SOC	0
c1	RSOC	1
c2	SOC	2
c3	RSOC	3

Calling `model.Remove(c0)` will result in the following shift:

Constraint	Type	Index
c1	RSOC	0
c2	SOC	1
c3	RSOC	2

Parameters

cone (`OPTVCone`) – Conic constraint to be removed.

Returns

None.

Return type

None

Raises

- ***OPTVErrCode.NOT_IN_MODEL*** – Cone constraint is not part of any model.
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – Cone constraint is part of a different model than the one removing.
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** – Index of the cone constraint is larger than the number of cone constraints in the model. This may occur if the cone constraint needs to be flushed to the model.

Reset()

Reset model to an unsolved state.

`OPTVModel.`**`Reset`**`(`*clearAll=false*`)`

This function will clear the internal solution information. By default, this includes solution values, LP basis values, and the model status. The `clearAll` flag allows for clearing additional model attribute values (MIP warm start values, MIP branch priority, LP prohibited).

Parameters

`clearAll` (*`bool`*) – Clear additional information. (Default is `false`).

Returns

None.

Return type

None

Example

```
model.Reset()
```

GetObjSA()

LP sensitivity analysis of objective function.

See the *SA Quick Start Guide*.

`OPTVModel.`**`GetObjSA`**(*down*, *up*, *list*)

Parameters

- **down** (*list[float]*) – Smallest objective coefficient value at which the current optimal basis would remain optimal.
- **up** (*list[float]*) – Largest objective coefficient value at which the current optimal basis would remain optimal.
- **list** (*list[int]*) – Indices of variables for which to return objective coefficient analysis information.

Returns

Sensitivity information successfully populated.

Return type

bool

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

Example

```
# Objective coefficient sensitivity info, within which the current␣
↪optimal basis would remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.Optimize()
cols = [0, 1, 2]
objLower = []
objUpper = []
result = model.GetObjSA(objLower, objUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, objLower[i], objUpper[i]))
```

GetVarLbSA()

LP sensitivity analysis of variable lower bounds.

See the *SA Quick Start Guide*.

`OPTVModel.`**`GetVarLbSA`**(*down*, *up*, *list*)

Parameters

- **down** (*list[float]*) – Smallest lower bound value at which the current optimal basis would remain optimal.
- **up** (*list[float]*) – Largest lower bound value at which the current optimal basis would remain optimal.
- **list** (*list[int]*) – Indices of variables for which to return lower bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
bool

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

Example

```
# Lower bound sensitivity info, within which the current optimal basis␣
↪would remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.Optimize()
cols = [0, 1, 2]
lbLower = []
lbUpper = []
result = model.GetVarLbSA(lbLower, lbUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, lbLower[i], lbUpper[i]))
```

GetVarUbSA()

LP sensitivity analysis of variable upper bounds.

See the *SA Quick Start Guide*.

`OPTVModel.`**`GetVarUbSA`**(*down*, *up*, *list*)

Parameters

- **down** (*list[float]*) – Smallest upper bound value at which the current optimal basis would remain optimal.

- **up** (*list[float]*) – Largest upper bound value at which the current optimal basis would remain optimal.
- **list** (*list[int]*) – Indices of variables for which to return upper bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
bool

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

Example

```
# Upper bound sensitivity info, within which the current optimal basis
↪would remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.Optimize()
cols = [0, 1, 2]
ubLower = []
ubUpper = []
result = model.GetVarUbSA(ubLower, ubUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, ubLower[i], ubUpper[i]))
```

GetConstrLbSA()

LP sensitivity analysis of constraint lower bounds.

See the *SA Quick Start Guide*.

`OPTVModel.GetConstrLbSA(`*down*, *up*, *list*`)`

Parameters

- **down** (*list[float]*) – Smallest lower bound value at which the current optimal basis would remain optimal.
- **up** (*list[float]*) – Largest lower bound value at which the current optimal basis would remain optimal.
- **list** (*list[int]*) – Indices of constraints for which to return lower bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
bool

Return values

- **false** – SA was not successful.

- **true** – SA was successful.

Example

```
# LHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")
model.Optimize()
rows = [0, 1]
lhsLower = []
lhsUpper = []
result = model.GetConstrLbSA(lhsLower, lhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, lhsLower[i], lhsUpper[i]))
```

GetConstrUbSA()

LP sensitivity analysis of constraint upper bounds.

See the *SA Quick Start Guide*.

`OPTVModel.`**`GetConstrUbSA`**`(`*down*, *up*, *list*`)`

Parameters

- **down** (*list[float]*) – Smallest upper bound value at which the current optimal basis would remain optimal.
- **up** (*list[float]*) – Largest upper bound value at which the current optimal basis would remain optimal.
- **list** (*list[int]*) – Indices of constraints for which to return upper bound analysis information.

Returns
Sensitivity information successfully populated.

Return type
bool

Return values

- **false** – SA was not successful.
- **true** – SA was successful.

Example

```
# RHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
```

(continues on next page)

(continued from previous page)

```
model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")
model.Optimize()
rows = [0, 1]
rhsLower = []
rhsUpper = []
result = model.GetConstrUbSA(rhsLower, rhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, rhsLower[i], rhsUpper[i]))
```

SetCoef()

Set the coefficient of a variable in a given constraint.

`OPTVModel.`**`SetCoef`**`(`*con*, *var*, *newValue*`)`

Parameters

- **con** (`OPTVConstr`) – Constraint to be modified.
- **var** (`OPTVVar`) – Variable whose coefficient we wish to modify.
- **newValue** (*float*) – New coefficient value.

Returns
None.

Return type
None

Raises

- ***OPTVErrCode.INVALID_ARGUMENT*** – Constraint or variable is not present in the model, or `newValue` is `NaN`.
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – No inner model data, or constraint linear expression is unavailable.

Example

```
c0 = mdoel.AddConstr(x+y, 1, 10, "c0")
# c0: 1 <= x+2*y <= 10
model.SetCoef(c0, y, 2)
```

SetObjective()

Set a linear objective function.

`OPTVModel.`**`SetObjective`**`(`*expr*, *sense=OPTVSense.MINIMIZE*`)`

This function clears the current objective (as opposed to adding to it). If no `sense` parameter is provide, the solver will default to *`OPTVSense.MINIMIZE`*.

Parameters

- **expr** (`OPTVLinExpr`/`OPTVQuadExpr`) – Objective function to be set.
- **sense** (`OPTVSense`) – [**optional**, defaults to *`OPTVSense.MINIMIZE`*] Minimize or maximize. See *`OPTVSense`*.

Returns
None.

Return type
None

Example

```
model.SetObjective(x+y+z, OPTVSense.MINIMIZE)
model.SetObjective(x*y+z, OPTVSense.MAXIMIZE)
```

SetObjectiveN()

Set a linear multi-objective function.

`OPTVModel.SetObjectiveN(`*objId*, *expr*, *name*, *priority*, *weight*, *absTolerance*, *relTolerance*`)`

This function clears the current multi-objective (as opposed to adding to it). Sense of the multi-objective is determined by the sign of `weight` parameter.

Parameters

- **objId** (*int*) – Id of the multi-objective to be set.
- **expr** (OPTVLinExpr) – Objective function to be set.
- **name** (*str*) – Name of the multi-objective to be set.
- **priority** (*int*) – Priority of the multi-objective to be set (multi-objectives with greater priority are processed earlier).
- **weight** (*float*) – [**optional**, defaults to `1.0`] Weight of the multi-objective to be set (used to determine the sense of the multi-objective and to process multiple multi-objectives with the same priority).
- **absTolerance** (*float*) – [**optional**, defaults to `0.0`] Absolute tolerance of the multi-objective (determines maximal absolute violation of the multi-objective in the following optimization passes).
- **relTolerance** (*float*) – [**optional**, defaults to `0.0`] Relative tolerance of the multi-objective (determines maximal relative violation of the multi-objective in the following optimization passes).

Returns
None.

Return type
None

Example

```
model.SetObjectiveN(0, x+y+0.3, "obj1", 1, 1.0, 0.0, 0.0)
model.SetObjectiveN(1, x+y+z, "obj2", 2, 1.5, 1.0, 0.5)
```

Set()

Parameters

Set a solver parameter for the model. Once a model has been created, this interface provides the ability to control parameters specifically for this model. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Parameters*.

OPTVModel.**Set**(*param*, *value*)

Parameters

- **param** (OPTVIntParam | OPTVDblParam | OPTVStrParam) – Model parameter to be set.
- **value** (*int* | *float* | *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
model.Set(OPTVIntParam.PRESOLVE_LEVEL, 0)
model.Set(OPTVStrParam.RESULT_FILE, sol_file)
model.Set("resultFile", "example2.sol")
```

Attributes

Set the value of a model attribute. Will not be reflected internally until *OPTVModel.Update()* is called.

See *OptVerse MP Attributes*.

OPTVModel.**Set**(*attr*, *value*)

Parameters

- **attr** (OPTVIntAttr | OPTVDblAttr | OPTVStrAttr) – Model attribute to be set.
- **value** (*int* | *float* | *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_ATTRIBUTE – An unknown attribute is provided.

Example

```
model.Set(OPTVIntAttr.OBJ_SENSE, OPTVSense.MAXIMIZE)
model.Set(OPTVDblAttr.OBJ_OFFSET, 2.5)
model.Set(OPTVStrAttr.OBJ_N_NAME, "obj2")
```

Update()

Update the internal model.

`OPTVModel.Update()`

This function will perform a model update, or lazy-construction (if necessary). Calling this functions is **required** before changes to the model will take effect. This function will be called, internally, whenever it is required that the internal model reflects the model in the SDK. For example, users **do not** need to call this before calling *OPTVModel.Optimize()*, since the function will be called internally.

Returns
None.

Return type
None

Example

```
model.Update()
```

WriteIIS()

Write the computed IIS to file.

See *IIS Quick Start Guide* for more details on IIS.

`OPTVModel.WriteIIS(`*fileName*`)`

Parameters
fileName (*str*) – Name of `.ilp` file to be written. Must include `.ilp` extension (i.e `iis.ilp`).

Returns
None.

Return type
None

Raises
OPTVErrCode.ERROR_FILE_WRITE – Unsupported file type, or no IIS computed.

Note

This function will **not** compute an IIS. If the IIS was not previously computed, this function will throw an exception.

example

```
if model.ComputeIIS():
    model.WriteIIS("iis.ilp")
else:
    print("IIS computation failed")
```

Write()

Write model/basis data to file.

`OPTVModel.`**`Write`**(*fileName*)

See *File Formats*.

Table 18: Supported Write Types

.mps	.lp	.bas	.ilp

Table 19: Supported Write Compression

.gz	.z	.Z

Parameters

`fileName` (*str*) – File name, as C++ string.

Returns

None.

Return type

None

Raises

`OPTVErrCode.ERROR_FILE_WRITE` – File name or extension is empty, or error.

Example

```
model.Write("out.mps")
mdoel.Write("out.gz")
```

OPTVTempConstr

class OPTVTempConstr

OptVerse temporary constraint object. The `OPTVTempConstr` class serves as a placeholder for intermediate results in constraint expressions formed with overloaded operators. This class does not contains any member functions. Instead, comparison operations (==, <=, >=) on expression and variable types (*OPTVVar*, *OPTVLinExpr*, *OPTVQuadExpr*) return `OPTVTempConstr` objects. These are typically consumed immediately by constraint-adding methods and are not intended for storage in user-defined variables.

The `OPTVTempConstr` object allows you to create several different types of constraints:

- **Linear Constraint**: A linear constraint is defined as an expression of the form `Expr1 sense Expr2`, where `Expr1` and `Expr2` can be *OPTVVar* objects, *OPTVLinExpr* objects, or numeric constants, and `sense` is one of the comparison operators: ==, <=, or >=. For example, both `x + y <= 1 + z` and `x + y == 5` are valid linear constraints.
- **Quadratic Constraint**: A quadratic constraint is defined as an expression of the form `Expr1 sense Expr2`, where `Expr1` and `Expr2` can be *OPTVVar* objects, *OPTVLinExpr* objects, *OPTVQuadExpr*, or numeric constants, and `sense` is one of the comparison operators: ==, <=, or >=. For example, both `x*y <= 1 + z` and `x*x + y*y <= z*z` are valid quadratic constraints. Note that at least one of `Expr1` or `Expr2` must be an *OPTVQuadExpr*; otherwise, the constraint is considered linear.
- **Absolute Value Constraint**: This constraint is expressed as `x == Abs_(y)`, with `x` and `y` required to be *OPTVVar* objects.

- **Logical Constraint**: This constraint is expressed as `x == And_(y)` or `x == Or_(y)`. Here, `x` is a binary *OPTVVar* object, and `y` is a list of *OPTVVar* objects.
- **Min or Max Constraint**: This constraint is expressed as `x == Max_(y)` or `x == Min_(y)`. Here, `x` is a *OPTVVar* object, and `y` is a list of *OPTVVar* objects.

The following examples illustrate this:

```
model.AddConstr(x + y >= 1, name="c0")

model.AddConstr(x*x + y*y <= z, name="c1")

model.AddConstr(x == Abs_(y), name="c2")

model.AddConstr(z == And_([x, y, w]), name="c3")

model.AddConstr(z == Max_([x, y, w]), name="c4")
```

OPTVException

class OPTVException

OptVerse exception class.

This class represents exceptions thrown by the OptVerse solver. It provides access to both an error code (*OPTVErrCode*) and a descriptive message.

Use *OPTVException.GetErrorCode()* to determine the specific error type, and *OPTVException.GetMessage()* to get a human-readable description.

OPTVException.**GetMessage**()

Get error message from exception.

Returns
Exception error message.

Return type
str

OPTVException.**GetErrorCode**()

Get error code from exception.

Returns
Exception error code.

Return type
OPTVErrCode

For more details on *OPTVErrCode* refer to *OptVerse Error Code*.

OPTVErrCode

OPTVErrCode is used to generate a specific *OPTVException* object and provide the corresponding error details.

class OPTVErrCode

OptVerse error code.

ERROR

Generic error message, occasionally thrown by internal solver calls.

WARNING

Generic warning message, occasionally thrown by internal solver calls.

UNKNOWN

Encountered unknown error.

INVALID_ARGUMENT

A value provided to a function did not meet the specification.

For example: if we provide an undefined variable type when setting *OPTVCharAttr.VAR_TYPE*, or dividing a linear expression by zero.

UNKNOWN_ATTRIBUTE

An attribute was queried or set for an object of which it does not belong.

For example: *OPTVConstr.Get(OPTVIntAttr.NUM_VARS)* will throw this error, since the *OPTVIntAttr.NUM_VARS* corresponds to an attribute of the constraints.

DATA_NOT_AVAILABLE

An attribute was queried when the solution is not available.

For example: *OPTVVar.Get(OPTVDblAttr.X)* before calling *OPTVModel.Optimize()*.

INDEX_OUT_OF_RANGE

The index used to access an object is outside the range of allowed values.

For example: If a model has 2 variables and a user calls *model.GetVar(10)*.

UNKNOWN_PARAMETER

A parameter was queried or set that is not defined.

VALUE_OUT_OF_RANGE

A value for a function parameter is outside of it's allowed value (ex. *OPTVSense*).

ERROR_FILE_READ

An error occurred while reading a file.

ERROR_FILE_WRITE

An error occurred while writing a file.

LICENSE_CHECK_FAILED

A license file could not be detected. License files are set using the environment variable, OPTV_LICENSE_FILE.

See the *Installation Guide*.

INVALID_MODEL_INPUT

Attempted to solve a model with invalid input.

NOT_IMPLEMENTED

Attempted to use functionality which is not implemented.

NOT_IN_MODEL

A variable or constraint unassociated with a model was used.

MATRIX_NOT_PSD

A quadratic expression whose matrix representation is not positive semi-definite was provided to the QCQP solver.

This may be the result of either a quadratic constraint or quadratic objective in a QCQP problem. This error will not occur if a quadratic objective is provided to a problem strictly with linear constraints.

OPTVSense

class OPTVSense

Objective sense. The default value is 1, which indicates minimization.

MINIMIZE

Minimize problem (+1)

Type
int

Value
1

MAXIMIZE

Maximize problem (-1)

Type
int

Value
-1

OPTVConeType

class OPTVConeType

Cone type, for cone constraints.

The cone type parameter specifies the geometric constraint, C, that the *affine expression* must satisfy in cone constraints.

SOC

Second-order cones

A second-order cone, C_s, is defined as:

$$C_s^n = \left\{ x \in \mathbb{R}^n \mid x_1 \geq \sqrt{x_2^2 + x_3^2 + \cdots + x_n^2} \right\}.$$

RSOC

Rotated second-order cone.

A rotated second-order cone, C_r, is defined as:

$$C_r^n = \left\{ x \in \mathbb{R}^n \mid 2x_1 x_2 \geq x_3^2 + \cdots + x_n^2,\ x_1, x_2 \geq 0 \right\}.$$

See *OPTVCone*.

OPTVTune

OptVerse tuning tool Class, which is used to search optimal parameter configuration of OptVerse given an instance.

class OPTVTune

OPTVTune()

Constructor of OptVerse tuning tool class.

OPTVTune(*targets: OPTVModel | List[OPTVModel]*, *definedSpaces: List[OPTVTuneSpace]*, *maxTime: int*, *timeLimit: int*, *numSuggestion: int*, *indicator: str*, *bkptFile: str*, *bkptDir: str*, *logDir: str*, *resultDir: str*)

Parameters

- **targets** – Target models for tuning
- **definedSpaces** – Parameter search space
- **maxTime** – Max allowable time for the whole tuning experiment
- **timeLimit** – Max allowable time for solve a single model
- **numSuggestion** – Number of suggested parameter settings per round
- **inidcator** – Tuning indicator, optional values:
  - `time`: Elapsed time for optimization
  - `gap`: Final gap (only for MIP models)
  - `solve`: Solvability (for a batch of tuning target models)
- **bkptFile** – Path of breakpoint file used for resuming an interrupted tuning experiment, start a new tuning experiment if this path is empty.
- **bkptDir** – Folder to save the breakpoint file of current tuning experiment, save nothing if no folder directory is given.
- **logDir** – Folder to save the tuning log, save nothing if no folder directory is given.
- **resultDir** – Folder to save the tuning results, save nothing if no folder directory is given.

Returns
None.

SetFixedParams()

Method for setting fixed parameters during tuning experiment.

OPTVTune.**SetFixedParams**(*self*, *fixedParams: Dict*)

Parameters
fixedParams (*Dict*) – Parameters to be fixed and their values

Returns
None.

Run()

Method for starting a tuning experiment.

`OPTVTune.Run()`

Returns
None.

GetBestParams()

Method for getting the optimal parameters.

`OPTVTune.GetBestParams(self):`

Returns:

`nRound: int`: The tuning round in which optimal parameter setting is found

`paramID: int`: Index of the optimal parameter setting

`bestParams: Dict`: Optimal parameters and their values

OPTVTuneSpace

Class of pre-defined search space of OptVerse parameters

`class OPTVTuneSpace`

- `PRESOLVE`: Search space of parameters in Presolving.
- `SEARCH`: Search space of parameters in Branching Search.
- `SEPARATOR`: Search space of parameters in Separation.
- `HEURISTIC`: Search space of parameters in Heuristic.
- `CONFLICT`: Search space of parameters in Conflict Analysis.
- `PROPAGATION`: Search space of parameters in Domain Propagation.
- `LP`: Search space of parameters in LP.

OptVerse Constants

`OPTV_INF`

Infinity value.

Type
float

Value
`1.0e+100`

OPTV_UNDF

Undefined value.

Type
float

Value
`1.0e+101`

Variable Types

OPTV_BINARY

Binary variable type.

Type
str

Value
`'B'`

OPTV_INTEGER

Normal integer variable type.

Type
str

Value
`'I'`

OPTV_CONTINUOUS

Continuous variable type.

Type
str

Value
`'C'`

Model Statuses

OPTV_UNKNOWN

Unknown model status (Default).

Type
int

Value
`0`

OPTV_OPTIMAL

Model solved to optimality. An optimal solution is available for querying.

Type
int

Value
1

OPTV_INFEASIBLE

Model proven to be infeasible.

Type
int

Value
2

OPTV_INF_OR_UNBD

Model proven to be infeasible or unbounded.

Type
int

Value
3

OPTV_UNBOUNDED

Model proven to be unbounded.

Type
int

Value
4

OPTV_TIME_LIMIT

Optimization time exceeded limit set by *OPTVDblParam.TIME_LIMIT*.

Type
int

Value
5

OPTV_MEM_LIMIT

Optimization memory exceeded limit set by *OPTVIntParam.MEM_LIMIT*.

Type
int

Value
6

OPTV_INTERRUPTED

Optimization terminated by user (ex. through Ctrl+C).

Type
int

Value
7

Variable and Constraint Basis Statuses

OPTV_BASIC

Variable or constraint is basic.

Type
int

Value
0

OPTV_NONBASIC_LOWER

Variable or constraint is non-basic at its lower bound.

Type
int

Value
-1

OPTV_NONBASIC_UPPER

Variable or constraint is non-basic at its upper bound.

Type
int

Value
-2

OPTV_SUPERBASIC

Variable or constraint is superbasic.

Type
int

Value
-3

General Constraint Types

OPTV_GENCONSTR_AND

Resultant variable is non-zero if all supplied variables are non-zero.

Type
int

Value
0

OPTV_GENCONSTR_OR

Resultant variable is non-zero if any of the supplied variables are non-zero.

Type
int

Value
1

OPTV_GENCONSTR_INDICATOR

Binary variable is non-zero if and only if the associated logical condition is satisfied.

Type
int

Value
2

OPTV_GENCONSTR_ABS

Absolute value constraint: `x == Abs(y)`.

Type
int

Value
3

OPTV_GENCONSTR_NORM

Euclidean norm constraint: `x == Norm(y1, y2, ...)`.

Type
int

Value
4

OPTV_GENCONSTR_MIN

Minimum constraint: `x == Min(y1, y2, ...)`.

Type
int

Value
5

OPTV_GENCONSTR_MAX

Maximum constraint: `x == Max(y1, y2, ...)`.

Type
int

Value
6

Constraint Sense

OPTV_SENSE_LESS

Less than or equal to.

Type
str

Value
<

OPTV_SENSE_EQUAL

Equal to.

Type
str

Value
=

OPTV_SENSE_GREATER

Greater than or equal to.

Type
str

Value
>

CP Infinity

OPTV_CP_INF

Infinity value for CP solver.

Type
int

Value
`int64_t max`

OPTVCallback

class OPTVCallback

OptVerse callback object.

This class serves as an abstract base class, from which users may derive their own callback subclass. The function *OPTVCallback.RunCallback()* will be called periodically through solver execution. Depending on the callback type, various data may be queried throughout the optimization process.

A user may provide a callback to a given problem through the `OPTVModel.SetCallback()` function. Setting multiple callbacks is allowed.

Consider the following example, in which two user supplied callbacks are provided to a model. In this example, one callback is used to query solution information, while the other serves to provide MIP node information.

```
class SDKMipSolutionCallback(OPTVCallback):
    def __init__(self, callback, x):
        super().__init__(callback)
        self.varToWatch = x

    def RunCallback(self):
        print("Callback 1: ")
        print("Time: ", self.GetInfo(OPTVCBDblInfo.RUN_TIME), end=", ")
        print("dual bound: " , self.GetInfo(OPTVCBDblInfo.MIP_DUAL_BOUND), end=", ")
        print("new incumbent: ", self.GetInfo(OPTVCBDblInfo.MIP_INCUMBENT_OBJ), end=", ")
        print("new incumbent sol @ i: ", self.GetSolution(self.varToWatch), end= ", ")
        print("type: ", self.GetType())

class SDKMipNodeCallback(OPTVCallback):
    def __init__(self, callback, x):
            super().__init__(callback)
            self.varToWatch = x

    def RunCallback(self):
        print("Callback 2: ")
        print("n solution: ", self.GetInfo(OPTVCBIntInfo::MIP_NUM_SOLUTION), end=", ")
        print("Time: ", self.GetInfo(OPTVCBDblInfo::RUN_TIME), end=", ")
        print(self.SetSolution(self.varToWatch, 0))
        print("type: ", self.GetType())

        self.Abort()

if __name__ == "__main__":

    env = OPTVEnv()
    model = OPTVModel(env, "cb_demo.mps")

    model.Update()
    x1 = model.GetVarByName("x1")

    sdkMipSolutionCallback = SDKMipSolutionCallback(OPTVCBType.MIP_SOLUTION, x1)
    sdkMipNodeCallback = SDKMipNodeCallback(OPTVCBType.MIP_NODE, x1)

    model.SetCallback(sdkMipSolutionCallback)
    model.SetCallback(sdkMipNodeCallback)

    model.Optimize()
```

Abort()

Interrupt the optimization.

OPTVCallback.**Abort**()

Returns
None.

Return type
None

GetInfo()

OPTVCallback.**GetInfo**(*info*)

Parameters
info (OPTVCBDblInfo / OPTVCBIntInfo / OPTVCBLongInfo / OPTVCBStrInfo) – Callback data to query.

Returns
Callback data value.

Return type
float|int|str

Raises

- *OPTVErrCode.DATA_NOT_AVAILABLE* – Queried data unavailable to callback type.
- *OPTVErrCode.INVALID_ARGUMENT* – Unknown callback data type was queried.

GetNodeRel()

OPTVCallback.**GetNodeRel**(*var*)

Only available if callback type is *OPTVCBType.MIP_NODE*.

Parameters
var (OPTVVar) – Variable to get node relaxation solution value for.

Returns
Node relaxation solution value for variable `var`.

Return type
float

Raises

- *OPTVErrCode.NOT_IN_MODEL* – `var` is not part of a model. This may occur if `var` is removed, or if the model has not been flushed.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Node relaxation data is not available, or callback type is not *OPTVCBType.MIP_NODE*.

OPTVCallback.**GetNodeRel**()

Only available if callback type is *OPTVCBType.MIP_NODE*.

Returns
List containing node relaxation solution at current node for model.

Return type
list[float]

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Node relaxation data is not available, or callback type is not *OPTVCBType.MIP_NODE*.

GetSolution()

OPTVCallback.**GetSolution**(*var*)

Only available if callback type is *OPTVCBType.MIP_SOLUTION*.

Parameters
var (OPTVVar) – Variable to get current incumbent solution data for.

Returns
Current incumbent solution value for variable var.

Return type
float

Raises

- *OPTVErrCode.NOT_IN_MODEL* – var is not part of a model. This may occur if var is removed, or if the model has not been flushed.
- *OPTVErrCode.DATA_NOT_AVAILABLE* – Solution data is not available, or callback type is not *OPTVCBType.MIP_SOLUTION*.

OPTVCallback.**GetSolution**()

Only available if callback type is *OPTVCBType.MIP_SOLUTION*.

Returns
List containing all current incumbent solution data for model.

Return type
list[float]

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Solution data is not available, or callback type is not *OPTVCBType.MIP_SOLUTION*.

GetType()

Returns the type of the callback. See *OPTVCBType*.

OPTVCallback.**GetType**()

Returns
Type of the callback.

Return type
OPTVCBType

RunCallback()

This is a virtual function to be overridden when creating user-derived callback class. This function will be executed on callback steps to allow users to obtain callback information and perform other logic.

OPTVCallback.**RunCallback**()

Returns
None.

Return type
None

SetSolution()

OPTVCallback.**SetSolution**(*var*, *value*)

Only available if callback type is *OPTVCBType.MIP_NODE*.

Parameters

- **var** (OPTVVar) – Variable whose solution we wish to set.
- **value** – Solution value to set.

Returns
None.

Return type
None

Raises

- ***OPTVErrCode.NOT_IN_MODEL*** – `var` is not part of a model. This may occur if `var` is removed, or if the model has not been flushed.
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** – Solution data is not available, or callback type is not *OPTVCBType.MIP_NODE*.

OPTVCallback.**SetSolution**(*values*)

Only available if callback type is *OPTVCBType.MIP_NODE*.

Parameters
values (*list[float]*) – List containing solution values to be set for entire model.

Returns
None.

Return type
None

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Solution data is not available, or callback type is not *OPTVCBType.MIP_NODE*.

UseSolution()

OPTVCallback.**UseSolution**()

After importing solution values using *OPTVCallback.SetSolution()*, you can optionally call *UseSolution* from within a *OPTVCBType.MIP_NODE* callback. This allows the solver to immediately use the imported values to generate a heuristic solution.

Returns
New candidate solution objective value or, *OPTV_INF* if the solution is infeasible or there is no new incumbent found.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – Callback type is not *OPTVCBType.MIP_NODE*.

OptVerse Callback Information

OPTVCBDblInfo

class OPTVCBDblInfo

Table 20: OptVerse Double Callback Information

Name	Description
MIP_DUAL_BOUND	Current dual bound.
MIP_INCUMBENT_OBJ	New incumbent solution.
MIP_PRIMAL_BOUND	Current primal bound.
RUN_TIME	Elapsed time for optimization.
SIMPLEX_OBJ	Simplex objective value.

MIP_DUAL_BOUND

MIP_DUAL_BOUND
Current dual bound. Available in all callback types except *OPTVCBType.MESSAGE*.

MIP_INCUMBENT_OBJ

MIP_INCUMBENT_OBJ

Objective value of new incumbent solution. Available in only in callbacks of type *OPTVCBType.MIP_SOLUTION*.

MIP_PRIMAL_BOUND

MIP_PRIMAL_BOUND

Current primal bound. Available in all callback types except *OPTVCBType.MESSAGE*.

RUN_TIME

RUN_TIME

Elapsed optimization time. Available in all callback types.

SIMPLEX_OBJ

SIMPLEX_OBJ

Simplex objective value. Only available in callbacks of type *OPTVCBType.SIMPLEX*.

OPTVCBIntInfo

class OPTVCBIntInfo

Table 21: OptVerse Integer Callback Information

Name	Description
MIP_NODE_STATUS	Node LP model status.
MIP_NUM_SOLUTION	Number of feasible solutions found.
SIMPLEX_NUM_ITERS	Number of simplex iterations.

MIP_NODE_STATUS

MIP_NODE_STATUS

Node LP model status. Only available in *OPTVCBType.MIP_NODE*.

MIP_NUM_SOLUTION

MIP_NUM_SOLUTION

Number of feasible solutions found. Available in all callback types except *OPTVCBType.MESSAGE*.

SIMPLEX_NUM_ITERS

SIMPLEX_NUM_ITERS

Simplex iteration count. Only available in callbacks of type *OPTVCBType.SIMPLEX*.

OPTVCBLongInfo

class OPTVCBLongInfo

Table 22: OptVerse Long Callback Information

Name	Description
MIP_NUM_NODE	Number of nodes explored.
MIP_NUM_OPENNODE	Number of open nodes left.

MIP_NUM_NODE

MIP_NUM_NODE

Number of nodes explored. Available in all callback types except *OPTVCBType.MESSAGE*.

MIP_NUM_OPENNODE

MIP_NUM_OPENNODE

Number of open nodes left. Only available in callbacks of type *OPTVCBType.MIP*.

OPTVCBStrInfo

class OPTVCBStrInfo

Table 23: OptVerse String Callback Information

Name	Description
MESSAGE	Next log message.

MESSAGE

MESSAGE

The next log message to be printed. Only available in callbacks of type *OPTVCBType.MESSAGE*.

OPTVCBType

OptVerse distinguishes the functionality of different callbacks through the `OPTVCBType` enumerator. Each type is triggered under a specific context. A callback type is assigned upon instantiation of the callback class and may not be changed.

class OPTVCBType

MESSAGE

Triggered prior to printing a log message.

MIP

Triggered at various points during MIP evaluation.

MIP_NODE

Triggered after each node LP evaluation.

MIP_SOLUTION

Triggered after each non-incumbent solution is found.

SIMPLEX

Triggered during simplex optimization.

General Constraint Helper Function

OptVerse general constraint helper functions. These helper functions are designed to assist in the formulation of general constraints within OptVerse models. They are typically used in combination with overloaded operators and the *OPTVModel.AddConstr()* or *OPTVModel.AddConstrs()* methods, enabling users to construct complex constraint expressions efficiently.

And_()

Implements a logical `AND` relationship between a target binary variable and a list of source binary variables.

And_(*vars_list*)

Parameters
vars_list (*list[OPTVVar]*) – List of variables that must be non-zero for resultant to be non-zero.

Returns
General expression object.

Return type
OPTVGenExpr

Raises
TypeError – Invalid argument type in variables list.

Example

```
c0 = model.AddConstr(x3 == And_([x1, x2]), "and")
```

Or_()

Implements a logical OR relationship between a target binary variable and a list of source binary variables.

Or_(*vars_list*)

Parameters
vars_list (*list[OPTVVar]*) – Array of variables that must be non-zero for resultant to be non-zero.

Returns
General expression object.

Return type
OPTVGenExpr

Raises
TypeError – Invalid argument type in variables list.

Example

```
c0 = model.AddConstr(x3 == Or_([x1, x2]), "or")
```

Abs_()

Used to express the absolute value of a variable in terms of an additional decision variable.

Abs_(*arg_var*)

Parameters
arg_var (OPTVVar) – The variable for which the absolute value will be taken.

Returns
General expression object.

Return type
OPTVGenExpr

Example

```
c0 = model.AddConstr(x1 == Abs_(x2), "abs")
```

Norm_()

Used to express the norm of a variable vector in terms of an additional decision variable.

Norm_(*vars_list*, *which*)

Parameters

- **vars_list** (*list[OPTVVar]*) – An array of variables on which the vector norm is computed.
- **which** (*int*) – Select the norm type to use. Valid options are: -1 for the L-infinity norm (also denoted as L^{∞} norm), 0 for the L^0 norm, 1 for the L^1 norm.

Returns
General expression object.

Return type
OPTVGenExpr

Raises
TypeError – Invalid argument type in variables list.

Example

```
c0 = model.AddConstr(x3 == Norm_([x1, x2]), "norm")
```

Min_()

Used to express the minimum function in terms of a new decision variable, which may also include a constant term.

Min_(*vars_list*, *constant=OPTV_INF*)

Parameters

- **vars_list** (*list[OPTVVar]*) – An array of variables from which the minimum value is taken.
- **constant** (*float*) – [**optional**] A constant value to be included in the list of arguments for the MIN operation.

Returns
General expression object.

Return type
OPTVGenExpr

Raises
TypeError – Invalid argument type in variables list.

Example

```
c0 = model.AddConstr(x3 == Min_([x1, x2]), "min")
```

Max_()

Used to express the maximum function in terms of a new decision variable, which may also include a constant term.

Max_(*vars_list*, *constant=-OPTV_INF*)

Parameters

- **vars_list** (*list[OPTVVar]*) – An array of variables from which the maximum value is taken.
- **constant** (*float*) – [**optional**] A constant value to be included in the list of arguments for the MAX operation.

Returns
General expression object.

Return type
OPTVGenExpr

Raises
TypeError – Invalid argument type in variables list.

Example

```
c0 = model.AddConstr(x3 == Max_([x1, x2]), "min")
```

Global Functions

OptVerse global functions. Unlike other methods in the OptVerse Python API, these functions can be invoked directly without a OptVerse object instance.

OPTVMultiDict()

This function splits a single dictionary into multiple dictionaries. Each key in the input dictionary must map to a list of exactly n values. The function returns a tuple: the first element is a list of shared keys, followed by n separate dictionaries, one for each position in the value lists.

OPTVMultiDict(*data*)

Parameters
data (*dict*) – A dictionary with keys mapped to lists of values.

Returns
A list with shared keys first, then the split dictionaries.

Return type
list

Example

```
keys, dict1, dict2 = OPTVMultiDict( {
    "key1": [1, 2],
    "key2": [1, 3],
    "key3": [1, 4] } )
```

OPTVQuickSum()

This function provides a more efficient way than Python's standard `sum` to build large OptVerse expressions (such as *OPTVLinExpr* or *OPTVQuadExpr*). It accepts any iterable of expression terms.

While OPTVQuickSum is significantly faster than Python's standard `sum`, it is not the most efficient method for constructing large expressions. For maximum performance, use *AddTerms*.

OPTVQuickSum(*data*)

Parameters
data (*Iterable*) – Iterable of terms to add. The terms can be constants, *OPTVVar* objects, *OPTVLinExpr* objects, or *OPTVQuadExpr* objects.

Returns
An expression representing the sum of the terms in the input iterable.

Return type
OPTVLinExpr | *OPTVQuadExpr*

Example

```
expr = OPTVQuickSum([2*x, 3*y+1, 4*z*z])
expr = OPTVQuickSum(model.GetVars())
```

8.3 Enum

8.3.1 C++ Enum

Parameter Enums

Parameters correspond to internal settings used by the solver. Parameters may be set either through an *OPTVEnv* object prior to model creation, or through an *OPTVModel* object after model creation.

The complete list of parameters can be found in the *Parameters*.

enum class **OPTVIntParam**

This enum is used to get or set int-valued parameters.

enum class **OPTVDblParam**

This enum is used to get or set double-valued parameters.

enum class **OPTVStrParam**

This enum is used to get or set string-valued parameters.

Attribute Enums

These enums are used to get or set the MP solver attributes.

For example, users can get or set an attribute using *OPTVModel::Get* or *OPTVModel::Set* .

The complete list of attributes can be found in the *Attributes* section.

enum class **OPTVIntAttr**

This enum is used to get or set int-valued attributes.

enum class **OPTVDblAttr**

This enum is used to get or set double-valued attributes.

enum class **OPTVStrAttr**

This enum is used to get or set string-valued attributes.

enum class **OPTVCharAttr**

This enum is used to get or set char-valued attributes.

enum class **OPTVLongAttr**

This enum is used to get or set long-valued attributes.

enum class **OPTVBoolAttr**

This enum is used to get or set bool-valued attributes.

8.3.2 Java Enum

Parameter Enums

Parameters correspond to internal settings used by the solver. Parameters may be set either through an *OPTVEnv* object prior to model creation, or through an *OPTVModel* object after model creation.

The complete list of parameters can be found in the *Parameters*.

class OPTVIntParam

This class is used to get or set int-valued parameters.

class OPTVDblParam

This class is used to get or set double-valued parameters.

class OPTVStrParam

This class is used to get or set string-valued parameters.

Attribute Enums

These enums are used to get or set the MP solver attributes.

For example, users can get or set an attribute using *OPTVModel.get* or *OPTVModel.set* .

The complete list of attributes can be found in the *Attributes* section.

class OPTVIntAttr

This class is used to get or set int-valued attributes.

class OPTVDblAttr

This class is used to get or set double-valued attributes.

class OPTVStrAttr

This class is used to get or set string-valued attributes.

class OPTVCharAttr

This class is used to get or set char-valued attributes.

class OPTVLongAttr

This class is used to get or set long-valued attributes.

class OPTVBoolAttr

This class is used to get or set bool-valued attributes.

8.3.3 Python Enum

Parameter Enums

Parameters correspond to internal settings used by the solver. Parameters may be set either through an *OPTVEnv* object prior to model creation, or through an *OPTVModel* object after model creation.

The complete list of parameters can be found in the *Parameters*.

class OPTVIntParam

This class is used to get or set int-valued parameters.

class OPTVDblParam

This class is used to get or set double-valued parameters.

class OPTVStrParam

This class is used to get or set string-valued parameters.

Attribute Enums

These enums are used to get or set the MP solver attributes.

For example, users can get or set an attribute using *OPTVModel.Get* or *OPTVModel.Set* .

The complete list of attributes can be found in the *Attributes* section.

class OPTVIntAttr

This class is used to get or set int-valued attributes.

class OPTVDblAttr

This class is used to get or set double-valued attributes.

class OPTVStrAttr

This class is used to get or set string-valued attributes.

class OPTVCharAttr

This class is used to get or set char-valued attributes.

class OPTVLongAttr

This class is used to get or set long-valued attributes.

class OPTVBoolAttr

This class is used to get or set bool-valued attributes.

8.4 OptVerse CP Solver API References

8.4.1 C++ API Reference Manual

OPTVCPModel

class **OPTVCPModel**

The OPTVCPModel offers a foundational framework for formulating constraint programming problems.

Using the OPTVCPModel class, you are able to:

- define decision variables,
- impose constraints,
- and specify optimization goals.

Once the model is fully defined, it can be solved efficiently using the *OPTVCPSolver*, thereby achieving optimal decisions under complex constraint conditions.

NewIntVar()

Adds an integer variable to the model.

OPTVCPModel::**NewIntVar**(int64_t lb, int64_t ub, const std::string &name)

This function will add an integer variable to the model.

Parameters

- **lb** – Lower bound of variable.
- **ub** – Upper bound of variable.
- **name** – Variable name.

Returns

Variable object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – `name` is empty, a numeric value is `NaN`.

NewIntVarFromDomain()

Adds an integer variable to the model and specify its domain.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). For example, when domain $= \{1, 2, 5, 9\}$, it indicates that the variable's domain consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the variable can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

Note

The domain vector must satisfy the following constraints:

- The number of elements must be even (each pair defines one interval).
- Intervals must be strictly non-overlapping and non-adjacent (no shared or consecutive endpoints).

OPTVCPModel::**NewIntVarFromDomain**(const std::vector<int64_t> &domain, const std::string &name)

This function will add an integer variable to the model and specify its domain.

Parameters

- **domain** – The variable's valid value range.
- **name** – Variable name.

Returns

Variable object added to the model.

Throws

OPTVErrCode::*INVALID_ARGUMENT* – Either `domain` is empty, `name` is empty, or `domain` includes a `NaN`.

NewBoolVar()

Adds a boolean variable to the model. The value of a boolean variable can only be 0 or 1.

OPTVCPModel::**NewBoolVar**(const std::string &name)

This function will add a boolean variable to the model.

Parameters
name – Variable name.

Returns
Variable object added to the model.

NewConstant()

Adds a variable to the model, which represents a constant value.

OPTVCPModel::**NewConstant**(int64_t value)

This function will add a constant variable to the model.

Parameters
value – The value of the variable.

Returns
Variable object added to the model.

AddLinearConstraint()

Adds a single linear constraint to the model.

OPTVCPModel::**AddLinearConstraint**(const *OPTVCPLinExpr* &expr, int64_t lhs, int64_t rhs)

This function will add a single linear constraint to the model.

Parameters

- **expr** – Linear expression describing the constraint.
- **lhs** – Lower bound of constraint.
- **rhs** – Upper bound of constraint.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – The values of parameters `lhs` and `rhs` are invalid.

OPTVCPModel::**AddLinearConstraint**(const *OPTVCPTempConstr* &constr)

This method accepts a *OPTVCPTempConstr* object as the primary argument and optionally a name for the constraint as the second argument. The *OPTVCPTempConstr* object represents a constraint formed by applying comparison operators (e.g., <=, >=, or ==) between expression objects and/or numeric values.

Parameters
constr – *OPTVCPTempConstr* object that expresses constraint.

Returns
Constraint object added to the model.

Warning

A constraint can only have a single comparison operator. While `1 <= x + y <= 2` may look like a valid constraint, `AddLinearConstraint` won't accept it.

AddLinearConstraintInDomain()

Adds a single constraint to the model which enforces the result of the given linear expression must belong to the specified `domain`.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). For example, when domain $= \{1, 2, 5, 9\}$, it indicates that the constraint's domain consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the linear expression can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

Note

The domain vector must satisfy the following constraints:

- The number of elements must be even (each pair defines one interval).
- Intervals must be strictly non-overlapping and non-adjacent (no shared or consecutive endpoints).

OPTVCPModel::**AddLinearConstraintInDomain**(const *OPTVCPLinExpr* &expr, const std::vector<int64_t> &domain)

This function will add a single linear constraint to the model such that the resulting value of the expression is restricted to the specified `domain`.

Parameters

- **expr** – Linear expression describing the constraint.
- **domain** – The valid range of values for the result of the linear expression.

Returns

Constraint object added to the model.

Throws

- *OPTVErrCode*::*INVALID_MODEL_INPUT* – The number of elements in the domain vector is odd.
- *OPTVErrCode*::*INVALID_ARGUMENT* – The domain vector contains null elements.

AddBoolOr()

Adds a single boolean `OR` constraint to the model.

An `OR` constraint of the form:

$$x_1 \vee x_2 \vee \cdots \vee x_n = 1$$

enforces that at least one of the boolean variables $x_1, x_2, \ldots, x_n$ is equal to 1.

OPTVCPModel::**AddBoolOr**(const std::vector<*OPTVCPVar*> &literals)

This function will add a single boolean OR constraint to the model.

Parameters
literals – A vector of variables, all of which must be binary.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddBoolAnd()

Adds a single boolean AND constraint to the model.

An AND constraint of the form:

$$x_1 \wedge x_2 \wedge \cdots \wedge x_n = 1$$

enforces that all boolean variables $x_1, x_2, \ldots, x_n$ are equal to 1.

OPTVCPModel::**AddBoolAnd**(const std::vector<*OPTVCPVar*> &literals)

This function will add a single boolean AND constraint to the model.

Parameters
literals – A vector of variables, all of which must be binary.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddBoolXor()

Adds a single boolean XOR constraint to the model.

An XOR constraint of the form:

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n = 1$$

enforces that an odd number of boolean variables $x_1, x_2, \ldots, x_n$ are equal to 1.

OPTVCPModel::**AddBoolXor**(const std::vector<*OPTVCPVar*> &literals)

This function will add a single boolean XOR constraint to the model.

Parameters
literals – A vector of variables, all of which must be binary.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddAtLeastOne()

The function is equivalent to *OPTVCPModel::AddBoolOr()*.

OPTVCPModel::**AddAtLeastOne**(const std::vector<*OPTVCPVar*> &literals)

The function will add a single constraint to the model, ensuring that at least one variable in the sequence is non-zero.

Parameters
literals – A vector of variables, all of which must be binary.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddAtMostOne()

Adds a single linear constraint to the model which enforces that no more than one variable in the given sequence can be assigned the value 1.

OPTVCPModel::**AddAtMostOne**(const std::vector<*OPTVCPVar*> &literals)

This function will add a constraint to the model which enforces that no more than one variable in the given sequence can take a non-zero value.

Parameters
literals – A vector of variables, all of which must be binary.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddExactlyOne()

Adds a single constraint to the model which enforces precisely one variable in the provided boolean variable list is assigned the value 1, while all other variables are assigned 0.

OPTVCPModel::**AddExactlyOne**(const std::vector<*OPTVCPVar*> &literals)

This function enforces an "exactly one" constraint over a list of boolean variables. It enforces exactly one variable in the list is set to 1, while all other variables are constrained to 0.

Parameters
literals – A vector of variables, all of which must be binary.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddImplication()

Expresses the logical implication $x \rightarrow y$. This means variable y must be true if variable x is true, but variable y can be either true or false when variable x is false.

OPTVCPModel::**AddImplication**(const std::vector<*OPTVCPVar*> &literals)

This function adds a logical implication constraint to the model.

Parameters

literals – A vector of variables, all of which must be binary.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Either some of the provided variables are not binary, or the parameter length does not equal 2.

AddAllowedAssignments()

Adds a single table constraint to the model. The input contains the expressions ($exprs$) and tuples of values ($tuples$). The size of each tuple should equal the number of expressions provided. A table constraint enforces all variables must be assigned to one of the tuple of values.

For example, for table constraint `AddAllowedAssignments({x1, x2}, {{0, 0}, {1, 0}, {1, 1}})` where *x1* and *x2* are binary, values can be represented as follows:

x1	x2
0	0
1	0
1	1

OPTVCPModel::**AddAllowedAssignments**(const std::vector<*OPTVCPLinExpr*> &exprs, const std::vector<std::vector<int64_t>> &tuples)

This function will add a single table constraint to the model.

Parameters

- **exprs** – Vector of expressions of the constraint.
- **tuples** – Vector of tuples of possible values.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Number of expressions is zero, or one of the expressions is not affine, or one of tuples is not the same size as expressions.

AddForbiddenAssignments()

Adds a single negated table constraint to the model. The input contains the expressions ($exprs$) and tuples of values ($tuples$). The size of each tuple should equal the number of expressions provided. A negated table constraint enforces all variables must not be assigned to any one tuple.

For example, for negated table constraint `AddForbiddenAssignments({x1, x2}, {{0, 1}})` where *x1* and *x2* are binary, values can be represented as follows:

x1	x2
0	1

OPTVCPModel`::`**`AddForbiddenAssignments`**(const std::vector<*OPTVCPLinExpr*> &exprs, const std::vector<std::vector<int64_t>> &tuples)

This function will add a single negated table constraint to the model.

Parameters

- **`exprs`** – Vector of expressions of the constraint.
- **`tuples`** – Vector of tuples of impossible values.

Returns

Constraint object added to the model.

Throws

OPTVErrCode`::`*INVALID_MODEL_INPUT* – Number of expressions is zero, or one of the expressions is not affine, or one of tuples is not the same size as expressions.

AddInterval()

Adds a single interval constraint to the model. An interval constraint enforces $start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*.

OPTVCPModel`::`**`AddInterval`**(const *OPTVCPLinExpr* &start, const *OPTVCPLinExpr* &size, const *OPTVCPLinExpr* &end)

This function will add a single interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **`start`** – Affine expression which represents the start of the interval.
- **`size`** – Affine expression which represents the size of the interval.
- **`end`** – Affine expression which represents the end of the interval.

Returns

Constraint object added to the model.

Throws

OPTVErrCode`::`*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddFixedSizeInterval()

Adds a single fixed size interval constraint to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*. This function is an alternative to the *OPTVCPModel::AddInterval()* function in the case where the size of the interval is known in advance.

OPTVCPModel::**AddFixedSizeInterval**(const *OPTVCPLinExpr* &start, int64_t size)

This function will add a single fixed-size interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** – Affine expression which represents the start of the interval.
- **size** – Constant value which represents the size of the interval.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Start expression has more than one variable.

AddOptionalInterval()

Adds a single optional interval constraint to the model. An optional interval constraint enforces $isPresent \rightarrow start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*. This function is an alternative to the *OPTVCPModel::AddInterval()* function in the case when an enforcement literal (boolean variable or its negation) is required. The constraint will only be active if the enforcement literal evaluates to true.

OPTVCPModel::**AddOptionalInterval**(const *OPTVCPLinExpr* &start, const *OPTVCPLinExpr* &size, const *OPTVCPLinExpr* &end, const *OPTVCPVar* &isPresent)

This function will add a single interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** – Affine expression which represents the start of the interval.
- **size** – Affine expression which represents the size of the interval.
- **end** – Affine expression which represents the end of the interval.
- **isPresent** – Enforcement literal (boolean variable or its negation).

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddOptionalFixedSizeInterval()

Adds an optional fixed size interval constraint to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*. This function is an alternative to the *OPTVCPModel::AddInterval()* function in the case where the size of the interval is known in advance and an enforcement literal (boolean variable or its negation) is required. The constraint will only be active if the enforcement literal evaluates to true.

OPTVCPModel::**AddOptionalFixedSizeInterval**(const *OPTVCPLinExpr* &start, int64_t size, const *OPTVCPVar* &isPresent)

This function will add a single fixed-size interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** – Affine expression which represents the start of the interval.
- **size** – Constant value which represents the size of the interval.
- **isPresent** – Enforcement literal (boolean variable or its negation).

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Start expression has more than one variable.

AddIntervalSeries()

Adds a series of interval constraints to the model. An interval constraint enforces $start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*.

OPTVCPModel::**AddIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<*OPTVCPLinExpr*> &sizes, const std::vector<*OPTVCPLinExpr*> &ends)

This function will add a series of interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** – Vector of affine expressions which represents the start of the intervals.
- **sizes** – Vector of affine expressions which represents the size of the intervals.
- **ends** – Vector of affine expressions which represents the end of the intervals.

Returns

Vector of constraint objects added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable or provided vectors have different sizes.

AddFixedSizeIntervalSeries()

Adds a series of fixed size interval constraints to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*. This function is an alternative to the *OPTVCPModel::AddIntervalSeries()* function in the case where the sizes of the intervals are known in advance.

OPTVCPModel::**AddFixedSizeIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<int64_t> &sizes)

This function will add a series of fixed-size interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** – Vector of affine expressions which represents the start of the intervals.
- **sizes** – Vector of constant values which represents the size of the intervals.

Returns

Vector of constraint objects added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable or provided vectors have different sizes.

AddOptionalIntervalSeries()

Adds a series of optional interval constraints to the model. An optional interval constraint enforces $isPresent \rightarrow start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*. This function is an alternative to the *OPTVCPModel::AddIntervalSeries()* function in the case when enforcement literals (boolean variables or their negations) are required. The constraints will only be active if the corresponding enforcement literal evaluates to be true.

OPTVCPModel::**AddOptionalIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<*OPTVCPLinExpr*> &sizes, const std::vector<*OPTVCPLinExpr*> &ends, const std::vector<*OPTVCPVar*> &arePresent)

This function will add a series of interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** – Vector of affine expressions which represents the start of the intervals.
- **sizes** – Vector of affine expressions which represents the size of the intervals.
- **ends** – Vector of affine expressions which represents the end of the intervals.
- **arePresent** – Vector of enforcement literals (boolean variables or their negations).

Returns

Vector of constraint objects added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable or provided vectors have different sizes.

AddOptionalFixedSizeIntervalSeries()

Adds a series of optional fixed size interval constraints to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap*, *No Overlap 2D*, or *Cumulative*. This function is an alternative to the *OPTVCPModel::AddIntervalSeries()* function in the case where the sizes of the intervals are known in advance and enforcement literals (boolean variables or their negations) are required. The constraints will only be active if the corresponding enforcement literal evaluates to true.

OPTVCPModel::**AddOptionalFixedSizeIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<int64_t> &sizes, const std::vector<*OPTVCPVar*> &arePresent)

This function will add a series of fixed-size interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** – Vector of affine expressions which represents the start of the intervals.
- **sizes** – Vector of constant values which represents the size of the intervals.
- **arePresent** – Vector of enforcement literals (boolean variables or their negations).

Returns

Vector of constraint objects added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable or provided vectors have different sizes.

AddNoOverlap()

Adds a single no overlap constraint to the model. A no overlap constraint enforces all `intervals` contained in this constraint do not overlap.

OPTVCPModel::**AddNoOverlap**(const std::vector<*OPTVCPConstr*> &intervals)

This function will add a single no overlap constraint to the model.

Parameters

intervals – Vector of *intervals* which should not overlap.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided constraints are not *interval* constraints from the model.

AddNoOverlap2D()

Adds a single no overlap 2D constraint to the model. A no overlap 2D constraint enforces all rectangles contained in this constraint do not overlap.

OPTVCPModel::**AddNoOverlap2D**(const std::vector<*OPTVCPConstr*> &xIntervals, const std::vector<*OPTVCPConstr*> &yIntervals)

This function will add a single no overlap 2D constraint to the model.

Parameters

- **xIntervals** – Vector of `intervals` on the X-axis.
- **yIntervals** – Vector of `intervals` on the Y-axis.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided constraints are not *interval* constraints from the model or provided vectors have different sizes.

AddCumulative()

Adds a single cumulative constraint to the model. A cumulative constraint enforces the sum of `demands` of all the `intervals` that overlap at any time point cannot exceed the capacity.

OPTVCPModel::**AddCumulative**(const std::vector<*OPTVCPConstr*> &intervals, const std::vector<*OPTVCPLinExpr*> &demands, *OPTVCPLinExpr* capacity)

This function will add a single cumulative constraint to the model.

Parameters

- **intervals** – Vector of *intervals*.
- **demands** – Vector of *demands* for each *interval*.
- **capacity** – Capacity of the constraint.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided constraints are not *interval* constraints from the model, `intervals` and `demands` have different sizes or capacity expression has more than one variable.

AddAllDifferent()

Adds a single all-different constraint to the model. An all-different constraint enforces all expressions within a given set are assigned distinct values.

OPTVCPModel::**AddAllDifferent**(const std::vector<*OPTVCPLinExpr*> &expressions)

This function will add a single all-different constraint to the model.

Parameters
expressions – A vector of affine expressions, all of which must have at most one variable.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

OPTVCPModel::**AddAllDifferent**(const std::vector<*OPTVCPVar*> &vars)

This function will add a single all-different constraint to the model.

Parameters
vars – A vector of variables.

Returns
Constraint object added to the model.

AddElement()

Adds a single element constraint to the model. An element constraint enforces that: target $=$ expressions$_{\text{index}}$.

OPTVCPModel::**AddElement**(const *OPTVCPLinExpr* &index, const std::vector<*OPTVCPLinExpr*> &expressions, const *OPTVCPLinExpr* &target)

This function will add a single element constraint to the model.

Parameters

- **index** – Index of the expression, which is equal to `target`.
- **expressions** – Vector of `expressions`.
- **target** – Expression which is equal to the `expressions[index]`.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddCircuit()

Adds a single circuit constraint to the model. A circuit constraint enforces that the input graph contains a Hamiltonian circuit.

OPTVCPModel::**AddCircuit**(const std::vector<std::tuple<int, int, *OPTVCPVar*>> &arcs)

This function will add a single circuit constraint to the model.

Parameters
arcs – Vector of arcs of the graph. An arc is a tuple of tail node, head node, and boolean literal. The arc is selected in the circuit if the literal is true.

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddMultipleCircuit()

Adds a single multiple-circuit constraint to the model. A multiple-circuit constraint should satisfy the following properties:

- For each node except node 0, the number of outgoing arcs must be equal to 1;
- For each node except node 0, the number of incoming arcs must be equal to 1;
- For node 0, the number of outgoing arcs must be equal to the number of incoming arcs;
- There are no duplicate arcs;
- Self-arcs are not allowed for node 0;
- All cycles of the graph must include node 0.

OPTVCPModel::**AddMultipleCircuit**(const std::vector<std::tuple<int, int, *OPTVCPVar*>> &arcs)

This function will add a single multiple-circuit constraint to the model.

Parameters

arcs – Vector of arcs of the graph. An arc is a tuple of tail node, head node, and boolean literal. The arc is selected in the circuit if the literal is true.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided variables are not binary.

AddMaxEquality()

Adds a single max equality constraint to the model.

A max equality constraint enforces that: target $= \max(\text{expressions})$.

OPTVCPModel::**AddMaxEquality**(const *OPTVCPLinExpr* &target, const std::vector<*OPTVCPLinExpr*> &expressions)

This function will add a single max equality constraint to the model.

Parameters

- **target** – Expression which is equal to the maximum of the *expressions*.
- **expressions** – Vector of *expressions*.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddMinEquality()

Adds a single min equality constraint to the model. A min equality constraint enforces that: target $= \min(\text{expressions})$.

OPTVCPModel::**AddMinEquality**(const *OPTVCPLinExpr* &target, const std::vector<*OPTVCPLinExpr*> &expressions)

This function will add a single min equality constraint to the model.

Parameters

- **target** – Expression which is equal to the minimum of the *expressions*.
- **expressions** – Vector of *expressions*.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddAbsEquality()

Adds a single absolute equality constraint to the model. An absolute equality constraint enforces that: target $= |\text{expressions}|$.

OPTVCPModel::**AddAbsEquality**(const *OPTVCPLinExpr* &target, const *OPTVCPLinExpr* &expr)

This function will add a single absolute equality constraint to the model.

Parameters

- **target** – Expression which is equal to the absolute value of the `expr`.
- **expr** – Expression, the absolute value of which equals the `target`.

Returns

Constraint object added to the model.

Throws

OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddMultiplicationEquality()

Adds a single multiplication equality constraint to the model. A multiplication equality constraint enforces that: target $= \prod_{i=0}^{n} \text{expressions}_i$.

OPTVCPModel::**AddMultiplicationEquality**(const *OPTVCPLinExpr* &target, const std::vector<*OPTVCPLinExpr*> &expressions)

This function will add a single multiplication equality constraint to the model.

Parameters

- **target** – Expression which is equal to the product of all *expressions*.
- **expressions** – Vector of multiplied expressions.

Returns

Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddDivisionEquality()

Adds a single division equality constraint to the model. A division equality constraint enforces that: `target = num / denom`.

OPTVCPModel::**AddDivisionEquality**(const *OPTVCPLinExpr* &target, const *OPTVCPLinExpr* &num, const *OPTVCPLinExpr* &denom)

This function will add a single division equality constraint to the model.

Parameters

- **target** – Expression which is equal to the division of *num* by *denom*.
- **num** – Numerator expression
- **denom** – Denominator expression

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddModuloEquality()

Adds a single modulo equality constraint to the model. A modulo equality constraint enforces that: `target = expr % mod`.

OPTVCPModel::**AddModuloEquality**(const *OPTVCPLinExpr* &target, const *OPTVCPLinExpr* &expr, const *OPTVCPLinExpr* &mod)

This function will add a single modulo equality constraint to the model.

Parameters

- **target** – Expression which is equal to the remainder of division after *expr* is divided by *mod*.
- **expr** – dividend expression
- **mod** – divisor expression

Returns
Constraint object added to the model.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – Some of the provided expressions have more than one variable.

AddInverse()

Adds a single inverse constraint to the model. An inverse constraint enforces that: `variables[i] == j` and `inverseVariables[j] == i`.

OPTVCPModel`::`**`AddInverse`**(const std::vector<*OPTVCPVar*> &variables, const std::vector<*OPTVCPVar*> &inverseVariables)

This function will add a single inverse constraint to the model.

Parameters

- **`variables`** – Array of variables
- **`inverseVariables`** – Array of inverse variables

Returns

Constraint object added to the model.

Throws

OPTVErrCode`::`*INVALID_MODEL_INPUT* – Some of the provided arrays are empty or the sizes of the arrays do not match.

AddDecisionStrategy()

Adds a search strategy to the model.

OPTVCPModel`::`**`AddDecisionStrategy`**(const std::vector<*OPTVCPVar*> &vars, *OPTVCPVariableSelectionStrategy* varStrategy, *OPTVCPDomainReductionStrategy* domainStrategy)

This function will add a search strategy to the model.

Parameters

- **`vars`** – Vector of variables this strategy will assign.
- **`varStrategy`** – Heuristic to choose the next variable to assign.
- **`domainStrategy`** – Heuristic to reduce the domain of the selected variable.

Throws

OPTVErrCode`::`*INDEX_OUT_OF_RANGE* – Some of the provided variables are negative or not in the model.

Maximize()

Sets the model's objective to maximize the value of a linear expression.

Note

Setting no objective or a constant objective results in a feasibility problem.

When no objective is specified, the objective value is considered to be 0. If a constant is used as the objective, the objective value will be equal to that constant.

OPTVCPModel::**Maximize**(const *OPTVCPLinExpr* &expr)

This function sets the objective sense of the model to maximization and specifies the objective function accordingly.

Parameters
expr – Objective function to be set.

Returns
None.

Minimize()

Sets the model's objective to minimize the value of a linear expression.

Note

Setting no objective or a constant objective results in a feasibility problem.

When no objective is specified, the objective value is considered to be 0. If a constant is used as the objective, the objective value will be equal to that constant.

OPTVCPModel::**Minimize**(const *OPTVCPLinExpr* &expr)

This function sets the objective sense of the model to minimization and specifies the objective function accordingly.

Parameters
expr – Objective function to be set.

Returns
None.

HasObjective()

Checks whether the model has an objective function set.

OPTVCPModel::**HasObjective**()

Returns
Indicates whether the model has an objective function configured.

Return values

- **true** – An objective function has been set.
- **false** – No objective function has been set.

ClearObjective()

Clears the previously set objective function in the model.

OPTVCPModel::**ClearObjective**()

Returns
None.

IsMaximize()

Checks whether the model's objective sense is maximization.

OPTVCPModel::**IsMaximize**()

Returns
Whether the model is a maximization problem.

Return values

- **true** – The model is a maximization problem.
- **false** – The model is a minimization problem or has no objective set.

GetVar()

Returns a specific variable based on its position.

OPTVCPModel::**GetVar**(int i) const

Parameters
i – Index of the variable in the model.

Returns
Variable at index i.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – The index is negative or greater than the number of variables in the model.

GetVars()

Returns all variables in the model.

OPTVCPModel::**GetVars**() const

Returns
Vector containing all variables in the model.

GetConstr()

Returns a specific constraint based on its position.

OPTVCPModel::**GetConstr**(int i) const

Parameters
i – Index of the constraint in the model.

Returns
Constraint at index i.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – The index is negative or greater than the number of constraints in the model.

GetConstrs()

Returns all constraints in the model.

OPTVCPModel::**GetConstrs**() const

Returns
Vector containing all constraints in the model.

GetVarDomain()

Returns the current set of possible values for the specified variable, represented as a domain vector.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). All intervals in the domain are non-overlapping and non-adjacent, and the number of elements in the domain vector is always even.

For example, when the result is specified as domain $= \{1, 2, 5, 9\}$, it represents that the valid range of the variable consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the variable can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

OPTVCPModel::**GetVarDomain**(int i) const

Parameters
i – Index of the variable in the model.

Returns
A vector containing the variable's domain.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – The index is negative or greater than the number of variables in the model.

GetLinearConstraintVars()

Returns the indices (in the model) of all variables appearing in the specified linear constraint.

OPTVCPModel::**GetLinearConstraintVars**(int i) const

Parameters
i – Index of the constraint in the model.

Returns
A vector containing the indices of all variables in the linear constraint at index **i**.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – The queried constraint is not a linear constraint.

GetLinearConstraintCoeffs()

Returns the variable coefficients in a linear constraint.

OPTVCPModel::**GetLinearConstraintCoeffs**(int i) const

Parameters
i – Index of the constraint in the model.

Returns
A vector of coefficients for all variables in the linear constraint.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – The queried constraint is not a linear constraint.

GetLinearConstraintDomain()

Returns the effective domain of the expression in a linear constraint.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). All intervals in the domain are non-overlapping and non-adjacent, and the number of elements in the domain vector is always even.

For example, when the result is specified as domain $= \{1, 2, 5, 9\}$, it represents that the valid range of the linear expression consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the linear expression can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

OPTVCPModel::**GetLinearConstraintDomain**(int i) const

Parameters
i – Index of the constraint in the model.

Returns
A vector containing the constraint's domain.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – The queried constraint is not a linear constraint.

GetEnforcementLiterals()

If certain variables being true causes other variables or constraints to be `activated`, such variables are called `enforcement literals`. For example:

$$(a \wedge b \wedge c) \rightarrow \text{constraint} \tag{8.1}$$

$$x \rightarrow y \tag{8.2}$$

Equation (1) means that the constraint is only activated when all variables a, b, c are true. See *OPTVCPConstr::OnlyEnforceIf()*.

Equation (2) means that if variable x is true, then variable y must be true. See *OPTVCPModel::AddImplication()*.

In this context, variables a, b, c, x are enforcement literals.

This function returns the indices of the enforcement literals associated with the specified constraint.

OPTVCPModel`::GetEnforcementLiterals(`int i`) const`

Parameters
`i` – Index of the constraint in the model.

Returns
A vector containing the indices of the enforcement literals for the specified constraint.

GetConstraintLiterals()

Returns the values of the literals associated with the specified constraint.

OPTVCPModel`::GetConstraintLiterals(`int i`) const`

Parameters
`i` – Index of the constraint in the model.

Returns
A vector containing the values of the literals for the specified constraint.

Throws
OPTVErrCode`::`*INVALID_MODEL_INPUT* – The queried constraint is not a boolean constraint.

GetConstraintName()

Returns the name of the specified linear constraint.

OPTVCPModel`::GetConstraintName(`int i`) const`

Parameters
`i` – Index of the constraint in the model.

Returns
The name of the constraint located at index `i`.

Set()

Parameters

Sets a solver parameter for the model. Once a model has been created, this interface provides the ability to control parameters specifically for this model.

OPTVCPModel::**Set**(*OPTVIntParam* param, int value)

Parameters

- **param** – Integer type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVCPModel::**Set**(*OPTVDblParam* param, double value)

Parameters

- **param** – Double type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

OPTVCPModel::**Set**(const std::string &paramName, const std::string &value)

Parameters

- **param** – String type parameter.
- **value** – Value to be set.

Returns
None.

Throws
OPTVErrCode::*UNKNOWN_PARAMETER* – An unknown parameter is provided.

Read()

Read model/basis data from file.

OPTVCPModel::**Read**(const std::string &fileName)

Generally speaking, this function serves for reading model data. There is additional support for reading model files; however, the preferred method for reading model files is through the constructor. See *File Formats*.

Table 24: Supported Read Types

.mps	.lp	.fzn

Table 25: Supported Read Compression

.gz	.z	.Z

Parameters
fileName – File name, as C++ string.

Returns
None.

Throws
OPTVErrCode::*ERROR_FILE_READ* – File name or extension is empty. File is incorrectly formatted.

Write()

Write model data to file.

OPTVCPModel::**Write**(const std::string &fileName)

See *File Formats*.

Table 26: Supported Write Types

.fzn

Table 27: Supported Write Compression

.gz	.z	.Z

Parameters
fileName – File name, as C++ string.

Returns
None.

Throws
OPTVErrCode::*ERROR_FILE_WRITE* – File name or extension is empty, or error.

OPTVCPSolver

class **OPTVCPSolver**

The OPTVCPSolver is used to solve constraint programming problems.

Solve()

Solves input constraint programming model.

OPTVCPSolver::**Solve**(*OPTVCPModel* &model)

This function solves input constraint programming model and returns the solving status.

Parameters
model – Constraint programming model.

Returns
Solving status.

GetObjectiveValue()

Returns the objective value of the last solved model.

OPTVCPSolver::**GetObjectiveValue**()

This function returns the objective value of the last solved model.

Returns
Objective value of the last solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the value.

GetBestObjectiveBound()

Returns the best objective bound of the solved model.

OPTVCPSolver::**GetBestObjectiveBound**()

This function returns the best objective bound of the last solved model.

Returns
Best objective bound of the last solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the value.

GetValue()

Returns the value of the given object from the last solved model.

OPTVCPSolver::**GetValue**(const *OPTVCPVar* &var)

This function returns the value of the specified variable from the last solved model.

Parameters
var – Variable from the last solved model.

Returns
The value of the specified variable from the solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the objective value or variable does not apply to the current model.

OPTVCPSolver::**GetValue**(const *OPTVCPLinExpr* &expr)

This function returns the value of the specified expression from the last solved model.

Parameters
expr – Expression that includes variables from the last solved model.

Returns
The value of the specified expression from the solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the objective value or a variable does not apply to the current model.

GetValues()

Returns the values of the given variables from the last solved model.

OPTVCPSolver::**GetValues**(const std::vector<*OPTVCPVar*> &vars)

This function returns the values of the given variables from the last solved model.

Parameters
vars – Vector of variables from the last solved model.

Returns
Vector of values of the given variables from the last solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the objective value or any of variables does not apply to the current model.

GetBooleanValue()

Returns the value of the given boolean variable from the last solved model.

OPTVCPSolver::**GetBooleanValue**(const *OPTVCPVar* &var)

This function returns the boolean value of the specified variable from the last solved model.

Parameters
var – Boolean variable from the last solved model.

Returns
The value of the specified boolean variable from the last solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the objective value, provided variable is not binary, or variable does not apply to the current model.

GetBooleanValues()

Returns the values of the given boolean variables from the last solved model.

OPTVCPSolver::**GetBooleanValues**(const std::vector<*OPTVCPVar*> &vars)

This function returns the values of the given boolean variables from the last solved model.

Parameters
vars – Vector of boolean variables from the last solved model.

Returns
Vector of values of the given boolean variables from the last solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the objective value, any of provided variables is not binary, or any of variables does not apply to the current model.

GetSolution()

Returns the values of all variables of the last solved model.

OPTVCPSolver::**GetSolution**()

This function returns the values of all variables of the last solved model.

Returns
Vector of values of the last solved model.

Throws
OPTVErrCode::*DATA_NOT_AVAILABLE* – The model must be solved before getting the value.

OPTVCPVar

class **OPTVCPVar**

OptVerse CP variable object. Used to access variable data from a model. Variables should not be created through their constructor. Instead, a user should add a variable to a model using the *OPTVCPModel::NewIntVar()*, *OPTVCPModel::NewIntVarFromDomain()*, or *OPTVCPModel::NewBoolVar()* interfaces.

Index()

Returns the index of the variable in a model.

OPTVCPVar::**Index**() const

Returns
Index of the variable in the model.

Return values

- **-1** – Variable is not in a model.
- **>=0** – Index of the variable in the model.

GetName()

Returns the name of the variable.

OPTVCPVar::**GetName**() const

Returns
Variable name.

Throws
OPTVErrCode::*NOT_IN_MODEL* – Variable is not in a model.

GetDomain()

Returns the domain of the variable.

OPTVCPVar::**GetDomain**() const

Returns
Domain of the variable.

SameAs()

Determines if two variables are actually the same underlying object.

OPTVCPVar::**SameAs**(const *OPTVCPVar* &v2) const

Parameters
v2 – Variable to check sameness against.

Returns
Variable sameness.

Return values

- **true** – Variables are the same.
- **false** – Variables are not the same.

IsBinary()

Checks whether the variable is binary.

OPTVCPVar::**IsBinary**() const

Returns
Whether the variable is binary.

Throws
OPTVErrCode::*NOT_IN_MODEL* – Variable is not in a model.

OPTVCPConstr

class **OPTVCPConstr**

OptVerse CP constraint object. Used to access constraint data from a model. Constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVCPModel* interfaces.

OnlyEnforceIf()

Adds enforcement literals (boolean variables or their negations) to the constraint. The constraint will only be active if all enforcement literals evaluate to true.

OPTVCPConstr::**OnlyEnforceIf**(const std::vector<*OPTVCPVar*> &literals)

Parameters
literals – Vector of enforcement literals (boolean variables or their negations).

Throws

- *OPTVErrCode*::*NOT_IN_MODEL* – Constraint is not in a model.
- *OPTVErrCode*::*INVALID_MODEL_INPUT* – Some of the variables are not binary.

WithName()

Sets the name of the constraint.

OPTVCPConstr::**WithName**(const std::string &name)

Parameters
name – Name of the constraint.

Throws
OPTVErrCode::*NOT_IN_MODEL* – Constraint is not in a model.

Index()

Returns the index of the constraint in its model.

OPTVCPConstr::**Index**() const

Returns
Index of constraint.

Return values

- **-1** – Constraint is not in a model.
- **>=0** – Constraint index in a model.

GetName()

Gets the name of the constraint.

OPTVCPConstr::**GetName**() const

Returns
Name of the constraint.

Throws
OPTVErrCode::*NOT_IN_MODEL* – Constraint is not in a model.

GetType()

Returns the type of the constraint.

OPTVCPConstr::**GetType**() const

Returns
Type of the constraint.

Throws
OPTVErrCode::*NOT_IN_MODEL* – Constraint is not in a model.

SameAs()

Determines if two constraints are actually the same underlying object.

OPTVCPConstr::**SameAs**(const *OPTVCPConstr* &c2) const

Parameters
c2 – Constraint to check sameness against.

Returns
Constraint sameness.

Return values

- **true** – Constraints are the same.
- **false** – Constraints are not the same.

OPTVCPLinExpr

class **OPTVCPLinExpr**

The *OPTVCPLinExpr* class is used to construct linear constraints and objective functions. An object of this type is constructed from variables and constant terms. The expressions are accessed as constructed. That is, an expression `1 * x + 2 * y + 3 * x` will be accessed as:

Index	Coef	Variable
0	1	x
1	2	y
2	3	x

AddTerms()

Adds terms to the end of the linear expression.

OPTVCPLinExpr::**AddTerms**(const std::vector<*OPTVCPVar*> &vars, const std::vector<double> &coeffs)

This function will add the terms $\text{coeffs}_i * \text{vars}_i$, $\forall i \in [0, \text{vars.size()}]$ to the end of the expression.

Parameters

- **vars** – Vector of variables.
- **coeffs** – Vector of coefficient values.

Returns
None.

Throws
OPTVErrCode::*INVALID_MODEL_INPUT* – `vars` and `coeffs` sizes do not match.

Clear()

Clears the inner "terms" container, such that *OPTVCPLinExpr::Size()* will return `0`. Additionally, resets the expression such that it is numerically equivalent to `0`.

OPTVCPLinExpr::**Clear**()

Returns
None.

GetCoef()

Returns the coefficient for a variable at a given index in the expression.

OPTVCPLinExpr::**GetCoef**(int i) const

Parameters
i – Index of the variable in the expression.

Returns
Coefficient of variable at position `i`.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If `i` $\notin [0,$ `this->Size()`$]$.

GetVar()

Returns the variable at a specified index in the expression.

OPTVCPLinExpr::**GetVar**(int i) const

Parameters
i – Index of the variable in the expression.

Returns
Variable at position `i`.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If `i` $\notin [0,$ `this->Size()`$]$.

GetConstant()

Returns the constant term from the expression.

OPTVCPLinExpr::**GetConstant**() const

Returns
Constant value in expression.

SameAs()

Determines if two linear expressions are actually the same underlying expression.

OPTVCPLinExpr::**SameAs**(const *OPTVCPLinExpr* &e2) const

Parameters
e2 – Linear expression to check sameness against.

Returns
Expression sameness.

Return values

- **true** – Expressions are the same.
- **false** – Expressions are not the same.

Size()

Returns the size of the expression.

OPTVCPLinExpr::**Size**() const

This value corresponds to the number of elements in the expression, regardless of whether the element is repeated. For example, the expression `x+y` has `Size = 2`. The expression `x+y+x` has `Size = 3`.

Returns
Size of the linear expression.

Remove()

Remove term(s) from the expression.

OPTVCPLinExpr::**Remove**(int i)

Removes the term at a specified index from the expression.

Parameters
i – Index of term to be removed.

Returns
None.

Throws
OPTVErrCode::*INDEX_OUT_OF_RANGE* – If i $\notin [0,$ this->Size()$]$.

OPTVCPLinExpr::**Remove**(const *OPTVCPVar* &v)

Removes any instance of a variable from the expression.

Parameters
v – Variable to be removed.

Returns
Whether at least one term removed.

Return values

- **true** – One or more terms was removed.
- **false** – No terms were removed.

Throws
OPTVErrCode::*NOT_IN_MODEL* – Variable is not in a model.

OPTVCPTempConstr

class **OPTVCPTempConstr**

OptVerse CP temporary constraint object. The OPTVCPTempConstr class serves as a placeholder for intermediate results in constraint expressions formed with overloaded operators. This class does not contain any member functions. Instead, comparison operations (==, !=, <=, >=) on expression and variable types (*OPTVCPVar*, *OPTVCPLinExpr*) return OPTVCPTempConstr objects. These are typically consumed immediately by constraint-adding methods and are not intended for storage in user-defined variables.

The OPTVCPTempConstr object allows you to create linear constraints:

- **Linear Constraint**: A linear constraint is defined as an expression of the form Expr1 sense Expr2, where Expr1 and Expr2 can be *OPTVCPVar* objects, *OPTVCPLinExpr* objects, or numeric constants, and sense is one of the comparison operators: ==, !=, <=, or >=. For example, both x + y <= 1 + z and x + y == 5 are valid linear constraints.

The following example illustrates this:

```
model.AddLinearConstraint(x + y >= 1)
```

OPTVCPVariableSelectionStrategy

Variable selection strategy of the CP Solver.

enum class **OPTVCPVariableSelectionStrategy**

enumerator **CHOOSE_FIRST**
Choose the first variable from variable list.

enumerator **CHOOSE_LOWEST_MIN**
Choose the variable with the minimum lower bound.

enumerator **CHOOSE_HIGHEST_MAX**
Choose the variable with the maximum upper bound.

enumerator **CHOOSE_MIN_DOMAIN_SIZE**
Choose the variable with the minimum domain range.

enumerator **CHOOSE_MAX_DOMAIN_SIZE**
Choose the variable with the maximum domain range.

OPTVCPDomainReductionStrategy

Domain reduction strategy of the CP Solver.

enum class **OPTVCPDomainReductionStrategy**

enumerator **SELECT_MIN_VALUE**
Select the lower bound.

enumerator **SELECT_MAX_VALUE**
Select the upper bound.

enumerator **SELECT_LOWER_HALF**
Select the bound value calculated by `bound = lb + (ub - lb) / 2`.

enumerator **SELECT_UPPER_HALF**
Select the bound value calculated by `bound = ub - (ub - lb) / 2`.

enumerator **SELECT_MEDIAN_VALUE**
Select the median value of the variable.

enumerator **SELECT_RANDOM_HALF**
Select random half.

OPTVCPStatus

CP model status code.

OPTV_CP_UNKNOWN

Value

`0`

Description

Solver could not find a solution.

`OPTV_CP_MODEL_INVALID`

Value

1

Description

The model is invalid.

`OPTV_CP_FEASIBLE`

Value

2

Description

Solver found a feasible solution.

`OPTV_CP_INFEASIBLE`

Value

3

Description

Solver proved the model is infeasible.

`OPTV_CP_OPTIMAL`

Value

4

Description

Solver found an optimal solution.

`OPTV_CP_USER_INTERRUPT`

Value

5

Description

Solver was interrupted by the user.

8.4.2 Python API Reference Manual

OPTVCPModel

class OPTVCPModel

The OPTVCPModel offers a foundational framework for formulating constraint programming problems.

Using the OPTVCPModel class, you are able to:

- define decision variables,
- impose constraints,
- and specify optimization goals.

Once the model is fully defined, it can be solved efficiently using the *OPTVCPSolver*, thereby achieving optimal decisions under complex constraint conditions.

NewIntVar()

Adds an integer variable to the model.

OPTVCPModel.**NewIntVar**(*lb=0*, *ub=OPTV_INF*, *name=''*)

This function will add an integer variable to the model.

Parameters

- **lb** (*int*) – Lower bound of variable.
- **ub** (*int*) – Upper bound of variable.
- **name** (*str*) – Variable name.

Returns

Variable object added to the model.

Return type

OPTVCPVar

Raises

OPTVErrCode.INVALID_ARGUMENT – name is empty, a numeric value is NaN.

Example

```
x = model.NewIntVar(2, 5, "x")
```

NewIntVarFromDomain()

Adds an integer variable to the model and specify its domain.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). For example, when domain $= \{1, 2, 5, 9\}$, it indicates that the variable's domain consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the variable can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

Note

The domain vector must satisfy the following constraints:

- The number of elements must be even (each pair defines one interval).
- Intervals must be strictly non-overlapping and non-adjacent (no shared or consecutive endpoints).

`OPTVCPModel.`**`NewIntVarFromDomain`**`(`*domain*, *name=''*`)`

This function will add an integer variable to the model and specify its domain.

The `domain` argument can be provided in either of the following ways:

- As multiple arguments: `NewIntVarFromDomain(a, b, c)`
- As a single iterable: `NewIntVarFromDomain([a, b, c])` or `NewIntVarFromDomain((a, b, c))`

Parameters

- **`domain`** – The variable's valid value range.
- **`name`** (*str*) – Variable name.

Returns

Variable object added to the model.

Return type

OPTVCPVar

Raises

`OPTVErrCode.INVALID_ARGUMENT` – Either `domain` is empty, `name` is empty, or `domain` includes a `NaN`.

Example

```
x = model.NewIntVarFromDomain(a, b, c, name="x")

y = model.NewIntVarFromDomain([a, b, c], name="y")
```

NewBoolVar()

Adds a boolean variable to the model. The value of a boolean variable can only be 0 or 1.

`OPTVCPModel.`**`NewBoolVar`**`(`*name*`)`

This function will add a boolean variable to the model.

Parameters

`name` (*str*) – Variable name.

Returns

Variable object added to the model.

Return type

OPTVCPVar

Example

```
model = OPTVCPModel()

x = model.NewBoolVar("x")
```

NewConstant()

Adds a variable to the model, which represents a constant value.

OPTVCPModel.**NewConstant**(*value*)

This function will add a constant variable to the model.

Parameters
value (*int*) – The value of the variable.

Returns
Variable object added to the model.

Return type
OPTVCPVar

Example

```
model = OPTVCPModel()

x = model.NewConstant(5)
```

AddLinearConstraint()

Adds a single linear constraint to the model.

OPTVCPModel.**AddLinearConstraint**(*expr*, *lhs=0*, *rhs=OPTV_INF*)

This function will add a single linear constraint to the model.

Parameters

- **expr** (OPTVCPVar / OPTVCPLinExpr) – Linear expression describing the constraint.
- **lhs** (*float*) – Lower bound of constraint.
- **rhs** (*float*) – Upper bound of constraint.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – The values of parameters `lhs` and `rhs` are invalid.

Example

```
model.AddLinearConstraint(2 * x - y, lhs = 4, rhs = 15)
```

OPTVCPModel.**AddLinearConstraint**(*constr*)

This method accepts a *OPTVCPTempConstr* object as the primary argument and optionally a name for the constraint as the second argument. The *OPTVCPTempConstr* object represents a constraint formed by applying comparison operators (e.g., <=, >=, or ==) between expression objects and/or numeric values.

Parameters
constr (OPTVCPTempConstr) – *OPTVCPTempConstr* argument.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Example

```
model.AddLinearConstraint(2 * x - y <= 15)
```

Warning

A constraint can only have a single comparison operator. While `1 <= x + y <= 2` may look like a valid constraint, `AddLinearConstraint` won't accept it.

AddLinearConstraintInDomain()

Adds a single constraint to the model which enforces the result of the given linear expression must belong to the specified `domain`.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). For example, when domain $= \{1, 2, 5, 9\}$, it indicates that the constraint's domain consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the linear expression can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

Note

The domain vector must satisfy the following constraints:

- The number of elements must be even (each pair defines one interval).
- Intervals must be strictly non-overlapping and non-adjacent (no shared or consecutive endpoints).

OPTVCPModel.**AddLinearConstraintInDomain**(*expr*, **domain*)

This function will add a single linear constraint to the model such that the resulting value of the expression is restricted to the specified `domain`.

The `domain` argument can be provided in either of the following ways:

- As multiple arguments: `AddLinearConstraintInDomain(expr, a, b, c)`
- As a single iterable: `AddBoolXor(expr, [a, b, c])` or `AddBoolXor(expr, (a, b, c))`

Parameters

- **expr** (OPTVCPVar / OPTVCPLinExpr) – Linear expression describing the constraint.
- **domain** – The valid range of values for the result of the linear expression.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises

- ***OPTVErrCode.INVALID_MODEL_INPUT*** – The number of elements in the domain is odd.
- ***OPTVErrCode.INVALID_ARGUMENT*** – The domain contains null elements.

Example

```
model.AddLinearConstraintInDomain(3 * x - y + z, [3, 4, 6, 10, 12, 14])
model.AddLinearConstraintInDomain(3 * x - y + z, 3, 4, 6, 10, 12, 14)
```

AddBoolOr()

Adds a single boolean OR constraint to the model.

An OR constraint of the form:

$$x_1 \vee x_2 \vee \cdots \vee x_n = 1$$

enforces that at least one of the boolean variables $x_1, x_2, \ldots, x_n$ is equal to 1.

OPTVCPModel.**AddBoolOr**(**literals*)

This function will add a single boolean OR constraint to the model.

The `literals` argument can be provided in either of the following ways:

- As multiple arguments: `AddBoolOr(a, b, c)`
- As a single iterable: `AddBoolOr([a, b, c])` or `AddBoolOr((a, b, c))`

Parameters
literals – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided variables are not binary.

Example

```
model.AddBoolOr(a, b, c)

model.AddBoolOr([a, b, c])
```

AddBoolAnd()

Adds a single boolean AND constraint to the model.

An AND constraint of the form:

$$x_1 \wedge x_2 \wedge \cdots \wedge x_n = 1$$

enforces that all boolean variables $x_1, x_2, \ldots, x_n$ are equal to 1.

OPTVCPModel.**AddBoolAnd**(**literals*)

This function will add a single boolean AND constraint to the model.

The literals argument can be provided in either of the following ways:

- As multiple arguments: AddBoolAnd(a, b, c)
- As a single iterable: AddBoolAnd([a, b, c]) or AddBoolAnd((a, b, c))

Parameters
literals – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided variables are not binary.

Example

```
model.AddBoolAnd(a, b, c)

model.AddBoolAnd([a, b, c])
```

AddBoolXor()

Adds a single boolean XOR constraint to the model.

An XOR constraint of the form:

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n = 1$$

enforces that an odd number of boolean variables $x_1, x_2, \ldots, x_n$ are equal to 1.

OPTVCPModel.**AddBoolXor**(**literals*)

This function will add a single boolean XOR constraint to the model.

The literals argument can be provided in either of the following ways:

- As multiple arguments: AddBoolXor(a, b, c)
- As a single iterable: AddBoolXor([a, b, c]) or AddBoolXor((a, b, c))

Parameters
literals – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided variables are not binary.

Example

```
model.AddBoolXor(a, b, c)

model.AddBoolXor([a, b, c])
```

AddAtLeastOne()

The function is equivalent to *OPTVCPModel.AddBoolOr()*.

OPTVCPModel.**AddAtLeastOne**(**literals*)

The function will add a single constraint to the model, ensuring that at least one variable in the sequence is non-zero.

The `literals` argument can be provided in either of the following ways:

- As multiple arguments: `AddAtLeastOne(a, b, c)`
- As a single iterable: `AddAtLeastOne([a, b, c])` or `AddAtLeastOne((a, b, c))`

Parameters
literals – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided variables are not binary.

Example

```
model.AddAtLeastOne(a, b, c)

model.AddAtLeastOne([a, b, c])
```

AddAtMostOne()

Adds a single linear constraint to the model which enforces that no more than one variable in the given sequence can be assigned the value 1.

OPTVCPModel.**AddAtMostOne**(**literals*)

This function will add a constraint to the model which enforces that no more than one variable in the given sequence can take a non-zero value.

The `literals` argument can be provided in either of the following ways:

- As multiple arguments: `AddAtMostOne(a, b, c)`
- As a single iterable: `AddAtMostOne([a, b, c])` or `AddAtMostOne((a, b, c))`

Parameters
literals – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided variables are not binary.

Example

```
model.AddAtMostOne(a, b, c)

model.AddAtMostOne([a, b, c])
```

AddExactlyOne()

Adds a single constraint to the model which enforces precisely one variable in the provided boolean variable list is assigned the value 1, while all other variables are assigned 0.

OPTVCPModel.**AddExactlyOne**(**literals*)

This function enforces an "exactly one" constraint over a list of boolean variables. It enforces exactly one variable in the list is set to `1`, while all other variables are constrained to `0`.

The `literals` argument can be provided in either of the following ways:

- As multiple arguments: `AddExactlyOne(a, b, c)`
- As a single iterable: `AddExactlyOne([a, b, c])` or `AddExactlyOne((a, b, c))`

Parameters
literals – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided variables are not binary.

Example

```
model.AddExactlyOne(a, b, c)

model.AddExactlyOne([a, b, c])
```

AddImplication()

Expresses the logical implication $x \rightarrow y$. This means variable y must be true if variable x is true, but variable y can be either true or false when variable x is false.

`OPTVCPModel.`**`AddImplication`**`(`*literals*`)`

This function adds a logical implication constraint to the model.

The `literals` argument can be provided in either of the following ways:

- As multiple arguments: `AddImplication(a, b)`
- As a single iterable: `AddImplication([a, b])` or `AddImplication((a, b))`

Parameters
`literals` – One or more binary variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Either some of the provided variables are not binary, or the parameter length does not equal 2.

Example

```
model.AddImplication(a, b)

model.AddImplication([a, b])
```

AddAllowedAssignments()

Adds a single table constraint to the model. The input contains the expressions ($exprs$) and tuples of values ($tuples$). The size of each tuple should be equal to the size of expressions. A table constraint enforces all variables must be assigned to one of the tuple of values.

For example, for table constraint `AddAllowedAssignments([x1, x2], [[0, 0], [1, 0], [1, 1]])` where *x1* and *x2* are binary, values can be represented as follows:

x1	x2
0	0
1	0
1	1

`OPTVCPModel.`**`AddAllowedAssignments`**`(`*exprs*, *tuples*`)`

This function will add a single table constraint to the model.

Parameters

- **`exprs`** (*`iterable`*) – An iterable of expressions.
- **`tuples`** (*`iterable`*) – An iterable of tuples of integers.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Number of expressions is zero, or one of the expressions is not affine, or one of tuples is not the same size as expressions.

Example

```
model.AddAllowedAssignments([x1, b2], [[1, 2], [3, 4]])
```

AddForbiddenAssignments()

Adds a single negated table constraint to the model. The input contains the expressions ($exprs$) and tuples of values ($tuples$). The size of each tuple should be equal to the size of expressions. A negated table constraint enforces all variables must not be assigned to any one tuple.

For example, for negated table constraint `AddForbiddenAssignments([x1, x2], [[0, 1]])` where *x1* and *x2* are binary, values can be represented as follows:

x1	x2
0	1

OPTVCPModel.**AddForbiddenAssignments**(*exprs*, *tuples*)

This function will add a single negated table constraint to the model.

Parameters

- **exprs** (*iterable*) – An iterable of expressions.
- **tuples** (*iterable*) – An iterable of tuples of integers.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Number of expressions is zero, or one of the expressions is not affine, or one of tuples is not the same size as expressions.

Example

```
model.AddForbiddenAssignments([x1, b2], [[1, 2], [3, 4]])
```

AddInterval()

Adds a single interval constraint to the model. An interval constraint enforces $start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap* or *Cumulative*.

OPTVCPModel.**AddInterval**(*start*, *size*, *end*)

This function will add a single interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** (*number* | OPTVCPVar | OPTVCPLinExpr) – Affine expression which represents the start of the interval.

- **size** (*number* | OPTVCPVar | OPTVCPLinExpr) – Affine expression which represents the size of the interval.
- **end** (*number* | OPTVCPVar | OPTVCPLinExpr) – Affine expression which represents the end of the interval.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddInterval(2 * x, 2 * y + 1, 1)
```

AddFixedSizeInterval()

Adds a single fixed size interval constraint to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap* or *Cumulative*. This function is an alternative to the *OPTVCPModel.AddInterval()* function in the case where the size of the interval is known in advance.

OPTVCPModel.**AddFixedSizeInterval**(*start*, *size*)

This function will add a single fixed-size interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** (*number* | OPTVCPVar | OPTVCPLinExpr) – Affine expression which represents the start of the interval.
- **size** (*int*) – Constant value which represents the size of the interval.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Start expression has more than one variable.

Example

```
model.AddFixedSizeInterval(2 * x2, 7)

model.AddFixedSizeInterval(x2, 7)
```

AddOptionalInterval()

Adds a single optional interval constraint to the model. An optional interval constraint enforces $isPresent \rightarrow start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap* or *Cumulative*. This function is an alternative to the *OPTVCPModel.AddInterval* function in the case when an enforcement literal (boolean variable or its negation) is required. The constraint will only be active if the enforcement literal evaluates to true.

OPTVCPModel.**AddOptionalInterval**(*start*, *size*, *end*, *isPresent*)

This function will add a single interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** (OPTVCPVar / OPTVCPLinExpr) – Affine expression which represents the start of the interval.
- **size** (OPTVCPVar / OPTVCPLinExpr) – Affine expression which represents the size of the interval.
- **end** (OPTVCPVar / OPTVCPLinExpr) – Affine expression which represents the end of the interval.
- **isPresent** (OPTVCPVar) – Enforcement literal (boolean variable or its negation).

Returns

Constraint object added to the model.

Return type

OPTVCPConstr

Raises

OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddOptionalInterval(5 * x + 6, 3, 4 * y + 2, z)
```

AddOptionalFixedSizeInterval()

Adds an optional fixed size interval constraint to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap* or *Cumulative*. This function is an alternative to the *OPTVCPModel.AddInterval* function in the case where the size of the interval is known in advance and an enforcement literal (boolean variable or its negation) is required. The constraint will only be active if the enforcement literal evaluates to true.

OPTVCPModel.**AddOptionalFixedSizeInterval**(*start*, *size*, *isPresent*)

This function will add a single fixed-size interval constraint to the model. The expression passed to this function can contain at most one variable.

Parameters

- **start** (OPTVCPVar / OPTVCPLinExpr) – Affine expression which represents the start of the interval.
- **size** (*int*) – Constant value which represents the size of the interval.
- **isPresent** (OPTVCPVar) – Enforcement literal (boolean variable or its negation).

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Start expression has more than one variable.

Example

```
model.AddOptionalFixedSizeIntervalSeries([x, 4 * y], [6, 10], [b1, b2])
```

AddIntervalSeries()

Adds a series of interval constraints to the model. An interval constraint enforces $start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap* or *Cumulative*.

OPTVCPModel.**AddIntervalSeries**(*starts*, *sizes*, *ends*)

This function will add a series of interval constraints to the model. If an expression is passed to this function, it can contain no more than one variable.

Parameters

- **starts** (*iterable*) – An iterable of *OPTVCPVar* or *OPTVCPLinExpr* objects representing the start of the intervals.
- **sizes** (*iterable*) – An iterable of *OPTVCPVar* or *OPTVCPLinExpr* objects representing the size of the intervals.
- **ends** (*iterable*) – An iterable of *OPTVCPVar* or *OPTVCPLinExpr* objects representing the end of the intervals.

Returns
List of constraint objects added to the model.

Return type
list[*OPTVCPConstr*]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddIntervalSeries([2 * x, 2 * y], [2 * y + 1, 3], [13, z])
```

AddFixedSizeIntervalSeries()

Adds a series of fixed size interval constraints to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap* or *Cumulative*. This function is an alternative to the *OPTVCPModel.AddIntervalSeries()* function in the case where the sizes of the intervals are known in advance.

OPTVCPModel.**AddFixedSizeIntervalSeries**(*starts*, *sizes*)

This function will add a series of fixed-size interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** (*iterable*) – An iterable of *OPTVCPVar* or *OPTVCPLinExpr* objects representing the start times of the intervals.
- **sizes** (*iterable*) – An iterable of numeric values representing the size of the intervals.

Returns
List of constraint objects added to the model.

Return type
list[*OPTVCPConstr*]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
start1, start2 = x + 1, 15 * y

model.AddFixedSizeIntervalSeries([start1, start2], [3, 5])
```

AddOptionalIntervalSeries()

Adds a series of optional interval constraints to the model. An optional interval constraint enforces $isPresent \rightarrow start + size = end$. Interval constraints serve as the basis for other constraints such as *No Overlap* or *Cumulative*. This function is an alternative to the *OPTVCPModel.AddIntervalSeries()* function in the case when enforcement literals (boolean variables or their negations) are required. The constraints will only be active if the corresponding enforcement literal evaluates to be true.

OPTVCPModel.**AddOptionalIntervalSeries**(*starts*, *sizes*, *ends*, *arePresent*)

This function will add a series of interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** (*iterable*) – An iterable of numbers, *OPTVCPVar*, or *OPTVCPLinExpr* objects representing the the start of the intervals.
- **sizes** (*iterable*) – An iterable of numbers, *OPTVCPVar*, or *OPTVCPLinExpr* objects representing the the size of the intervals.
- **ends** (*iterable*) – An iterable of numbers, *OPTVCPVar*, or *OPTVCPLinExpr* objects representing the the end of the intervals.
- **arePresent** (*iterable*) – An iterable of enforcement literals (boolean variables or their negations).

Returns
List of constraint objects added to the model.

Return type
list[*OPTVCPConstr*]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
tmp3 = model.AddOptionalIntervalSeries([2 * x, 2 * y], [x, 3], [8, 3 *␣
↪y], [b1, b2])
```

AddOptionalFixedSizeIntervalSeries()

Adds a series of optional fixed size interval constraints to the model. Fixed size interval constraints themselves are meaningless, but they can serve as the basis for other constraints such as *No Overlap* or *Cumulative*. This function is an alternative to the *OPTVCPModel.AddIntervalSeries()* function in the case where the sizes of the intervals are known in advance and enforcement literals (boolean variables or their negations) are required. The constraints will only be active if the corresponding enforcement literal evaluates to true.

OPTVCPModel.**AddOptionalFixedSizeIntervalSeries**(*starts*, *sizes*, *arePresent*)

This function will add a series of fixed-size interval constraints to the model. Expressions passed to this function can contain no more than one variable.

Parameters

- **starts** (*iterable*) – An iterable of numbers, *OPTVCPVar*, or *OPTVCPLinExpr* objects representing the start times of the intervals.
- **sizes** (*iterable*) – An iterable of integers which represents the size of the intervals.
- **arePresent** (*iterable*) – An iterable of enforcement literals (boolean variables or their negations).

Returns

List of constraint objects added to the model.

Return type

list[*OPTVCPConstr*]

Raises

OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddOptionalFixedSizeIntervalSeries([x, 4 * y], [6, 10], [b1, b2])
```

AddNoOverlap()

Adds a single no overlap constraint to the model. A no overlap constraint enforces all `intervals` contained in this constraint do not overlap.

OPTVCPModel.**AddNoOverlap**(*intervals*)

This function will add a single no overlap constraint to the model.

Parameters

intervals (*iterable*) – An iterable of *OPTVCPConstr* which should not overlap.

Returns

Constraint object added to the model.

Return type

OPTVCPConstr

Raises

OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided constraints are not *interval* constraints from the model.

Example

```
tmp1 = model.AddIntervalSeries([2 * x, 2 * y], [2 * y + 1, 3], [13, z])
tmp2 = model.AddFixedSizeIntervalSeries([y + 1, 2 * z], [3, 5])
model.AddNoOverlap([tmp1, tmp2])
```

AddCumulative()

Adds a single cumulative constraint to the model. A cumulative constraint enforces the sum of `demands` of all the `intervals` that overlap at any time point cannot exceed the capacity.

OPTVCPModel.**AddCumulative**(*intervals*, *demands*, *capacity*)

This function will add a single cumulative constraint to the model.

Parameters

- **intervals** (*iterable*) – An iterable of *OPTVCPConstr*.
- **demands** (*iterable*) – An iterable of *OPTVCPLinExpr*.
- **capacity** (*number* | OPTVCPVar | OPTVCPLinExpr) – Capacity of the constraint.

Returns

Constraint object added to the model.

Return type

OPTVCPConstr

Raises

OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided constraints are not *interval* constraints from the model, or capacity expression has more than one variable.

Example

```
start1, size1, end1 = 2 * x2, 2 * x1 + 1, 9

start2, size2, end2 = 5 * b3 + 6, 3, 4 * x1 + 2

start3 size3, end3 = 10, 5, 15

demand1, demand2, demand3, capacity = 4 * x1, 8, 9, 20

interval1 = model.AddInterval(start1, size1, end1)
interval2 = model.AddInterval(start2, size2, end2)
interval3 = model.AddInterval(start3, size3, end3)

cumulative = model.AddCumulative([interval1, interval2, interval3],␣
↪[demand1, demand2, demand3], capacity)
```

AddAllDifferent()

Adds a single all-different constraint to the model. An all-different constraint enforces all expressions within a given set are assigned distinct values.

OPTVCPModel.**AddAllDifferent**(*expressions*)

This function will add a single all-different constraint to the model. The elements of the parameter `expressions` can be numbers, *OPTVCPVar*, or *OPTVCPLinExpr*, but each elements may contain at most one variable.

Parameters
expressions (*list*) – A list of affine expressions, all of which must have at most one variable.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddAllDifferent([x+1, y, z])
```

OPTVCPModel.**AddAllDifferent**(*vars*)

This function will add a single all-different constraint to the model.

Parameters
vars (*list*) – A list of variables.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Example

```
model.AddAllDifferent([y, z])
```

AddElement()

Adds a single element constraint to the model. An element constraint enforces that: target $=$ expressions$_{\text{index}}$.

OPTVCPModel.**AddElement**(*index*, *expressions*, *target*)

This function will add a single element constraint to the model.

Parameters

- **index** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – The index of the selected expression in the array. It must be an affine expression (a * var + b).
- **expressions** (*Iterable[int | float | OPTVCPVar | OPTVCPLinExpr]*) – A iterable of affine expressions.

- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – The affine expression (a * var + b) constrained to be equal to the selected expression.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([0, 3, 5, 7, 15, 15], name="x2")
x3 = model.NewBoolVar("x3")
element = model.AddElement(x3, [2 * x2, 4 * x2 + 3], x1 + 4)
```

AddMaxEquality()

Adds a single max equality constraint to the model.

A max equality constraint enforces that: target $=$ max(expressions).

OPTVCPModel.**AddMaxEquality**(*target*, *expressions*)

This function will add a single max equality constraint to the model.

Parameters

- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – An expression which is equal to the maximum value of the `expressions`.
- **expressions** (*Iterable[int* | *float* | OPTVCPVar | OPTVCPLinExpr*]*) – Iterable of number, variables, or linear expressions.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddMaxEquality(3 * x2 + 7, [2 * x1 + 4, 3 * x1 - 1, 7 * x2 - 6])
```

AddMinEquality()

Adds a single min equality constraint to the model. A min equality constraint enforces that: target $=$ min(expressions).

OPTVCPModel.**AddMinEquality**(*target*, *expressions*)

This function will add a single min equality constraint to the model.

Parameters

- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – Expression which is equal to the minimum of the `expressions`.
- **expressions** (*Iterable[int | float | OPTVCPVar | OPTVCPLinExpr]*) – Iterable of number, variables, or linear expressions.

Returns

Constraint object added to the model.

Return type

OPTVCPConstr

Raises

OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddMinEquality(3 * x2 + 7, [2 * x1 + 4, 3 * x1 - 1, 7 * x2 - 6])
```

AddAbsEquality()

Adds a single absolute equality constraint to the model. An absolute equality constraint enforces that: target $=$ |expressions|.

OPTVCPModel.**AddAbsEquality**(*target*, *expr*)

This function will add a single absolute equality constraint to the model.

Parameters

- **target** (OPTVCPVar | OPTVCPLinExpr) – A variable or linear expression that will be constrained to equal the absolute value of `expr`.
- **expr** (OPTVCPVar | OPTVCPLinExpr) – A variable or linear expression representing the absolute value of `target`.

Returns

Constraint object added to the model.

Return type

OPTVCPConstr

Raises

OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
model.AddAbsEquality(x + 1, y) # x + 1 = Abs(y)
```

AddMultiplicationEquality()

Adds a single multiplication equality constraint to the model. A multiplication equality constraint enforces that: target $= \prod_{i=0}^{n}$ expressions$_i$.

OPTVCPModel.**AddMultiplicationEquality**(*target*, **expressions*)

This function will add a single multiplication equality constraint to the model.

Parameters

- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – Expression which is equal to the product of all expressions.
- ***expressions** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – One or more numbers, variables, or linear expressions.

Returns
Constraint object added to the model.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INVALID_MODEL_INPUT – Some of the provided expressions have more than one variable.

Example

```
multEquality = model.AddMultiplicationEquality(4 * x1, [2 * x1, x2 + 1])
```

AddDecisionStrategy()

Adds a search strategy to the model.

OPTVCPModel.**AddDecisionStrategy**(*vars*, *varStrategy*, *domainStrategy*)

This function will add a search strategy to the model.

Parameters

- **vars** (*Iterable*[OPTVCPVar]) – Iterable of variables this strategy will assign.
- **varStrategy** (OPTVCPVariableSelectionStrategy) – Heuristic to choose the next variable to assign.
- **domainStrategy** (OPTVCPDomainReductionStrategy) – Heuristic to reduce the domain of the selected variable.

Returns
None.

Return type
None

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – Some of the provided variables are negative or not in the model.

Example

```
model.AddDecisionStrategy([x, y, z], OPTVCPVariableSelectionStrategy.
↪CHOOSE_MIN_DOMAIN_SIZE, OPTVCPDomainReductionStrategy.SELECT_MIN_
↪VALUE)
```

Maximize()

Sets the model's objective to maximize the value of a linear expression.

Note

Setting no objective or a constant objective results in a feasibility problem.

When no objective is specified, the objective value is considered to be 0. If a constant is used as the objective, the objective value will be equal to that constant.

`OPTVCPModel.`**`Maximize`**`(`*`expr`*`)`

This function sets the objective sense of the model to maximization and specifies the objective function accordingly.

Parameters
expr (*number* | OPTVCPVar | OPTVCPLinExpr) – Objective function to be set.

Returns
None.

Return type
None

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")

model.Maximize(x1 + x2)
```

Minimize()

Sets the model's objective to minimize the value of a linear expression.

Note

Setting no objective or a constant objective results in a feasibility problem.

When no objective is specified, the objective value is considered to be 0. If a constant is used as the objective, the objective value will be equal to that constant.

`OPTVCPModel.`**`Minimize`**`(`*`expr`*`)`

This function sets the objective sense of the model to minimization and specifies the objective function accordingly.

Parameters
expr (*number* | OPTVCPVar | OPTVCPLinExpr) – Objective function to be set.

Returns
None.

Return type
None

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")

model.Minimize(x1 + x2)
```

HasObjective()

Checks whether the model has an objective function set.

OPTVCPModel.**HasObjective**()

Returns
Indicates whether the model has an objective function configured.

Return type
bool

Return values

- **True** – An objective function has been set.
- **False** – No objective function has been set.

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")
model.HasObjective() # False

model.Minimize(x1 + x2)
model.HasObjective() # True

model.ClearObjective()
model.HasObjective() # False
```

ClearObjective()

Clears the previously set objective function in the model.

`OPTVCPModel.`**`ClearObjective`**`()`

Returns
None.

Return type
None

Example

```
model.ClearObjective()
```

IsMaximize()

Checks whether the model's objective sense is maximization.

`OPTVCPModel.`**`IsMaximize`**`()`

Returns
Whether the model is a maximization problem.

Return type
bool

Return values

- **`True`** – The model is a maximization problem.
- **`False`** – The model is a minimization problem or has no objective set.

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")
model.IsMaximize() # False

model.Maximize(x1 + x2)
model.IsMaximize() # True

model.ClearObjective()
model.IsMaximize() # False
```

GetVar()

Returns a specific variable based on its position.

`OPTVCPModel.GetVar(i)`

Parameters
i (*int*) – Index of the variable in the model.

Returns
Variable at index `i`.

Return type
OPTVCPVar

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – The index is negative or greater than the number of variables in the model.

Example

```
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.GetVar(0) # b1
model.GetVar(1) # b2
model.GetVar(2) # b3
model.GetVar(3) # b4
```

GetVars()

Returns all variables in the model.

`OPTVCPModel.GetVars()`

Returns
A list containing all variables in the model.

Return type
list[*OPTVCPVar*]

Example

```
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.GetVars() # [b1, b2, b3, b4]
```

GetConstr()

Returns a specific constraint based on its position.

OPTVCPModel.**GetConstr**(*i*)

Parameters
i (*int*) – Index of the constraint in the model.

Returns
Constraint at index i.

Return type
OPTVCPConstr

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – The index is negative or greater than the number of constraints in the model.

Example

```
constr = model.GetConstr(0) # Get the constraint at index 0.
```

GetConstrs()

Returns all constraints in the model.

OPTVCPModel.**GetConstrs**()

Returns
A list containing all constraints in the model.

Return type
list[*OPTVCPConstr*]

Example

```
model.GetConstrs()
```

GetVarDomain()

Returns the current set of possible values for the specified variable, represented as a domain vector.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). All intervals in the domain are non-overlapping and non-adjacent, and the number of elements in the domain vector is always even.

For example, when the result is specified as domain $= \{1, 2, 5, 9\}$, it represents that the valid range of the variable consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the variable can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

OPTVCPModel.**GetVarDomain**(*i*)

Parameters
i (*int*) – Index of the variable in the model.

Returns
A list containing the variable's domain.

Return type
list[float]

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – The index is negative or greater than the number of variables in the model.

Example

```
b1 = model.NewBoolVar("b1")
model.GetVarDomain(b1.Index()) # [0, 1]

x2 = model.NewIntVarFromDomain([1, 2, 5, 6, 7, 15], name="x2")
model.GetVarDomain(x2.Index()) # [1, 2, 5, 6, 7, 15]
```

GetLinearConstraintVars()

Returns the indices (in the model) of all variables appearing in the specified linear constraint.

OPTVCPModel.**GetLinearConstraintVars**(*i*)

Parameters
i (*int*) – Index of the constraint in the model.

Returns
A list containing the indices of all variables in the linear constraint at index `i`.

Return type
list[int]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – The queried constraint is not a linear constraint.

Example

```
constr = model.AddLinearConstraint(2 * x - y < 15)
model.GetLinearConstraintVars(constr.Index()) == [x.Index(), y.Index()]
↪# True
```

GetLinearConstraintCoeffs()

Returns the variable coefficients in a linear constraint.

OPTVCPModel.**GetLinearConstraintCoeffs**(*i*)

Parameters
i (*int*) – Index of the constraint in the model.

Returns
A list of coefficients for all variables in the linear constraint.

Return type
list[float]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – The queried constraint is not a linear constraint.

Example

```
constr = model.AddLinearConstraint(2 * x - y < 15)
model.GetLinearConstraintCoeffs(constr.Index()) # [2, -1]
```

GetLinearConstraintDomain()

Returns the effective domain of the expression in a linear constraint.

The domain is defined by a sequence of integers, where every two consecutive elements represent a closed interval $[a, b]$ (with $a \leq b$). All intervals in the domain are non-overlapping and non-adjacent, and the number of elements in the domain vector is always even.

For example, when the result is specified as domain $= \{1, 2, 5, 9\}$, it represents that the valid range of the linear expression consists of two intervals: domain $= [1, 2] \cup [5, 9]$. Therefore, the linear expression can take one of the following values: $\{1, 2, 5, 6, 7, 8, 9\}$.

`OPTVCPModel.GetLinearConstraintDomain(`*i*`)`

Parameters
i (*int*) – Index of the constraint in the model.

Returns
A list containing the constraint's domain.

Return type
list[float]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – The queried constraint is not a linear constraint.

Example

```
constr = model.AddLinearConstraint(2 * x - y < 15)
model.GetLinearConstraintDomain(lin1.Index()) # [-OPTV_INF, 15]
```

GetEnforcementLiterals()

If certain variables being true causes other variables or constraints to be `activated`, such variables are called `enforcement literals`. For example:

$$(a \wedge b \wedge c) \rightarrow \text{constraint} \tag{8.3}$$

$$x \rightarrow y \tag{8.4}$$

Equation (1) means that the constraint is only activated when all variables a, b, c are true. See *OPTVCPConstr.OnlyEnforceIf()*.

Equation (2) means that if variable x is true, then variable y must be true. See *OPTVCPModel.AddImplication()*.

In this context, variables a, b, c, x are enforcement literals.

This function returns the indices of the enforcement literals associated with the specified constraint.

`OPTVCPModel.GetEnforcementLiterals(`*i*`)`

Parameters
i (*int*) – Index of the constraint in the model.

Returns
A list containing the indices of the enforcement literals for the specified constraint.

Return type
list[int]

Example

```
boolOr = model.AddBoolOr(b1, b2, b4)
model.GetEnforcementLiterals(boolOr.Index()) == []
boolOr.OnlyEnforceIf([b3, b4])
model.GetEnforcementLiterals(boolOr.Index()) == [b3.Index(), b4.Index()]
```

GetConstraintLiterals()

Returns the values of the literals associated with the specified constraint.

`OPTVCPModel.GetConstraintLiterals(i)`

Parameters
i (*int*) – Index of the constraint in the model.

Returns
A list containing the values of the literals for the specified constraint.

Return type
list[int]

Raises
OPTVErrCode.INVALID_MODEL_INPUT – The queried constraint is not a boolean constraint.

Example

```
boolOr = model.AddBoolOr(b1, b2, b4)
model.GetConstraintLiterals(boolOr.Index()) == [b1.Index(), b2.Index(),␣
↪b4.Index()]
```

GetConstraintName()

Returns the name of the specified linear constraint.

`OPTVCPModel.GetConstraintName(i)`

Parameters
i (*int*) – Index of the constraint in the model.

Returns
The name of the constraint located at index **i**.

Return type
str

Example

```
lin_constr = model.AddLinearConstraint(-x1 + 4 * x2 + 11 - 3, lhs = 2,␣
↪rhs = 15)
```

(continues on next page)

(continued from previous page)

```
lin_constr.WithName("c0")
model.GetConstraintName(lin_constr.Index()) # c0
```

Set()

Parameters

Sets a solver parameter for the model. Once a model has been created, this interface provides the ability to control parameters specifically for this model.

`OPTVCPModel.Set(`*param*, *value*`)`

Parameters

- **param** (OPTVIntParam / OPTVDblParam / *str*) – Model parameter to be set.
- **value** (*int* / *float* / *str*) – Value to be set.

Returns
None.

Return type
None

Raises
OPTVErrCode.UNKNOWN_PARAMETER – An unknown parameter is provided.

Example

```
model.Set(OPTVIntParam.CP_THREADS, 3)
model.Set("CP_GAP", "0.01")
```

OPTVCPSolver

class OPTVCPSolver

The OPTVCPSolver is used to solve constraint programming problems.

Solve()

Solves input constraint programming model.

`OPTVCPSolver.Solve(`*model*`)`

This function solves input constraint programming model and returns the solving status.

Parameters
model (OPTVCPModel) – Constraint programming model.

Returns
Solving status.

Return type
int

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)

lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15)
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, [3, 4, 6, 10,
↪ 12, 14])

model.Minimize(b1 + b2 + b3 + b4 + 2 * x1 + 2 * x2)
status = solver.Solve(model)
```

GetObjectiveValue()

Returns the objective value of the last solved model.

`OPTVCPSolver.GetObjectiveValue()`

This function returns the objective value of the last solved model.

Returns

Objective value of the last solved model.

Return type

float

Raises

OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the value.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
objVal = solver.GetObjectiveValue()
```

GetBestObjectiveBound()

Returns the best objective bound of the solved model.

OPTVCPSolver.**GetBestObjectiveBound**()

This function returns the best objective bound of the last solved model.

Returns
Best objective bound of the last solved model.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the value.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
objBnd = solver.GetBestObjectiveBound()
```

GetValue()

Returns the value of the given object from the last solved model.

OPTVCPSolver.**GetValue**(*expr*)

This function returns the value of the specified variable or expression from the last solved model.

Parameters
expr (OPTVCPVar/OPTVCPLinExpr) – A variable (*OPTVCPVar*) or linear expression (*OPTVCPLinExpr*) composed of variables from the last solved model.

Returns
The value of the specified variable or expression from the last solved model.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the objective value or any of variables does not apply to the current model.

Example

```
env = OPTVEnv()
model = OPTVCPModel(env)
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
```

(continues on next page)

(continued from previous page)

```
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)

lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15)
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, [3, 4, 6, 10,
↪ 12, 14])

model.Minimize(b1 + b2 + b3 + b4 + 2 * x1 + 2 * x2)
status = solver.Solve(model)

resB1 = solver.GetValue(b1)
```

GetValues()

Returns the values of the given variables from the last solved model.

`OPTVCPSolver.GetValues`(*vars*)

This function returns the values of the given variables from the last solved model.

Parameters

vars (*iterable*) – One or more variables from the last solved model.

Returns

A list of values of the given variables from the last solved model.

Return type

list[float]

Raises

OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the objective value or any of variables does not apply to the current model.

Example

```
env = OPTVEnv()
model = OPTVCPModel(env)
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)

lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15)
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, [3, 4, 6, 10,
↪ 12, 14])
```

(continues on next page)

(continued from previous page)

```
model.Minimize(b1 + b2 + b3 + b4 + 2 * x1 + 2 * x2)
status = solver.Solve(model)

result = solver.GetValues(b1, b2, b4)
```

GetBooleanValue()

Returns the value of the given boolean variable from the last solved model.

OPTVCPSolver::GetBooleanValue(var)

This function returns the boolean value of the specified variable from the last solved model.

Parameters
var (OPTVCPVar) – Boolean variable from the last solved model.

Returns
The value of the specified boolean variable from the last solved model.

Return type
float

Raises
OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the objective value, provided variable is not binary, or variable does not apply to the current model.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
result = solver.GetBooleanValue(b1)
```

GetBooleanValues()

Returns the values of the given boolean variables from the last solved model.

OPTVCPSolver.**GetBooleanValues**(**vars*)

This function returns the values of the given boolean variables from the last solved model.

Parameters
vars – One or more boolean variables from the last solved model.

Returns
A list of values of the given boolean variables from the last solved model.

Return type
list[float]

Raises

OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the objective value, any of provided variables is not binary, or any of variables does not apply to the current model.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
result = solver.GetBooleanValues(b1, b2, b3, b4)
```

GetSolution()

Returns the values of all variables of the last solved model.

OPTVCPSolver.**GetSolution**()

This function returns the values of all variables of the last solved model.

Returns

A list of values of the last solved model.

Return type

list[int]

Raises

OPTVErrCode.DATA_NOT_AVAILABLE – The model must be solved before getting the value.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
result = solver.GetSolution()
```

OPTVCPVar

class OPTVCPVar

OptVerse CP variable object. Used to access variable data from a model. Variables should not be created through their constructor. Instead, a user should add a variable to a model using the *OPTVCPModel.NewIntVar*, *OPTVCPModel.NewIntVarFromDomain*, or *OPTVCPModel.NewBoolVar* interfaces.

Index()

Returns the index of the variable in a model.

OPTVCPVar.**Index**()

Returns
Index of the variable in the model.

Return type
int

Return values

- **-1** – Variable is not in a model.
- **>=0** – Index of the variable in the model.

Example

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

print(x2.Index())  # 1
print(x1.Index())  # 0
```

GetName()

Returns the name of the variable.

OPTVCPVar.**GetName**()

Returns
Variable name.

Return type
str

Raises
OPTVErrCode.NOT_IN_MODEL – Variable is not in a model.

Example

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

print(x2.GetName())  # "x1"
print(x1.GetName())  # "x2"
```

GetDomain()

Returns the domain of the variable.

OPTVCPVar.**GetDomain**()

Returns
Domain of the variable.

Return type
list[int]

Example

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

print(x2.GetDomain())  # [0, 3, 5, 7, 15, 15]
print(x1.GetDomain())  # [2, 5]
```

SameAs()

Determines if two variables are actually the same underlying object.

OPTVCPVar.**SameAs**(*v2*)

Parameters
v2 (OPTVCPVar) – Variable to check sameness against.

Returns
Variable sameness.

Return type
bool

Return values

- **True** – Variables are the same.
- **False** – Variables are not the same.

Example

```
model = OPTVCPModel(env)

b1 = model.NewBoolVar("b1")
x1 = model.NewIntVar(2, 5, "x1")

print(b1.SameAs(x1)) # False
```

IsBinary()

Checks whether the variable is binary.

`OPTVCPVar.IsBinary()`

Returns
Whether the variable is binary.

Return type
bool

Raises
OPTVErrCode.NOT_IN_MODEL – Variable is not in a model.

Example

```
model = OPTVCPModel(env)

b1 = model.NewBoolVar("b1")
x1 = model.NewIntVar(2, 5, "x1")

print(b1.IsBinary()) # True
print(x1.IsBinary()) # False
```

OPTVCPLinExpr

class OPTVCPLinExpr

The *OPTVCPLinExpr* class is used to construct linear constraints and objective functions. An object of this type is constructed from variables and constant terms. The expressions are accessed as constructed. That is, an expression `1 * x + 2 * y + 3 * x` will be accessed as:

Index	Coef	Variable
0	1	x
1	2	y
2	3	x

AddTerms()

Adds terms to the end of the linear expression.

OPTVCPLinExpr.**AddTerms**(*coeffs*, *vars*)

This function will add the terms coeffs$_i$ $*$ vars$_i$, $\forall i \in [0, \text{vars.size()}]$ to the end of the expression.

Parameters

- **coeffs** (*float|iterable*) – One or more coefficient values.
- **vars** (*iterable*) – One or more variables from the last solved model.

Returns
None.

Return type
None

Raises
OPTVErrCode.INVALID_MODEL_INPUT – `vars` and `coeffs` sizes do not match.

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")
x3 = model.NewIntVar(0, 5, "x3")
x4 = model.NewIntVar(0, 3, "x4")

expr = x1 + 1 + x2
expr.AddTerms([2, 3], [x3, x4]) # x1 + x2 + 2*x3 + 3*x4 + 1
```

Clear()

Clears the inner "terms" container, such that *OPTVCPLinExpr.Size()* will return `0`. Additionally, resets the expression such that it is numerically equivalent to `0`.

OPTVCPLinExpr.**Clear**()

Returns
None.

Return type
None

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr = x1 + 1 + x2

expr.Clear()
```

GetCoef()

Returns the coefficient for a variable at a given index in the expression.

OPTVCPLinExpr.**GetCoef**(*i*)

Parameters
i (*int*) – Index of the variable in the expression.

Returns
Coefficient of variable at position i.

Return type
float

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If i $\notin [0,$ this->Size()$]$.

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr = 3*x1 + 1 - x2

print(expr.GetCoef(0)) # 3
print(expr.GetCoef(1)) # -1
```

GetVar()

Returns the variable at a specified index in the expression.

OPTVCPLinExpr.**GetVar**(*i*)

Parameters
i (*int*) – Index of the variable in the expression.

Returns
Variable at position i.

Return type
OPTVCPVar

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If i $\notin [0,$ this->Size()$]$.

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr = 3*x2 + 1 - x1
```

(continues on next page)

(continued from previous page)

```
print(expr.GetVar(0).SameAs(x2)) # True
print(expr.GetVar(1).SameAs(x1)) # True
```

GetConstant()

Returns the constant term from the expression.

OPTVCPLinExpr.**GetConstant**()

Returns
Constant value in expression.

Return type
float

Example

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr1 = 3*x1 + 1 - x2
expr2 = x1 + x2

print(expr1.GetConstant()) # 1
print(expr2.GetConstant()) # 0
```

SameAs()

Determines if two linear expressions are actually the same underlying expression.

OPTVCPLinExpr.**SameAs**(*e2*)

Parameters
e2 (OPTVCPLinExpr) – Linear expression to check sameness against.

Returns
Expression sameness.

Return type
bool

Return values

- **True** – Expressions are the same.
- **False** – Expressions are not the same.

Example

```
expr1 = x1 + x2
expr2 = x1 + 1
expr3 = expr1 + expr2
```

(continues on next page)

(continued from previous page)

```
expr3.SameAs(x1 + x2 + x1 + 1) # True
```

Size()

Returns the size of the expression.

`OPTVCPLinExpr.Size()`

This value corresponds to the number of elements in the expression, regardless of whether the element is repeated. For example, the expression `x+y` has `Size = 2`. The expression `x+y+x` has `Size = 3`.

Returns
Size of the linear expression.

Return type
int

Example

```
expr1 = x1 + x2
expr2 = x1 + 1
expr3 = x1 + x2 + x1 + 1

print(expr1.Size()) # 2
print(expr2.Size()) # 1
print(expr3.Size()) # 3
```

Remove()

Remove term(s) from the expression.

`OPTVCPLinExpr.Remove(i)`

Removes the term at a specified index from the expression.

Parameters
i (*int*) – Index of term to be removed.

Returns
None.

Return type
None

Raises
OPTVErrCode.INDEX_OUT_OF_RANGE – If `i` $\notin [0, \texttt{this->Size()}]$.

Example

```
expr = 3*x2 + 1 - x1

expr.Remove(1)
print(expr) # 3*x2 + 1
```

OPTVCPLinExpr.**Remove**(*v*)

Removes any instance of a variable from the expression.

Parameters
v (OPTVCPVar) – Variable to be removed.

Returns
Whether at least one term removed.

Return type
bool

Return values

- **True** – One or more terms was removed.
- **False** – No terms were removed.

Raises
OPTVErrCode.NOT_IN_MODEL – If variable is not in a model.

Example

```
expr = 3*x2 + 1 - x1

expr.Remove(x2)
print(expr) # 1 - x1
```

OPTVCPConstr

class OPTVCPConstr

OptVerse CP constraint object. Used to access constraint data from a model. Constraints should not be created through their constructor. Instead, a user should add a constraint to a model using the *OPTVCPModel* interfaces.

OnlyEnforceIf()

Adds enforcement literals (boolean variables or their negations) to the constraint. The constraint will only be active if all enforcement literals evaluate to true.

OPTVCPConstr.**OnlyEnforceIf**(*literals*)

Parameters
literals (OPTVCPVar | *iterable*) – One or more boolean variables (or their negations).

Returns
None.

Return type
None

Raises

- *OPTVErrCode.NOT_IN_MODEL* – Constraint is not in a model.
- *OPTVErrCode.INVALID_MODEL_INPUT* – Some of the variables are not binary.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")ref-man/EN/source/manual_text/optverse_api/
↪cp-sat/python_api/OPTVCPModel/get_enforcement_literals.rst
b4 = model.NewBoolVar("b4")

boolAnd = model.AddBoolAnd(b1, ~b2)
boolAnd.OnlyEnforceIf(b3, b4)

model.GetEnforcementLiterals(boolAnd) == [b3.Index(), b4.Index()]
```

WithName()

Sets the name of the constraint.

`OPTVCPConstr.`**`WithName`**`(`*name*`)`

Parameters
name (*str*) – Name of the constraint.

Returns
None.

Return type
None

Raises
OPTVErrCode.NOT_IN_MODEL – Constraint is not in a model.

Example

```
model = OPTVCPModel()
domain = [1, 2, 5, 6, 7, 15]
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domain, name="x2")

constr = model.AddLinearConstraint(-x1 + 4 * x2 + 11 - 3, lhs = 2, rhs␣
↪= 15)

constr.WithName("c0")

print(constr.GetName()) # c0
```

Index()

Returns the index of the constraint in its model.

`OPTVCPConstr.`**`Index`**`()`

Returns
Index of constraint.

Return type
int

Return values

- **-1** – Constraint is not in a model.
- **>=0** – Constraint index in a model.

Example

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")

boolAnd = model.AddBoolAnd(b1, ~b2)
boolOr = model.AddBoolOr(b1, b2)

print(boolAnd.Index()) # 0
print(boolOr.Index())  # 1
```

GetName()

Gets the name of the constraint.

OPTVCPConstr.**GetName**()

Returns
Name of the constraint.

Return type
str

Raises
OPTVErrCode.NOT_IN_MODEL – Constraint is not in a model.

Example

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

table = model.AddAllowedAssignments([x1, x2], [[1, 2], [3, 4]])
table.WithName("table")

print(table.GetName()) # table
```

GetType()

Returns the type of the constraint.

OPTVCPConstr.**GetType**()

Returns
Type of the constraint.

Return type
OPTVCBType

Raises
OPTVErrCode.NOT_IN_MODEL – Constraint is not in a model.

Example

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

table = model.AddAllowedAssignments([x1, x2], [[1, 2], [3, 4]])
interval1 = model.AddInterval(2 * x2, 2 * x1 + 1, 9)
boolXor = model.AddBoolXor(b1, b2, b3, b4)
AMO = model.AddAtMostOne(b1, b2, b3)
EO = model.AddExactlyOne(b1, b2, b3)
allDiff = model.AddAllDifferent(2 * x1 + 3, 11 * x2 + 12, 15, b2)

table.GetType() == OPTVCPConstraintType.TABLE
interval1.GetType() == OPTVCPConstraintType.INTERVAL
boolXor.GetType() == OPTVCPConstraintType.BOOL_XOR
AMO.GetType() == OPTVCPConstraintType.AT_MOST_ONE
EO.GetType() == OPTVCPConstraintType.EXACTLY_ONE
allDiff.GetType() == OPTVCPConstraintType.ALL_DIFF
```

SameAs()

Determines if two constraints are actually the same underlying object.

OPTVCPConstr.**SameAs**(*c2*)

Parameters
c2 (OPTVCPConstr) – Constraint to check sameness against.

Returns
Constraint sameness.

Return type
bool

Return values

- **True** – Constraints are the same.
- **False** – Constraints are not the same.

Example

```
model = OPTVCPModel()
domain = [1, 2, 5, 6, 7, 15]
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domain, name="x2")

constr = model.AddLinearConstraint(-x1 + 4 * x2 + 11 - 3, lhs = 2, rhs␣
↪= 15)

model.GetConstr(0).SameAs(constr)
```

OPTVCPTempConstr

class OPTVCPTempConstr

OptVerse CP temporary constraint object. The `OPTVCPTempConstr` class serves as a placeholder for intermediate results in constraint expressions formed with overloaded operators. This class does not contain any member functions. Instead, comparison operations (==, !=, <=, >=) on expression and variable types (*OPTVCPVar*, *OPTVCPLinExpr*) return `OPTVCPTempConstr` objects. These are typically consumed immediately by constraint-adding methods and are not intended for storage in user-defined variables.

The `OPTVCPTempConstr` object allows you to create linear constraints:

- **Linear Constraint**: A linear constraint is defined as an expression of the form `Expr1 sense Expr2`, where `Expr1` and `Expr2` can be *OPTVCPVar* objects, *OPTVCPLinExpr* objects, or numeric constants, and `sense` is one of the comparison operators: ==, !=, <=, or >=. For example, both `x + y <= 1 + z` and `x + y == 5` are valid linear temporary constraints.

The following example illustrates this:

```
model.AddLinearConstraint(x + y >= 1)
```

OPTVCPVariableSelectionStrategy

Variable selection strategy of the CP Solver.

class OPTVCPVariableSelectionStrategy

CHOOSE_FIRST

Type: int

Choose the first variable from variable list.

CHOOSE_LOWEST_MIN

Type: int

Choose the variable with the minimum lower bound.

CHOOSE_HIGHEST_MAX

Type: int

Choose the variable with the maximum upper bound.

CHOOSE_MIN_DOMAIN_SIZE

Type: int

Choose the variable with the minimum domain range.

CHOOSE_MAX_DOMAIN_SIZE

Type: int

Choose the variable with the maximum domain range.

OPTVCPDomainReductionStrategy

Domain reduction strategy of the CP Solver.

class OPTVCPDomainReductionStrategy

SELECT_MIN_VALUE

Type: int

Select the lower bound.

SELECT_MAX_VALUE

Type: int

Select the upper bound.

SELECT_LOWER_HALF

Type: int

Select the bound value calculated by `bound = lb + (ub - lb) / 2`.

SELECT_UPPER_HALF

Type: int

Select the bound value calculated by `bound = ub - (ub - lb) / 2`.

SELECT_MEDIAN_VALUE

Type: int

Select the median value of the variable.

SELECT_RANDOM_HALF

Type: int

Select random half.

OPTVCPStatus

CP model status code.

OPTV_CP_UNKNOWN

Solver could not find a solution.

Type
int

Value
0

OPTV_CP_MODEL_INVALID

The model is invalid.

Type
int

Value
1

OPTV_CP_FEASIBLE

Solver found a feasible solution.

Type
int

Value
2

OPTV_CP_INFEASIBLE

Solver proved the model is infeasible.

Type
int

Value
3

OPTV_CP_OPTIMAL

Solver found an optimal solution.

Type
int

Value
4

`OPTV_CP_USER_INTERRUPT`

Solver was interrupted by the user.

Type
int

Value
5

CHAPTER

NINE

SDK RECORDINGS

The OptVerse SDK supports the creation of "recording" files, enabling users to record a series of SDK commands into a file. With these recordings, a user may "replay" the recording through the `optverse-replay` binary.

Recording files will compress any necessary input data (such as `.mps` files), meaning that only the recording file is needed to rerun the API calls.

These files enable customers to easily test deployment scenarios, and more importantly, provide OptVerse team members with additional information when providing user support.

9.1 Record

To enable the creation of a recording file, a user must set the *OPTVStrParam::REPLAY_FILE* parameter to be a non-empty string. If this parameter is an empty string, no recording will be produced (default behaviour).

Unlike other parameters, this parameter **can not** be set through an *OPTVModel* object. This parameter must be set through the environment, and it must be prior to construction of an *OPTVModel* object.

```
/**
 * This script illustrates how to generate a recording file.
 * This is accomplished through the `OPTVStrParam::REPLAY_FILE` parameter in the OPTVEnv␣
↪object.
 *
 * The recording file produced by this script can be rerun using optverse-replay.
 */
#include "optv_c++.h"
#include <iostream>

using namespace std;

int main()
{
    try {
        OPTVEnv env("lp1.log");
        env.Set(OPTVStrParam::REPLAY_FILE, "optv.rec");
        OPTVModel model(env);

        // add variables to the model, all variables are continuous
        OPTVVar x = model.AddVar(0, 1, -1, OPTV_CONTINUOUS, "x");
        OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_CONTINUOUS, "y");
        OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z");
```

(continues on next page)

(continued from previous page)

```
        // add constraint to the model
        OPTVConstr c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0");
        OPTVConstr c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1");

        model.Optimize();
        model.Write("lp1.lp");

        if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
            cout << "Best solution: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;

            cout << "x = " << x.Get(OPTVDblAttr::X) << endl;
            cout << "y = " << y.Get(OPTVDblAttr::X) << endl;
            cout << "z = " << z.Get(OPTVDblAttr::X) << endl;
        } else {
            cout << "No feasible solution available!" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Note

Presently, SDK Recording and Replay only works with one recording per environment.

9.2 Replay

To replay an OptVerse recording file, simply provide the path to the recording file to the `optverse-replay` binary. The `optverse-replay` binary will then execute the recorded commands in sequence.

Note

Presently, SDK Recording and Replay only works with one recording per environment.

CHAPTER

TEN

NETWORK FLOW PLANNING PROBLEMS

Network flow planning problems are a special class of linear programming problems. This chapter will provide a brief explanation of these problems. The OptVerse solver can automatically recognize the structure of network flow planning problems and use more efficient algorithms (network simplex method) for solving them.

10.1 Problem Definition

A linear programming problem is called a **network flow planning problem** if it satisfies the following conditions:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b, \\ & l \leq x \leq u, \end{aligned}$$

where

$$\begin{aligned} c &\in \mathbb{R}^m \\ b &\in \mathbb{R}^n \\ A &\in \{-1, 0, 1\}^{n \times m} \end{aligned}$$

and the problem satisfies the following conditions:

- Each column of the coefficient matrix A has only one 1 and one -1 (i.e., all other coefficients are zero).
- $\sum_i b_i = 0, i \in \{1, \ldots, n\}$

Here, n is the number of constraints, and m is the number of variables.

10.2 Problem Example

- Objective function:

$$\min x_1 + x_2 + x_3 + x_4$$

- Constraints:

$$\begin{bmatrix} 1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \\ -5 \end{bmatrix}$$

- Variable bounds:

$$
\begin{aligned}
0 \leq x_1 \leq 4, \\
0 \leq x_2 \leq 2, \\
0 \leq x_3 \leq 4, \\
0 \leq x_4 \leq 10
\end{aligned}
$$